Chinith: Algorithm Specifications of Signature Schemes SM4th, uBlockith, and Vistrutith

v0.0.2

Weihan Li[1], Yuchen Wang[1], Zhelei Zhou[1], Cheng Hong[1], and Tao Wei[1]

Ant Group, Hangzhou, China
{liweihan.lwh, tianwu.wyc, zhouzhelei.zzl, vince.hc, lenx.wei}@antgroup.com

October 30, 2026

Changelog

November 2, 2026. Reassessed the QROM query bounds, including signing-query budgets and reduction-resource costs.

October 30, 2026. Corrections addressing public feedback:

- **Final challenge.** The original implementation omitted $\tilde{a}_0$, the constant-term response of the QuickSilver proof, from the final Fiat–Shamir hash. Signing and verification now include all response components $\tilde{a}_0, \ldots, \tilde{a}_{d-1}$ in this hash.
- **Signing loop.** The original specification's grinding test was reversed and the opening rejection reused the counter. Now, the opening follows grinding success such that every failed trial advances the counter and exhaustion returns $\perp$. The per-trial success probability changes from $2^{-w_{\mathsf{grind}}}$ to $\alpha\, 2^{-w_{\mathsf{grind}}}$, where α is the conditional opening acceptance probability.
- **Witnesses.** The EM encryption-witness slice changes from $\boldsymbol{w}[0\,..\,\ell_{\mathsf{enc}})$ to $\boldsymbol{w}[N_{\mathsf{block}}\,..\,\ell_{\mathsf{enc}})$. For uBlockith-EM and Vistrutith, ExtendWitness changes from arguments (sk_2, sk_1) to (sk_1, sk_2).
- **Coefficients and masks.** Vistrutith treated different constraint families as different polynomial coefficients. Each family now contributes its complete coefficient vectors at degrees $0, 1, 2$. The uBlockith specification's response masks change from $(u_0^*, u_1^* + v_0^*, v_1^*)$ to $(v_0^*, u_0^* + v_1^*, u_1^*)$.
- **Specification constants.** In the uBlock key-schedule permutation PK (Table 10), the entries at indices $(8, 9, 12, 13, 14, 15)$ change from $(14, 6, 1, 3, 11, 9)$ to $(1, 14, 9, 11, 3, 6)$. For SM4th-ti, the little-endian bytes of the SM4 subfield-embedding constant α_8 (Table 12) change from `0b d7 82 9e 1c c5 6e 4a a4 52 ca 47 5a 3a 72 e0 70 13 a2 00` to `d5 1d 5b 51 62 cb 88 67 b3 c6 6e 3e 1f aa 3b 35 70 69 9c 13`.
- **Reference implementation.** The reference implementation now evaluates S-boxes and finite-field operations without secret-indexed lookup tables by default.
- **Intel SM4 testing.** We tested the optimized implementations of SM4th and SM4th-EM on an Intel processor with native SM4 instructions.

Contents

Chinith: Algorithm Specifications of Signature Schemes SM4th, uBlockith, and Vistrutith .. 1
Weihan Li, Yuchen Wang, Zhelei Zhou, Cheng Hong, and Tao Wei
Changelog 1
1 Introduction 4
2 Algorithm Description: An Overview 5
2.1 Notations 6
2.2 Generating VOLE Correlations from VOLE Commitments 6
2.3 Zero-Knowledge Proof Phase 8
2.4 Putting Things Together 9
3 Design Rationale 9
3.1 Mathematical Hard Problems and Hardness Assumptions 9
3.2 Parameters and Criteria for Parameter Selection 10
3.3 Key and Signature Encodings 16
3.4 Analysis of Failure Probability 16
3.5 Statements of Differences 16
3.6 Statements of Additional Functionalities 17
4 Algorithm Description: Cryptographic Primitives 17
4.1 The SM4 Block Cipher 17
4.1.1 The Key Expansion Routine of SM4 19
4.1.2 The Encryption Routine of SM4 20
4.2 The uBlock Block Cipher 20
4.2.1 The Key Expansion Routine of uBlock 22
4.2.2 The Encryption Routine of uBlock 23
4.3 The Ballet Block Cipher 24
4.4 The Vistrutah Block Cipher 24
4.4.1 The Encryption Routine of Vistrutah 27
4.5 Pseudo-Random Generators 28
4.6 Linear Universal Hash Functions 30
5 Algorithm Description: General Building Blocks 33
5.1 VOLE-in-the-Head Functions 33
5.1.1 Batch All-but-One Vector Commitments 33
5.1.2 Seed Expansions and Conversion to VOLE 37
5.1.3 Construction of VOLE Commitments 40
5.2 Zero-Knowledge Operations over VOLE Commitments 42
6 Algorithm Description: the SM4th Signature Scheme 43
6.1 Zero-Knowledge Building Blocks for SM4th 43
6.1.1 Building Blocks for Non-EM Variants 46
6.2 Witness Extension 52
6.3 Deriving Constraints for the Key Expansion 53
6.4 Deriving Constraints for the Encryption 55
6.5 Constraints for the OWF 57
6.6 Construction of SM4th 60
6.6.1 Encoding and Input Lengths 60
6.6.2 Key Generation 60
6.6.3 Signing 62
6.6.4 Verifying 64
7 Algorithm Description: the uBlockith Signature Scheme 65
7.1 Zero-Knowledge Building Blocks for uBlockith 65
7.2 Witness Extension 67
7.3 Deriving Constraints for the Key Expansion 68

7.4 Deriving Constraints for the Encryption . . . 68
7.5 Constraints for the OWF . . . 70
7.6 Construction of uBlockith . . . 73
7.6.1 Key Generation . . . 73
7.6.2 Signing . . . 73
7.6.3 Verifying . . . 73
8 Algorithm Description: the Vistrutith Signature Scheme . . . 74
8.1 Zero-Knowledge Building Blocks for Vistrutith . . . 74
8.2 Witness Extension . . . 85
8.3 Deriving Constraints for the Encryption . . . 86
8.4 Complete OWF Constraints . . . 91
8.5 Construction of Vistrutith . . . 93
8.5.1 Key Generation . . . 93
8.5.2 Signing . . . 94
8.5.3 Verifying . . . 96
9 Security Statements and Analysis . . . 96
9.1 Theoretical Security . . . 96
9.1.1 Security Targets and Model . . . 96
9.1.2 Building Blocks . . . 98
9.1.3 One-Wayness and Public-Key Pseudorandomness of Block Ciphers . . . 101
9.1.4 Security Analysis of VOLE Commitments . . . 102
9.1.5 EUF-CMA Security in the ROM . . . 105
9.1.6 EUF-CMA Security in the QROM . . . 107
9.1.7 Conditional Query Bounds . . . 111
9.1.8 Reference Attack-Cost Models . . . 112
9.2 Practical Security against Known Attacks . . . 114
9.2.1 Exhaustive and quantum search for an OWF preimage . . . 114
9.2.2 Brute Force the PRG and Multi-Key Attacks . . . 115
9.2.3 Attack Soundness of the ZK Protocol . . . 116
10 Performance Evaluation . . . 116
10.1 Performance Analysis . . . 116
10.2 Detailed Configuration of Implementation Platforms . . . 117
10.3 Acceleration Techniques . . . 119
10.3.1 Acceleration Techniques of PRGs . . . 119
10.3.2 Acceleration Techniques of Field Operations . . . 119
10.4 Performance Test Results . . . 121

1 Introduction

This document describes and specifies Chinith[1], including SM4th, uBlockith and Vistrutith, a family of signature schemes based on block ciphers and zero-knowledge proofs (ZKPs) [1] following the "in-the-head" paradigm, targeting 128-bit, 256-bit, and 512-bit classical security, respectively. These schemes are designed to achieve post-quantum security using only information-theoretic and symmetric-key cryptographic primitives. Their security is bound to the security of the Chinese national standard SM3 hash function [2] and the underlying block ciphers. In particular, the block ciphers employed in Chinith include the 128-bit-block Chinese national standard SM4 [3], as well as the 256-bit-block uBlock [4] and Ballet [5], which are solely designed by Chinese researchers and were the only two first-prize winners in the block-cipher category of the Chinese Cryptographic Algorithm Design Competition[2]. For the instantiation targeting 512-bit classical security, we currently use Vistrutah [6], a cipher with a 512-bit block size. Future revisions may instead adopt a suitable Chinese-designed symmetric primitive as the NGCC program advances[3].

Let $E_{\boldsymbol{k}}(\boldsymbol{x})$ be a one-way function (OWF) with input binary vector $\boldsymbol{x}$ and secret input $\boldsymbol{k}$. In our case, it can be the encryption algorithm of a block cipher with key $\boldsymbol{k}$ and plaintext $\boldsymbol{x}$. A signature is generated by a ZKP, in which the signer acts as the prover and proves to the receiver (*i.e.*, verifier) that it knows $\boldsymbol{k}$ such that $E_{\boldsymbol{k}}(\boldsymbol{x}) = \boldsymbol{y}$ for public $\boldsymbol{x}, \boldsymbol{y}$ and $E(\cdot)$. Such a ZKP is typically originally a public-coin interactive argument and can be converted into non-interactive via the Fiat-Shamir transformation [8], where the public-coin challenges are generated by hashing each-round transcripts with the hashes modeled as random oracles. Here, the public key is $(\boldsymbol{x}, \boldsymbol{y})$, the secret key is $(\boldsymbol{x}, \boldsymbol{k})$, and the signature is the ZKP proof with the message embedded in the first-round transcript. Technically, the ZKPs are built based on the *Vector Oblivious Linear Evaluation in the Head* (VOLEitH) [9, 10] framework, and specifically, a variant of the QuickSilver protocol [11].

We give reductions for existential unforgeability against chosen-message attacks (EUF-CMA) in the random oracle model (ROM) and the quantum-accessible random oracle model (QROM) under the primitive assumptions stated in Section 3. We also provide a quantitative analysis of these reductions and a reference concrete-security analysis against known attacks.

Related works and comparisons. Our signature schemes are similar to other post-quantum signature schemes via the "in-the-Head" techniques. The original work is PICNIC [12], which is constructed from MPC-in-the-Head (MPCith) [13] and using LowMC as OWF [14]. The MPCith framework is later improved by the threshold-computation-in-the-head (TCitH) [15] framework. Subsequent works combine the MPCith/TCitH framework with various OWFs to build signatures with different trade-offs, such as Banquet [16] from AES, Mirath [17] from the Min-Rank problem, MQOM [18] from the multivariate quadratic problem, PERK [19] from the permuted kernel problem, and RYDE [20] or SDitH [21] from syndrome decoding. Recently, VOLEitH [10] is proposed, which can lead to smaller signatures sizes due to the more efficient repetition method to achieve target soundness, such as FAEST [22] from AES and ReSolveD [23] from syndrome decoding.

There also exist other post-quantum signatures based on different assumptions and techniques, such as lattice-based Dilithium [24], Falcon [25], and HAWK [26], code-based CROSS [27] and LESS [28], multivariate-based MAYO [29], isogeny-based SQIsign [30], and solely hash-based SPHINCS$^+$ [31]. Our signatures achieve different trade-offs in security and performance.

Compared to them, our most promising feature is that the security of our signature schemes relies solely on symmetric ciphers and does not depend on any number-theoretic or structured hardness assumptions. Among the related works, FAEST and SPHINCS$^+$ also achieve this property. However, this advantage comes at the cost that our scheme (as well as FAEST and SPHINCS$^+$) is expected to have worse performance than those schemes based on structured hardness assumptions. Nevertheless, our schemes feature smaller signature sizes, faster signing time and comparable verification time compared with the standardized SLH-DSA version of SPHINCS$^+$.

[1] CHINese block ciphers In The Head

[2] https://www.cacrnet.org.cn/site/content/854.html

[3] Although the NGCC block-cipher submission requirements [7] do not specify 512-bit blocks, the required parameter set with 256-bit blocks and 512-bit keys can in principle meet our target security level through a suitable multi-block OWF construction.

Outline of this document. In Section 2, we present an overview of the algorithms, which illustrates the main ideas of the design. Section 3 gives the rationales for the design, including assumptions and hard problems, parameters, and their criteria.

Section 4 introduces the cryptographic primitives used our schemes, including block ciphers, PRGs, and universal hashes. Section 5 specifies several general building blocks of all signature schemes, including VOLE commitments and zero-knowledge operations over VOLE commitments.

Section 6, Section 7, and Section 8 propose the concrete construction of SM4th, uBlockith, and Vistrutith, respectively.

Section 9 analyzes the security of our schemes, including theoretical security, concrete security, and practical security against known attacks. Section 10 presents signature-size analysis, implementation platforms, instruction-set acceleration, and performance results.

2 Algorithm Description: An Overview

This section provides a high-level overview of the proposed signature schemes. Let E be a public block-cipher-based OWF. In the standard OWF, the public key is defined as $\mathsf{pk} = (\boldsymbol{x}, \boldsymbol{y})$ with $\boldsymbol{y} = E_{\boldsymbol{k}}(\boldsymbol{x})$, where $\boldsymbol{x}$ is public and $\boldsymbol{k}$ is secret. In the Even–Mansour OWF, the public key is again $\mathsf{pk} = (\boldsymbol{x}, \boldsymbol{y})$ with $\boldsymbol{y} = E_{\boldsymbol{x}}(\boldsymbol{k}) \oplus \boldsymbol{k}$, where $\boldsymbol{x}$ is the public block-cipher key and $\boldsymbol{k}$ is the secret input block. In the concrete algorithms below, the secret key is encoded as $\mathsf{sk} = (\boldsymbol{x}, \boldsymbol{k})$, so the second component is always the secret witness/key component. A signature is generated via ZKP, where the signer acts as the prover and proves to the receiver (or verifier) that it knows $\boldsymbol{k}$ satisfying the corresponding OWF relation for public $\boldsymbol{x}$, $\boldsymbol{y}$, and $E(\cdot)$.

The proposed signatures use a variant of QuickSilver [11] as the underlying ZKP. Such ZKPs assume a preprocessing phase to generate a sub-field VOLE correlation, which, saying of length ℓ, is defined by a random bit vector $\boldsymbol{u} \in \mathbb{F}_2^{\ell}$, a random field-element vector $\boldsymbol{v} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{\ell}$, a random global key $\Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, and local keys $\boldsymbol{q} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{\ell}$ such that for $i \in [0, \ell)$,

$$q_i = u_i \cdot \Delta + v_i. \tag{1}$$

The values u_i and v_i should be known only to the prover, while Δ should be picked by the verifier, and q_i should be given to the verifier. In our signature schemes, such VOLE correlations are produced by the VOLEitH technique [10], where the prover generates u_i, v_i and commits to them using a VOLE commitment. The commitment opening lets the verifier reconstruct the q_i values satisfying Equation (1) for a random Δ without learning anything extra. In the complete protocol, the prover binds the consistency and constraint responses before the verifier reveals the final challenge Δ and requests the opening (*cf.*, Section 2.4).

VOLE correlations in Equation (1) can be alternatively seen as the prover committing a bit u_i with commitment q_i and openings u_i and v_i. This commitment scheme is perfectly hiding due to the mask $v[i]$. Two openings $(u[i], v[i])$ and $(u'[i], v'[i])$ to the same $q[i]$ satisfy $q[i] = u[i]\Delta + v[i] = u'[i]\Delta + v'[i]$. If $u'[i] \neq u[i]$, they determine

$$\Delta = \frac{v[i] - v'[i]}{u'[i] - u[i]}. \tag{2}$$

Thus, without information about the uniformly random Δ, an attempted distinct opening succeeds with probability $2^{-\lambda_{\mathbb{F}}}$. The commitment scheme has linear homomorphism such that if q_i and q_j are commitments to u_i and u_j, respectively, then $q_i + q_j$ is exactly the commitment to $u_i + u_j$. Further, $q_i \cdot q_j = u_i u_j \cdot \Delta^2 + (u_i v_j + u_j v_i)\Delta + v_i v_j$ can be seen as a degree-2 commitment to $u_i \cdot u_j$ such that the degree-2 term stores the secret and the degree-1 or degree-0 term is used for masking.

This section is outlined as follows. Section 2.2 describes how to generate and open VOLE commitments. Section 2.3 introduces the interactive zero-knowledge proof phase given VOLE correlations. In Section 2.4, we show how VOLE commitments and ZKP are combined into an non-interactive zero-knowledge argument.

2.1 Notations

Let $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ denote a finite field with $2^{\lambda_\mathbb{F}}$ elements, where $\lambda_\mathbb{F}$ is an integer. Let $f(x) \in \mathbb{F}[x]$ denote a polynomial over $\mathbb{F}$ in the variable x. We construct $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ explicitly as $\mathbb{F}_2[z]/f(z)$, where $f(z) \in \mathbb{F}_2[z]$ is an irreducible polynomial of degree $\lambda_\mathbb{F}$. For the fields used in VOLE and universal hashing, the defining polynomials are

$$\begin{aligned} f_{64}(z) &= z^{64} + z^4 + z^3 + z + 1, & f_{128}(z) &= z^{128} + z^7 + z^2 + z + 1, \\ f_{160}(z) &= z^{160} + z^5 + z^3 + z^2 + 1, & f_{256}(z) &= z^{256} + z^{10} + z^5 + z^2 + 1, \\ f_{512}(z) &= z^{512} + z^8 + z^5 + z^2 + 1. \end{aligned} \tag{3}$$

We write $\alpha_e = z \bmod f_e(z)$ for the polynomial-basis element of $\mathbb{F}_{2^e}$. We also write $\mathrm{GF}(2^e)$ for a finite field with 2^e elements.

For a bit string $\boldsymbol{b} \in \{0,1\}^{8n}$, its byte encoding is $\mathrm{OS}(\boldsymbol{b})[j] = \sum_{t=0}^{7} b[8j+t]2^t$ for $j \in [0,n)$. A field element $a = \sum_{t=0}^{e-1} a_t \alpha_e^t$ is encoded as $\mathrm{FE}(a) = \mathrm{OS}(a_0 \| \cdots \| a_{e-1})$. The functions LE32, BE32, and BE64 encode integers in 4 little-endian bytes, 4 big-endian bytes, and 8 big-endian bytes, respectively. Their inverses parse byte strings in the same order. All hash inputs are byte strings obtained by these encodings and concatenation.

Ordinary lowercase letters such as a denote scalars (integers or field elements), according to the context. Bold lowercase letters such as $\boldsymbol{a} = (a_0, \ldots, a_{n-1}) \in \mathbb{F}^n$ denote vectors of length n. Bold uppercase letters like $\boldsymbol{A}$ denote matrices. Constant vectors may omit boldface when explicitly declared. For a vector $\boldsymbol{a}$, $a[i]$ and a_i denote its i-th entry, and $\boldsymbol{a}[i \,..\, j)$ contains entries $a[i], \ldots, a[j-1]$, while $\boldsymbol{a}[i \,..\, j]$ also includes $a[j]$. We use $\boldsymbol{a} \in (\mathbb{F}^m)^n$ for n sub-vectors of length m, *i.e.*, $\boldsymbol{a} = (\boldsymbol{a}[0], \ldots, \boldsymbol{a}[n-1])$. Here, $\boldsymbol{a}[i]$ denotes the i-th sub-vector $\boldsymbol{a}_i$. For word-valued traces, $\boldsymbol{z}[i]$ denotes word i, and $\boldsymbol{z}[i][j]$ denotes its byte j.

We use $\cdot$ to represent multiplications between field elements or scalar multiplications between field elements and vectors. We use $\log a$ to denote the base-2 logarithm of an integer a. The set $\{1, \ldots, n\}$ is denoted by $[n]$ for integer n. We use $[m,n]$ and $[m,n)$ to represent the sets $\{m, m+1, \ldots, n\}$ and $\{m, m+1, \ldots, n-1\}$ for non-negative integers $m < n$, respectively. The symbol $\oplus$ represents the bitwise exclusive-OR (XOR) operation over vectors. Additionally, $\lll i$ denotes circular left rotation by i positions. For block-cipher words, the rotation acts on the integer represented under the encoding specified for that cipher. The notation $\boldsymbol{v}(\lll i)_{32}$ applies this rotation independently to each 32-bit word.

2.2 Generating VOLE Correlations from VOLE Commitments

Generating VOLE correlations from all-but-one vector commitments. We describe how to produce a VOLE correlation of length ℓ over $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ via a all-but-one vector commitment, where the prover commits to N seeds via a single hash commitment, and can later open $N-1$ seeds with a communication complexity of $O(\log N)$ and a concrete communication of $(\log N + 2) \cdot N_{\mathsf{block}}$ bits. This all-but-one vector commitment is built from the Goldreich-Goldwasser-Micali (GGM) [32] tree technique, which builds a complete binary tree with $N = 2^{\lambda_\mathbb{F}}$ leaves, where, starting from the root node with a random N_{block}-bit seed, the two children of any node are defined by evaluating the parent seed with a length-doubling PRG that outputs two new seeds. Each random bits of these N leaves are used as the randomness in a BAVC.LeafCommit algorithm, to derive a new seed sd and a leaf commitment com. The seed sd is then expanded into ℓ bits via another PRG, contributing to both $\boldsymbol{u}$ and $\boldsymbol{v}$ in Equation (1). More concretely, suppose the N committed seeds $\mathsf{sd}_0, \ldots, \mathsf{sd}_{N-1}$ are expanded to N bit strings $\boldsymbol{r}_0, \ldots, \boldsymbol{r}_{N-1} \in \{0,1\}^\ell$. Then, $\boldsymbol{u}$ and $\boldsymbol{v}$ are defined as

$$\boldsymbol{u} = \sum_{i \in [0,N)} \boldsymbol{r}_i, \quad \boldsymbol{v} = \sum_{i \in [0,N)} i \cdot \boldsymbol{r}_i, \tag{4}$$

where the multiplication is over the field $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$.

Consider a verifier who learns seeds sd_i for all $i \neq j^*$ for some index $j^* \in [0, N)$. Note that as $N = 2^{\lambda_{\mathbb{F}}}$, j^* can be uniquely mapped into a field element $\Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Then, the verifier can compute

$$\boldsymbol{q} = \sum_{i \in [0,N)} (j^* - i) \cdot \boldsymbol{r}_i = j^* \cdot \sum_{i \in [0,N)} \boldsymbol{r}_i - \sum_{i \in [0,N)} i \cdot \boldsymbol{r}_i = j^* \cdot \boldsymbol{u} - \boldsymbol{v}. \tag{5}$$

Note that the field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ has 2-linearity, so Equation (5) is consistent with Equation (1).

Generating τ small VOLE correlations from batch all-but-one vector commitments. There exists one efficiency problem of the "one-tree" approach above: directly generating VOLE correlations over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ requires $O(2^{\lambda_{\mathbb{F}}})$ calls of PRG, which is exponential and unacceptable. To solve this problem, VOLEitH first generates multiple VOLE instances over smaller finite fields and then combines them into ones over the target large field. Specifically, VOLEith creates τ_0 VOLE instances over field $\mathbb{F}_{2^{k_0}}$ and τ_1 VOLE instances over field $\mathbb{F}_{2^{k_1}}$. If the prover commits the *same* $\boldsymbol{u}$ in each instance and $\tau_0 k_0 + \tau_1 k_1 = \lambda_{\mathbb{F}} - w_{\mathsf{grind}}$, concatenation gives $\lambda_{\mathbb{F}} - w_{\mathsf{grind}}$ columns. We append w_{grind} zero columns to both matrices and the same number of zero bits to the concatenated challenge, as required by grinding, to obtain a VOLE correlation over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. More details can be found in this section later.

The vector commitment above can be directly constructed by independent $\tau = \tau_0 + \tau_1$ GGM trees, but the concrete communication would be increased to $\tau \cdot (\log N + n_{\mathsf{leafcom}}) \cdot N_{\mathsf{block}}$ bits, where $n_{\mathsf{leafcom}} N_{\mathsf{block}}$ is the length of a leaf commitment. We use the batch all-but-one vector commitment (BAVC) technique [33] to optimize the communication. Specifically, the prover interleaves multiple vector commitments such that the i-th leaves of multiple trees become neighbors instead of the original neighbored i-th and $(i+1)$-th leaves of a single tree. When opening all-but-τ leaves, the concrete communication depends on how much the τ active paths in the interleaved GGM tree overlap. In the best case, when the τ unopened leaves form an aligned subtree of size τ, the number of revealed internal seeds is $\log(L/\tau)$. For equal-length vectors, where $L = \tau N$, this becomes $\log N$, and hence the total communication is $(\log N + \tau n_{\mathsf{leafcom}}) \cdot N_{\mathsf{block}}$ bits. If the τ unopened leaves do not align perfectly into such a subtree, the opening size would be larger. In the worst case, the τ active paths in the interleaved GGM tree have little overlap, so the one-tree BAVC opening provides essentially no asymptotic improvement over using τ independent trees, with a total communication $(\tau \log N + \tau n_{\mathsf{leafcom}}) \cdot N_{\mathsf{block}}$ bits. In our setting, the τ unopened leaves are determined by the verifier's challenge, and therefore the actual BAVC opening size is a random variable depending on the overlap pattern of the corresponding active paths in the interleaved tree.

To control the concrete communication, VOLEith introduces a threshold parameter T_{open}, which upper-bounds the number of internal seeds that may be revealed in a valid opening. Concretely, after deriving a challenge, the prover computes the associated set of unopened leaves and the resulting opening size. If this size is at most T_{open}, the prover proceeds normally; otherwise, the challenge is discarded and a fresh one is derived. Based on the Fiat-Shamir transformation, this retry step can be implemented non-interactively by appending a counter to the transcript and re-hashing until a challenge satisfying the threshold is obtained. This rejection-sampling technique [33] reduces the opening size and is covered by the security analysis of the resulting Fiat–Shamir construction.

From τ small VOLEs to a large VOLE. For illustration, suppose the parties hold τ independent small-field VOLE correlations $\{(\boldsymbol{q}_i, \Delta_i, \boldsymbol{u}_i, \boldsymbol{v}_i)\}_{i \in [0,\tau)}$ with a common field width k and $w_{\mathsf{grind}} = 0$, where $\boldsymbol{u}_i \in \mathbb{F}_2^{\ell}$, $\boldsymbol{v}_i \in \mathbb{F}_{2^k}^{\ell}$, and $\boldsymbol{q}_i = \boldsymbol{u}_i \cdot \Delta_i + \boldsymbol{v}_i$. For convenience, we represent each $\boldsymbol{v}_i$ as a binary matrix $\boldsymbol{V}_i \in \{0,1\}^{\ell \times k}$ instead of a vector over $\mathbb{F}_{2^k}$, where the j-th row is the bit decomposition of the j-th element of $\boldsymbol{v}_i$.

Let the bit decomposition of $\Delta_i \in \mathbb{F}_{2^k}$ be $(\delta_0^{(i)}, \ldots, \delta_{k-1}^{(i)}) \in \{0,1\}^k$. Then, the verifier's output can be written as an $\ell \times k$ binary matrix $\boldsymbol{Q}_i = \left(\boldsymbol{u}_i \cdot \delta_0^{(i)} \parallel \cdots \parallel \boldsymbol{u}_i \cdot \delta_{k-1}^{(i)}\right) + \boldsymbol{V}_i$, where $\parallel$ denotes the horizontal concatenation of column vectors.

To concatenate these τ small VOLEs into a single VOLE over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ with $\lambda_{\mathbb{F}} = \tau k$, it is necessary that all correlations use the same binary vector $\boldsymbol{u}$. Let us choose $\boldsymbol{u} = \boldsymbol{u}_0$. For each $i \in [1, \tau)$, the prover sends the correction value $\boldsymbol{c}_i = \boldsymbol{u}_i - \boldsymbol{u}_0$. Using $\boldsymbol{c}_i$, the verifier subtracts $\delta_j^{(i)} \cdot \boldsymbol{c}_i$ from column

j of $\boldsymbol{Q}_i$ for every $j \in [0,k)$, so that the i-th VOLE now uses the common vector $\boldsymbol{u} = \boldsymbol{u}_0$. Denote the corrected instances by $\{(\boldsymbol{u}, \boldsymbol{V}_i, \boldsymbol{Q}_i, \Delta_i)\}_{i\in[0,\tau)}$.

The prover and verifier then form the concatenated matrices $\boldsymbol{V} = \left(\boldsymbol{V}_0 \parallel \cdots \parallel \boldsymbol{V}_{\tau-1}\right) \in \{0,1\}^{\ell\times\tau k}$, and $\boldsymbol{Q} = \left(\boldsymbol{Q}_0 \parallel \cdots \parallel \boldsymbol{Q}_{\tau-1}\right) \in \{0,1\}^{\ell\times\tau k}$, and hence $\boldsymbol{V}, \boldsymbol{Q} \in \{0,1\}^{\ell\times\lambda_{\mathbb{F}}}$. They also concatenate the bit decompositions of the Δ_i's to define

$$\Delta = \left(\delta_0^{(0)}, \ldots, \delta_{k-1}^{(0)}, \delta_0^{(1)}, \ldots, \delta_{k-1}^{(1)}, \ldots, \delta_0^{(\tau-1)}, \ldots, \delta_{k-1}^{(\tau-1)}\right) \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}.$$

Viewing the j-th row of $\boldsymbol{V}$ and $\boldsymbol{Q}$ as an element of $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, we obtain a single large-field VOLE correlation $\boldsymbol{Q}[j] = \boldsymbol{V}[j] + \boldsymbol{u}[j] \cdot \Delta$ for $j \in [0,\ell)$. Thus, τ VOLE instances over $\mathbb{F}_{2^k}$ are combined into one VOLE instance over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$.

VOLE consistency check. A malicious prover may send inconsistent correlation value $\boldsymbol{c}_i$. To guarantee the consistency, the prover must additionally prove that the same $\boldsymbol{u}$ is used across all instances. This is done using the VOLE consistency check from [22, 34], where the verifier challenges the prover to open a random linear universal hash function applied to $\boldsymbol{u}$ and $\boldsymbol{V}$. To this end, let $\boldsymbol{H}$ be the linear hash matrix related to the verifier's challenge. The prover sends $\tilde{\boldsymbol{u}} = \boldsymbol{H}\boldsymbol{u}$ and binds $\tilde{\boldsymbol{V}} = \boldsymbol{H}\boldsymbol{V}$ before the final challenge Δ. After reconstructing $\boldsymbol{Q}$, the verifier computes $\tilde{\boldsymbol{V}}' = \boldsymbol{H}\boldsymbol{Q} - \Delta \cdot \tilde{\boldsymbol{u}}$ and checks it against the bound value.

Padding the extended witness due to VOLE consistency check. The above description assumes $\boldsymbol{u}$ is of length ℓ bits, which is the same as the extended witness for the OWF. To achieve zero knowledge, the extended witness would be increased by a masked sub-witness of length $(\lambda_{\mathbb{F}}+B)$ bits, as the VOLE consistency check reveals a $(\lambda_{\mathbb{F}}+B)$-bit linear combination of $\boldsymbol{u}$ to the verifier.

2.3 Zero-Knowledge Proof Phase

After generating the VOLE correlations, the prover (or signer) and verifier (or receiver) would invoke the interactive ZKP phase based on QuickSilver [11] to prove the knowledge of secret key consistent with the public key. Below, we assume that the prover and verifier hold random VOLE correlations $\{(q_i, \Delta, u_i, v_i)\}_{i\in[0,\ell)}$ such that $q_i, v_i, \Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, $u \in \mathbb{F}_2$, and $q_i = u_i \cdot \Delta + v_i$. The proof phase proceeds in three main stages.

Witness extension. The prover takes the secret key $\mathsf{sk} = (\boldsymbol{x}, \boldsymbol{k})$ as inputs, and executes the block cipher key expansion and encryption routines to compute $\boldsymbol{y} = E_{\boldsymbol{k}}(\boldsymbol{x})$. Specified bits related to the S-box inputs and outputs are extracted during the execution, and are then recorded as the extended witness $\boldsymbol{w} \in \{0,1\}^{\ell}$. After extracting $\boldsymbol{w}$, the prover commits to it by sending $\boldsymbol{d} = \boldsymbol{u}+\boldsymbol{w}$ to the verifier. This allows the verifier to update the VOLE commitments to $\boldsymbol{w}$, *i.e.*, $q'_i = q_i + d_i \cdot \Delta = w_i \cdot \Delta + v_i$ for all $i \in [0,\ell)$.

Evaluating constraints. Next, the parties evaluate all constraints over the committed extended witness. Suppose there are C constraints determined by functions $f_1, \ldots, f_C$, where each function can be represented as a degree-d circuit. Then, the goal of the ZKP is showing that for all $i \in [C]$, $f_i(\boldsymbol{w}) = 0$. The circuit evaluation follows a gate-by-gate manner. As the circuit is public and the VOLE commitments have linear homomorphism, the prover and verifier can evaluate constraitns for addition gates freely. For multiplication gates, the prover sees its (w_i, v_i) as the coefficients of a secret degree-1 polynomial $\rho_i(X) = w_i X + v_i$. Then, multiplying two VOLE commitments for w_i, w_j is done by multiplying the polynomials $\rho_i(X)\rho_j(X)$, obtaining a degree-2 secret polynomial. Meanwhile, the verifier can multiply its VOLE values q_i and q_j by computing $q_i \cdot q_j = \rho_i(\Delta) \cdot \rho_j(\Delta)$. As the polynomial degree increases, the prover needs to additionally send terms $w_i v_j + w_j v_i$ and $v_i v_j$ to the verifier for the consistency check. The above procedure can be generalized for any low-degree polynomials, as long as the secrets are aligned (by increasing degrees if necessary) in the highest coefficient. The highest degree among $f_i(\boldsymbol{X})$ is a parameter in our schemes.

Proving constraints. After evaluating all constraints gate-by-gate over the VOLE-committed extended witness, the prover must convince the verifier that every constraint output equals zero, without leaking any information about the witness itself. Recall that each constraint (supposing degree-2), when evaluated over VOLE commitments, yields a secret polynomial of the form $\rho(X) =$

$a_0 + a_1 X + zX^2$. If every constraint is satisfied, linearity gives $\tilde{q} = \tilde{a}_0 + \tilde{a}_1 \Delta$, where the leading coefficient z vanishes if and only if the constraint is satisfied. The prover holds the plaintext coefficient vectors (a_0, a_1), while the verifier holds $\rho(\Delta)$.

To batch all C constraints $\{(a_0^{(i)}, a_1^{(i)}), \rho_i(\Delta)\}$ into a single one, the verifier picks a random challenge chall. The prover then invokes another linear universal hash (called as ZKHash) with chall as a public seed to compress each coefficient vector into a single field element. For each coefficient vector of length C, ZKHash combines two evaluations of a polynomial of degree at most $C-1$, using the challenge-derived seeds specified in Section 4. This produces 2 compact hash outputs $\tilde{a}_0, \tilde{a}_1 \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$. The verifier applies ZKHash to obtain a corresponding value $\tilde{q} \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$. By the linearity of both ZKHash and the underlying VOLE structure, it holds that $\tilde{q} = \tilde{a}_0 + \tilde{a}_1 \cdot \Delta$ if and only if every constraint output vanishes (*i.e.*, $z = 0$). The verifier accepts if $\tilde{q} - \tilde{a}_1 \cdot \Delta$ matches the prover-supplied $\tilde{a}_0$. This proving phase can be naturally generalized into the high-degree case such that for degree-d constraints, the prover computes $\tilde{a}_0, \ldots, \tilde{a}_{d-1}$ from $\{(a_0^{(i)}, a_1^{(i)}, \ldots, a_{d-1}^{(i)})\}_{i \in [C]}$ and, in the interactive protocol, sends all d values to the verifier before the final challenge. The verifier checks whether $\tilde{q} - \tilde{a}_{d-1} \cdot \Delta^{d-1} \cdots - \tilde{a}_1 \cdot \Delta$ matches the prover-supplied $\tilde{a}_0$.

Padding the extended witness due to ZKHash. Like the VOLE consistency check, to achieve zero knowledge, the extended witness would be increased by a masked sub-witness of length $(d-1) \cdot \lambda_\mathbb{F}$ bits, as the ZKHash reveals a $(d-1) \cdot \lambda_\mathbb{F}$-bit linear combination of $\{(a_0^{(i)}, a_1^{(i)}, \ldots, a_{d-1}^{(i)})\}_{i \in [C]}$.

2.4 Putting Things Together

In our construction, VOLE commitments and the QuickSilver-style ZKP are combined into a 3-challenge interactive argument. At a high level, the protocol consists of the following stages.

First, the prover and verifier generate VOLE-based commitments and correction values. The verifier then sends the first challenge chall_1, which is used to derive the VOLE consistency check. After receiving chall_1, the prover answers this check and, at the same time, commits to the extended witness derived from the secret key and the OWF evaluation. Next, the verifier sends a second challenge chall_2. Using this challenge, the prover evaluates the OWF constraints over the VOLE-committed witness and generates the proof for the OWF ZKP phase.

Finally, the verifier sends the third challenge chall_3, *i.e.*, Δ, which determines which parts of the VOLE commitment must be opened. The prover returns the requested VOLE opening information, and the verifier checks both the VOLE consistency and the correctness of the compressed constraint proof. If all checks pass, the verifier accepts.

To obtain a non-interactive signature scheme, we apply the Fiat–Shamir transformation in the random oracle model. Instead of receiving challenges from the verifier, the signer derives them by hashing the transcript incrementally. More precisely, letting μ be the hash of the signature message and the public key, the challenges are defined as $\mathsf{chall}_i = H(\mathsf{chall}_{i-1} \,\|\, \mathsf{msg}_i)$, where msg_i denotes the prover's i-th message and $\mathsf{chall}_0 \leftarrow \mu$.

A straightforward Fiat–Shamir transcript would include all prover messages. However, some parts of the early messages can be reconstructed from the final challenge chall_3 together with the VOLE opening. Therefore, the signature can be represented more compactly by keeping only the information that is not reconstructible, while still allowing the verifier to recompute the full transcript, reconstruct all challenges, and verify that the resulting transcript is consistent.

3 Design Rationale

This section describes the parameters for our signature schemes, including the main parameter param, the OWF parameter $\mathsf{param}_{\mathsf{OWF}}$, and the VOLE parameter $\mathsf{param}_{\mathsf{VOLE}}$.

3.1 Mathematical Hard Problems and Hardness Assumptions

Mathematical hard problems. Our signature schemes do not rely on any number-theoretic or structured hardness assumptions. We use the block-cipher PRP security, pseudorandomness of

the actual restricted-key public-key distributions, almost-injectivity of the leaf PRGs, and hash functions in the stated PRF and random-oracle models.

Hardness assumptions. We make the following hardness assumptions in the proposed signatures.

First, we rely on the PRP security of the underlying block ciphers in both the classical and quantum models, with explicit computation and oracle-query budgets as in Equation (39). We also assume that block ciphers and the Even-Mansour constructions [35] of block ciphers have one-wayness properties, which have been implied in previous works [36, 37]. The lossy-key proof additionally uses the joint public-key pseudorandomness experiment in Equation (46), with the secret key uniform over $k[0]k[1] = 0$ and the public block uniform and independent. For the standard construction, Lemma 8 gives a PRP reduction with a square-root loss. For the EM construction, this public-key pseudorandomness is a separate assumption.

Second, we rely on the security of PRG-based leaf commitments in the VOLE commitments, which use $2N_{\mathsf{block}}$-bit outputs of block ciphers on N_{block}-bit seeds. The almost-injectivity advantages are retained explicitly in each classical or quantum model. This leaf commitment is adapted from FAEST-EM variants for better performance, which can be changed as standard hash functions or the universal-hash-based leaf commitment in non-EM variants of FAEST [22].

Finally, to make the ZKP non-interactive, we rely on the ROM and QROM. Although idealized models, they are widely used in cryptography, including the NIST selected standardized post-quantum signature schemes ML-DSA, Falcon [25], and SLH-DSA [31].

3.2 Parameters and Criteria for Parameter Selection

This section introduces the main parameter tuple param and the OWF parameters $\mathsf{param}_{\mathsf{OWF}}$, VOLE parameters $\mathsf{param}_{\mathsf{VOLE}}$, and auxiliary parameters $\mathsf{param}_{\mathsf{other}}$ derived from it, including hashes and IVs, together with the criteria for parameter selection.

Main parameter. The main parameter param, which determines $\mathsf{param}_{\mathsf{OWF}}$ and $\mathsf{param}_{\mathsf{VOLE}}$, is

$$\mathsf{param} = (\lambda, \lambda_{\mathsf{Q}}, \lambda_{\mathbb{F}}, \tau, w_{\mathsf{grind}}, T_{\mathsf{open}}, d, B, \mathrm{OWF}, \ell, \lambda_{\mathsf{iv}}, N_{\mathsf{block}}).$$

The entries of the main tuple are defined below.

- λ, security parameter targeting for a specified classical security, as required by ICCS.
- λ_{Q}, security parameter targeting for a specified quantum security, as required by ICCS.
- $\lambda_{\mathbb{F}}$, the logarithm of the field size. This is determined by λ and λ_{Q} and the round-by-round soundness of signature schemes.
- τ, the total number of sub-GGM trees in the BAVC technique, as introduced in Section 2.2.
- w_{grind}. Grinding (a lightweight proof-of-work), a technique practically applied in ZKPs and blockchain, is used to make the Fiat–Shamir challenge grinding expensive for a malicious prover. After committing, the prover searches for a counter whose transcript hash has zeros in the last w_{grind} challenge-bit positions. Any change to commitments forces repeating this work, while the verifier checks the nonce with a single hash. A grinding parameter w_{grind} is estimated to provide w_{grind}-bit security.
- T_{open}, the upper bound of the number of revealed seeds in the opening of BAVC, as introduced in Section 2.2.
- d, the highest degree of polynomial constraints in the ZKP for OWFs. This is determined by the proving method for OWFs, and we choose the method with the optimized signature size.
- B, used for extra length padding for the VOLE consistency check. This is fixed as 16 to meet the target security,
- OWF. The OWF is either $y = E_k(x)$ (the standard variant) or $y = E_x(k) \oplus k$ (the EM variant), where E is the encryption function of a specified block cipher. (x, y) are the public key, and the concrete secret-key encoding is (x, k) so that the second component is the secret witness/key component. The signer proves that it knows the witness k such that $y = E_k(x)$ or $y = E_x(k) \oplus k$ holds for public inputs (x, y, E) via a ZKP.
- ℓ, the extended witness length for the underlying ZKP, which is determined by the OWF and the proving method.

– λ_{iv}, the length of initialization vectors (IVs), which is no more than the block size of PRGs.
– N_{block}, the block size used in PRGs.

Table 1 lists the main parameters for all proposed variants of SM4th, uBlockith, and Vistrutith. Key and signature sizes are derived in Section 3.3. Specifically, the key words of variants and their meanings are as follows.

– SM4th, uBlockith, or Vistrutith: the block cipher used as the OWF in the scheme.
– "EM" or no "EM": whether the scheme adopts the EM or non-EM construction as the OWF.
– "s" or "f": whether the variant focuses on *s*maller signatures or *f*aster running times.
– "128", "256", or "512": the targeted classical security level, in bits.
– "lo" or "ti": whether the variant uses the loose or tight parameter configuration.

Below we give necessary explanations.

Choice of loose and tight variants with different fields, λ_{iv}, and PRGs. For SM4th with 128-bit classical security, we give two sets of schemes, loose (-lo) variants and tight (-ti) variants, with different λ_{iv}, PRGs, and finite fields. We include the ti variants as a conservative design option, using wider IVs and PRG seeds and a larger proof field to provide additional margin in the security analysis. The loose variants use smaller fields and native SM4 PRGs for efficiency. The complete conditional proof budgets are stated in Section 9.1.7.

SM4th-lo uses SM4 as PRGs with $\lambda_{\mathsf{iv}} = 128$, and SM4th-ti uses Ballet as PRGs with $\lambda_{\mathsf{iv}} = 256$, where the size-$\lambda_{\mathsf{iv}}$ IV contributes to the plaintext of block-cipher-based PRGs. In the CTR construction of PRG, the IV is embedded in one cipher-input block, requiring $\lambda_{\mathsf{iv}} \leq N_{\mathsf{block}}$. Meanwhile, SM4th-lo uses a 128-bit finite field in contrast to SM4th-ti using a larger 160-bit finite field.

These parameter choices are guided by the following ICCS requirements:

1. The upper bound of signing queries is $Q_{\mathsf{sig}} = 2^{80}$.
2. Each key pair shall support signing and verifying at least 2^{64} distinct messages.
3. The concrete security (in bits) in the quantum model corresponding to the classical 128-bit security should be 80 bits.

FAEST-v2 analyzes its AES-based PRG with at most 2^{64} signatures per key [22]. At this budget, all 128-bit signature IVs remain distinct with constant probability, as used in its analysis of PRG search attacks across multiple signatures [22]. We adopt the same conservative reference budget for the lo variants of SM4th and SM4th-EM. Our multi-key evaluation considers 2^{16} independently generated keys, giving a total budget of 2^{80} signing queries. At 2^{80} signatures per key, IV repetition alone does not establish that the lo variants fail to meet the security target.

At 2^{80} signatures per key, a 160-bit IV has the same birthday scale as a 128-bit IV at 2^{64} signatures in FAEST. IV lengths above 160 bits provide additional collision margin at this signing budget. The ti variants use a 256-bit IV with Ballet-256/256 to keep the repeated-IV probability below 2^{-96} at 2^{80} signatures per key in the random-IV model.[4] The same estimate applies to uBlockith, uBlockith-EM, and Vistrutith, which also use 256-bit IVs.

For the field size, our theoretical security proof in the QROM contains a term of the form $Q^2/2^{\lambda_{\mathbb{F}}}$ in the round-by-round soundness error of the ZKP, where Q denotes the number of random-oracle queries and $\lambda_{\mathbb{F}}$ is the bit size of the finite field. This term comes from the probability that the adversary guesses a bad value of Δ in the VOLE correlation of Equation (1). Therefore, **in the tight variants, we set $\lambda_{\mathbb{F}} = 160$**, to reduce this field-size term. **For the loose variants, we instead use $\lambda_{\mathbb{F}} = 128$**, and consider the concrete security against best known attacks. Currently, we are not aware of any quantum attack of guessing Δ that performs better than the Grover algorithm. Using the same depth-limited cost model as in [38–40], we estimate the depth-limited cost of this Grover search is approximately $2^{160}/\mathsf{MAXDEPTH}$ (Details can be found in Section 9.2.3). For the concrete value $\mathsf{MAXDEPTH} = 2^{56}$ used in our evaluation, the resulting attack cost is approximately $2^{104} > 2^{80}$, for this particular search strategy and circuit-cost model.

[4] Let $P_{\mathsf{coll}}(s, b)$ denote the probability of a repeated IV among s signatures with independently uniform b-bit preprocessing IVs and $\mathsf{iv} = \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}})$, where $\mathsf{H}_4 : \{0,1\}^b \to \{0,1\}^b$ is a random oracle. A pair of derived IVs coincides with probability $2^{-b} + (1 - 2^{-b})2^{-b}$, so $P_{\mathsf{coll}}(s, b) \leq \binom{s}{2}\left(2^{1-b} - 2^{-2b}\right) < s^2 2^{-b}$.

Choice of τ, w_{grind}, and T_{open}. These parameters are chosen jointly for the BAVC opening and Fiat–Shamir challenge space. Let $m = \lambda_{\mathbb{F}} - w_{\mathsf{grind}}$. For a fixed τ, the implementation splits the m challenge-index bits almost evenly among the τ sub-GGM trees by setting $k = \lfloor m/\tau \rfloor + 1$, $\tau_1 = m \bmod \tau$, and $\tau_0 = \tau - \tau_1$. Thus, τ_1 trees have depth k and τ_0 trees have depth $k-1$, so that $\tau_1 k + \tau_0(k-1) = m$ and the total challenge-index space is $\prod_i N_i = 2^{\lambda_{\mathbb{F}} - w_{\mathsf{grind}}}$. The signer searches for a nonce satisfying both the w_{grind} zero-bit condition and the BAVC opening bound; with opening acceptance rate α, the expected search cost is approximately $2^{w_{\mathsf{grind}}}/\alpha$ hash trials, as detailed in Section 3.4. Finally, T_{open} is selected as the fixed upper bound on the number of sibling seeds included in the BAVC opening. If n_h denotes the number of nodes in the union of the root-to-hidden-leaf paths, the implementation accepts only openings satisfying $n_h - 2\tau + 1 \le T_{\mathsf{open}}$. In summary, smaller τ reduces the signature size, while larger τ leads to faster running times. Smaller T_{open} reduces the signature size, whereas larger T_{open} increases the acceptance probability of the grinding loop. All these concrete values are chosen to balance signature size and signing/verifying cost for each parameter set.

Our concrete choices keep the small-GGM tree depths close to the corresponding FAEST instances and use T_{open} to balance opening size and acceptance rate. For the 128-bit loose variants, we adapt the parameters from FAEST. For the 128-bit SM4th-ti schemes, after enlarging the proof field to $\lambda_{\mathbb{F}} = 160$, keeping the original FAEST values $\tau = 11$ and $\tau = 16$ would make the BAVC leaf budget much larger. We therefore increase τ so that the maximum tree depth is preserved: the small variants use $\tau = 14$, for which $\lambda_{\mathbb{F}} - w_{\mathsf{grind}} = 153$ is split into trees of depth at most 11, matching the FAEST-128s depth, while the fast variants use $\tau = 21$, for which the maximum depth is 8, matching the FAEST-128f depth. We keep $w_{\mathsf{grind}} = 7$ for SM4th-s and $w_{\mathsf{grind}} = 8$ for SM4th-f variants. With these choices, we set $T_{\mathsf{open}} = 131$ for SM4th-d3-128s and $T_{\mathsf{open}} = 132$ for SM4th-EM-d2-128s. For the fast variants, we set $T_{\mathsf{open}} = 138$ for SM4th-d3-128f and $T_{\mathsf{open}} = 140$ for SM4th-EM-d2-128f.

For the 256-bit uBlockith schemes, we keep the FAEST-256-style BAVC split directly. The small variants use $\tau = 22$ and $w_{\mathsf{grind}} = 6$, so we have $\tau_1 = 8$, $\tau_0 = 14$, and $k = 12$, giving trees of depth at most 12 and leaf budget $8 \cdot 2^{12} + 14 \cdot 2^{11}$, matching the FAEST-256s depth. The fast variants use $\tau = 32$ and $w_{\mathsf{grind}} = 8$, so we have $\tau_1 = 24$, $\tau_0 = 8$, and $k = 8$, giving leaf budget $24 \cdot 2^8 + 8 \cdot 2^7$, matching the FAEST-256f depth. With these choices, we set $T_{\mathsf{open}} = 245$ for uBlockith-d3-256s and $T_{\mathsf{open}} = 246$ for uBlockith-d3-256f. For the EM variants, the witness length and hence $\widehat{\ell}$ are smaller, so we use the slightly tighter bounds $T_{\mathsf{open}} = 218$ for uBlockith-EM-d3-256s and $T_{\mathsf{open}} = 234$ for uBlockith-EM-d3-256f to further reduce signature size.

For the 512-bit instantiation, the choices of τ, w_{grind}, and T_{open} for Vistrutith then follow the same FAEST-style depth-preserving rule, using the FAEST-256 instances as the reference point. The small variant uses $\tau = 44$, approximately twice the FAEST-256s value $\tau = 22$, so that the maximum small-tree depth remains 12. We set $w_{\mathsf{grind}} = 5$ for this variant because increasing it to 6 would only slightly reduce the leaf budget but would double the grinding factor $2^{w_{\mathsf{grind}}}$, which is less attractive for the more expensive 512-bit field operations. The fast variant uses $\tau = 64$, matching twice the FAEST-256f value $\tau = 32$, and keeps $w_{\mathsf{grind}} = 8$, giving trees of depth at most 8 and a much smaller BAVC leaf budget. Finally, we set $T_{\mathsf{open}} = 480$ for the small variant and $T_{\mathsf{open}} = 481$ for the fast variant.

One-way functions and their parameters. We introduce OWFs used in SM4th, uBlockith, and Vistrutith and extra parameters $\mathsf{param}_{\mathsf{OWF}}$ related to these OWFs.

Our signatures use $y = E_k(x)$ as OWF where E is the encryption routine of the underlying block cipher, x is the plaintext, y is the ciphertext, and k is the key of corresponding block ciphers. The public key is (x, y), and the concrete secret-key encoding is (x, k). In SM4th-EM and uBlockith-EM, the one-way function is $y = E_x(k) \oplus k$[5], where x is the key, and k is the plaintext of corresponding block ciphers. The public key is (x, y), and the concrete secret-key encoding is again (x, k). As Vistrutah does not involve any non-linear operations in its key-schedule routine, we do not construct EM variants of Vistrutith.

[5] As long as $E_x(k)$ can be modeled as a random permutation, $y = E_x(k) \oplus k$ is indeed a OWF. To the best of our knowledge, no attacks are known against full round SM4 [41] or uBlock [42] that would undermine this construction.

Table 1: Concrete values of the main parameter param, where "s" means a variant with smaller signature sizes but slower running times, and "f" means a variant with faster running times and larger signature sizes

Scheme	OWF $E_k(x)$ or $E_x(k) \oplus k$	$(\lambda, \lambda_{\mathsf{Q}})$	$\lambda_{\mathbb{F}}$	ℓ	τ	w_{grind}	T_{open}	B	d	λ_{iv}	N_{block}
SM4th-d3-128s-lo	$\text{SM4}_k(x)$	(128, 80)	128	1808	11	7	102	16	3	128	128
SM4th-d3-128f-lo		(128, 80)	128	1808	16	8	110	16	3	128	128
SM4th-EM-d2-128s-lo	$\text{SM4}_x(k) \oplus k$	(128, 80)	128	1024	11	7	103	16	2	128	128
SM4th-EM-d2-128f-lo		(128, 80)	128	1024	16	8	112	16	2	128	128
SM4th-d3-128s-ti	$\text{SM4}_k(x)$	(128, 80)	160	1808	14	7	131	16	3	256	256
SM4th-d3-128f-ti		(128, 80)	160	1808	21	8	138	16	3	256	256
SM4th-EM-d2-128s-ti	$\text{SM4}_x(k) \oplus k$	(128, 80)	160	1024	14	7	132	16	2	256	256
SM4th-EM-d2-128f-ti		(128, 80)	160	1024	21	8	140	16	2	256	256
uBlockith-d3-256s	$\text{uBlock}_k(x)$	(256, 128)	256	4608	22	6	245	16	3	256	256
uBlockith-d3-256f		(256, 128)	256	4608	32	8	246	16	3	256	256
uBlockith-EM-d3-256s	$\text{uBlock}_x(k) \oplus k$	(256, 128)	256	3072	22	6	218	16	3	256	256
uBlockith-EM-d3-256f		(256, 128)	256	3072	32	8	234	16	3	256	256
Vistrutith-d3-512s	$\text{Vistrutah}_k(x)$	(512, 256)	512	6912	44	5	480	16	3	256	512
Vistrutith-d3-512f		(512, 256)	512	6912	64	8	481	16	3	256	512

Table 2 describes the parameters of these one-way functions used in ZKP, which is denoted as

$$\mathsf{param}_{\mathsf{OWF}} = (R, S_{\mathsf{ke}}, S_{\mathsf{enc}}, \ell_{\mathsf{ke}}, \ell_{\mathsf{enc}}, \ell, C).$$

The meanings of $\mathsf{param}_{\mathsf{OWF}}$ are as follows.

- R: the number of rounds for either the key expansion routine or the encryption routine;
- S_{ke}: the number of secret-dependent key-schedule S-box evaluations represented in the ZK relation (zero for EM variants, whose key schedule is public);
- S_{enc}: the number of secret-dependent encryption S-box evaluations represented in the ZK relation;
- ℓ_{ke}: the length of extended witness in bits for the key expansion routine;
- ℓ_{enc}: the length of extended witness in bits for the encryption routine;
- ℓ: the total length of extended witness in bits;
- C: the number of constraints in OWFs.

VOLE-in-the-head parameters. Table 3 describes the VOLE-in-the-head parameter $\mathsf{param}_{\mathsf{VOLE}}$, which is defined as

$$\mathsf{param}_{\mathsf{VOLE}} = (\tau_1, \tau_0, k, L, \{N_i\}_{i=0}^{\tau-1}, B, \hat{\ell}, N_{\mathsf{block}}).$$

The meanings of $\mathsf{param}_{\mathsf{VOLE}}$ are as follows.

- τ_1: the number of *larger* small-VOLE instances, computed as $\tau_1 = (\lambda_{\mathbb{F}} - w_{\mathsf{grind}}) \bmod \tau$;
- τ_0: the number of *smaller* small-VOLE instances, computed as $\tau_0 = \tau - \tau_1$;
- k: the bit-length of the larger small-VOLE instances, computed as $k = \lfloor (\lambda_{\mathbb{F}} - w_{\mathsf{grind}})/\tau \rfloor + 1$, so that the bit-length of the smaller instances is $k - 1$;
- N_i: the field size of the i-th small-VOLE instance, equivalently the number of leaves in the i-th GGM tree, defined as

$$N_i = \begin{cases} 2^k & \text{if } i < \tau_1, \\ 2^{k-1} & \text{otherwise;} \end{cases}$$

Table 2: Concrete values of the OWF parameter $\mathsf{param}_{\mathsf{OWF}}$ for all parameter sets.

Scheme	OWF	R	S_{ke}	S_{enc}	ℓ_{ke}	ℓ_{enc}	ℓ	C
SM4th-d3-128s-lo	$\mathrm{SM4}_k(x)$	32	128	128	1024	784	1808	453
SM4th-d3-128f-lo		32	128	128	1024	784	1808	453
SM4th-EM-d2-128s-lo	$\mathrm{SM4}_x(k) \oplus k$	32	0	128	0	1024	1024	257
SM4th-EM-d2-128f-lo		32	0	128	0	1024	1024	257
SM4th-d3-128s-ti	$\mathrm{SM4}_k(x)$	32	128	128	1024	784	1808	453
SM4th-d3-128f-ti		32	128	128	1024	784	1808	453
SM4th-EM-d2-128s-ti	$\mathrm{SM4}_x(k) \oplus k$	32	0	128	0	1024	1024	257
SM4th-EM-d2-128f-ti		32	0	128	0	1024	1024	257
uBlockith-d3-256s	$\mathrm{uBlock}_k(x)$	24	384	1536	1792	2816	4608	4609
uBlockith-d3-256f		24	384	1536	1792	2816	4608	4609
uBlockith-EM-d3-256s	$\mathrm{uBlock}_x(k) \oplus k$	24	0	1536	0	3072	3072	3073
uBlockith-EM-d3-256f		24	0	1536	0	3072	3072	3073
Vistrutith-d3-512s	$\mathrm{Vistrutah}_k(x)$	18	0	1152	0	6912	6912	1729
Vistrutith-d3-512f		18	0	1152	0	6912	6912	1729

– L: the total number of leaves across all GGM trees, given by

$$L = \sum_{i=0}^{\tau-1} N_i = \tau_1 \cdot 2^k + \tau_0 \cdot 2^{k-1}.$$

Then, the VOLE challenge space is $\prod_{i=0}^{\tau-1} N_i = 2^{\lambda_{\mathbb{F}} - w_{\mathsf{grind}}}$.
– B: the extra bit-length padding for the VOLE consistency check, fixed at $B = 16$;
– $\hat{\ell}$: the extended witness length including extra masks for the VOLE consistency check and ZKHash, computed as $\hat{\ell} = \ell + d\lambda_{\mathbb{F}} + B$.
– N_{block}: the block size of the used block ciphers as PRGs.

Hash functions. Let λ denote the security parameter, $\lambda_{\mathbb{F}}$ the finite-field size, and λ_{iv} the IV length in Table 1. The hash functions are

$$\begin{aligned}
&\mathsf{H}_1 : \{0,1\}^* \longrightarrow \{0,1\}^{2N_{\mathsf{block}}}, \quad && \mathsf{H}_2^0 : \{0,1\}^* \longrightarrow \{0,1\}^{2\lambda},\\
&\mathsf{H}_2^1 : \{0,1\}^* \longrightarrow \{0,1\}^{5\lambda_{\mathbb{F}}+64}, && \mathsf{H}_2^2 : \{0,1\}^* \longrightarrow \{0,1\}^{3\lambda_{\mathbb{F}}+64},\\
&\mathsf{H}_2^3 : \{0,1\}^* \longrightarrow \{0,1\}^{\lambda_{\mathbb{F}}}, && \mathsf{H}_3 : \{0,1\}^* \longrightarrow \{0,1\}^{N_{\mathsf{block}}+\lambda_{\mathsf{iv}}},\\
&\mathsf{H}_4 : \{0,1\}^{\lambda_{\mathsf{iv}}} \longrightarrow \{0,1\}^{\lambda_{\mathsf{iv}}}.
\end{aligned}$$

Here, H_1 hashes the leaf-commitment vectors and their resulting digests. H_2^0 derives μ. H_2^1, H_2^2, H_2^3 derive the three Fiat–Shamir challenges. The function H_3 derives the root seed and preprocessing IV from $\mathsf{sk}_2 \| \mu \| \rho$, where $\rho \in \{0,1\}^{\xi}$ and

$$\xi = \max\{384, \lambda + 128\} = \begin{cases} 384, & \lambda \in \{128, 256\}, \\ 640, & \lambda = 512. \end{cases}$$

Hence, SM4th and uBlockith use 384 bits of signing randomness, whereas Vistrutith uses 640 bits. The function H_4 maps the preprocessing IV to the public IV such that $\mathsf{iv} = \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}})$.

Protocol domains. For parameter set p, let

$$D_{\mathsf{p}} = \mathrm{ASCII}(\texttt{Chinith/} \| p),$$

Table 3: Concrete values of the VOLE-in-the-head parameter $\mathsf{param}_{\mathsf{VOLE}}$ for all parameter sets

Scheme	τ_1	τ_0	k	L	$\{N_i\}_{i=0}^{\tau-1}$	B	$\hat{\ell}$	N_{block}
SM4th-d3-128s-lo	0	11	12	22528	11×2^{11}	16	2208	128
SM4th-d3-128f-lo	8	8	8	3072	8×2^{8}, 8×2^{7}	16	2208	128
SM4th-EM-d2-128s-lo	0	11	12	22528	11×2^{11}	16	1296	128
SM4th-EM-d2-128f-lo	8	8	8	3072	8×2^{8}, 8×2^{7}	16	1296	128
SM4th-d3-128s-ti	13	1	11	27648	13×2^{11}, 1×2^{10}	16	2304	256
SM4th-d3-128f-ti	5	16	8	3328	5×2^{8}, 16×2^{7}	16	2304	256
SM4th-EM-d2-128s-ti	13	1	11	27648	13×2^{11}, 1×2^{10}	16	1360	256
SM4th-EM-d2-128f-ti	5	16	8	3328	5×2^{8}, 16×2^{7}	16	1360	256
uBlockith-d3-256s	8	14	12	61440	8×2^{12}, 14×2^{11}	16	5392	256
uBlockith-d3-256f	24	8	8	7168	24×2^{8}, 8×2^{7}	16	5392	256
uBlockith-EM-d3-256s	8	14	12	61440	8×2^{12}, 14×2^{11}	16	3856	256
uBlockith-EM-d3-256f	24	8	8	7168	24×2^{8}, 8×2^{7}	16	3856	256
Vistrutith-d3-512s	23	21	12	137216	23×2^{12}, 21×2^{11}	16	8464	512
Vistrutith-d3-512f	56	8	8	15360	56×2^{8}, 8×2^{7}	16	8464	512

where p is the string in Table 4 and the encoding contains no terminating zero byte. The message digest and root-seed derivation use

$$\mu = \mathsf{H}_2^0(D_{\mathsf{p}} \| \mathsf{pk}_1 \| \mathsf{pk}_2 \| \mathsf{msg}), \qquad (r, \mathsf{iv}^{\mathsf{pre}}) = \mathsf{H}_3(\mathsf{sk}_2 \| \mu \| \rho).$$

Table 4: Parameter-set strings for D_{p}

Scheme	p (ASCII)
SM4th-d3-128s-lo	`sm4th_d3_128s_loose`
SM4th-d3-128f-lo	`sm4th_d3_128f_loose`
SM4th-EM-d2-128s-lo	`sm4th_em_d2_128s_loose`
SM4th-EM-d2-128f-lo	`sm4th_em_d2_128f_loose`
SM4th-d3-128s-ti	`sm4th_d3_128s_tight`
SM4th-d3-128f-ti	`sm4th_d3_128f_tight`
SM4th-EM-d2-128s-ti	`sm4th_em_d2_128s_tight`
SM4th-EM-d2-128f-ti	`sm4th_em_d2_128f_tight`
uBlockith-d3-256s	`ublockith_d3_256s`
uBlockith-d3-256f	`ublockith_d3_256f`
uBlockith-EM-d3-256s	`ublockith_em_d3_256s`
uBlockith-EM-d3-256f	`ublockith_em_d3_256f`
Vistrutith-d3-512s	`vistrutith_d3_512s`
Vistrutith-d3-512f	`vistrutith_d3_512f`

SM3 output expansion. The specification uses hash functions supplied in `API_PKC.zip` [43]: `sm3hash`, `pseudohash`, and `pseudoXOF` in `auxfunc.c`. The NGCC requirements [44] state that these auxiliary functions are used only for correctness verification and preliminary performance testing without complete security guarantee. For byte-aligned inputs, `sm3hash` produces 256 bits, `pseudohash` produces 512, 768, or 1024 bits, and `pseudoXOF` produces other requested lengths.

3.3 Key and Signature Encodings

For every parameter set, the public key is $\mathrm{OS}(\boldsymbol{x}) \| \mathrm{OS}(\boldsymbol{y})$ and the secret key is $\mathrm{OS}(\boldsymbol{x}) \| \mathrm{OS}(\boldsymbol{k})$, each of length $2\lambda/8$ bytes. A signature concatenates the entries in Table 5, using OS for bit strings, FE for field elements, and LE32 for the counter. The masked witness is denoted by $\boldsymbol{d}_{\mathsf{w}}$.

Table 5: Signature byte layout for all parameter sets

Field	Byte length
$\boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}$	$(\tau-1)\hat{\ell}/8$
$\tilde{\boldsymbol{u}}$	$(\lambda_{\mathbb{F}} + B)/8$
$\boldsymbol{d}_{\mathsf{w}}$	$\ell/8$
$\tilde{a}_1, \ldots, \tilde{a}_{d-1}$	$(d-1)\lambda_{\mathbb{F}}/8$
decom_I	$(2\tau + T_{\mathsf{open}})N_{\mathsf{block}}/8$
chall_3	$\lambda_{\mathbb{F}}/8$
$\mathsf{iv}^{\mathsf{pre}}$	$\lambda_{\mathsf{iv}}/8$
ctr	4

The opening decom_I follows the formats of BAVC.Open. The total signature length in bytes is

$$\mathsf{sigsize}(\mathsf{param}) = \frac{\tau\hat{\ell} + (2\tau + T_{\mathsf{open}})N_{\mathsf{block}} + \lambda_{\mathbb{F}} + \lambda_{\mathsf{iv}} + 32}{8}. \tag{6}$$

In chall_2, the matrix $\tilde{\boldsymbol{V}}$ or $\tilde{\boldsymbol{V}}'$ is encoded by concatenating all $\lambda_{\mathbb{F}}$ columns in increasing order, including the zero columns. In chall_3, the coefficients are encoded in the order $\tilde{a}_0, \tilde{a}_1, \ldots, \tilde{a}_{d-1}$.

3.4 Analysis of Failure Probability

A counter value succeeds when the last w_{grind} challenge bits are zero and the BAVC opening contains at most T_{open} seeds. Let α be the fraction of index tuples $I \in \prod_{i=0}^{\tau-1}[0, N_i)$ satisfying the opening bound. For uniform independent hash outputs, one trial succeeds with probability $p = \alpha 2^{-w_{\mathsf{grind}}}$, giving a mean of $1/p = 2^{w_{\mathsf{grind}}}/\alpha$ trials without the counter limit. The signer tests the 2^{32} counter values in increasing order and returns $\perp$ if all trials fail. Thus,

$$\Pr[\mathsf{Fail}] = (1 - \alpha 2^{-w_{\mathsf{grind}}})^{2^{32}} \le \exp\left(-\alpha 2^{32-w_{\mathsf{grind}}}\right).$$

For SM4th-d3-128s-lo and SM4th-EM-d2-128s-lo, an approximate counting of the path unions defined by BAVC.PosInTree gives $\alpha \approx 0.0238090743$ and $\alpha \approx 0.0349024779$, respectively. With $w_{\mathsf{grind}} = 7$, the corresponding means are approximately 5376.10 and 3667.36 hash trials, and the failure bounds are below $2^{-1152569}$ and $2^{-1689587}$, respectively.

3.5 Statements of Differences

Our signature schemes follow the high-level VOLE-in-the-head framework of FAEST, but differ from FAEST in several important aspects. FAEST instantiates its schemes with AES and aligns the main security parameter across the construction: for a λ-bit security level, it uses an AES variant with a λ-bit key as both the OWF and the PRG, performs the ZKP over $\mathbb{F}_{2^\lambda}$, and chooses hash output lengths linear in λ. In contrast, Chinith decouples these roles when necessary and replaces the AES-based components with primitives that are better aligned with Chinese cryptographic standards and with the algebraic structure required by our proofs.

First, the SM4th variants keep the OWF key width $\lambda = 128$ and target quantum parameter $\lambda_{\mathsf{Q}} = 80$, while their PRG seed widths are $N_{\mathsf{block}} = 128$ for loose and $N_{\mathsf{block}} = 256$ for tight. The tight variants use $\lambda_{\mathbb{F}} = 160$ to reduce the $Q^2 2^{-\lambda_{\mathbb{F}}}$ field term, with this field used in VOLE,

QuickSilver, ZKHash, and field-valued Fiat–Shamir challenges. The loose variants use $\lambda_{\mathbb{F}} = 128$ for efficiency and compare specified challenge-search attacks in a separate circuit-cost model. The full security bound, including signing-randomness and primitive losses, is given in Section 9.1.7.

Second, we distinguish the lengths of IVs between the loose and tight variants. For the SM4th-lo variants, denoted by the suffix "lo", we keep the SM4-based PRG and its 128-bit IV for better performance and compatibility with SM4 hardware acceleration. The ti variants use a 256-bit IV with Ballet-256/256 to retain this conservative IV-separation estimate at 2^{80} signatures per key, as discussed in Section 3.2.

Third, we replace and redesign the OWF and PRG instantiations. For the 128-bit instantiations, SM4 provides the OWF, while SM4 is used as the PRG in the loose variants and Ballet is used as the PRG in the tight variants. For the 256-bit instantiations, uBlock is used as the OWF, while Ballet is used as a software-friendly PRG. For the 512-bit instantiations, we use Vistrutah, which provides a large-block primitive suitable for the target security level.

Fourth, we redesign the zero-knowledge constraint systems for the underlying block ciphers. FAEST is tailored to AES and its algebraic representation, whereas Chinith must handle the different nonlinear layers of SM4, uBlock, and Vistrutah.

Fifth, our hash functions are instantiated with SM3 or SM3-based extendable-output functions as required by NGCC.

Finally, we adopt the PRG-based leaf commitments introduced for FAEST-EM and use them in both our EM and non-EM variants. This reduces the cost of leaf commitments, while their binding security is expressed by the almost-injectivity property in Definition 6.

Overall, Chinith keeps the modular VOLE-in-the-head architecture of FAEST, but replaces and retunes the concrete primitives, field sizes, hash lengths, commitment mechanisms, and ZK constraint systems to match the selected block ciphers and the desired concrete security levels.

3.6 Statements of Additional Functionalities

This section specifies the additional functionalities required by NGCC of ICCS.

First, our signature schemes support messages of up to 2^{63} bits, since the complete hash input, including the parameter-set domain D_{p} and public key, remains shorter than the 2^{64}-bit input limit of SM3 [2]. However, we note that the submitted C implementations expose a byte-array API and process these hash inputs as in-memory buffers before invoking the SM3/SM3-XOF based hash routines. Therefore, the practical maximum message length of the current implementations is bounded by the machine word size, address space, and available memory.

Second, our evaluation assigns each key a signing budget of 2^{64} for the lo variants of SM4th and SM4th-EM, and 2^{80} for the remaining parameter sets, as specified in Equation (66).

4 Algorithm Description: Cryptographic Primitives

This section introduces the symmetric cryptographic primitives used in our signature schemes.

4.1 The SM4 Block Cipher

The SM4 algorithm [3] is a Chinese standard block cipher with a 128-bit block size and a 128-bit key size, running in $R = 32$ rounds for both the key expansion and encryption/decryption routines. In SM4th, SM4 supplies the OWF. Its encryption routine also supplies the PRG in the loose variants, while the tight variants use Ballet for the PRG. An SM4 word with bytes (b_0, b_1, b_2, b_3) has integer value $\sum_{j=0}^{3} b_j 2^{8(3-j)}$. Words and bytes are concatenated in increasing index order. Within a byte b, bit j is $(b \gg j) \bmod 2$, and its field representation is $\sum_{j=0}^{7} b_j t^j$. The vectors in the S-box matrix formulas are column vectors in this bit order. SM4 uses the following building blocks.

The SM4 S-box. S-box is a non-linear function mapping 8 bits to 8 bits. It is originally given as a substitution table in Table 6. Its algebraic representation is analyzed in [45] with details below.

Given a $\boldsymbol{x}$ of 8 bits, the S-box function $S(\cdot) : \mathbb{F}_2^8 \to \mathbb{F}_2^8$ is defined as

$$S(\boldsymbol{x}) = \boldsymbol{A} \cdot I(\boldsymbol{A} \cdot \boldsymbol{x} + \boldsymbol{c}) + \boldsymbol{c}. \tag{7}$$

Here, $I(\boldsymbol{x})$ is the patched inversion over $\mathbb{F}_{2^8}$. It first transforms $\boldsymbol{x}$ into a field element $x \in \mathbb{F}_{2^8}$, and then outputs the inverse of x if $x \neq 0$, otherwise outputs 0. The construction of $\boldsymbol{A} \in \mathbb{F}_2^{8\times 8}$ and $\boldsymbol{c} \in \mathbb{F}_2^8$ are as follows:

$$\boldsymbol{A} = \begin{pmatrix} 1&1&1&0&0&1&0&1 \\ 1&1&1&1&0&0&1&0 \\ 0&1&1&1&1&0&0&1 \\ 1&0&1&1&1&1&0&0 \\ 0&1&0&1&1&1&1&0 \\ 0&0&1&0&1&1&1&1 \\ 1&0&0&1&0&1&1&1 \\ 1&1&0&0&1&0&1&1 \end{pmatrix}, \quad \boldsymbol{c} = (1,1,0,0,1,0,1,1)^{\mathsf{T}}. \tag{8}$$

The irreducible polynomial is

$$m(t) = t^8 + t^7 + t^6 + t^5 + t^4 + t^2 + 1 \in \mathbb{F}_2[t]. \tag{9}$$

In SM4th, we also need the inverse matrix of $\boldsymbol{A}$, which is

$$\boldsymbol{A^{-1}} = \begin{pmatrix} 1&1&0&0&0&0&1&0 \\ 0&1&1&0&0&0&0&1 \\ 1&0&1&1&0&0&0&0 \\ 0&1&0&1&1&0&0&0 \\ 0&0&1&0&1&1&0&0 \\ 0&0&0&1&0&1&1&0 \\ 0&0&0&0&1&0&1&1 \\ 1&0&0&0&0&1&0&1 \end{pmatrix}. \tag{10}$$

Table 6: The SM4 S-box Substitution Table

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	D6	90	E9	FE	CC	E1	3D	B7	16	B6	14	C2	28	FB	2C	05
1	2B	67	9A	76	2A	BE	04	C3	AA	44	13	26	49	86	06	99
2	9C	42	50	F4	91	EF	98	7A	33	54	0B	43	ED	CF	AC	62
3	E4	B3	1C	A9	C9	08	E8	95	80	DF	94	FA	75	8F	3F	A6
4	47	07	A7	FC	F3	73	17	BA	83	59	3C	19	E6	85	4F	A8
5	68	6B	81	B2	71	64	DA	8B	F8	EB	0F	4B	70	56	9D	35
6	1E	24	0E	5E	63	58	D1	A2	25	22	7C	3B	01	21	78	87
7	D4	00	46	57	9F	D3	27	52	4C	36	02	E7	A0	C4	C8	9E
8	EA	BF	8A	D2	40	C7	38	B5	A3	F7	F2	CE	F9	61	15	A1
9	E0	AE	5D	A4	9B	34	1A	55	AD	93	32	30	F5	8C	B1	E3
A	1D	F6	E2	2E	82	66	CA	60	C0	29	23	AB	0D	53	4E	6F
B	D5	DB	37	45	DE	FD	8E	2F	03	FF	6A	72	6D	6C	5B	51
C	8D	1B	AF	92	BB	DD	BC	7F	11	D9	5C	41	1F	10	5A	D8
D	0A	C1	31	88	A5	CD	7B	BD	2D	74	D0	12	B8	E5	B4	B0
E	89	69	97	4A	0C	96	77	7E	65	B9	F1	09	C5	6E	C6	84
F	18	F0	7D	EC	3A	DC	4D	20	79	EE	5F	3E	D7	CB	39	48

The $T(\cdot)$ function. $T(\cdot) : \mathbb{F}_2^{32} \to \mathbb{F}_2^{32}$ is a composite substitution function composed of S-boxes and a linear transformation $L(\cdot) : \mathbb{F}_2^{32} \to \mathbb{F}_2^{32}$. Two linear transformations with slight differences,

denoted as $L'(\cdot)$ and $L(\cdot)$, are used in the key expansion and encryption routines, respectively. The function $T(\cdot)$ includes two phases as below.

– S-box phases. Assume the input is $\boldsymbol{a} = (\boldsymbol{a}_0, \boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3) \in (\mathbb{F}_2^8)^4$, and the output is $\boldsymbol{b} = (\boldsymbol{b}_0, \boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3) \in (\mathbb{F}_2^8)^4$. Then we have

$$(\boldsymbol{b}_0, \boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3) = (S(\boldsymbol{a}_0), S(\boldsymbol{a}_1), S(\boldsymbol{a}_2), S(\boldsymbol{a}_3)).$$

Define a function SubWord$(\cdot)$: $\mathbb{F}_2^{32} \to \mathbb{F}_2^{32}$ such that SubWord$(\boldsymbol{a}) = \boldsymbol{b}$.

– Linear transformations $L(\cdot)$ and $L'(\cdot)$. Assume the input is $\boldsymbol{b} = (\boldsymbol{b}_0, \boldsymbol{b}_1, \boldsymbol{b}_2, \boldsymbol{b}_3) \in (\mathbb{F}_2^8)^4$, and the output is $\boldsymbol{c} = (\boldsymbol{c}_0, \boldsymbol{c}_1, \boldsymbol{c}_2, \boldsymbol{c}_3) \in (\mathbb{F}_2^8)^4$. In the encryption routine, we have

$$\boldsymbol{c} = L(\boldsymbol{b}) = \boldsymbol{b} \oplus (\boldsymbol{b} \lll 2) \oplus (\boldsymbol{b} \lll 10) \oplus (\boldsymbol{b} \lll 18) \oplus (\boldsymbol{b} \lll 24). \tag{11}$$

In the key expansion routine, we have

$$\boldsymbol{c} = L'(\boldsymbol{b}) = \boldsymbol{b} \oplus (\boldsymbol{b} \lll 13) \oplus (\boldsymbol{b} \lll 23). \tag{12}$$

In SM4th, we also need the inversions of $L(\cdot)$ and $L'(\cdot)$:

$$\begin{aligned} \boldsymbol{b} = L^{-1}(\boldsymbol{c}) = \boldsymbol{c} &\oplus (\boldsymbol{c} \lll 2) \oplus (\boldsymbol{c} \lll 4) \oplus (\boldsymbol{c} \lll 8) \oplus (\boldsymbol{c} \lll 12) \oplus (\boldsymbol{c} \lll 14) \oplus (\boldsymbol{c} \lll 16) \\ &\oplus (\boldsymbol{c} \lll 18) \oplus (\boldsymbol{c} \lll 22) \oplus (\boldsymbol{c} \lll 24) \oplus (\boldsymbol{c} \lll 30) \end{aligned} \tag{13}$$

and

$$\begin{aligned} \boldsymbol{b} = L'^{-1}(\boldsymbol{c}) = \boldsymbol{c} &\oplus (\boldsymbol{c} \lll 2) \oplus (\boldsymbol{c} \lll 4) \oplus (\boldsymbol{c} \lll 8) \\ &\oplus (\boldsymbol{c} \lll 11) \oplus (\boldsymbol{c} \lll 12) \oplus (\boldsymbol{c} \lll 14) \oplus (\boldsymbol{c} \lll 17) \\ &\oplus (\boldsymbol{c} \lll 22) \oplus (\boldsymbol{c} \lll 23) \oplus (\boldsymbol{c} \lll 24) \oplus (\boldsymbol{c} \lll 30) \oplus (\boldsymbol{c} \lll 31). \end{aligned} \tag{14}$$

4.1.1 The Key Expansion Routine of SM4

The key expansion algorithm SM4.KeyExpansion takes the SM4 initial key $\boldsymbol{k} \in \{0,1\}^{128}$ as inputs, and outputs an expanded key $\bar{\boldsymbol{k}} \in (\{0,1\}^{32})^{R+4}$, which is used in the encryption routine. SM4.KeyExpansion also uses the system parameters FK and fixed parameters CK as in Table 7.

Table 7: SM4 system parameters $\{FK_i\}_{i\in[0,4)}$ and fixed parameters $\{CK_i\}_{i\in[0,32)}$

i	0	1	2	3
FK_i	0xA3B1BAC6	0x56AA3350	0x677D9197	0xB27022DC

i	0	1	2	3
CK_i	0x00070E15	0x1C232A31	0x383F464D	0x545B6269
CK_{i+4}	0x70777E85	0x8C939AA1	0xA8AFB6BD	0xC4CBD2D9
CK_{i+8}	0xE0E7EEF5	0xFC030A11	0x181F262D	0x343B4249
CK_{i+12}	0x50575E65	0x6C737A81	0x888F969D	0xA4ABB2B9
CK_{i+16}	0xC0C7CED5	0xDCE3EAF1	0xF8FF060D	0x141B2229
CK_{i+20}	0x30373E45	0x4C535A61	0x686F767D	0x848B9299
CK_{i+24}	0xA0A7AEB5	0xBCC3CAD1	0xD8DFE6ED	0xF4FB0209
CK_{i+28}	0x10171E25	0x2C333A41	0x484F565D	0x646B7279

SM4.KeyExpansion takes the 128-bit initial key $\boldsymbol{k} = (\boldsymbol{k}_0, \boldsymbol{k}_1, \boldsymbol{k}_2, \boldsymbol{k}_3)$ as input and derives a sequence of round keys. First, the initial words are mixed with the system parameters FK_i. Then, for each round i, a temporary value is computed from the previous words and the constant CK_i, followed by the nonlinear transformation SubWord and the linear transformation $L'(\cdot)$. Finally, the next expanded key word is generated by XORing this result with $\bar{\boldsymbol{k}}[i]$.

SM4.KeyExpansion($\boldsymbol{k}, \{FK_i\}_{i\in[0,4)}, \{CK_i\}_{i\in[0,32)}, \mathsf{param}$)

INPUT: $\boldsymbol{k} = (\boldsymbol{k}_0, \boldsymbol{k}_1, \boldsymbol{k}_2, \boldsymbol{k}_3) \in (\{0,1\}^{32})^4$
OUTPUT: $\bar{\boldsymbol{k}} \in (\{0,1\}^{32})^{R+4}$

new $\bar{\boldsymbol{k}} \in (\{0,1\}^{32})^{R+4}$
new $\mathsf{tmp} \in \{0,1\}^{32}$ // temporary values
for $i \in [0..4)$ **do**
    $\bar{\boldsymbol{k}}[i] \leftarrow \boldsymbol{k}[i] \oplus FK_i$.
end for
for $i \in [0\,..\,R)$ **do**
    $\mathsf{tmp} \leftarrow \bar{\boldsymbol{k}}[i+1] \oplus \bar{\boldsymbol{k}}[i+2] \oplus \bar{\boldsymbol{k}}[i+3] \oplus CK_i$
    $\mathsf{tmp} \leftarrow \mathsf{SubWord}(\mathsf{tmp})$
    $\mathsf{tmp} \leftarrow L'(\mathsf{tmp})$ // $L'(\cdot)$ is defined in Equation (12)
    $\bar{\boldsymbol{k}}[i+4] \leftarrow \bar{\boldsymbol{k}}[i] \oplus \mathsf{tmp}$
end for
return $\bar{\boldsymbol{k}}$

4.1.2 The Encryption Routine of SM4

The encryption algorithm SM4.Encryption takes a 128-bit plaintext $\boldsymbol{in}$ and the expanded key $\bar{\boldsymbol{k}}$ as input and returns a 128-bit ciphertext $\boldsymbol{out}$. It first initializes the internal state $\boldsymbol{z}$ with the 4 32-bit words of $\boldsymbol{in}$. Then, in each round i, a temporary value tmp is computed from $\boldsymbol{z}[i+1]$, $\boldsymbol{z}[i+2]$, $\boldsymbol{z}[i+3]$, and the round key $\bar{k}[i+4]$, followed by the nonlinear transformation SubWord and the linear transformation $L(\cdot)$. Finally, the next state word is obtained as $\boldsymbol{z}[i+4] = \boldsymbol{z}[i] \oplus \mathsf{tmp}$, and the ciphertext is formed by reversing the order of the last four state words.

SM4.Encryption($\boldsymbol{in}, \bar{\boldsymbol{k}}, \mathsf{param}$)

INPUT: $\boldsymbol{in} \in (\{0,1\}^{32})^4, \bar{\boldsymbol{k}} \in (\{0,1\}^{32})^{4+R}$
OUTPUT: $\boldsymbol{out} \in (\{0,1\}^{32})^4$

new $\boldsymbol{out} \in (\{0,1\}^{32})^4$
new $\boldsymbol{z} \in (\{0,1\}^{32})^{R+4}$
new $\mathsf{tmp} \in \{0,1\}^{32}$
for $i \in [0\,..\,4)$ **do**
    $\boldsymbol{z}[i] \leftarrow \boldsymbol{in}[i]$
end for
for $i \in [0\,..\,R)$ **do**
    $\mathsf{tmp} \leftarrow \boldsymbol{z}[i+1] \oplus \boldsymbol{z}[i+2] \oplus \boldsymbol{z}[i+3] \oplus \bar{\boldsymbol{k}}[i+4]$
    $\mathsf{tmp} \leftarrow \mathsf{SubWord}(\mathsf{tmp})$
    $\mathsf{tmp} \leftarrow L(\mathsf{tmp})$ // $L(\cdot)$ is defined in Equation (11)
    $\boldsymbol{z}[i+4] \leftarrow \boldsymbol{z}[i] \oplus \mathsf{tmp}$
end for
return $(\boldsymbol{z}[35], \boldsymbol{z}[34], \boldsymbol{z}[33], \boldsymbol{z}[32])$

4.2 The uBlock Block Cipher

The uBlock algorithm [4] is a family of block ciphers with block sizes and key sizes of 128 or 256 bits. It is designed by Wu, Zhang, Zheng, and Li, all Chinese researchers. This document only involves the uBlock variant with a 256-bit block size and a 256-bit key size, and we use the term uBlock to represent such a variant for ease of notation. uBlock runs in $R = 24$ rounds for both the key expansion and encryption/decryption routines. In uBlockith, we use the key expansion and encryption routines of uBlock to construct the OWF, while we use the software-friendly Ballet [5] to construct the PRGs. Recall that uBlock involves the following building blocks.

The uBlock S-box. S-box is a function mapping 4 bits to 4 bits. Its substitution table version and inverse are presented in Table 8. In uBlockith, we also need their algebraic representations.

Table 8: The uBlock substitution table and its inverse

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	7	4	9	C	B	A	D	8	F	E	1	6	0	3	2	5
$S^{-1}(x)$	C	A	E	D	1	F	B	0	7	2	5	4	3	6	9	8

Given the input $(x_0, x_1, x_2, x_3) \in \mathbb{F}_2^4$, the output (y_0, y_1, y_2, y_3) of S-boxes is computed as

$$\begin{aligned} y_0 &= 1 \oplus x_0 \oplus x_2x_3, \\ y_1 &= 1 \oplus x_0 \oplus x_1 \oplus x_0x_1 \oplus x_0x_2 \oplus x_0x_1x_2 \oplus x_0x_3 \oplus x_2x_3, \\ y_2 &= 1 \oplus x_1 \oplus x_0x_1 \oplus x_2 \oplus x_1x_2x_3, \\ y_3 &= x_1 \oplus x_2 \oplus x_3 \oplus x_1x_2. \end{aligned} \tag{15}$$

For the inverse S-box, given the input (y_0, y_1, y_2, y_3), the output (z_0, z_1, z_2, z_3) is computed as

$$\begin{aligned} z_0 &= y_0y_1 \oplus y_2 \oplus y_3 \oplus y_0y_3 \oplus y_2y_3 \oplus y_0y_1y_3, \\ z_1 &= y_0 \oplus y_1 \oplus y_3 \oplus y_0y_3, \\ z_2 &= 1 \oplus y_0 \oplus y_2 \oplus y_0y_1, \\ z_3 &= 1 \oplus y_2 \oplus y_3 \oplus y_0y_2 \oplus y_1y_2 \oplus y_2y_3 \oplus y_0y_2y_3. \end{aligned} \tag{16}$$

Clearly, both the S-box and inverse S-box can be represented as degree-3 binary circuits.

Like SM4, we define two functions SubDWord$(\cdot)$ and SubQWord$(\cdot)$. For the former, assume the input is $\boldsymbol{a} = (\boldsymbol{a}_0, \boldsymbol{a}_1, \ldots, \boldsymbol{a}_{15}) \in (\mathbb{F}_2^4)^{16}$. Then we have

$$\mathsf{SubDWord}(\boldsymbol{a}) = (S(\boldsymbol{a}_0), S(\boldsymbol{a}_1), \ldots, S(\boldsymbol{a}_{15})). \tag{17}$$

For the latter, assume the input is $\boldsymbol{b} = (\boldsymbol{b}_0, \boldsymbol{b}_1, \ldots, \boldsymbol{b}_{31}) \in (\mathbb{F}_2^4)^{32}$. Then we have

$$\mathsf{SubQWord}(\boldsymbol{b}) = (S(\boldsymbol{b}_0), S(\boldsymbol{b}_1), \ldots, S(\boldsymbol{b}_{31})). \tag{18}$$

Vector permutations $PL(\cdot)$ and $PR(\cdot)$. Both $PL(\cdot)$ and $PR(\cdot)$ are byte-wise permutations on 16 bytes, as listed in Table 9. For example, $PL(\cdot)$ is defined as

$$PL : (\{0,1\}^8)^{16} \to (\{0,1\}^8)^{16},$$

which maps $\boldsymbol{y} = (\boldsymbol{y}_0, \boldsymbol{y}_1, \ldots, \boldsymbol{y}_{15})$ to $\boldsymbol{z} = (\boldsymbol{z}_0, \boldsymbol{z}_1, \ldots, \boldsymbol{z}_{15})$, where

$$\boldsymbol{z}_0 = \boldsymbol{y}_2, \quad \boldsymbol{z}_1 = \boldsymbol{y}_7, \quad \boldsymbol{z}_2 = \boldsymbol{y}_8, \quad \boldsymbol{z}_3 = \boldsymbol{y}_{13}, \quad \boldsymbol{z}_4 = \boldsymbol{y}_3, \quad \boldsymbol{z}_5 = \boldsymbol{y}_6, \quad \boldsymbol{z}_6 = \boldsymbol{y}_9, \quad \boldsymbol{z}_7 = \boldsymbol{y}_{12},$$

$$\boldsymbol{z}_8 = \boldsymbol{y}_1, \quad \boldsymbol{z}_9 = \boldsymbol{y}_4, \quad \boldsymbol{z}_{10} = \boldsymbol{y}_{15}, \quad \boldsymbol{z}_{11} = \boldsymbol{y}_{10}, \quad \boldsymbol{z}_{12} = \boldsymbol{y}_{14}, \quad \boldsymbol{z}_{13} = \boldsymbol{y}_{11}, \quad \boldsymbol{z}_{14} = \boldsymbol{y}_5, \quad \boldsymbol{z}_{15} = \boldsymbol{y}_0.$$

Table 9: The uBlock Permutation Table

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$PL(i)$	2	7	8	13	3	6	9	12	1	4	15	10	14	11	5	0
$PR(i)$	6	11	1	12	9	4	2	15	7	0	13	10	14	3	8	5

Vector permutation $PK(\cdot)$. $PK(\cdot)$ is a nibble-wise permutation on 32 nibbles, as listed in Table 10. For this permutation, octets follow their serialized order and the high nibble of each octet precedes its low nibble. Bits inside a nibble are ordered from least to most significant. For a serialized bit vector $\boldsymbol{x} \in \{0,1\}^{128}$, define

$$\nu(j,b) = 8\lfloor j/2 \rfloor + 4(1 - (j \bmod 2)) + b, \qquad j \in [0,32), \quad b \in [0,4). \tag{19}$$

The nibble vector $\boldsymbol{y}$ has bits $\boldsymbol{y}[j][b] = \boldsymbol{x}[\nu(j,b)]$. If $\boldsymbol{z} = PK(\boldsymbol{y})$ serializes as $\boldsymbol{t} \in \{0,1\}^{128}$, its bits are $\boldsymbol{t}[\nu(j,b)] = \boldsymbol{x}[\nu(PK(j),b)]$. The same index map defines PK on VOLE commitments. It is defined as

$$PK : (\{0,1\}^4)^{32} \to (\{0,1\}^4)^{32}, \tag{20}$$

which maps $\boldsymbol{y} = (\boldsymbol{y}[0], \ldots, \boldsymbol{y}[31])$ to $\boldsymbol{z} = (\boldsymbol{z}[0], \ldots, \boldsymbol{z}[31])$ as follows:

$$\begin{array}{cccc}
\boldsymbol{z}[0] = \boldsymbol{y}[10] & \boldsymbol{z}[1] = \boldsymbol{y}[5] & \boldsymbol{z}[2] = \boldsymbol{y}[15] & \boldsymbol{z}[3] = \boldsymbol{y}[0] \\
\boldsymbol{z}[4] = \boldsymbol{y}[2] & \boldsymbol{z}[5] = \boldsymbol{y}[7] & \boldsymbol{z}[6] = \boldsymbol{y}[8] & \boldsymbol{z}[7] = \boldsymbol{y}[13] \\
\boldsymbol{z}[8] = \boldsymbol{y}[1] & \boldsymbol{z}[9] = \boldsymbol{y}[14] & \boldsymbol{z}[10] = \boldsymbol{y}[4] & \boldsymbol{z}[11] = \boldsymbol{y}[12] \\
\boldsymbol{z}[12] = \boldsymbol{y}[9] & \boldsymbol{z}[13] = \boldsymbol{y}[11] & \boldsymbol{z}[14] = \boldsymbol{y}[3] & \boldsymbol{z}[15] = \boldsymbol{y}[6] \\
\boldsymbol{z}[16] = \boldsymbol{y}[24] & \boldsymbol{z}[17] = \boldsymbol{y}[25] & \boldsymbol{z}[18] = \boldsymbol{y}[26] & \boldsymbol{z}[19] = \boldsymbol{y}[27] \\
\boldsymbol{z}[20] = \boldsymbol{y}[28] & \boldsymbol{z}[21] = \boldsymbol{y}[29] & \boldsymbol{z}[22] = \boldsymbol{y}[30] & \boldsymbol{z}[23] = \boldsymbol{y}[31] \\
\boldsymbol{z}[24] = \boldsymbol{y}[16] & \boldsymbol{z}[25] = \boldsymbol{y}[17] & \boldsymbol{z}[26] = \boldsymbol{y}[18] & \boldsymbol{z}[27] = \boldsymbol{y}[19] \\
\boldsymbol{z}[28] = \boldsymbol{y}[20] & \boldsymbol{z}[29] = \boldsymbol{y}[21] & \boldsymbol{z}[30] = \boldsymbol{y}[22] & \boldsymbol{z}[31] = \boldsymbol{y}[23]
\end{array}$$

Table 10: The uBlock key schedule permutation $PK(\cdot)$

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$PK(i)$	10	5	15	0	2	7	8	13	1	14	4	12	9	11	3	6
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$PK(i)$	24	25	26	27	28	29	30	31	16	17	18	19	20	21	22	23

The $T(\cdot)$ function. uBlock uses $T(\cdot) : \mathbb{F}_2^{64} \to \mathbb{F}_2^{64}$ as a linear transformation that operates on each nibble of its input independently. It multiplies every nibble by $\beta = t \bmod m(t)$, encoded as `0x2`, in $\mathbb{F}_{2^4} = \mathbb{F}_2[t]/(m(t))$, where

$$m(t) = t^4 + t + 1 \in \mathbb{F}_2[t].$$

For an input $\boldsymbol{x} = (\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_{15})$ with $\boldsymbol{x}_i \in \{0,1\}^4$, we first transform $\boldsymbol{x}_i$ into a field element $x_i \in \mathbb{F}_{2^4}$, and then define

$$T(\boldsymbol{x}) = (\beta \cdot x_0,\ \beta \cdot x_1,\ \ldots,\ \beta \cdot x_{15}). \tag{21}$$

For a nibble with least-significant-bit-first coordinates $(y_0, \ldots, y_3)$, representing $\sum_{i=0}^{3} y_i \beta^i$, multiplication by β gives output coordinates $(z_0, \ldots, z_3)$ as

$$z_0 = y_3, \quad z_1 = y_0 \oplus y_3, \quad z_2 = y_1, \quad z_3 = y_2. \tag{22}$$

4.2.1 The Key Expansion Routine of uBlock

The key expansion algorithm uBlock.KeyExpansion takes the uBlock initial key $\boldsymbol{k} \in \{0,1\}^{256}$ as inputs, and outputs an expanded key $\bar{\boldsymbol{k}} \in (\{0,1\}^{256})^{R+1}$, which is used in the encryption routine. uBlock.KeyExpansion uses the 32-bit round constants RC_i in Table 11, indexed by $i \in [1, R]$. Their 64-bit encodings are

$$\widehat{RC}[i] = \mathrm{BE32}(RC_i) \,\|\, \texttt{00 00 00 00} \in \{0,1\}^{64}, \tag{23}$$

Table 11: uBlock round constants $\{RC_i\}_{i\in[24]}$

i	1	2	3	4
RC_i	0x988CC9DD	0xF0E4A1B5	0x21357064	0x8397D2C6
RC_{i+4}	0xC7D39682	0x4F5B1E0A	0x5E4A0F1B	0x7C682D39
RC_{i+8}	0x392D687C	0xB3A7E2F6	0xA7B3F6E2	0x8E9ADFCB
RC_{i+12}	0xDCC88D99	0x786C293D	0x30246175	0xA1B5F0E4
RC_{i+16}	0x8296D3C7	0xC5D19480	0x4A5E1B0F	0x55410410
RC_{i+20}	0x6B7F3A2E	0x17034652	0xEFFBBEAA	0x1F0B4E5A

where BE32 writes the most significant octet first and the four appended octets are zero. Both ordinary key expansion and its VOLE constraints use this encoding.

Specifically, the key $\boldsymbol{k}$ is first split into four 64-bit double words $(\boldsymbol{k}_0, \boldsymbol{k}_1, \boldsymbol{k}_2, \boldsymbol{k}_3)$, and the initial round key is set as $\bar{\boldsymbol{k}}[0] \leftarrow \boldsymbol{k}$. At each round $i \in [1\,..\,R]$, the state is updated as follows:

$$\begin{aligned}(\boldsymbol{k}_0,\, \boldsymbol{k}_1) &\leftarrow PK(\boldsymbol{k}_0,\, \boldsymbol{k}_1),\\ \boldsymbol{k}_2 &\leftarrow \boldsymbol{k}_2 \oplus \mathsf{SubDWord}(\boldsymbol{k}_0 \oplus \widehat{RC}[i]),\\ \boldsymbol{k}_3 &\leftarrow \boldsymbol{k}_3 \oplus T(\boldsymbol{k}_1),\\ \boldsymbol{k} &\leftarrow (\boldsymbol{k}_2,\, \boldsymbol{k}_3,\, \boldsymbol{k}_1,\, \boldsymbol{k}_0),\end{aligned}$$

after which the round key is extracted as $\bar{\boldsymbol{k}}[i] \leftarrow \boldsymbol{k}$.

$\mathsf{uBlock.KeyExpansion}(\boldsymbol{k},\, \{RC_i\}_{i\in[R]},\, PK(\cdot),\, \mathsf{param})$

INPUT: $\boldsymbol{k} = (\boldsymbol{k}_0^{(0)}, \boldsymbol{k}_1^{(0)}, \boldsymbol{k}_2^{(0)}, \boldsymbol{k}_3^{(0)}) \in (\{0,1\}^{64})^4$, round constants $\{RC_i\}_{i\in[R]}$, permutation PK
OUTPUT: $\bar{\boldsymbol{k}} \in (\{0,1\}^{256})^{R+1}$

new $\bar{\boldsymbol{k}} \in (\{0,1\}^{256})^{R+1}$
$\bar{\boldsymbol{k}}[0] \leftarrow (\boldsymbol{k}_0^{(0)}, \boldsymbol{k}_1^{(0)}, \boldsymbol{k}_2^{(0)}, \boldsymbol{k}_3^{(0)})$
for $i \in [1\,..\,R]$ **do**
    new $(\boldsymbol{k}_0^{(i)}, \boldsymbol{k}_1^{(i)}, \boldsymbol{k}_2^{(i)}, \boldsymbol{k}_3^{(i)})$
    $(\boldsymbol{k}_0^{(i)}, \boldsymbol{k}_1^{(i)}) \leftarrow PK(\boldsymbol{k}_0^{(i-1)}, \boldsymbol{k}_1^{(i-1)})$ // $PK(\cdot)$ is defined in Equation (20)
    $\boldsymbol{k}_2^{(i)} \leftarrow \boldsymbol{k}_2^{(i-1)} \oplus \mathsf{SubDWord}(\boldsymbol{k}_0^{(i)} \oplus \widehat{RC}[i])$ // SubDWord$(\cdot)$ is defined in Equation (17)
    $\boldsymbol{k}_3^{(i)} \leftarrow \boldsymbol{k}_3^{(i-1)} \oplus T(\boldsymbol{k}_1^{(i)})$ // $T(\cdot)$ is defined in Equation (21)
    $\bar{\boldsymbol{k}}[i] \leftarrow (\boldsymbol{k}_2^{(i)}, \boldsymbol{k}_3^{(i)}, \boldsymbol{k}_1^{(i)}, \boldsymbol{k}_0^{(i)})$
    $(\boldsymbol{k}_0^{(i)}, \boldsymbol{k}_1^{(i)}, \boldsymbol{k}_2^{(i)}, \boldsymbol{k}_3^{(i)}) \leftarrow (\boldsymbol{k}_2^{(i)}, \boldsymbol{k}_3^{(i)}, \boldsymbol{k}_1^{(i)}, \boldsymbol{k}_0^{(i)})$
end for
return $\bar{\boldsymbol{k}}$

4.2.2 The Encryption Routine of uBlock

The encryption algorithm uBlock.Encryption takes a 256-bit plaintext $\boldsymbol{in}$ and the expanded key $\bar{\boldsymbol{k}} \in (\{0,1\}^{256})^{R+1}$ as input, and outputs a 256-bit ciphertext $\boldsymbol{out}$. It first splits $\boldsymbol{in}$ into two 128-bit branches $(\boldsymbol{x}_0, \boldsymbol{x}_1)$. Then, in each round $i \in [0\,..\,R)$, the round key $\bar{\boldsymbol{k}}[i]$ is divided into two 128-bit subkeys $(\boldsymbol{k}_0, \boldsymbol{k}_1)$, which are XORed with $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$, respectively, before applying the nonlinear function SubQWord. After that, the two state branches are updated through a sequence of XOR operations and word-wise cyclic left shifts by 4, 8, 8, and 20 bits, followed by the vector permutations $PL(\cdot)$ and $PR(\cdot)$ applied to $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$, respectively. Finally, after R rounds, the ciphertext is obtained as $\boldsymbol{out} = \bar{\boldsymbol{k}}[R] \oplus (\boldsymbol{x}_0 \,\|\, \boldsymbol{x}_1)$.

uBlock.Encryption($\boldsymbol{in}, \bar{\boldsymbol{k}}$, param)

INPUT: $\boldsymbol{in} \in \{0,1\}^{256}$, $\bar{\boldsymbol{k}} \in (\{0,1\}^{256})^{R+1}$
OUTPUT: $\boldsymbol{out} \in \{0,1\}^{256}$

new $\boldsymbol{x}_0, \boldsymbol{x}_1 \in \{0,1\}^{128}$
new $\boldsymbol{k}_0, \boldsymbol{k}_1 \in \{0,1\}^{128}$
$(\boldsymbol{x}_0, \boldsymbol{x}_1) \leftarrow \boldsymbol{in}$
for $i \in [0\,..\,R)$ **do**
    $(\boldsymbol{k}_0, \boldsymbol{k}_1) \leftarrow \bar{\boldsymbol{k}}[i]$
    $\boldsymbol{x}_0 \leftarrow \mathsf{SubQWord}(\boldsymbol{x}_0 \oplus \boldsymbol{k}_0)$ // defined in Equation (18)
    $\boldsymbol{x}_1 \leftarrow \mathsf{SubQWord}(\boldsymbol{x}_1 \oplus \boldsymbol{k}_1)$
    $\boldsymbol{x}_1 \leftarrow \boldsymbol{x}_1 \oplus \boldsymbol{x}_0$
    $\boldsymbol{x}_0 \leftarrow \boldsymbol{x}_0 \oplus (\boldsymbol{x}_1(\lll 4)_{32})$ // word-wise cyclic left shifts
    $\boldsymbol{x}_1 \leftarrow \boldsymbol{x}_1 \oplus (\boldsymbol{x}_0(\lll 8)_{32})$
    $\boldsymbol{x}_0 \leftarrow \boldsymbol{x}_0 \oplus (\boldsymbol{x}_1(\lll 8)_{32})$
    $\boldsymbol{x}_1 \leftarrow \boldsymbol{x}_1 \oplus (\boldsymbol{x}_0(\lll 20)_{32})$
    $\boldsymbol{x}_0 \leftarrow \boldsymbol{x}_0 \oplus \boldsymbol{x}_1$
    $\boldsymbol{x}_0 \leftarrow PL(\boldsymbol{x}_0)$ // defined in Table 9
    $\boldsymbol{x}_1 \leftarrow PR(\boldsymbol{x}_1)$ // defined in Table 9
end for
$\boldsymbol{out} \leftarrow \bar{\boldsymbol{k}}[R] \oplus (\boldsymbol{x}_0 \,\|\, \boldsymbol{x}_1)$
return $\boldsymbol{out}$

4.3 The Ballet Block Cipher

The Ballet algorithm [5] is a family of block ciphers with block sizes and key sizes of 128 or 256 bits. It is designed by Cui, Wang, Fan, Hu, Fu and Huang, all Chinese researchers. This document only involves the Ballet variant with a 256-bit block size and a 256-bit key size, and we use the term Ballet to represent such a variant for ease of notation. Ballet is software-friendly and we use Ballet as PRGs in uBlockith and SM4th-ti. The concrete variant has 74 rounds. The following description uses 64-bit words, addition modulo 2^{64}, and rotations within a word. For a pair of words, define

$$G(p,q) = \big((p \ll 3) \oplus (q \gg 61) \oplus (p \ll 5) \oplus (q \gg 59), (q \ll 3) \oplus (p \gg 61) \oplus (q \ll 5) \oplus (p \gg 59)\big),$$

where every shift is truncated to 64 bits.

Ballet.KeyExpansion(K)

INPUT: A 32-byte key K
OUTPUT: 148 round-key words $\mathsf{rk}[0], \ldots, \mathsf{rk}[147]$

$(k_1, k_0, k_3, k_2) \leftarrow (\mathrm{BE64}^{-1}(K[8j..8j+8)))_{j=0}^{3}$
for $m \in [0, 37)$ **do**
    $\mathsf{rk}[4m] \leftarrow k_0$, $\mathsf{rk}[4m+1] \leftarrow k_1$
    $(k_0, k_1) \leftarrow (k_0, k_1) \oplus G(k_2, k_3) \oplus (2m, 0)$
    $\mathsf{rk}[4m+2] \leftarrow k_2$, $\mathsf{rk}[4m+3] \leftarrow k_3$
    $(k_2, k_3) \leftarrow (k_2, k_3) \oplus G(k_0, k_1) \oplus (2m+1, 0)$
end for
return rk

4.4 The Vistrutah Block Cipher

The Vistrutah algorithm [6] is a family of larger block ciphers with block sizes and key sizes of 256 or 512 bits. It is designed by Avanzi, Chakraborthi, Chakraborty and List. We use the variant with a block size and key size of 512 bits, and call it Vistrutah in this document for ease of notation. Vistrutah offers a long version with 9 two-round steps and a short version with 6 two-round steps.

Ballet.Encryption(P, rk)

INPUT: A 32-byte plaintext P and round keys rk
OUTPUT: A 32-byte ciphertext

$(a, b, c, e) \leftarrow (\mathrm{BE64}^{-1}(P[8j..8j+8)))_{j=0}^{3}$
for $m \in [0, 37)$ **do**
  $t \leftarrow b \oplus c$
  $b \leftarrow b \oplus \mathsf{rk}[4m+1]$, $c \leftarrow c \oplus \mathsf{rk}[4m]$
  $b \leftarrow b \lll 6$, $e \leftarrow e \lll 15$, $a \leftarrow a \lll 6$
  $a \leftarrow a + (t \lll 9)$, $e \leftarrow e + (t \lll 14)$
  $t \leftarrow a \oplus e$
  $b \leftarrow b + (t \lll 9)$
  $e \leftarrow e \oplus \mathsf{rk}[4m+2]$, $a \leftarrow a \oplus \mathsf{rk}[4m+3]$
  $c \leftarrow c \lll 15$, $c \leftarrow c + (t \lll 14)$
end for
return $\mathrm{BE64}(b) \| \mathrm{BE64}(a) \| \mathrm{BE64}(e) \| \mathrm{BE64}(c)$

We use the long version and write $R = 18$ for its number of AES rounds. On a high level, Vistrutah iterates a step function that applies two AES rounds to each 128-bit block of the state, followed by a state-wide cell permutation. We next introduce the building blocks of AES and Vistrutah.

The AES State. The state of AES is represented as a 4×4 array of bytes, denoted by $\boldsymbol{s}$. Each byte of the state is written as $s_{r,c}$, where $0 \le r < 4$ and $0 \le c < 4$ denote the row and column indices, respectively. Hence, the state is of the form

$$\boldsymbol{s} = \begin{pmatrix} \boldsymbol{s}_{0,0} & \boldsymbol{s}_{0,1} & \boldsymbol{s}_{0,2} & \boldsymbol{s}_{0,3} \\ \boldsymbol{s}_{1,0} & \boldsymbol{s}_{1,1} & \boldsymbol{s}_{1,2} & \boldsymbol{s}_{1,3} \\ \boldsymbol{s}_{2,0} & \boldsymbol{s}_{2,1} & \boldsymbol{s}_{2,2} & \boldsymbol{s}_{2,3} \\ \boldsymbol{s}_{3,0} & \boldsymbol{s}_{3,1} & \boldsymbol{s}_{3,2} & \boldsymbol{s}_{3,3} \end{pmatrix}.$$

The state is interpreted in column-major order. That is, when viewing $\boldsymbol{s}$ as a 16-byte string, one reads the entries column by column:

$$\boldsymbol{s} = \boldsymbol{s}_{0,0}, \boldsymbol{s}_{1,0}, \boldsymbol{s}_{2,0}, \boldsymbol{s}_{3,0}, \boldsymbol{s}_{0,1}, \boldsymbol{s}_{1,1}, \boldsymbol{s}_{2,1}, \boldsymbol{s}_{3,1}, \boldsymbol{s}_{0,2}, \boldsymbol{s}_{1,2}, \boldsymbol{s}_{2,2}, \boldsymbol{s}_{3,2}, \boldsymbol{s}_{0,3}, \boldsymbol{s}_{1,3}, \boldsymbol{s}_{2,3}, \boldsymbol{s}_{3,3}.$$

The AES S-box. The AES S-box is a non-linear function mapping 8 bits to 8 bits. Given an input vector $\boldsymbol{x} = (x_0, x_1, \ldots, x_7) \in \mathbb{F}_2^8$, the S-box function SubByte$(\cdot)$: $\mathbb{F}_2^8 \to \mathbb{F}_2^8$ is defined as

$$S(\boldsymbol{x}) = \boldsymbol{A} \cdot I(\boldsymbol{x}) + \boldsymbol{c}, \tag{24}$$

where $I(\boldsymbol{x})$ is the patched inversion over $\mathrm{GF}(2^8)$. It first transforms $\boldsymbol{x}$ into a field element $x \in \mathrm{GF}(2^8)$, and then outputs the inverse of x if $x \neq 0$, otherwise outputs 0. The form of $\boldsymbol{A}$ and $\boldsymbol{c}$ is

$$\boldsymbol{A} = \begin{pmatrix} 1&0&0&0&1&1&1&1 \\ 1&1&0&0&0&1&1&1 \\ 1&1&1&0&0&0&1&1 \\ 1&1&1&1&0&0&0&1 \\ 1&1&1&1&1&0&0&0 \\ 0&1&1&1&1&1&0&0 \\ 0&0&1&1&1&1&1&0 \\ 0&0&0&1&1&1&1&1 \end{pmatrix}, \quad \boldsymbol{c} = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}.$$

The irreducible polynomial is

$$m(t) = t^8 + t^4 + t^3 + t + 1 \in \mathbb{F}_2[t]. \tag{25}$$

For an AES state $\boldsymbol{s}$, define SubBytes$(\boldsymbol{s})$ such that each byte is applied via SubByte independently.

Accordingly, the inverse affine component of the AES S-box is given by

$$\boldsymbol{u} = \boldsymbol{A}^{-1} \cdot \boldsymbol{y} + \boldsymbol{c}',$$

where

$$\boldsymbol{A}^{-1} = \begin{pmatrix} 0&0&1&0&0&1&0&1 \\ 1&0&0&1&0&0&1&0 \\ 0&1&0&0&1&0&0&1 \\ 1&0&1&0&0&1&0&0 \\ 0&1&0&1&0&0&1&0 \\ 0&0&1&0&1&0&0&1 \\ 1&0&0&1&0&1&0&0 \\ 0&1&0&0&1&0&1&0 \end{pmatrix}, \quad \boldsymbol{c}' = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

Equalivently, for each bit position $0 \le j < 8$, one has

$$u_j = y_{(j-1) \bmod 8} \oplus y_{(j-3) \bmod 8} \oplus y_{(j-6) \bmod 8} \oplus c'_j, \tag{26}$$

where $c'_j = 1$ if $j \in \{0, 2\}$, and $c'_j = 0$ otherwise.

ShiftRows. The ShiftRows transformation cyclically shifts the bytes in each row of the AES state to the left by different offsets. For AES with a 4×4 state, the first row is not shifted, while the second, third, and fourth rows are shifted to the left by 1, 2, and 3 byte positions, respectively.

More precisely, for a state

$$\boldsymbol{s} = \begin{pmatrix} \boldsymbol{s}_{0,0} & \boldsymbol{s}_{0,1} & \boldsymbol{s}_{0,2} & \boldsymbol{s}_{0,3} \\ \boldsymbol{s}_{1,0} & \boldsymbol{s}_{1,1} & \boldsymbol{s}_{1,2} & \boldsymbol{s}_{1,3} \\ \boldsymbol{s}_{2,0} & \boldsymbol{s}_{2,1} & \boldsymbol{s}_{2,2} & \boldsymbol{s}_{2,3} \\ \boldsymbol{s}_{3,0} & \boldsymbol{s}_{3,1} & \boldsymbol{s}_{3,2} & \boldsymbol{s}_{3,3} \end{pmatrix},$$

the output of ShiftRows is

$$\mathsf{ShiftRows}(\boldsymbol{s}) = \begin{pmatrix} \boldsymbol{s}_{0,0} & \boldsymbol{s}_{0,1} & \boldsymbol{s}_{0,2} & \boldsymbol{s}_{0,3} \\ \boldsymbol{s}_{1,1} & \boldsymbol{s}_{1,2} & \boldsymbol{s}_{1,3} & \boldsymbol{s}_{1,0} \\ \boldsymbol{s}_{2,2} & \boldsymbol{s}_{2,3} & \boldsymbol{s}_{2,0} & \boldsymbol{s}_{2,1} \\ \boldsymbol{s}_{3,3} & \boldsymbol{s}_{3,0} & \boldsymbol{s}_{3,1} & \boldsymbol{s}_{3,2} \end{pmatrix}.$$

MixColumns. The MixColumns transformation is linear over $\mathbb{F}_{2^8}$ and acts independently on each column of the AES state. More precisely, each column of the state is viewed as a vector of four bytes over $\mathrm{GF}(2^8)$, and is multiplied by a fixed 4×4 matrix over $\mathrm{GF}(2^8)$.

Let $\boldsymbol{s}' = \mathsf{MixColumns}(\boldsymbol{s})$. Then, for each column index $0 \le c < 4$, it holds that

$$\begin{pmatrix} \boldsymbol{s}'_{0,c} \\ \boldsymbol{s}'_{1,c} \\ \boldsymbol{s}'_{2,c} \\ \boldsymbol{s}'_{3,c} \end{pmatrix} = \begin{pmatrix} \mathtt{02} & \mathtt{03} & \mathtt{01} & \mathtt{01} \\ \mathtt{01} & \mathtt{02} & \mathtt{03} & \mathtt{01} \\ \mathtt{01} & \mathtt{01} & \mathtt{02} & \mathtt{03} \\ \mathtt{03} & \mathtt{01} & \mathtt{01} & \mathtt{02} \end{pmatrix} \begin{pmatrix} \boldsymbol{s}_{0,c} \\ \boldsymbol{s}_{1,c} \\ \boldsymbol{s}_{2,c} \\ \boldsymbol{s}_{3,c} \end{pmatrix}.$$

Here, the bytes of the state are interpreted as elements of $\mathrm{GF}(2^8)$. The constants $\{\mathtt{01}\}$, $\{\mathtt{02}\}$, and $\mathtt{03}$ denote field elements in $\mathrm{GF}(2^8)$, where $\{\mathtt{01}\} = 1$, $\{\mathtt{02}\} = \alpha$, and $\mathtt{03} = \alpha + 1$, with α being the generator of $\mathrm{GF}(2^8)$ such that $m(\alpha) = 0$ in Equation (25).

We also need the inverse operation InvMixColumns. Let $\boldsymbol{s} = \mathsf{InvMixColumns}(\boldsymbol{s}')$. Then, for each column index $0 \le c < 4$, it holds that

$$\begin{pmatrix} \boldsymbol{s}_{0,c} \\ \boldsymbol{s}_{1,c} \\ \boldsymbol{s}_{2,c} \\ \boldsymbol{s}_{3,c} \end{pmatrix} = \begin{pmatrix} \mathtt{0e} & \mathtt{0b} & \mathtt{0d} & \mathtt{09} \\ \mathtt{09} & \mathtt{0e} & \mathtt{0b} & \mathtt{0d} \\ \mathtt{0d} & \mathtt{09} & \mathtt{0e} & \mathtt{0b} \\ \mathtt{0b} & \mathtt{0d} & \mathtt{09} & \mathtt{0e} \end{pmatrix} \begin{pmatrix} \boldsymbol{s}'_{0,c} \\ \boldsymbol{s}'_{1,c} \\ \boldsymbol{s}'_{2,c} \\ \boldsymbol{s}'_{3,c} \end{pmatrix}.$$

Here, the bytes of the state are interpreted as elements of $\mathrm{GF}(2^8)$. The constants $\mathtt{09}$, $\mathtt{0b}$, $\mathtt{0d}$, and $\mathtt{0e}$ denote field elements in $\mathrm{GF}(2^8)$, where

$$\mathtt{09} = \alpha^3 + 1, \qquad \mathtt{0b} = \alpha^3 + \alpha + 1, \qquad \{0d\} = \alpha^3 + \alpha^2 + 1, \qquad \mathtt{0e} = \alpha^3 + \alpha^2 + \alpha.$$

AESRound and AESFinalRound. Given $\boldsymbol{x}, \boldsymbol{k} \in (\mathbb{F}_2^8)^{16}$, define $\mathsf{AESRound}(\boldsymbol{x}, \boldsymbol{k})$ as

$$\mathsf{AESRound}(\boldsymbol{x}, \boldsymbol{k}) = \mathsf{MixColumns}(\mathsf{ShiftRows}(\mathsf{SubBytes}(\boldsymbol{x}))) \oplus \boldsymbol{k}.$$

Similarly, define the final AES round $\mathsf{AESFinalRound}$ omitting the $\mathsf{MixColumns}$ layer, *i.e.*,

$$\mathsf{AESFinalRound}(\boldsymbol{x}, \boldsymbol{k}) = \mathsf{ShiftRows}(\mathsf{SubBytes}(\boldsymbol{x})) \oplus \boldsymbol{k}.$$

We next introduce the building blocks of Vistrutah.

RotByte. $\mathsf{Vistrutah.RotByte}_s(\cdot)$ denotes the cyclic rotation of a 128-bit branch by s bytes.

$\mathsf{Vistrutah.RotByte}_s(\boldsymbol{x})$

INPUT: A 128-bit string $\boldsymbol{x} = (\boldsymbol{x}[0], \ldots, \boldsymbol{x}[15])$ represented as 16 bytes, and a shift value $s \in \{0, \ldots, 15\}$
OUTPUT: A 128-bit string $\boldsymbol{y} = (\boldsymbol{y}[0], \ldots, \boldsymbol{y}[15])$

for $i = 0$ to 15 **do**
    $\boldsymbol{y}[i] \leftarrow \boldsymbol{x}[(i + s) \bmod 16]$
end for
return $\boldsymbol{y}$

Mix. $\mathsf{Vistrutah.Mix}$ denotes the byte-wise inter-branch permutation over the 512-bit state.

$\mathsf{Vistrutah.Mix}(\boldsymbol{s}_0, \boldsymbol{s}_1, \boldsymbol{s}_2, \boldsymbol{s}_3)$

INPUT: Four 128-bit branches $\boldsymbol{s}_0, \boldsymbol{s}_1, \boldsymbol{s}_2, \boldsymbol{s}_3$, each viewed as 16 bytes
OUTPUT: Four 128-bit branches $(\boldsymbol{s}'_0, \boldsymbol{s}'_1, \boldsymbol{s}'_2, \boldsymbol{s}'_3)$

Parse $\boldsymbol{s}_b = (\boldsymbol{s}_b[0], \ldots, \boldsymbol{s}_b[15]) \in (\mathbb{F}_2^8)^{16}$ for each $b \in [0, 3]$
Let $\pi_{\mathsf{mix}} = (0, 1, 2, 3, 8, 9, 10, 11, 4, 5, 6, 7, 12, 13, 14, 15)$.
new $\boldsymbol{t}[0\,..\,63] \in (\mathbb{F}_2^8)^{64}$
for $j = 0$ to 15 **do**
    $u \leftarrow \pi_{\mathsf{mix}}[j]$
    $\boldsymbol{t}[4j + 0] \leftarrow s_0[u]$
    $\boldsymbol{t}[4j + 1] \leftarrow s_1[u]$
    $\boldsymbol{t}[4j + 2] \leftarrow s_2[u]$
    $\boldsymbol{t}[4j + 3] \leftarrow s_3[u]$
end for
Parse $\boldsymbol{s}'_0 \| \boldsymbol{s}'_1 \| \boldsymbol{s}'_2 \| \boldsymbol{s}'_3 \leftarrow \boldsymbol{t}[0] \| \boldsymbol{t}[1] \| \cdots \| \boldsymbol{t}[63]$.
return $(\boldsymbol{s}'_0, \boldsymbol{s}'_1, \boldsymbol{s}'_2, \boldsymbol{s}'_3)$

Shuffle. The $\mathsf{Vistrutah.Shuffle}$ transformation is a byte permutation on a 256-bit input. It takes a 256-bit string represented as 32 bytes and rearranges these bytes according to a fixed permutation.

4.4.1 The Encryption Routine of Vistrutah

The key schedule routine of Vistrutah does not involve any S-boxes, so we describe the key schedule and encryption routines in one function named as $\mathsf{Vistrutah.Encryption}$.

$\mathsf{Vistrutah.Encryption}$ operates on a 512-bit input $\boldsymbol{in}$, which is parsed into 4 128-bit branches $\boldsymbol{in}_0, \boldsymbol{in}_1, \boldsymbol{in}_2, \boldsymbol{in}_3$. The 512-bit initial key $\boldsymbol{k}$ is parsed into 4 128-bit branches $\boldsymbol{k}_0, \boldsymbol{k}_1, \boldsymbol{k}_2, \boldsymbol{k}_3$.

First, the fixed branch keys are defined by $FK_0 = \boldsymbol{k}_0$ and $FK_1 = \boldsymbol{k}_1$, while $FK_2 \,\|\, FK_3 = \mathsf{Vistrutah.Shuffle}(\boldsymbol{k}_2 \,\|\, \boldsymbol{k}_3)$. The initial round-key state is then set as $RK_0 = FK_1$, $RK_1 = FK_0$, $RK_2 = FK_3$, and $RK_3 = FK_2$. Encryption starts with a whitening step, where each input branch is xored with the corresponding round-key branch. The resulting state branches are then independently processed by one application of $\mathsf{AESRound}$ using the fixed keys FK_0, FK_1, FK_2, FK_3.

Vistrutah.Shuffle($\boldsymbol{x}$)

INPUT: A 256-bit string $\boldsymbol{x} = (\boldsymbol{x}[0], \ldots, \boldsymbol{x}[31])$, each is a byte
OUTPUT: A 256-bit string $\boldsymbol{y} = (\boldsymbol{y}[0], \ldots, \boldsymbol{y}[31])$, each is a byte

1: Let
$$\pi_{\mathsf{Shuffle}} = (30, 29, 8, 23, 10, 9, 20, 3, 22, 21, 0, 31, 2, 1, 28, 11, 14, 13, 24, 7, 26, 25, 4, 19, 6, 5, 16, 15, 18, 17, 12, 27).$$
2: **for** $i = 0$ to 31 **do**
3: $\quad \boldsymbol{y}[i] \leftarrow \boldsymbol{x}[\pi_{\mathsf{Shuffle}}[i]]$
4: **end for**
5: **return** $\boldsymbol{y}$

In each intermediate step, the four state branches first undergo one zero-key AES-like round, then the inter-branch mixing layer Vistrutah.Mix, and then a round-key update. More precisely, RK_0 and RK_2 are rotated by 5 bytes, whereas RK_1 and RK_3 are rotated by 10 bytes. The updated round keys are xored into the four state branches, and in addition the round constant RC_{i-1} is injected into the first branch only. The round constants $RC_0, RC_1, \ldots, RC_7 \in \{0,1\}^{128}$ are defined as

$$\begin{aligned}
RC_0 &= \texttt{243F6A8885A308D313198A2E03707344},\\
RC_1 &= \texttt{A4093822299F31D0082EFA98EC4E6C89},\\
RC_2 &= \texttt{452821E638D01377BE5466CF34E90C6C},\\
RC_3 &= \texttt{C0AC29B7C97C50DD3F84D5B5B5470917},\\
RC_4 &= \texttt{9216D5D98979FB1BD1310BA698DFB5AC},\\
RC_5 &= \texttt{2FFD72DBD01ADFB7B8E1AFED6A267E96},\\
RC_6 &= \texttt{BA7C9045F12C7F9924A19947B3916CF7},\\
RC_7 &= \texttt{0801F2E2858EFC16636920D871574E69}.
\end{aligned}$$

After that, the four branches are again processed independently by AESRound with the fixed keys FK_0, FK_1, FK_2, FK_3.

After the last intermediate step, the round keys are rotated once more in the same manner, and the cipher terminates with a final AES-like layer AESFinalRound applied independently to the four 128-bit branches using RK_0, RK_1, RK_2, RK_3, respectively. The ciphertext is finally obtained as $\boldsymbol{out} = \boldsymbol{s}_0 \,\|\, \boldsymbol{s}_1 \,\|\, \boldsymbol{s}_2 \,\|\, \boldsymbol{s}_3$.

4.5 Pseudo-Random Generators

Our signature schemes use a block cipher E in CTR mode with a λ_{iv}-bit IV and a 32-bit tweak. The cipher E is SM4 for SM4th-lo, Ballet-256/256 for SM4th-ti and uBlockith, and Vistrutah-512 for Vistrutith. Its key and block lengths are N_{block} bits. The PRG encrypts $\lceil m/N_{\mathsf{block}} \rceil$ counter blocks and returns the first m bits of their concatenation.

The function BitDec(i, d) returns the d least-significant bits of i in increasing bit order, and NumRec is its inverse. Write the IV as $s = \lambda_{\mathsf{iv}}/32$ consecutive 32-bit strings $(\mathsf{iv}_0, \ldots, \mathsf{iv}_{s-1})$. The functions AddToUpperWord and AddToLowerWord add modulo 2^{32} to the last and first word, respectively, using little-endian byte order.

AddToUpperWord$((\mathsf{iv}_0, \ldots, \mathsf{iv}_{s-1}), \alpha)$

INPUT: $\mathsf{iv}_j \in \{0,1\}^{32}$, $s = \lambda_{\mathsf{iv}}/32$, $\alpha \in [0, 2^{32})$
OUTPUT: $\mathsf{iv}' \in \{0,1\}^{\lambda_{\mathsf{iv}}}$

1: $v \leftarrow (\mathsf{NumRec}(32, \mathsf{iv}_{s-1}) + \alpha) \bmod 2^{32}$
2: **return** $(\mathsf{iv}_0, \ldots, \mathsf{iv}_{s-2}, \mathsf{BitDec}(v, 32))$

Vistrutah.Encryption($\boldsymbol{in}, \boldsymbol{k}$)

INPUT: $\boldsymbol{in} \in \{0,1\}^{512}$, $\boldsymbol{k} \in \{0,1\}^{512}$
OUTPUT: $\boldsymbol{out} \in \{0,1\}^{512}$

new $\boldsymbol{in}_0, \boldsymbol{in}_1, \boldsymbol{in}_2, \boldsymbol{in}_3 \in \{0,1\}^{128} \leftarrow \boldsymbol{in}$
new $\boldsymbol{k}_0, \boldsymbol{k}_1, \boldsymbol{k}_2, \boldsymbol{k}_3 \in \{0,1\}^{128}$
new $FK_0, FK_1, FK_2, FK_3 \in \{0,1\}^{128}$
new $RK_0, RK_1, RK_2, RK_3 \in \{0,1\}^{128}$
new $\boldsymbol{s}_0, \boldsymbol{s}_1, \boldsymbol{s}_2, \boldsymbol{s}_3 \in \{0,1\}^{128}$
$(\boldsymbol{k}_0, \boldsymbol{k}_1, \boldsymbol{k}_2, \boldsymbol{k}_3) \leftarrow \boldsymbol{k}$
$FK_0 \leftarrow \boldsymbol{k}_0, \quad FK_1 \leftarrow \boldsymbol{k}_1$
$(FK_2 \,\|\, FK_3) \leftarrow$ Vistrutah.Shuffle$(\boldsymbol{k}_2 \,\|\, \boldsymbol{k}_3)$
$RK_0 \leftarrow FK_1, \quad RK_1 \leftarrow FK_0, \quad RK_2 \leftarrow FK_3, \quad RK_3 \leftarrow FK_2$
$\boldsymbol{s}_0 \leftarrow \boldsymbol{in}_0 \oplus RK_0$
$\boldsymbol{s}_1 \leftarrow \boldsymbol{in}_1 \oplus RK_1$
$\boldsymbol{s}_2 \leftarrow \boldsymbol{in}_2 \oplus RK_2$
$\boldsymbol{s}_3 \leftarrow \boldsymbol{in}_3 \oplus RK_3$
$\boldsymbol{s}_0 \leftarrow$ AESRound$(\boldsymbol{s}_0, FK_0)$
$\boldsymbol{s}_1 \leftarrow$ AESRound$(\boldsymbol{s}_1, FK_1)$
$\boldsymbol{s}_2 \leftarrow$ AESRound$(\boldsymbol{s}_2, FK_2)$
$\boldsymbol{s}_3 \leftarrow$ AESRound$(\boldsymbol{s}_3, FK_3)$
for $i \in [1 \,..\, R/2)$ **do**
    $\boldsymbol{s}_0 \leftarrow$ AESRound$(\boldsymbol{s}_0, \mathbf{0}^{128})$
    $\boldsymbol{s}_1 \leftarrow$ AESRound$(\boldsymbol{s}_1, \mathbf{0}^{128})$
    $\boldsymbol{s}_2 \leftarrow$ AESRound$(\boldsymbol{s}_2, \mathbf{0}^{128})$
    $\boldsymbol{s}_3 \leftarrow$ AESRound$(\boldsymbol{s}_3, \mathbf{0}^{128})$
    $(\boldsymbol{s}_0, \boldsymbol{s}_1, \boldsymbol{s}_2, \boldsymbol{s}_3) \leftarrow$ Vistrutah.Mix$(\boldsymbol{s}_0, \boldsymbol{s}_1, \boldsymbol{s}_2, \boldsymbol{s}_3)$
    $RK_0 \leftarrow$ Vistrutah.RotByte$_5(RK_0)$
    $RK_1 \leftarrow$ Vistrutah.RotByte$_{10}(RK_1)$
    $RK_2 \leftarrow$ Vistrutah.RotByte$_5(RK_2)$
    $RK_3 \leftarrow$ Vistrutah.RotByte$_{10}(RK_3)$
    $\boldsymbol{s}_0 \leftarrow \boldsymbol{s}_0 \oplus RK_0 \oplus RC_{i-1}$
    $\boldsymbol{s}_1 \leftarrow \boldsymbol{s}_1 \oplus RK_1$
    $\boldsymbol{s}_2 \leftarrow \boldsymbol{s}_2 \oplus RK_2$
    $\boldsymbol{s}_3 \leftarrow \boldsymbol{s}_3 \oplus RK_3$
    $\boldsymbol{s}_0 \leftarrow$ AESRound$(\boldsymbol{s}_0, FK_0)$
    $\boldsymbol{s}_1 \leftarrow$ AESRound$(\boldsymbol{s}_1, FK_1)$
    $\boldsymbol{s}_2 \leftarrow$ AESRound$(\boldsymbol{s}_2, FK_2)$
    $\boldsymbol{s}_3 \leftarrow$ AESRound$(\boldsymbol{s}_3, FK_3)$
end for
$RK_0 \leftarrow$ Vistrutah.RotByte$_5(RK_0)$
$RK_1 \leftarrow$ Vistrutah.RotByte$_{10}(RK_1)$
$RK_2 \leftarrow$ Vistrutah.RotByte$_5(RK_2)$
$RK_3 \leftarrow$ Vistrutah.RotByte$_{10}(RK_3)$
$\boldsymbol{s}_0 \leftarrow$ AESFinalRound$(\boldsymbol{s}_0, RK_0)$
$\boldsymbol{s}_1 \leftarrow$ AESFinalRound$(\boldsymbol{s}_1, RK_1)$
$\boldsymbol{s}_2 \leftarrow$ AESFinalRound$(\boldsymbol{s}_2, RK_2)$
$\boldsymbol{s}_3 \leftarrow$ AESFinalRound$(\boldsymbol{s}_3, RK_3)$
$\boldsymbol{out} \leftarrow \boldsymbol{s}_0 \,\|\, \boldsymbol{s}_1 \,\|\, \boldsymbol{s}_2 \,\|\, \boldsymbol{s}_3$
return $\boldsymbol{out}$

BitDec(i, d)

INPUT: $i \in [0, 2^d)$, $d \in \mathbb{N}$
OUTPUT: $(b_0, \ldots, b_{d-1}) \in \{0,1\}^d$

for $j \in [0, d)$ **do**
    $b_j \leftarrow \lfloor i/2^j \rfloor \bmod 2$
end for
return $(b_0, \ldots, b_{d-1})$

NumRec$(d, (b_0, \ldots, b_{d-1}))$

INPUT: $(b_0, \ldots, b_{d-1}) \in \{0,1\}^d$
OUTPUT: $i \in [0, 2^d)$
1: **return** $\sum_{j=0}^{d-1} b_j 2^j$

AddToLowerWord$((\mathsf{iv}_0, \ldots, \mathsf{iv}_{s-1}), \alpha)$

INPUT: $\mathsf{iv}_j \in \{0,1\}^{32}$, $s = \lambda_{\mathsf{iv}}/32$, $\alpha \in [0, 2^{32})$
OUTPUT: $\mathsf{iv}' \in \{0,1\}^{\lambda_{\mathsf{iv}}}$
1: $v \leftarrow (\mathsf{NumRec}(32, \mathsf{iv}_0) + \alpha) \bmod 2^{32}$
2: **return** $(\mathsf{BitDec}(v, 32), \mathsf{iv}_1, \ldots, \mathsf{iv}_{s-1})$

PRG$(\mathsf{sd}, \mathsf{iv}, \mathsf{twk}; m)$

INPUT: $\mathsf{sd} \in \{0,1\}^{N_{\mathsf{block}}}$, $\mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$, $\mathsf{twk} \in [0, 2^{32})$, $0 \le m \le 2^{32} N_{\mathsf{block}}$
OUTPUT: $\boldsymbol{s} \in \{0,1\}^m$
1: $h \leftarrow \lceil m / N_{\mathsf{block}} \rceil$, $\boldsymbol{s} \leftarrow \epsilon$
2: $\mathsf{iv}' \leftarrow \mathsf{AddToUpperWord}(\mathsf{iv}, \mathsf{twk})$
3: **for** $i \in [0, h)$ **do**
4: $\quad P_i \leftarrow \mathsf{AddToLowerWord}(\mathsf{iv}', i) \| 0^{N_{\mathsf{block}} - \lambda_{\mathsf{iv}}}$
5: $\quad \boldsymbol{s} \leftarrow \boldsymbol{s} \| E_{\mathsf{sd}}(P_i)$
6: **end for**
7: **return** $\boldsymbol{s}[0..m)$

4.6 Linear Universal Hash Functions

The VOLE consistency check and the constraint validity check are achieved via linear universal hash functions. We adapt the two functions from FAEST [22] with ϵ-almost universal and ϵ-uniform properties as defined below.

Definition 1 (ϵ-almost universal and uniform hash [22]). *A family of linear hash functions is a family of matrices $\mathcal{H} \subseteq \mathbb{F}_q^{r \times n}$. The family is ε-almost universal if for any non-zero $\boldsymbol{x} \in \mathbb{F}_q^n$,*

$$\Pr_{\boldsymbol{H} \leftarrow \mathcal{H}} \left[\boldsymbol{H}\boldsymbol{x} = \boldsymbol{0} \right] \le \varepsilon.$$

The family is ε-almost uniform, if for any non-zero $\boldsymbol{x} \in \mathbb{F}_q^n$ and for any $\boldsymbol{v} \in \mathbb{F}_q^r$,

$$\Pr_{\boldsymbol{H} \leftarrow \mathcal{H}} \left[\boldsymbol{H}\boldsymbol{x} = \boldsymbol{v} \right] \le \varepsilon.$$

Our hashes must also satisfy the following hiding property, and Lemma 1 gives a method to transform a uniform hash family into a hiding one.

Definition 2 (Hiding property of linear hash). *A matrix $\boldsymbol{H} \in \mathbb{F}_q^{r \times (n+h)}$ is $\mathbb{F}_q^n$-hiding if the distribution of $\boldsymbol{H}\boldsymbol{v}$ is independent from $\boldsymbol{v}_{[0..n)}$ when $\boldsymbol{v}_{[n..n+h)} \xleftarrow{\$} \mathbb{F}_q^h$. A hash family $\mathcal{H} \subseteq \mathbb{F}_q^{r \times (n+h)}$ is $\mathbb{F}_q^n$-hiding if every $\boldsymbol{H} \in \mathcal{H}$ is $\mathbb{F}_q^n$-hiding.*

Lemma 1 (Transforming a uniform linear hash to universal and hiding [22]). *Let $\mathcal{H} \subseteq \mathbb{F}_q^{r \times n}$ be an ϵ-almost uniform linear hash family. Define*

$$\mathcal{H}' = \{ [\boldsymbol{H}, \boldsymbol{I}_r] : \boldsymbol{H} \in \mathcal{H} \} \subseteq \mathbb{F}_q^{r \times (n+r)}.$$

For input $(\boldsymbol{x}, \boldsymbol{m}) \in \mathbb{F}_q^n \times \mathbb{F}_q^r$, the output is $\boldsymbol{H}\boldsymbol{x} + \boldsymbol{m}$, where $\boldsymbol{m}$ is sampled uniformly and independently of the data $\boldsymbol{x}$ and the selected matrix $\boldsymbol{H}$. Then $\mathcal{H}'$ is ϵ-almost universal and $\mathbb{F}_q^n$-hiding.

Proof. For a difference $(\boldsymbol{d}, \boldsymbol{e})$ with $\boldsymbol{d} \neq 0$, a collision requires $\boldsymbol{H}\boldsymbol{d} = -\boldsymbol{e}$, which has probability at most ϵ by almost uniformity. If $\boldsymbol{d} = 0$ and $\boldsymbol{e} \neq 0$, a collision is impossible. For every fixed $\boldsymbol{H}$ and $\boldsymbol{x}$, addition of the independent uniform $\boldsymbol{m}$ makes the output uniform in $\mathbb{F}_q^r$. □

We next give the concrete constructions of VOLEHash and ZKHash, used in the VOLE consistency check and the constraint validity check, respectively. Before presenting these constructions, we first introduce two auxiliary functions: ToField and ToBits. The ToField algorithm maps a bit string $\boldsymbol{x} \in \{0,1\}^{nk}$, for an integer $n \geq 1$, into a vector of field elements $\boldsymbol{y} \in \mathbb{F}_{2^k}^n$ using little-endian ordering. Conversely, the ToBits algorithm maps a vector of field elements $\boldsymbol{x} \in \mathbb{F}_{2^k}^n$, for an integer $n \geq 1$, into a bit string $\boldsymbol{y} \in \{0,1\}^{nk}$. For $n = 1$, ToField returns a scalar. In ToBits, omitted k and n are inferred from the input field and number of elements, counting a scalar as one element.

ToField$(\boldsymbol{x}, k)$

INPUT: $\boldsymbol{x} \in \{0,1\}^{nk}$
OUTPUT: $y \in \mathbb{F}_{2^k}$ if $n = 1$, or $\boldsymbol{y} \in \mathbb{F}_{2^k}^n$ otherwise

$\alpha_k \leftarrow z \bmod f_k(z)$ // Equation (3)
if $\boldsymbol{x} \in \{0,1\}^k$ **then**
  return $y \leftarrow \sum_{i=0}^{k-1} \boldsymbol{x}[i] \cdot \alpha_k^i$
else if $\boldsymbol{x} \in \{0,1\}^{nk}$ **then**
  new $\boldsymbol{y} \in \mathbb{F}_{2^k}^n$
  for $i \in [0..n)$ **do**
    $\boldsymbol{y}[i] \leftarrow \sum_{j=0}^{k-1} \boldsymbol{x}[ik + j] \cdot \alpha_k^j$
  end for
  return $\boldsymbol{y}$
else
  return $\perp$
end if

ToBits$(x; k, n)$

INPUT: $x \in \mathbb{F}_{2^k}$ or $x \in \mathbb{F}_{2^k}^n$
OUTPUT: $\boldsymbol{x} \in \{0,1\}^{nk}$

if $x \in \mathbb{F}_{2^k}$ **then**
  $x \leftarrow (x)$; $n \leftarrow 1$
end if
$\boldsymbol{x} \leftarrow \epsilon$
for $i \in [0..n)$ **do**
  Parse $x_0 + x_1\alpha_k + \cdots + x_{k-1}\alpha_k^{k-1} = x[i]$ for $x_j \in \mathbb{F}_2$
  $\boldsymbol{x}_i \leftarrow x_0 \| \cdots \| x_{k-1} \in \{0,1\}^k$
  $\boldsymbol{x} \leftarrow \boldsymbol{x} \| \boldsymbol{x}_i$
end for
return $\boldsymbol{x} \in \{0,1\}^{nk}$

VOLEHash. The VOLEHash algorithm verifies the consistency of VOLE correlations by operating on inputs in $\mathbb{F}_2^{\hat{\ell}}$, where $\hat{\ell} = \ell + d\lambda_{\mathbb{F}} + B$. Here, ℓ denotes the witness length, and $B = 16$ is a security parameter adapted from [10].

To construct the hash, we first map the seed sd into $(r_0, r_1, r_2, r_3, s, t) \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^5 \times \mathbb{F}_{2^{64}}$ via ToField. The input $\boldsymbol{x} \in \mathbb{F}_2^{\hat{\ell}}$ is split as $\boldsymbol{x} = \boldsymbol{x}_0 \| \boldsymbol{x}_1$ at bit $\ell + (d-1)\lambda_{\mathbb{F}}$, where $\boldsymbol{x}_0 \in \{0,1\}^{\hat{\ell} - \lambda_{\mathbb{F}} - B}$ is padded to a length that is a multiple of $\lambda_{\mathbb{F}}$, and $\boldsymbol{x}_1 \in \{0,1\}^{\lambda_{\mathbb{F}} + B}$. We then parse $\boldsymbol{x}_0$ via ToField in two ways: first as a vector $\hat{\boldsymbol{y}}$ of $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ elements, and second, after appending zeros to length $\ell_{64} = 64\lceil \ell'/64 \rceil$,

as a vector $\bar{\boldsymbol{y}}$ of $\mathbb{F}_{2^{64}}$ elements. Next, we compute

$$h_0 = \hat{y}_0 s^{\ell'/\lambda_{\mathbb{F}}-1} + \hat{y}_1 s^{\ell'/\lambda_{\mathbb{F}}-2} + \cdots + \hat{y}_{\ell'/\lambda_{\mathbb{F}}-2} s + \hat{y}_{\ell'/\lambda_{\mathbb{F}}-1} \quad \text{in } \mathbb{F}_{2^{\lambda_{\mathbb{F}}}},$$
$$h_1 = \bar{y}_0 t^{\ell_{64}/64-1} + \bar{y}_1 t^{\ell_{64}/64-2} + \cdots + \bar{y}_{\ell_{64}/64-2} t + \bar{y}_{\ell_{64}/64-1} \quad \text{in } \mathbb{F}_{2^{64}},$$

where

$$\ell' = \lambda_{\mathbb{F}} \cdot \left\lceil \frac{\hat{\ell} - \lambda_{\mathbb{F}} - B}{\lambda_{\mathbb{F}}} \right\rceil$$

is the padded length of $\boldsymbol{x}_0$. To combine these two hash values, we view h_1 as an element of $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ by zero-padding:

$$h_1' = \mathsf{ToField}(\mathsf{ToBits}(h_1) \| 0^{\lambda_{\mathbb{F}}-64}, \lambda_{\mathbb{F}}).$$

The hash values are then linearly combined in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ as

$$\begin{bmatrix} h_2 \\ h_3 \end{bmatrix} = \begin{bmatrix} r_0 & r_1 \\ r_2 & r_3 \end{bmatrix} \begin{bmatrix} h_0 \\ h_1' \end{bmatrix}.$$

Finally, we extract the first $\lambda_{\mathbb{F}} + B$ bits from the concatenation of h_2 and h_3, and XOR this with $\boldsymbol{x}_1$ to obtain the output

$$\boldsymbol{h} = (\mathsf{ToBits}(h_2) \| \mathsf{ToBits}(h_3)[0..B)) \oplus \boldsymbol{x}_1.$$

$\mathsf{VOLEHash}(\mathsf{sd}, (\boldsymbol{x}_0, \boldsymbol{x}_1) \in \{0,1\}^{\hat{\ell}-\lambda_{\mathbb{F}}-B} \times \{0,1\}^{\lambda_{\mathbb{F}}+B})$

Input: $\boldsymbol{x}_0 \in \{0,1\}^{\hat{\ell}-\lambda_{\mathbb{F}}-B}$, $\boldsymbol{x}_1 \in \{0,1\}^{\lambda_{\mathbb{F}}+B}$, $\mathsf{sd} \in \{0,1\}^{5\lambda_{\mathbb{F}}+64}$
Output: $\boldsymbol{h} \in \{0,1\}^{\lambda_{\mathbb{F}}+B}$

1: Parse $\mathsf{sd} = (\boldsymbol{r}_0 \,\|\, \boldsymbol{r}_1 \,\|\, \boldsymbol{r}_2 \,\|\, \boldsymbol{r}_3 \,\|\, \boldsymbol{s} \,\|\, \boldsymbol{t}) \in \{0,1\}^{5\lambda_{\mathbb{F}}+64}$ // $\lambda_{\mathbb{F}}$-bit r_i, s; 64-bit t
2: $r_i \leftarrow \mathsf{ToField}(\boldsymbol{r}_i, \lambda_{\mathbb{F}})$, **for** $i \in [0..3]$
3: $s \leftarrow \mathsf{ToField}(\boldsymbol{s}, \lambda_{\mathbb{F}})$
4: $t \leftarrow \mathsf{ToField}(\boldsymbol{t}, 64)$
5: $\ell' \leftarrow \lambda_{\mathbb{F}} \cdot \left\lceil (\hat{\ell} - \lambda_{\mathbb{F}} - B)/\lambda_{\mathbb{F}} \right\rceil$
6: $\boldsymbol{x}_0 \leftarrow \boldsymbol{x}_0 \,\|\, 0^{\ell'-(\hat{\ell}-\lambda_{\mathbb{F}}-B)}$ // pad to multiple of $\lambda_{\mathbb{F}}$
7: $\hat{\boldsymbol{y}} \leftarrow \mathsf{ToField}(\boldsymbol{x}_0, \lambda_{\mathbb{F}})$
8: $\ell_{64} \leftarrow 64\lceil \ell'/64 \rceil$
9: $\bar{\boldsymbol{y}} \leftarrow \mathsf{ToField}(\boldsymbol{x}_0 \| 0^{\ell_{64}-\ell'}, 64)$
10: $h_0 \leftarrow \sum_{i=0}^{\ell'/\lambda_{\mathbb{F}}-1} s^{\ell'/\lambda_{\mathbb{F}}-1-i} \cdot \hat{\boldsymbol{y}}[i]$ (in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$)
11: $h_1 \leftarrow \sum_{i=0}^{\ell_{64}/64-1} t^{\ell_{64}/64-1-i} \cdot \bar{\boldsymbol{y}}[i]$ (in $\mathbb{F}_{2^{64}}$)
12: $h_1' \leftarrow \mathsf{ToField}(\mathsf{ToBits}(h_1) \,\|\, 0^{\lambda_{\mathbb{F}}-64}, \lambda_{\mathbb{F}})$
13: $(h_2, h_3) \leftarrow (r_0 h_0 + r_1 h_1', r_2 h_0 + r_3 h_1')$
14: $\boldsymbol{h} \leftarrow (\mathsf{ToBits}(h_2) \,\|\, \mathsf{ToBits}(h_3)[0..B)) \oplus \boldsymbol{x}_1$
15: **return** $\boldsymbol{h}$

The following bound follows the polynomial-hash analysis of FAEST [22] using the degrees of the two input polynomials.

Lemma 2 (VOLE hash bound). *Let $n_V = \ell'/\lambda_{\mathbb{F}}$ and $n_{64} = \ell_{64}/64$, with the padded lengths defined in Algorithm 18. Then* VOLEHash *is an ϵ_V-almost universal hash family in $\mathbb{F}_2^{(\lambda_{\mathbb{F}}+B)\times\hat{\ell}}$, where*

$$\epsilon_V \le 2^{-\lambda_{\mathbb{F}}-B} + \frac{(n_V - 1)(n_{64} - 1)}{2^{\lambda_{\mathbb{F}}+64}}. \tag{27}$$

In particular, if $(n_V - 1)(n_{64} - 1) \le 2^{14}$, then

$$\epsilon_V \le 2^{-\lambda_{\mathbb{F}}-B}(1 + 2^{B-50}).$$

Furthermore, VOLEHash *is $\mathbb{F}_2^{\hat{\ell}-\lambda_{\mathbb{F}}-B}$-hiding.*

Proof. Zero-padding and either field parsing preserve a non-zero prefix difference. The two resulting polynomials have degrees at most $n_V - 1$ and $n_{64} - 1$. Their evaluation points are independent, so both evaluations vanish with probability at most $(n_V-1)(n_{64}-1)/2^{\lambda_{\mathbb{F}}+64}$. Otherwise, multiplication by the independent random 2×2 matrix gives a uniform pair of field elements, and its first $\lambda_{\mathbb{F}} + B$ bits are uniform. Adding the suffix gives Equation (27) and the hiding property. A difference confined to the suffix is always detected. □

ZKHash. The ZKHash algorithm compresses inputs in $\mathbb{F}^n_{2^{\lambda_{\mathbb{F}}}} \times \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ into a $\lambda_{\mathbb{F}}$-bit output. Given a seed $\mathsf{sd} \in \{0,1\}^{3\lambda_{\mathbb{F}}+64}$, we first parse it as $(r_0, r_1, s, t) \in \mathbb{F}^3_{2^{\lambda_{\mathbb{F}}}} \times \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ via ToField, where r_0, r_1, s are obtained by interpreting $\lambda_{\mathbb{F}}$-bit substrings as elements of $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, and t is derived from a 64-bit substring by zero-padding it to $\lambda_{\mathbb{F}}$ bits and then applying ToField.

The input is given as $(\boldsymbol{x}_0, x_1)$, where $\boldsymbol{x}_0 = (x_{0,0}, \ldots, x_{0,n-1}) \in \mathbb{F}^n_{2^{\lambda_{\mathbb{F}}}}$ and $x_1 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. The vector $\boldsymbol{x}_0$ is hashed twice using two independent algebraic accumulations over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$:

$$h_0 = x_{0,0}s^{n-1} + x_{0,1}s^{n-2} + \cdots + x_{0,n-2}s + x_{0,n-1},$$
$$h_1 = x_{0,0}t^{n-1} + x_{0,1}t^{n-2} + \cdots + x_{0,n-2}t + x_{0,n-1}.$$

That is, h_0 and h_1 are two polynomial evaluations of the same input vector under the field points s and t, respectively. Next, these two hash values are linearly combined using the coefficients r_0, r_1, and the extra field element x_1 is added as an offset to obtain $h = r_0h_0 + r_1h_1 + x_1$. Finally, the output of ZKHash is the $\lambda_{\mathbb{F}}$-bit representation of h, *i.e.*, $\boldsymbol{h} = \mathsf{ToBits}(h)$. We identify h with this fixed polynomial-basis bit encoding, so algebraic uses of ZKHash refer to the field element h and bit-string uses refer to $\mathsf{ToBits}(h)$. At byte-string transcript boundaries, this encoding is serialized as $\mathrm{FE}(h) = \mathrm{OS}(\mathsf{ToBits}(h))$.

$\mathsf{ZKHash}(\mathsf{sd}, (\boldsymbol{x}_0, x_1) \in \mathbb{F}^n_{2^{\lambda_{\mathbb{F}}}} \times \mathbb{F}_{2^{\lambda_{\mathbb{F}}}})$

// $\lambda_{\mathbb{F}}$-bit $\boldsymbol{r}_i, \boldsymbol{s}$; 64-bit $\boldsymbol{t}$
Parse $\mathsf{sd} = (\boldsymbol{r}_0 \| \boldsymbol{r}_1 \| \boldsymbol{s} \| \boldsymbol{t}) \in \{0,1\}^{3\lambda_{\mathbb{F}}+64}$
$r_i \leftarrow \mathsf{ToField}(\boldsymbol{r}_i, \lambda_{\mathbb{F}})$, **for** $i \in \{0,1\}$
$s \leftarrow \mathsf{ToField}(\boldsymbol{s}, \lambda_{\mathbb{F}})$
$t \leftarrow \mathsf{ToField}(\boldsymbol{t} \| 0^{\lambda_{\mathbb{F}}-64}, \lambda_{\mathbb{F}})$
$h_0 \leftarrow \sum_{i=0}^{n-1} s^{n-1-i} \cdot \boldsymbol{x}_0[i]$
$h_1 \leftarrow \sum_{i=0}^{n-1} t^{n-1-i} \cdot \boldsymbol{x}_0[i]$
$\boldsymbol{h} \leftarrow \mathsf{ToBits}(r_0h_0 + r_1 \cdot h_1 + x_1)$
return $\boldsymbol{h}$

Lemma 3 ([22]). ZKHash *is an* ϵ_{zk}*-almost universal hash family in* $\mathbb{F}^{1\times(n+1)}_{2^{\lambda_{\mathbb{F}}}}$ *for*

$$\epsilon_{\mathsf{zk}} = 2^{-\lambda_{\mathbb{F}}}(1 + 2^{-38})$$

if $n + 1 \le 2^{13}$. *Furthermore,* ZKHash *is* $\mathbb{F}^n_{2^{\lambda_{\mathbb{F}}}}$ *hiding.*

5 Algorithm Description: General Building Blocks

In this section, we describe the algorithms required for all signature schemes, including VOLEitH functions in Section 5.1 and operations for ZKPs in Section 5.2.

5.1 VOLE-in-the-Head Functions

5.1.1 Batch All-but-One Vector Commitments

This section describes the construction of BAVC, which is adapted from FAEST [22]. It includes three algorithms: BAVC.Commit, BAVC.Open and BAVC.Reconstruct.

We first give some auxiliary algorithms.

– BAVC.PosInTree(i, j) computes the unique position $\alpha \in [L-1\,..\,2L-1)$ of the j-th position in the i-th GGM tree. Recall that L, as defined in Table 3, is the number of all leaves in GGM trees, and N_i is the number of leaves in the i-th GGM tree.
– BAVC.LeafCommit$(r, \mathsf{iv}, \mathsf{twk})$ is the leaf commitment scheme for GGM trees. It returns the original seed r as leaf seed and uses the $2N_{\mathsf{block}}$-bit PRG output as the commitment.

BAVC.LeafCommit$(r, \mathsf{iv}, \mathsf{twk})$

INPUT: $r \in \{0,1\}^{N_{\mathsf{block}}}, \mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}, \mathsf{twk} \in \{0,1\}^{32}$
OUTPUT: $\mathsf{sd} \in \{0,1\}^{N_{\mathsf{block}}}, \mathsf{com} \in \{0,1\}^{2N_{\mathsf{block}}}$

1: $\mathsf{com} \leftarrow \mathsf{PRG}(r, \mathsf{iv}, \mathsf{twk}; 2N_{\mathsf{block}})$
2: $\mathsf{sd} \leftarrow r$
3: **return** $(\mathsf{sd}, \mathsf{com})$

BAVC.PosInTree(i, j)

INPUT: $i \in [0..\tau),\ j \in [0..N_i)$
OUTPUT: $\alpha \in [L-1..2L-1)$

1: **if** $j < 2^{k-1}$ **then**
2: **return** $L - 1 + \tau j + i$
3: **else**
4: **return** $L - 1 + \tau 2^{k-1} + \tau_1(j \bmod 2^{k-1}) + i$
5: **end if**

We next describe the BAVC algorithms.

BAVC.Commit. The BAVC.Commit algorithm takes as input a root seed $\mathsf{sd}_{\mathsf{tree}} \in \{0,1\}^{N_{\mathsf{block}}}$ and an initialization vector $\mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$, and outputs a commitment com, a decommitment string decom, and the collection of leaf seeds $\{\mathsf{sd}_{i,j}\}_{i\in[0\,..\,\tau),\, j\in[0\,..\,N_i)}$. Here $L = \sum_{i=0}^{\tau-1} N_i$ denotes the total number of leaves across all GGM trees.

It first sets the root node key as $k_0 \leftarrow \mathsf{sd}_{\mathsf{tree}}$. Then, for every internal node $\alpha \in [0\,..\,L-2]$, it expands k_α using a $2N_{\mathsf{block}}$-bit output of PRG to derive the two child keys $(k_{2\alpha+1}, k_{2\alpha+2}) \in \{0,1\}^{N_{\mathsf{block}}} \times \{0,1\}^{N_{\mathsf{block}}}$. In this way, the full binary tree with $2L-1$ nodes is deterministically generated from the root seed.

Next, for each layer $i \in [0\,..\,\tau)$ and each leaf index $j \in [0\,..\,N_i)$, the algorithm computes the corresponding tree position $\alpha \leftarrow \mathsf{BAVC.PosInTree}(i, j)$ and invokes $\mathsf{BAVC.LeafCommit}(k_\alpha, \mathsf{iv}, i+L-1)$ to obtain a leaf seed $\mathsf{sd}_{i,j}$ together with a leaf commitment $\mathsf{com}_{i,j}$. For every fixed i, all leaf commitments in that layer are concatenated and hashed to obtain

$$h_i \leftarrow \mathsf{H}_1(\mathsf{com}_{i,0} \,\|\, \ldots \,\|\, \mathsf{com}_{i,N_i-1}).$$

Finally, the global commitment is computed by hashing all layer hashes:

$$\mathsf{com} \leftarrow \mathsf{H}_1(h_0 \,\|\, \ldots \,\|\, h_{\tau-1}).$$

The decommitment information consists of all node keys and all leaf commitments, namely

$$\mathsf{decom} \leftarrow \left((k_\alpha)_{\alpha\in[0\,..\,2L-1)}, (\mathsf{com}_{i,j})_{i\in[0\,..\,\tau),\, j\in[0\,..\,N_i)}\right).$$

The algorithm outputs $(\mathsf{com}, \mathsf{decom}, (\mathsf{sd}_{i,j})_{i,j})$.

BAVC.Open. The algorithm $\mathsf{BAVC.Open}(\mathsf{decom}, I, \mathsf{param})$ takes as input the decommitment string decom and an index tuple $I = (\Delta_0, \ldots, \Delta_{\tau-1})$, where each $\Delta_i \in [0\,..\,N_i)$ specifies the hidden leaf

BAVC.Commit($\mathsf{sd}_{\mathsf{tree}}, \mathsf{iv}, \mathsf{param}$)

INPUT: $\mathsf{sd}_{\mathsf{tree}} \in \{0,1\}^{N_{\mathsf{block}}}, \mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$
OUTPUT: $\mathsf{com} \in \{0,1\}^{2N_{\mathsf{block}}}, \mathsf{decom}, \{\mathsf{sd}_{i,j} \in \{0,1\}^{N_{\mathsf{block}}}\}_{i\in[0\,..\,\tau), j\in[0\,..\,N_i)}$

$k_0 \leftarrow \mathsf{sd}_{\mathsf{tree}}$
for $\alpha \in [0\,..\,L-2]$ **do**
  $(k_{2\alpha+1}, k_{2\alpha+2}) \leftarrow \mathsf{PRG}(k_\alpha, \mathsf{iv}, \alpha; 2N_{\mathsf{block}})$
end for
for $i \in [0\,..\,\tau)$ **do**
  for $j \in [0\,..\,N_i)$ **do**
    $\alpha \leftarrow \mathsf{BAVC.PosInTree}(i, j)$
    $(\mathsf{sd}_{i,j}, \mathsf{com}_{i,j}) \leftarrow \mathsf{BAVC.LeafCommit}(k_\alpha, \mathsf{iv}, i + L - 1)$
  end for
  $h_i \leftarrow \mathsf{H}_1(\mathsf{com}_{i,0} \,\|\, \ldots \,\|\, \mathsf{com}_{i,N_i-1})$
end for
$\mathsf{com} \leftarrow \mathsf{H}_1(h_0 \,\|\, \ldots \,\|\, h_{\tau-1})$
$\mathsf{decom} \leftarrow \big((k_\alpha)_{\alpha\in[0\,..\,2L-1)}, (\mathsf{com}_{i,j})_{i\in[0\,..\,\tau),j\in[0\,..\,N_i)}\big)$
return $(\mathsf{com}, \mathsf{decom}, (\mathsf{sd}_{i,0}, \ldots, \mathsf{sd}_{i,N_i-1})_{i\in[0\,..\,\tau)})$

position in the i-th GGM tree. It outputs an opening string decom_I or $\perp$ if the requested opening exceeds the allowed size bound.

It first parses decom as $\mathsf{decom} = \big((k_\alpha)_{\alpha\in[0\,..\,2L-1)}, (\mathsf{com}_{i,j})_{i\in[0\,..\,\tau),\, j\in[0\,..\,N_i)}\big)$. The opening string is initialized with the τ selected leaf commitments, namely $\mathsf{decom}_I \leftarrow (\mathsf{com}_{0,\Delta_0}, \ldots, \mathsf{com}_{\tau-1,\Delta_{\tau-1}})$.

Next, the algorithm determines which tree nodes must remain hidden. To this end, it initializes a bit vector $S \in \{0,1\}^{2L-1}$ to zero and a counter $n_h \leftarrow 0$. Then, for each layer $i \in [0\,..\,\tau)$, it computes the corresponding leaf position $\alpha \leftarrow \mathsf{BAVC.PosInTree}(i, \Delta_i)$, marks this node in S, and increments n_h. After that, it repeatedly moves upward along the path to the root, marking every previously unmarked ancestor. In this way, S records exactly those nodes that lie on the union of the root-to-hidden-leaf paths, and n_h counts the number of such marked nodes.

The algorithm then checks whether the resulting opening is admissible. If $n_h - 2\tau + 1 > T_{\mathsf{open}}$, it returns $\perp$. Otherwise, it scans the internal nodes in reverse order, from $L-2$ down to 0. For each internal node i, it tests whether exactly one of its two children is marked, i.e., whether $S[2i+1] \oplus S[2i+2] = 1$. If so, the sibling key that is not on the marked path is appended to decom_I. Concretely, it sets $\alpha \leftarrow 2i + 1 + S[2i+1]$ and appends k_α to the opening.

Finally, the algorithm zero-pads decom_I to the fixed length $2N_{\mathsf{block}}\tau + T_{\mathsf{open}}N_{\mathsf{block}}$ bits and returns the resulting string. Thus, the output contains the selected leaf commitments followed by the ordered auxiliary tree seeds needed to reconstruct the revealed sibling subtrees, with any remaining space zero-padded.

BAVC.Reconstruct. The BAVC.Reconstruct algorithm describes the reconstruction procedure for the BAVC commitment from an opening decom_I corresponding to the queried index set $I = (\Delta_0, \ldots, \Delta_{\tau-1})$. The algorithm first parses the opening decom_I into the selected leaf commitments $(\mathsf{com}_{0,\Delta_0}, \ldots, \mathsf{com}_{\tau-1,\Delta_{\tau-1}})$ and the auxiliary seed sequence nodes, which lists the revealed sibling-subtree seeds in opening order, followed by zero padding. It then initializes a binary marker array $S \in \{0,1\}^{2L-1}$ to zero, where $S[\alpha] = 1$ marks a node whose seed remains hidden. For each selected hidden leaf, the corresponding tree position is marked, and the algorithm traverses the tree upwards to mark its ancestors. Whenever exactly one child of an internal node is marked, the algorithm consumes the next seed in nodes for the unmarked child. It returns $\perp$ if a required seed is missing or the unused suffix contains a non-zero bit.

After this consistency check, the algorithm expands the revealed sibling subtrees using the $2N_{\mathsf{block}}$-bit output PRG. More precisely, for every internal node i with $S[i] = 0$ and a known seed k_i, its two children are derived deterministically from that seed. Then, for each vector $i \in [0\,..\,\tau)$ and each position $j \in [0\,..\,N_i)$, the corresponding leaf commitment is obtained either directly from the opening (when $j = \Delta_i$) or by recomputing it from the reconstructed seed via BAVC.LeafCommit. The resulting leaf hashes are aggregated vector by vector as $h_i \leftarrow H_1(h_{i,0} \,\|\, \cdots \,\|\, h_{i,N_i-1})$, and finally combined into the global commitment $\mathsf{com} \leftarrow H_1(h_0 \,\|\, \cdots \,\|\, h_{\tau-1})$. The algorithm outputs

BAVC.Open($\mathsf{decom}, I = (\Delta_0, \ldots, \Delta_{\tau-1}), \mathsf{param}$)

INPUT: $\mathsf{decom} \in \{0,1\}^{(2L-1)N_{\mathsf{block}}+2N_{\mathsf{block}}L}, I \in [0\,..\,N_0) \times \cdots \times [0\,..\,N_{\tau-1})$
OUTPUT: $\mathsf{decom}_I \in \{0,1\}^{2N_{\mathsf{block}}\tau+T_{\mathsf{open}}N_{\mathsf{block}}} \cup \{\bot\}$

Parse $\mathsf{decom} \leftarrow \big((k_\alpha)_{\alpha\in[0\,..\,2L-1)}, (\mathsf{com}_{i,j})_{i\in[0\,..\,\tau),j\in[0\,..\,N_i)}\big)$ // Initialize opening with τ leaf commitments
$\mathsf{decom}_I \leftarrow (\mathsf{com}_{0,\Delta_0}, \ldots, \mathsf{com}_{\tau-1,\Delta_{\tau-1}})$ // Mark tree nodes that will not be revealed
$S \leftarrow 0^{2L-1} \in \{0,1\}^{2L-1}$
$n_h \leftarrow 0$ // number of marked hidden nodes
// Mark each hidden leaf and its ancestors, counting marked nodes in n_h
for $i \in [0\,..\,\tau)$ **do**
    $\alpha \leftarrow$ BAVC.PosInTree(i, Δ_i)
    $S[\alpha] \leftarrow 1$
    $n_h \leftarrow n_h + 1$
    while $\alpha > 0$ and $S[\lfloor(\alpha-1)/2\rfloor] = 0$ **do**
        $\alpha \leftarrow \lfloor(\alpha-1)/2\rfloor$
        $S[\alpha] \leftarrow 1$
        $n_h \leftarrow n_h + 1$
    end while
end for
if $n_h - 2\tau + 1 > T_{\mathsf{open}}$ **then**
    return $\bot$
end if
// Add tree nodes to opening information
for $i = L-2$ to 0 **do**
    // If exactly one child is marked, reveal the unmarked sibling seed
    if $S[2i+1] \oplus S[2i+2] = 1$ **then**
        $\alpha \leftarrow 2i + 1 + S[2i+1]$
        Append k_α to decom_I
    end if
end for
Zero-append decom_I to length $2N_{\mathsf{block}}\tau + T_{\mathsf{open}}N_{\mathsf{block}}$ bits
return decom_I

this reconstructed commitment together with all non-hidden leaf seeds. Hence, BAVC.Reconstruct deterministically recovers the full commitment view implied by the partial opening, while rejecting malformed openings that do not match a valid tree structure.

BAVC.Reconstruct($\mathsf{decom}_I, I = (\Delta_0, \ldots, \Delta_{\tau-1}), \mathsf{iv}; \mathsf{param}$)

INPUT: $\mathsf{decom}_I \in \{0,1\}^{2N_{\mathsf{block}}\tau + T_{\mathsf{open}}N_{\mathsf{block}}}$, $I \in [0\,..\,N_0) \times \cdots \times [0\,..\,N_{\tau-1})$, $\mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$
OUTPUT: $\mathsf{com} \in \{0,1\}^{2N_{\mathsf{block}}}$ and $L - \tau$ seeds in $\{0,1\}^{N_{\mathsf{block}}}$, or $\perp$

Parse $\mathsf{decom}_I = (\mathsf{com}_{0,\Delta_0}, \ldots, \mathsf{com}_{\tau-1,\Delta_{\tau-1}}, \mathsf{nodes})$
// Mark tree nodes that are not revealed
$S \leftarrow 0^{2L-1} \in \{0,1\}^{2L-1}$
for $i \in [0\,..\,\tau)$ **do**
    $\alpha \leftarrow$ BAVC.PosInTree(i, Δ_i)
    $S[\alpha] \leftarrow 1$
end for
// Walk up the tree, copying nodes from decom_I
for $i = L - 2$ to 0 **do**
    $S[i] \leftarrow S[2i+1] \vee S[2i+2]$
    // If exactly one child is marked, take the unmarked sibling seed from the opening
    if $S[2i+1] \oplus S[2i+2] = 1$ **then**
        if nodes is empty **then**
            return $\perp$
        end if
        $\alpha \leftarrow 2i + 1 + S[2i+1]$
        Parse $\mathsf{nodes} = (k_\alpha, \mathsf{nodes}')$
        $\mathsf{nodes} \leftarrow \mathsf{nodes}'$
    end if
end for
if nodes contains a non-zero bit **then**
    return $\perp$
end if
// Expand the tree
for $i \in [0\,..\,L-1)$ **do**
    if $S[i] = 0$ **then**
        $(k_{2i+1}, k_{2i+2}) \leftarrow$ PRG$(k_i, \mathsf{iv}, i; 2N_{\mathsf{block}})$
    end if
end for
for $i \in [0\,..\,\tau)$ **do**
    for $j \in [0\,..\,N_i)$ **do**
        $\alpha \leftarrow$ BAVC.PosInTree(i, j)
        if $S[\alpha] = 1$ **then**
            $h_{i,j} \leftarrow \mathsf{com}_{i,j}$
        else
            $(\mathsf{sd}_{i,j}, h_{i,j}) \leftarrow$ BAVC.LeafCommit$(k_\alpha, \mathsf{iv}, i + L - 1)$
        end if
    end for
    $h_i \leftarrow H_1(h_{i,0} \,\|\, \ldots \,\|\, h_{i,N_i-1})$
end for
$\mathsf{com} \leftarrow H_1(h_0 \,\|\, \ldots \,\|\, h_{\tau-1})$
return $(\mathsf{com}, ((\mathsf{sd}_{i,j})_{j \in [0\,..\,N_i) \setminus \{\Delta_i\}})_{i \in [0\,..\,\tau)})$

5.1.2 Seed Expansions and Conversion to VOLE

After running the BAVC, the j-th leaf in the i-th GGM tree is associated with a seed $\mathsf{sd}_{i,j} \in \{0,1\}^N_{\mathsf{block}}$ and a commitment $\mathsf{com}_{i,j}$. All these seeds are then expanded into a VOLE correlation

via ConvertToVOLE. Without loss of generality, we focus on a single tree with $N = 2^D$ leaves, where $D \in \{k, k-1\}$, and we neglect the description about the tree index.

Fix a binary basis $(e_0, \ldots, e_{D-1})$ of $\mathbb{F}_{2^D}$. For $i \in [0, N)$, define $\iota_D(i) = \sum_{j=0}^{D-1} \mathsf{BitDec}(i, D)_j e_j$; then $\iota_D(i \oplus h) = \iota_D(i) + \iota_D(h)$ for $i, h \in [0, N)$. Let $\mathsf{PRG}(\mathsf{sd}_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell}) \in \{0,1\}^{\hat{\ell}}$ denote the $\hat{\ell}$-bit PRG expansion of seed $\mathsf{sd}_i \in \{0,1\}^N_{\mathsf{block}}$. A VOLE correlation over $\mathbb{F}_{2^D}$ (viewing $\mathbb{F}_2 \subseteq \mathbb{F}_{2^D}$ in the natural way) targets

$$\boldsymbol{u} \;=\; \bigoplus_{i=0}^{N-1} \mathsf{PRG}(\mathsf{sd}_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell}) \in \mathbb{F}_2^{\hat{\ell}}, \qquad \boldsymbol{v} \;=\; \sum_{i=0}^{N-1} \iota_D(i) \cdot \mathsf{PRG}(\mathsf{sd}_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell}) \in \mathbb{F}_{2^D}^{\hat{\ell}}, \tag{28}$$

where the multiplication and the sum defining $\boldsymbol{v}$ are in $\mathbb{F}_{2^D}$, and each $\hat{\ell}$-bit block $\mathsf{PRG}(\mathsf{sd}_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell}) \in \mathbb{F}_2^{\hat{\ell}}$ is embedded coordinate-wise into $\mathbb{F}_{2^D}^{\hat{\ell}}$.

The algorithm ConvertToVOLE outputs $\boldsymbol{u} \in \{0,1\}^{\hat{\ell}}$ together with D vectors $\boldsymbol{v}_0, \ldots, \boldsymbol{v}_{D-1} \in \{0,1\}^{\hat{\ell}}$ such that

$$\boldsymbol{v} \;=\; \sum_{j=0}^{D-1} e_j \cdot \boldsymbol{v}_j,$$

Intuitively, $\boldsymbol{v}_j$ is the XOR of exactly those PRG outputs whose leaf index has j-th binary bit equal to 1.

Prover and verifier views. The procedure ConvertToVOLE is executed by both prover and verifier. The prover knows all seeds, hence $\mathsf{sd}_0 \neq \perp$. On the verifier side, given an index $\Delta \in [0\,..\,N)$ and all seeds sd_i for $i \neq \Delta$, the verifier aims to compute

$$\boldsymbol{q} \;=\; \sum_{\substack{i=0 \\ i \neq \Delta}}^{N-1} \iota_D(\Delta \oplus i) \cdot \mathsf{PRG}(\mathsf{sd}_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell}) \in \mathbb{F}_{2^D}^{\hat{\ell}}. \tag{29}$$

To align the verifier input with the all-but-one setting of ConvertToVOLE, the verifier first applies the permutation

$$\pi_\Delta(i) \;\leftarrow\; \mathsf{BitCom}\big(\mathsf{BitDec}(i, D) \oplus \mathsf{BitDec}(\Delta, D)\big),$$

where $\mathsf{BitCom} : \{0,1\}^D \to [0, N)$ is the integer bit composition function $\mathsf{BitCom}(b_0, \ldots, b_{D-1}) = \sum_{j \in [0, D-1]} 2^j \cdot b_j$. Hence, the permutation π_Δ is exactly the XOR map $i \mapsto i \oplus \Delta$ on D-bit indices. This mapping sends Δ to 0, so the missing seed is moved to position 0 in the permuted order.

The following lemma formalizes the consistency between the prover and verifier executions of ConvertToVOLE.

Lemma 4 (Correctness of ConvertToVOLE [22]). *Let $(\mathsf{sd}_0, \ldots, \mathsf{sd}_{N-1})$ and $(\mathsf{sd}'_0, \ldots, \mathsf{sd}'_{N-1})$ be seeds with $\mathsf{sd}_i \in \{0,1\}^N_{\mathsf{block}}$ and $\mathsf{sd}'_i \in \{0,1\}^N_{\mathsf{block}} \cup \{\perp\}$ such that, for some $\Delta \in [0\,..\,N)$,*

$$\mathsf{sd}'_0 = \perp, \qquad \mathsf{sd}'_i = \mathsf{sd}_{\pi_\Delta(i)} \text{ for all } i \in [1\,..\,N), \quad \text{where } \pi_\Delta(i) = \mathsf{BitCom}\big(\mathsf{BitDec}(i, D) \oplus \mathsf{BitDec}(\Delta, D)\big).$$

Let $(\delta_0, \ldots, \delta_{D-1}) \leftarrow \mathsf{BitDec}(\Delta, D)$, and define

$$(\boldsymbol{u}, \boldsymbol{v}_0, \ldots, \boldsymbol{v}_{D-1}) \leftarrow \mathsf{ConvertToVOLE}(\mathsf{sd}_0, \ldots, \mathsf{sd}_{N-1}; \hat{\ell}),$$
$$(\boldsymbol{u}', \boldsymbol{q}_0, \ldots, \boldsymbol{q}_{D-1}) \leftarrow \mathsf{ConvertToVOLE}(\mathsf{sd}'_0, \ldots, \mathsf{sd}'_{N-1}; \hat{\ell}).$$

Then, for all $j \in [0\,..\,D)$,

$$\boldsymbol{q}_j \;=\; \boldsymbol{v}_j \oplus \delta_j \cdot \boldsymbol{u}.$$

We follow the idea in [22] to give a proof for Lemma 4.

Proof. For readability, define $\hat{\ell}$-bit blocks

$$\boldsymbol{x}_i \;\leftarrow\; \begin{cases} \mathsf{PRG}(\mathsf{sd}_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell}) & \text{if } \mathsf{sd}_i \neq \perp, \\ 0^{\hat{\ell}} & \text{if } \mathsf{sd}_i = \perp, \end{cases} \qquad \boldsymbol{x}'_i \;\leftarrow\; \begin{cases} \mathsf{PRG}(\mathsf{sd}'_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell}) & \text{if } \mathsf{sd}'_i \neq \perp, \\ 0^{\hat{\ell}} & \text{if } \mathsf{sd}'_i = \perp. \end{cases}$$

ConvertToVOLE initializes $\boldsymbol{r}_{0,i} = \boldsymbol{x}_i$ (and analogously $\boldsymbol{r}'_{0,i} = \boldsymbol{x}'_i$).

(1) Correctness of $\boldsymbol{u}$. We first establish an invariant for the intermediate values $\boldsymbol{r}_{j,i}$. For every $j \in [0\,..\,D]$ and every $i \in [0\,..\,N/2^j)$,

$$\boldsymbol{r}_{j,i} \;=\; \bigoplus_{t=0}^{2^j-1} \boldsymbol{x}_{\,i\cdot 2^j+t}. \tag{30}$$

For $j = 0$, this holds by initialization. Assuming it holds for level j, Line 10 sets

$$\boldsymbol{r}_{j+1,i} = \boldsymbol{r}_{j,2i} \oplus \boldsymbol{r}_{j,2i+1} = \Big(\bigoplus_{t=0}^{2^j-1} \boldsymbol{x}_{(2i)\cdot 2^j+t}\Big) \oplus \Big(\bigoplus_{t=0}^{2^j-1} \boldsymbol{x}_{(2i+1)\cdot 2^j+t}\Big) = \bigoplus_{t=0}^{2^{j+1}-1} \boldsymbol{x}_{\,i\cdot 2^{j+1}+t},$$

which is exactly Equation (30) at level $j+1$. Therefore, Equation (30) holds for all levels. In particular, at the end (Line 13),

$$\boldsymbol{u} = \boldsymbol{r}_{D,0} = \bigoplus_{i=0}^{N-1} \boldsymbol{x}_i = \bigoplus_{i=0}^{N-1} \mathsf{PRG}(\mathsf{sd}_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell}).$$

(2) Correctness of $\boldsymbol{v}_j$. Fix $j \in [0\,..\,D)$. By Equation (30), each term $\boldsymbol{r}_{j,2i+1}$ equals the XOR of $\boldsymbol{x}_k$ over the interval

$$k \in \big[(2i+1)\cdot 2^j \,..\, (2i+2)\cdot 2^j - 1\big].$$

These intervals (over all $i \in [0\,..\,N/2^{j+1})$) form a disjoint partition of exactly the indices $k \in [0\,..\,N)$ whose j-th bit is 1. Hence, Line 9 computes

$$\boldsymbol{v}_j \;=\; \bigoplus_{i=0}^{N/2^{j+1}-1} \boldsymbol{r}_{j,2i+1} \;=\; \bigoplus_{\substack{k\in[0\,..\,N)\\ \mathsf{BitDec}(k,D)_j=1}} \boldsymbol{x}_k. \tag{31}$$

(3) Relating prover and verifier outputs. By the lemma assumptions, $\boldsymbol{x}'_0 = 0^{\hat{\ell}}$ and $\boldsymbol{x}'_i = \boldsymbol{x}_{\pi_\Delta(i)}$ for all $i \in [1\,..\,N)$. Applying Equation (31) to the verifier run gives

$$\boldsymbol{q}_j = \bigoplus_{\substack{i\in[0\,..\,N)\\ \mathsf{BitDec}(i,D)_j=1}} \boldsymbol{x}'_i = \bigoplus_{\substack{i\in[1\,..\,N)\\ \mathsf{BitDec}(i,D)_j=1}} \boldsymbol{x}_{\pi_\Delta(i)}.$$

Since $\pi_\Delta(i) = i \oplus \Delta$ is a bijection on $[0\,..\,N)$, we rename $k = i \oplus \Delta$ and note that

$$\begin{aligned} \mathsf{BitDec}(i, D)_j = 1 &\iff \mathsf{BitDec}(k \oplus \Delta, D)_j = 1 \\ &\iff \mathsf{BitDec}(k, D)_j = 1 \oplus \delta_j, \end{aligned}$$

where $(\delta_0, \ldots, \delta_{D-1}) = \mathsf{BitDec}(\Delta, D)$. Therefore,

$$\boldsymbol{q}_j = \bigoplus_{\substack{k\in[0\,..\,N)\\ \mathsf{BitDec}(k,D)_j=1\oplus\delta_j}} \boldsymbol{x}_k.$$

If $\delta_j = 0$, then $\boldsymbol{q}_j = \boldsymbol{v}_j$ by Equation (31). If $\delta_j = 1$, then $\boldsymbol{q}_j$ is the XOR of the complementary set $\mathsf{BitDec}(k, D)_j = 0$, hence

$$\boldsymbol{q}_j = \Big(\bigoplus_{k=0}^{N-1} \boldsymbol{x}_k\Big) \oplus \Big(\bigoplus_{\substack{k\in[0\,..\,N)\\ \mathsf{BitDec}(k,D)_j=1}} \boldsymbol{x}_k\Big) = \boldsymbol{u} \oplus \boldsymbol{v}_j.$$

In both cases we obtain $\boldsymbol{q}_j = \boldsymbol{v}_j \oplus \delta_j \cdot \boldsymbol{u}$, as claimed. □

ConvertToVOLE$(\mathsf{sd}_0, \ldots, \mathsf{sd}_{N-1}, \mathsf{iv}, \mathsf{twk}, \hat{\ell})$

INPUT: $N = 2^D$ for an integer $D \geq 1$, $\mathsf{sd}_0 \in \{0,1\}^{\lambda}_{\mathsf{block}} \cup \{\perp\}$, $\mathsf{sd}_i \in \{0,1\}^{\lambda}_{\mathsf{block}}$ for all $i \in [1, N)$, $\mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$, $\mathsf{twk} \in \{0,1\}^{32}$, $\hat{\ell} \in \mathbb{N}$

OUTPUT: $\boldsymbol{u}, \boldsymbol{v}_0, \ldots, \boldsymbol{v}_{\log N - 1} \in \{0,1\}^{\hat{\ell}}$

$D \leftarrow \log N$
$\boldsymbol{r}_{0,0} \leftarrow 0^{\hat{\ell}}$ **if** $\mathsf{sd}_0 = \perp$ **else** PRG$(\mathsf{sd}_0, \mathsf{iv}, \mathsf{twk}; \hat{\ell})$
for $i \in [1 \,..\, N)$ **do**
    $\boldsymbol{r}_{0,i} \leftarrow$ PRG$(\mathsf{sd}_i, \mathsf{iv}, \mathsf{twk}; \hat{\ell})$
end for
$\boldsymbol{v}_0 = \cdots = \boldsymbol{v}_{D-1} \leftarrow 0^{\hat{\ell}}$
for $j \in [0 \,..\, D)$ **do**
    for $i \in [0 \,..\, N/2^{j+1})$ **do**
        $\boldsymbol{v}_j \leftarrow \boldsymbol{v}_j \oplus \boldsymbol{r}_{j,2i+1}$
        $\boldsymbol{r}_{j+1,i} \leftarrow \boldsymbol{r}_{j,2i} \oplus \boldsymbol{r}_{j,2i+1}$
    end for
end for
$\boldsymbol{u} \leftarrow \boldsymbol{r}_{D,0}$
return $\boldsymbol{u}, \boldsymbol{v}_0, \ldots, \boldsymbol{v}_{D-1}$

5.1.3 Construction of VOLE Commitments

Given the algorithms related to the BAVC, this section describes the construction of VOLE commitments, including VOLECommit and VOLEReconstruct. Before this, we first introduce the DecodeChall algorithm for challenge decomposition, which is used in VOLEReconstruct.

DecodeChall. The DecodeChall algorithm specifies the challenge decomposition procedure. Given a challenge $\mathsf{chall} \in \{0,1\}^{\lambda_{\mathbb{F}}}$, the algorithm decomposes it into τ sub-challenges $(\Delta_0, \ldots, \Delta_{\tau-1})$, which are later used in the reconstruction procedure VOLEReconstruct. The parameters τ, τ_0, τ_1, k, and w_{grind} determine how the challenge is partitioned. More precisely, the algorithm first checks that the last w_{grind} bits of chall are zero, and then parses the first $\lambda_{\mathbb{F}} - w_{\mathsf{grind}}$ bits as the challenge-index string. For each index $i \in [0, \tau)$, it determines the substring boundaries $(\mathsf{lo}, \mathsf{hi})$ according to whether i belongs to the first τ_1 positions or to the remaining ones. If $i < \tau_1$, then the i-th sub-challenge is extracted from a block of length k, with

$$\mathsf{lo} \leftarrow i \cdot k \qquad \text{and} \qquad \mathsf{hi} \leftarrow (i+1) \cdot k - 1.$$

Otherwise, letting $t \leftarrow i - \tau_1$, the algorithm extracts a block of length $k-1$, with

$$\mathsf{lo} \leftarrow \tau_1 k + t(k-1) \qquad \text{and} \qquad \mathsf{hi} \leftarrow \tau_1 k + (t+1)(k-1) - 1.$$

Hence, the first τ_1 sub-challenges are represented using k bits each, while the remaining τ_0 sub-challenges are represented using $k-1$ bits each. For each $i \in [0, \tau)$, the algorithm sets

$$\Delta_i \leftarrow \mathsf{NumRec}(\mathsf{hi} - \mathsf{lo} + 1, \mathsf{chall}[\mathsf{lo} \ldots \mathsf{hi}]),$$

using least-significant-bit-first order to obtain an integer in $[0 \,..\, N_i)$, and after all indices have been processed, it returns the full tuple $(\Delta_0, \ldots, \Delta_{\tau-1})$.

DecodeChall($\mathsf{chall}; \mathsf{param}$)

INPUT: $\mathsf{chall} \in \{0,1\}^{\lambda_{\mathbb{F}}}$
OUTPUT: $(\Delta_0, \ldots, \Delta_{\tau-1}) \in [0\,..\,N_0) \times \cdots \times [0\,..\,N_{\tau-1}) \cup \{\perp\}$

if $\mathsf{chall}[\lambda_{\mathbb{F}} - w_{\mathsf{grind}} \ldots \lambda_{\mathbb{F}} - 1] \neq 0^{w_{\mathsf{grind}}}$ **then**
  return $\perp$
end if
for $i \in [0, \tau)$ **do**
  if $i < \tau_1$ **then**
    $\mathsf{lo} \leftarrow i \cdot k$
    $\mathsf{hi} \leftarrow (i+1) \cdot k - 1$
  else
    $t \leftarrow i - \tau_1$
    $\mathsf{lo} \leftarrow \tau_1 k + t(k-1)$
    $\mathsf{hi} \leftarrow \tau_1 k + (t+1)(k-1) - 1$
  end if
  $\Delta_i \leftarrow \mathsf{NumRec}(\mathsf{hi} - \mathsf{lo} + 1, \mathsf{chall}[\mathsf{lo} \ldots \mathsf{hi}])$
end for
return $(\Delta_0, \ldots, \Delta_{\tau-1})$

VOLECommit. The VOLECommit algorithm describes the VOLE commitment procedure. Given a root seed $\mathsf{sd}_{\mathsf{tree}}$, an initialization vector iv, and the target output length $\hat{\ell}$, the algorithm first invokes BAVC.Commit to generate the BAVC commitment com, the corresponding decommitment information decom, and, for each index $i \in [0, \tau)$, a set of GGM seeds $(\mathsf{sd}_{i,0}, \ldots, \mathsf{sd}_{i,N_i-1})$. Then, for each VOLE instance i, the algorithm applies ConvertToVOLE to derive a vector $\boldsymbol{u}_i \in \{0,1\}^{\hat{\ell}}$ together with column vectors $\boldsymbol{v}_{i,0}, \ldots, \boldsymbol{v}_{i,k_i-1}$, which are assembled into a matrix $\boldsymbol{V}_i \in \mathbb{F}_2^{\hat{\ell} \times k_i}$. After concatenating all $\boldsymbol{V}_i$ and appending zero-padding of width w_{grind}, the algorithm obtains the global matrix $\boldsymbol{V} \in \mathbb{F}_2^{\hat{\ell} \times \lambda_{\mathbb{F}}}$. Finally, it sets $\boldsymbol{u} = \boldsymbol{u}_0$ and defines the correction vectors $\boldsymbol{c}_i = \boldsymbol{u} \oplus \boldsymbol{u}_i$ for all $i \in [1, \tau)$, so that all VOLE instances are linked to the same reference vector $\boldsymbol{u}$. The algorithm outputs the commitment com, the decommitment string decom, the correction vectors $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}$, and the resulting VOLE commitment values $(\boldsymbol{u}, \boldsymbol{V})$.

VOLECommit($\mathsf{sd}_{\mathsf{tree}}, \mathsf{iv}, \hat{\ell}, \mathsf{param}$)

INPUT: $\mathsf{sd}_{\mathsf{tree}} \in \{0,1\}^{N}_{\mathsf{block}}, \mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$
OUTPUT: $\mathsf{com} \in \{0,1\}^{2N_{\mathsf{block}}}, \mathsf{decom} \in \{0,1\}^{(2L-1)N_{\mathsf{block}}+2N_{\mathsf{block}}L}, \boldsymbol{c}_i, \boldsymbol{u} \in \{0,1\}^{\hat{\ell}}, \boldsymbol{V} \in \{0,1\}^{\hat{\ell} \times \lambda_{\mathbb{F}}}$

$(\mathsf{com}, \mathsf{decom}, (\mathsf{sd}_{i,0}, \ldots, \mathsf{sd}_{i,N_i-1})_{i \in [0,\tau)}) \leftarrow \mathsf{BAVC.Commit}(\mathsf{sd}_{\mathsf{tree}}, \mathsf{iv}, \mathsf{param})$
for $i \in [0\,..\,\tau)$ **do**
  $k_i \leftarrow k$ if $i < \tau_1$, and $k_i \leftarrow k - 1$ otherwise
  $(\boldsymbol{u}_i, \boldsymbol{v}_{i,0}, \ldots, \boldsymbol{v}_{i,k_i-1}) \leftarrow \mathsf{ConvertToVOLE}(\mathsf{sd}_{i,0}, \ldots, \mathsf{sd}_{i,N_i-1}, \mathsf{iv}, i + 2^{31}, \hat{\ell})$ // Recall that k_i is the bit-length of the current VOLE instance, and N_i is the number of leaves of the current GGM tree
  $\boldsymbol{V}_i \leftarrow [\boldsymbol{v}_{i,0}, \ldots, \boldsymbol{v}_{i,k_i-1}] \in \mathbb{F}_2^{\hat{\ell} \times k_i}$
end for
$\boldsymbol{V} \leftarrow [\boldsymbol{V}_0, \ldots, \boldsymbol{V}_{\tau-1}, \boldsymbol{0}_{\hat{\ell}, w_{\mathsf{grind}}}] \in \mathbb{F}_2^{\hat{\ell} \times \lambda_{\mathbb{F}}}$ // $\boldsymbol{0}_{\hat{\ell}, w_{\mathsf{grind}}}$ is $(\hat{\ell} \times w_{\mathsf{grind}})$ bits of zero-padding
$\boldsymbol{u} \leftarrow \boldsymbol{u}_0$
for $i \in [1\,..\,\tau)$ **do**
  $\boldsymbol{c}_i \leftarrow \boldsymbol{u} \oplus \boldsymbol{u}_i$
end for
return $(\mathsf{com}, \mathsf{decom}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \boldsymbol{u}, \boldsymbol{V})$

VOLEReconstruct. The VOLEReconstruct algorithm describes the VOLE reconstruction procedure from the BAVC opening information. Given the challenge chall_3, the partial decommitment decom_I, the correction vectors $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}$, and the initialization vector iv, the algorithm first

decodes chall_3 into the tuple $I = (\Delta_0, \ldots, \Delta_{\tau-1})$ using DecodeChall. If decoding fails, it returns $\perp$. Otherwise, it calls BAVC.Reconstruct to recover the commitment com and the indexed non-hidden leaf seeds, returning $\perp$ if reconstruction fails. For each instance $i \in [0, \tau)$, the recovered seeds are reordered according to the offset Δ_i, and the reordered seeds are then passed to ConvertToVOLE to derive $\boldsymbol{u}'_i$ and the vectors $\boldsymbol{q}_{i,0}, \ldots, \boldsymbol{q}_{i,k_i-1}$. Next, the algorithm computes the bit decomposition $(\delta_{i,0}, \ldots, \delta_{i,k_i-1})$ of Δ_i. If $i = 0$, it directly forms $\boldsymbol{Q}_i$ from the vectors $\boldsymbol{q}_{i,0}, \ldots, \boldsymbol{q}_{i,k_i-1}$. Otherwise, it adds the correction terms determined by the bits $\delta_{i,j}$ and the vector $\boldsymbol{c}_i$, thereby obtaining the corrected matrix $\boldsymbol{Q}_i$. Finally, it concatenates all matrices $\boldsymbol{Q}_i$ and appends zero-padding to obtain $\boldsymbol{Q} \in \mathbb{F}_2^{\hat{\ell} \times \lambda_{\mathbb{F}}}$. The algorithm outputs $(\mathsf{com}, \boldsymbol{Q})$.

VOLEReconstruct$(\mathsf{chall}_3, \mathsf{decom}_I, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv}, \hat{\ell}; \mathsf{param})$

Input: $\mathsf{chall}_3 \in \{0,1\}^{\lambda_{\mathbb{F}}}$, $\mathsf{decom}_I \in \{0,1\}^{2N_{\mathsf{block}}\tau + T_{\mathsf{open}}N_{\mathsf{block}}}$, $\boldsymbol{c}_i \in \{0,1\}^{\hat{\ell}}$, $\mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$

Output: $\mathsf{com} \in \{0,1\}^{2N_{\mathsf{block}}}$, $\boldsymbol{Q} \in \{0,1\}^{\hat{\ell} \times \lambda_{\mathbb{F}}}$, or $\perp$

$I = (\Delta_0, \ldots, \Delta_{\tau-1}) \leftarrow$ DecodeChall$(\mathsf{chall}_3; \mathsf{param})$
if $I = \perp$ **return** $\perp$
$\mathsf{rec} \leftarrow$ BAVC.Reconstruct$(\mathsf{decom}_I, I, \mathsf{iv}; \mathsf{param})$
if $\mathsf{rec} = \perp$ **return** $\perp$
Parse $\mathsf{rec} = (\mathsf{com}, ((\mathsf{sd}_{i,j})_{j \in [0\,..\,N_i) \setminus \{\Delta_i\}})_{i \in [0\,..\,\tau)})$
for $i \in [0 \ldots \tau)$ **do**
    $k_i \leftarrow k$ if $i < \tau_1$, and $k_i \leftarrow k - 1$ otherwise
    for $j \in [1 \ldots N_i)$ **do**
        $\mathsf{sd}'_{i,j} \leftarrow \mathsf{sd}_{i, j \oplus \Delta_i}$ // XOR j and Δ_i as bit-strings
    end for
    $(\boldsymbol{u}'_i, \boldsymbol{q}_{i,0}, \ldots, \boldsymbol{q}_{i,k_i-1}) \leftarrow$ ConvertToVOLE$(\perp, \mathsf{sd}'_{i,1}, \ldots, \mathsf{sd}'_{i,N_i-1}, \mathsf{iv}, i + 2^{31}; \hat{\ell})$
    $(\delta_{i,0}, \ldots, \delta_{i,k_i-1}) \leftarrow$ BitDec(Δ_i, k_i)
    if $i = 0$ **then**
        $\boldsymbol{Q}_i \leftarrow [\boldsymbol{q}_{i,0} \cdots \boldsymbol{q}_{i,k_i-1}] \in \mathbb{F}_2^{\hat{\ell} \times k_i}$ // stored in column major representation
    else
        $\boldsymbol{Q}_i \leftarrow [\boldsymbol{q}_{i,0} \cdots \boldsymbol{q}_{i,k_i-1}] + [\delta_{i,0} \cdot \boldsymbol{c}_i \cdots \delta_{i,k_i-1} \cdot \boldsymbol{c}_i] \in \mathbb{F}_2^{\hat{\ell} \times k_i}$
    end if
end for
$\boldsymbol{Q} \leftarrow [\boldsymbol{Q}_0 \cdots \boldsymbol{Q}_{\tau-1}\ \mathbf{0}_{\hat{\ell}, w_{\mathsf{grind}}}] \in \mathbb{F}_2^{\hat{\ell} \times \lambda_{\mathbb{F}}}$ // $\mathbf{0}$ is $(\hat{\ell} \times w_{\mathsf{grind}})$ bits of zero-padding
return $(\mathsf{com}, \boldsymbol{Q})$

5.2 Zero-Knowledge Operations over VOLE Commitments

We use the notation $\langle x \rangle_e^d$ to denote a degree-d VOLE commitment to a field element $x \in \mathbb{F}_{2^e}$, where $e = 1$ or $e = \lambda_{\mathbb{F}}$. Elements in $\mathbb{F}_2$ are naturally embedded into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. This means that the prover knows the coefficients of a secret polynomial $\rho_x(X) = \rho_0 + \rho_1 X + \cdots + \rho_d X^d$, where $\rho_d = x$ and $\rho_0, \ldots, \rho_{d-1} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, and the verifier holds the evaluation $\gamma_x = \rho_x(\Delta)$ for a challenge $\Delta \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. VOLE commitments support addition and multiplication operations from both the prover and verifier sides, which is shown in Fig. 1. For ease of notation, we omit e and d when they equal 1. We use $\langle \boldsymbol{x} \rangle_e^d$, where $\boldsymbol{x} = (x_0, \ldots, x_{n-1}) \in \mathbb{F}_{2^e}^n$, to mean the vector of VOLE commitments $\langle x_0 \rangle_e^d, \ldots, \langle x_{n-1} \rangle_e^d$.

Degree lifting. For $D \geq q$, define $\mathsf{Lift}_D(\langle x \rangle_e^q)$ as the degree-D commitment represented by

$$\rho'_x(X) = X^{D-q} \rho_x(X), \qquad \gamma'_x = \Delta^{D-q} \gamma_x. \tag{32}$$

Before adding commitments of different degrees, lift each summand to their maximum degree. A public field element c at degree D is represented by cX^D for the prover and $c\Delta^D$ for the verifier. Multiplication adds degrees; scalar multiplication preserves the degree. Every constraint appended to the constraint vector is lifted to the scheme degree d.

Prover: VOLE Com. operations	**Verifier: VOLE Com. operations**
// Represent $\langle x \rangle_e^d$ *as* $(x_0, \ldots, x_{d-1}, x)$ *// Holds* $\rho_x(X) = \sum_{i=0}^{d-1} x_i \cdot X^i + xX^d$	*// Represent* $\langle x \rangle_e^d$ *as* γ_x *// Holds the challenge* Δ
ADD: $\langle x \rangle_{e_1}^{d_1} + \langle y \rangle_{e_2}^{d_2} \to \langle z \rangle_e^d$ (where $d = \max(d_1, d_2), e = \max(e_1, e_2)$) $\rho_z(X) \leftarrow X^{d-d_1}\rho_x(X) + X^{d-d_2}\rho_y(X)$ **return** $(z_0, \ldots, z_d)$ (coefficients of ρ_z)	ADD: $\langle x \rangle^{d_1} + \langle y \rangle^{d_2} \to \langle z \rangle^d$ **return** $\Delta^{d-d_1} \cdot \gamma_x + \Delta^{d-d_2} \cdot \gamma_y$
ADD CONSTANT: $\langle x \rangle_e^d + c \to \langle z \rangle_e^d$ **return** $\rho_z(X) \leftarrow \rho_x(X) + X^d \cdot c$	ADD CONSTANT: $\langle x \rangle_e^d + c \to \langle z \rangle_e^d$ **return** $\gamma_z = \gamma_x + \Delta^d \cdot c$
MULTIPLY CONSTANT: $c \cdot \langle x \rangle_e^d \to \langle z \rangle_e^d$ $\rho_z(X) \leftarrow c \cdot \rho_x(X)$ **return** $(z_0, \ldots, z_{d-1}, z)$ (coefficients of ρ_z)	MULTIPLY CONSTANT: $c \cdot \langle x \rangle^d \to \langle z \rangle^d$ **return** $\gamma_z = c \cdot \gamma_x$
MULTIPLY: $\langle x \rangle_{e_1}^{d_1} \cdot \langle y \rangle_{e_2}^{d_2} \to \langle z \rangle_e^d$ $\rho_z(X) \leftarrow \rho_x(X)\rho_y(X)$ **return** $(z_0, \ldots, z_{d_1+d_2-1}, xy)$ (coeffs of ρ_z)	MULTIPLY: $\langle x \rangle^{d_1} \cdot \langle y \rangle^{d_2} \to \langle z \rangle^d$ **return** $\gamma_x \cdot \gamma_y$

Fig. 1: Operations upon VOLE commitments for the prover and verifier

ConstantToVOLE. The Prover.ConstantToVOLE produces a vector of degree-1 commitments for public $\boldsymbol{x} = (x_0, \ldots, x_{n-1}) \in \mathbb{F}_2^n$, storing the 0-string in the first component and the bit-string $\boldsymbol{x}$ in the second component. Conversely, Verifier.ConstantToVOLE multiplies $\boldsymbol{x}$ by the global challenge $\Delta \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$ and obtains the commitment $\langle x_0 \rangle, \ldots, \langle x_{n-1} \rangle$ from the verifier's side.

Prover.ConstantToVOLE($\boldsymbol{x} \in \mathbb{F}_2^n$)	Verifier.ConstantToVOLE($\boldsymbol{x} \in \mathbb{F}_2^n$)
1: **return** $(\langle x_0 \rangle, \ldots, \langle x_{n-1} \rangle) \leftarrow$ $((0, x_0), \ldots, (0, x_{n-1})) \in (\mathbb{F}_{2^{\lambda_\mathbb{F}}} \times \mathbb{F}_2)^n$	**1:** *// Holds* $\Delta \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$ **2:** **return** $(\langle x_0 \rangle, \ldots, \langle x_{n-1} \rangle) \leftarrow \boldsymbol{x} \cdot \Delta \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}^n$

Fig. 2: ConstantToVOLE for the signer and verifier

6 Algorithm Description: the SM4th Signature Scheme

This section presents the construction of SM4th, which uses two proving methods for OWFs in the non-EM and EM variants, respectively. The EM variant stores the private input and intermediate state words and derives the S-box input and output commitments from them, yielding degree-2 checks. The non-EM variant replaces selected key-schedule and encryption words with norm witnesses and reconstructs these words within a degree-3 constraint system.

6.1 Zero-Knowledge Building Blocks for SM4th

This section presents several VOLEitH-based ZKP building blocks for SM4 operations.

ByteCombine. The ByteCombine function combines a vector of committed 8 bits into one committed field element belonging to $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$, which is SM4-compatible as defined in Definition 3.

Definition 3 (SM4-Compatible Embedding Element). *Assume that* $8 \mid \lambda_{\mathbb{F}}$. *A field element* $\alpha_8 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ *is* SM4-compatible *if its minimal polynomial over* $\mathbb{F}_2$ *equals the* SM4 *irreducible polynomial as in Equation* (9). *That is to say,* α_8 *satisfies* $m(\alpha_8) = 0$ *in* $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$.

Define $\phi : \mathrm{GF}(2^8) \to \mathbb{F}_2[\alpha_8]$ by

$$\phi\left(\sum_{i=0}^{7} b_i t^i\right) = \sum_{i=0}^{7} b_i \alpha_8^i,$$

where $\mathrm{GF}(2^8) = \mathbb{F}_2[t]/(m(t))$ is the SM4 byte field.

Lemma 5 (Operation Compatibility of ByteCombine). *For* $a, b \in \mathbb{F}_2[\alpha_8]$, *the product* ab *computed in* $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ *lies in* $\mathbb{F}_2[\alpha_8]$ *and satisfies*

$$\phi^{-1}(ab) = \phi^{-1}(a) \cdot_8 \phi^{-1}(b),$$

where $\cdot_8$ *denotes multiplication in the SM4 byte field.*

Proof. Since α_8 has minimal polynomial $m(t)$, evaluation at α_8 induces the field isomorphism ϕ from $\mathbb{F}_2[t]/(m(t))$ onto the subfield $\mathbb{F}_2[\alpha_8]$. Thus its image is closed under multiplication and ϕ preserves products. □

By Lemma 5, byte-field multiplication constraints can be checked using QuickSilver on the corresponding embedded commitments.

Table 12 specifies the SM4-compatible generator α_8 for the proof field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ used in SM4th. The value is presented in: (i) hexadecimal little-endian byte representation, and (ii) expanded polynomial form in the standard basis $\{1, z, \ldots, z^{\lambda_{\mathbb{F}}-1}\}$. The fixed generators in Table 12 determine the byte embedding for each proof field.

Table 12: SM4-compatible generators α_8 for ByteCombine

$\lambda_{\mathbb{F}}$	Hexadecimal (little-endian)	Polynomial form in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
128	`b8 4c 7b 2e 8b a1 14 84 b9 44 1f b3 b4 95 a5 51`	$z^{126}+z^{124}+z^{120}+z^{119}+z^{117}+z^{114}+z^{112}+z^{111}+z^{108}+z^{106}+z^{104}+z^{103}+z^{101}+z^{100}+z^{98}+z^{95}+z^{93}+z^{92}+z^{89}+z^{88}+z^{84}+z^{83}+z^{82}+z^{81}+z^{80}+z^{78}+z^{74}+z^{71}+z^{69}+z^{68}+z^{67}+z^{64}+z^{63}+z^{58}+z^{52}+z^{50}+z^{47}+z^{45}+z^{40}+z^{39}+z^{35}+z^{33}+z^{32}+z^{29}+z^{27}+z^{26}+z^{25}+z^{22}+z^{21}+z^{20}+z^{19}+z^{17}+z^{16}+z^{14}+z^{11}+z^{10}+z^{7}+z^{5}+z^{4}+z^{3}$
160	`d5 1d 5b 51 62 cb 88 67 b3 c6 6e 3e 1f aa 3b 35 70 69 9c 13`	$z^{156}+z^{153}+z^{152}+z^{151}+z^{148}+z^{147}+z^{146}+z^{142}+z^{141}+z^{139}+z^{136}+z^{134}+z^{133}+z^{132}+z^{125}+z^{124}+z^{122}+z^{120}+z^{117}+z^{116}+z^{115}+z^{113}+z^{112}+z^{111}+z^{109}+z^{107}+z^{105}+z^{100}+z^{99}+z^{98}+z^{97}+z^{96}+z^{93}+z^{92}+z^{91}+z^{90}+z^{89}+z^{86}+z^{85}+z^{83}+z^{82}+z^{81}+z^{79}+z^{78}+z^{74}+z^{73}+z^{71}+z^{69}+z^{68}+z^{65}+z^{64}+z^{62}+z^{61}+z^{58}+z^{57}+z^{56}+z^{55}+z^{51}+z^{47}+z^{46}+z^{43}+z^{41}+z^{40}+z^{38}+z^{37}+z^{33}+z^{30}+z^{28}+z^{24}+z^{22}+z^{20}+z^{19}+z^{17}+z^{16}+z^{12}+z^{11}+z^{10}+z^{8}+z^{7}+z^{6}+z^{4}+z^{2}+1$

SquareAndCombine. The SquareAndCombine function takes commitments to the eight bits of an SM4 byte, computes commitments to the bits of its square in the SM4 byte field, and embeds the

ByteCombine$(\langle x_0 \rangle^D, \ldots, \langle x_7 \rangle^D; \lambda_{\mathbb{F}})$

INPUT: $D \geq 1$; Commitment $\langle x_i \rangle^D$ where $x_i \in \mathbb{F}_2$, for $i \in [0\,..\,7]$; the proof-field parameter $\lambda_{\mathbb{F}}$
OUTPUT: Commitment $\langle x \rangle^D_{\lambda_{\mathbb{F}}}$ to $x \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, containing an embedded $\mathbb{F}_{2^8}$ element

1: let $\alpha_8 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ // SM4-compatible basis element of $\mathbb{F}_{2^8}$ within $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
2: **return** $\langle x \rangle^D_{\lambda_{\mathbb{F}}} = \sum_{i=0}^{7} \langle x_i \rangle^D \cdot \alpha_8^i$

squared byte into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ using ByteCombine. Byte-field squaring is $\mathbb{F}_2$-linear and preserves the commitment degree. Specifically, given the generator $\beta \in \mathrm{GF}(2^8)$ such that $m(\beta) = 0$ as in Equation (9) and the input commitment $\langle x_0\rangle, \ldots, \langle x_7\rangle$ of bits $x_0, \ldots, x_7$, SquareAndCombine computes the commitments $\langle y_0\rangle, \ldots, \langle y_7\rangle$ of bits $y_0, \ldots, y_7$ such that $\sum_{i=0}^{7} y_i\beta^i = \sum_{i=0}^{7} x_i\beta^{2i}$. SquareAndCombine then runs ByteCombine with $\langle y_0\rangle, \ldots, \langle y_7\rangle$ as inputs. Using the SM4 irreducible polynomial as in Equation (9), $\langle y_0\rangle, \ldots, \langle y_7\rangle$ can be computed. For example, since $\beta^{10} = \beta^5 + \beta^4 + \beta^3 + \beta^2 + \beta$, $\langle x_5\rangle$ would contribute to $\{\langle y_i\rangle\}_{i\in[1,5]}$.

SquareAndCombine$(\langle x_0\rangle^D, \ldots, \langle x_7\rangle^D)$

INPUT: $D \geq 1$; Committed bits $\langle x_i\rangle^D$ for $i \in [0\,..\,7]$
OUTPUT: Commitment $\langle y\rangle^D_{\lambda_\mathbb{F}}$ to the squared field element $y \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$

1: $\langle y_0\rangle^D \leftarrow \langle x_0\rangle^D + \langle x_4\rangle^D$
2: $\langle y_1\rangle^D \leftarrow \langle x_5\rangle^D + \langle x_7\rangle^D$
3: $\langle y_2\rangle^D \leftarrow \langle x_1\rangle^D + \langle x_4\rangle^D + \langle x_5\rangle^D$
4: $\langle y_3\rangle^D \leftarrow \langle x_5\rangle^D + \langle x_6\rangle^D + \langle x_7\rangle^D$
5: $\langle y_4\rangle^D \leftarrow \langle x_2\rangle^D + \langle x_4\rangle^D + \langle x_5\rangle^D + \langle x_6\rangle^D$
6: $\langle y_5\rangle^D \leftarrow \langle x_4\rangle^D + \langle x_5\rangle^D + \langle x_6\rangle^D$
7: $\langle y_6\rangle^D \leftarrow \langle x_3\rangle^D + \langle x_4\rangle^D + \langle x_6\rangle^D$
8: $\langle y_7\rangle^D \leftarrow \langle x_4\rangle^D + \langle x_6\rangle^D$
9: **return** ByteCombine$(\langle y_0\rangle^D, \ldots, \langle y_7\rangle^D)$

Affine and InvAffine for T or T' functions. The T′.Affine function takes a quintuple of commitments $\{\langle \bar{k}_{i,j,k}\rangle, \langle \bar{k}_{i+1,j,k}\rangle, \langle \bar{k}_{i+2,j,k}\rangle, \langle \bar{k}_{i+3,j,k}\rangle, \langle \bar{k}_{i+4,j,k}\rangle\}_{j\in[0,4),k\in[0,8)}$ and the fixed parameter $CK_i \in \{0,1\}^{32}$ as inputs. It outputs the commitments to the bits $\{x_{j,0}, \ldots, x_{j,7}\}_{j\in[0,4)}$. For each j, $(x_{j,0}, \ldots, x_{j,7})$ works as the inputs of the patched inversion function in S-boxes during the key expansion routine. We retain the unused arguments $\{\langle \bar{k}_{i,j,k}\rangle, \langle \bar{k}_{i+4,j,k}\rangle\}$ in T′.Affine to provide a uniform interface for the calling procedure. For ease of notation, denote

$$\langle \bar{k}_{i,0,0}\rangle, \ldots, \langle \bar{k}_{i,0,7}\rangle, \langle \bar{k}_{i,1,0}\rangle, \ldots, \langle \bar{k}_{i,1,7}\rangle, \ldots, \langle \bar{k}_{i,3,7}\rangle, \langle \bar{k}_{i+1,0,0}\rangle, \ldots, \langle \bar{k}_{i+4,3,7}\rangle.$$

as $\langle \bar{\boldsymbol{k}}_{\mathsf{quin}}\rangle$. Use $\langle \bar{\boldsymbol{k}}_{\mathsf{quin}}[m\,..\,n]\rangle$ to represent the entries from the m-th position to the n-th one. Denote

$$\langle x_{0,0}\rangle, \ldots, \langle x_{0,7}\rangle, \langle x_{1,0}\rangle, \ldots, \langle x_{3,7}\rangle$$

as $\langle \boldsymbol{x}\rangle$.

T′.Affine$(\langle \bar{\boldsymbol{k}}_{\mathsf{quin}}\rangle, CK_i)$

INPUT: For some $i \in [0, 32)$, a commitment vector to bits $(\bar{k}_{i,j,k}, \bar{k}_{i+1,j,k}, \bar{k}_{i+2,j,k}, \bar{k}_{i+3,j,k}, \bar{k}_{i+4,j,k})$ for $j \in [0, 4)$ and $k \in [0, 8)$; the i-th fixed parameter CK_i
OUTPUT: $\langle \boldsymbol{x}\rangle$, commitments to $x_{j,k} \in \mathbb{F}_2$ for $j \in [0, 4)$ and $k \in [0, 8)$, working as inputs of $I(\cdot)$ during the key expansion routine

1: **new** $\langle \boldsymbol{x}\rangle$, commitments to $\boldsymbol{x} \in \mathbb{F}_2^{32}$
2: **new** $\langle \boldsymbol{t}\rangle$, commitments to $\boldsymbol{t} \in \mathbb{F}_2^{32}$ // temporary values
3: **new** $\langle \boldsymbol{ck}\rangle = (\langle ck_{0,0}\rangle, \ldots, \langle ck_{0,7}\rangle, \ldots \langle ck_{3,0}\rangle, \ldots, \langle ck_{3,7}\rangle) \leftarrow$ ConstantToVOLE(CK_i), where $ck_{j,k} \in \mathbb{F}_2$
4: $\langle \boldsymbol{t}\rangle \leftarrow \langle \bar{\boldsymbol{k}}_{\mathsf{quin}}[N_{\mathsf{word}}\,..\,2N_{\mathsf{word}}-1]\rangle + \langle \bar{\boldsymbol{k}}_{\mathsf{quin}}[2N_{\mathsf{word}}\,..\,3N_{\mathsf{word}}-1]\rangle + \langle \bar{\boldsymbol{k}}_{\mathsf{quin}}[3N_{\mathsf{word}}\,..\,4N_{\mathsf{word}}-1]\rangle + \langle \boldsymbol{ck}[0\,..\,N_{\mathsf{word}}-1]\rangle$
5: **for** $j \in [0\,..\,4)$ **do**
6: $\quad \langle \boldsymbol{x}[8j\,..\,8j+7]\rangle \leftarrow \boldsymbol{A}\cdot\langle \boldsymbol{t}[8j\,..\,8j+7]\rangle +$ ConstantToVOLE$(\boldsymbol{c})$, with $\boldsymbol{A}$ and $\boldsymbol{c}$ as defined in Equation (8), and $\cdot$ means multiplications in $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ after elements in $\boldsymbol{A}$ are embedded into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$
7: **end for**
8: **return** $\{\langle x_{j,0}\rangle, \ldots, \langle x_{j,7}\rangle\}_{j\in[0,4)}$

Similar to T′.Affine, T′.InvAffine takes $\langle \bar{\boldsymbol{k}}_{\mathsf{quin}} \rangle$ as inputs. It outputs the commitments to the bits $\{y_{j,0}, \ldots, y_{j,7}\}_{j\in[0,4)}$. For each j, $(y_{j,0}, \ldots, y_{j,7})$ works as the outputs of the patched inversion function in S-boxes during the key expansion routine.

T′.InvAffine($\langle \bar{\boldsymbol{k}}_{\mathsf{quin}} \rangle$)

Input: For some $i \in [0, 32)$, a commitment vector to bits $(\bar{k}_{i,j,k}, \bar{k}_{i+1,j,k}, \bar{k}_{i+2,j,k}, \bar{k}_{i+3,j,k}, \bar{k}_{i+4,j,k})$ for $j \in [0, 4)$ and $k \in [0, 8)$

Output: $\langle \boldsymbol{y} \rangle$, commitments to $y_{j,k} \in \mathbb{F}_2$ for $j \in [0, 4)$ and $k \in [0, 8)$, working as outputs of $I(\cdot)$ during the key expansion routine

1: **new** $\langle \boldsymbol{y} \rangle$, commitments to $\boldsymbol{y} \in \mathbb{F}_2^{32}$
2: **new** $\langle \boldsymbol{t} \rangle$, commitments to $\boldsymbol{t} \in \mathbb{F}_2^{32}$ // temporary values
3: $\langle \boldsymbol{t} \rangle \leftarrow \langle \bar{\boldsymbol{k}}_{\mathsf{quin}}[0 \,..\, N_{\mathsf{word}} - 1] \rangle + \langle \bar{\boldsymbol{k}}_{\mathsf{quin}}[4N_{\mathsf{word}} \,..\, 5N_{\mathsf{word}} - 1] \rangle$
4: $\langle \boldsymbol{t} \rangle \leftarrow L'^{-1}(\langle \boldsymbol{t} \rangle)$, which has the same operations over commitments as how L'^{-1} operates over vectors in $\mathbb{F}_2$ defined in Equation (14), *i.e.*, $L'^{-1}(\langle \boldsymbol{t} \rangle) = \langle \boldsymbol{t} \rangle \oplus (\langle \boldsymbol{t} \rangle \lll 2) \oplus (\langle \boldsymbol{t} \rangle \lll 4) \oplus (\langle \boldsymbol{t} \rangle \lll 8) \oplus (\langle \boldsymbol{t} \rangle \lll 11) \oplus (\langle \boldsymbol{t} \rangle \lll 12) \oplus (\langle \boldsymbol{t} \rangle \lll 14) \oplus (\langle \boldsymbol{t} \rangle \lll 17) \oplus (\langle \boldsymbol{t} \rangle \lll 22) \oplus (\langle \boldsymbol{t} \rangle \lll 23) \oplus (\langle \boldsymbol{t} \rangle \lll 24) \oplus (\langle \boldsymbol{t} \rangle \lll 30) \oplus (\langle \boldsymbol{t} \rangle \lll 31)$
5: **for** $j \in [0 \,..\, 4)$ **do**
6: $\quad \langle \boldsymbol{t}[8j \,..\, 8j + 7] \rangle \leftarrow \langle \boldsymbol{t}[8j \,..\, 8j + 7] \rangle + \mathsf{ConstantToVOLE}(\boldsymbol{c})$
7: $\quad \langle \boldsymbol{y}[8j \,..\, 8j + 7] \rangle \leftarrow \boldsymbol{A}^{-1} \cdot \langle \boldsymbol{t}[8j \,..\, 8j + 7] \rangle$, where $\boldsymbol{A}^{-1}$ is defined in Equation (10).
8: **end for**
9: **return** $\langle \boldsymbol{y} \rangle$

The T.Affine function takes a quintuple of commitments $\{\langle \bar{z}_{i,j,k} \rangle, \langle \bar{z}_{i+1,j,k} \rangle, \langle \bar{z}_{i+2,j,k} \rangle, \langle \bar{z}_{i+3,j,k} \rangle, \langle \bar{z}_{i+4,j,k} \rangle\}_{j\in[0,4),k\in[0,8)}$ and $\langle \bar{\boldsymbol{k}}_{\mathsf{quin}} \rangle$ as inputs. It outputs the commitments to bits $\{x_{j,0}, \ldots, x_{j,7}\}_{j\in[0,4)}$. For each j, $(x_{j,0}, \ldots, x_{j,7})$ works as the inputs of the patched inversion function in S-boxes during the encryption routine. Similar to $\langle \bar{\boldsymbol{k}}_{\mathsf{quin}} \rangle$, denote

$$\langle z_{i,0,0} \rangle, \ldots, \langle z_{i,0,7} \rangle, \langle z_{i,1,0} \rangle, \ldots, \langle z_{i,1,7} \rangle, \ldots, \langle z_{i,3,7} \rangle, \langle z_{i+1,0,0} \rangle, \ldots, \langle z_{i+4,3,7} \rangle.$$

as $\langle \boldsymbol{z}_{\mathsf{quin}} \rangle$.

T.Affine($\langle \boldsymbol{z}_{\mathsf{quin}} \rangle, \langle \bar{\boldsymbol{k}}_{\mathsf{quin}} \rangle$)

Input: For some $i \in [0, 32)$, a commitment vector to bits $(z_{i,j,k}, z_{i+1,j,k}, z_{i+2,j,k}, z_{i+3,j,k}, z_{i+4,j,k})$ and $(\bar{k}_{i,j,k}, \bar{k}_{i+1,j,k}, \bar{k}_{i+2,j,k}, \bar{k}_{i+3,j,k}, \bar{k}_{i+4,j,k})$, for $j \in [0, 4)$ and $k \in [0, 8)$

Output: $\langle \boldsymbol{x} \rangle$, commitments to $x_{j,k} \in \mathbb{F}_2$ for $j \in [0, 4)$ and $k \in [0, 8)$, working as inputs of $I(\cdot)$ during the encryption routine

1: **new** $\langle \boldsymbol{x} \rangle$, commitments to $\boldsymbol{x} \in \mathbb{F}_2^{32}$
2: **new** $\langle \boldsymbol{t} \rangle$, commitments to $\boldsymbol{t} \in \mathbb{F}_2^{32}$ // temporary values
3: $\langle \boldsymbol{t} \rangle \leftarrow \langle \boldsymbol{z}_{\mathsf{quin}}[N_{\mathsf{word}} \,..\, 2N_{\mathsf{word}} - 1] \rangle + \langle \boldsymbol{z}_{\mathsf{quin}}[2N_{\mathsf{word}} \,..\, 3N_{\mathsf{word}} - 1] \rangle + \langle \boldsymbol{z}_{\mathsf{quin}}[3N_{\mathsf{word}} \,..\, 4N_{\mathsf{word}} - 1] \rangle + \langle \bar{\boldsymbol{k}}_{\mathsf{quin}}[4N_{\mathsf{word}} \,..\, 5N_{\mathsf{word}} - 1] \rangle$
4: **for** $j \in [0 \,..\, 4)$ **do**
5: $\quad \langle \boldsymbol{x}[8j \,..\, 8j + 7] \rangle \leftarrow \boldsymbol{A} \cdot \langle \boldsymbol{t}[8j \,..\, 8j + 7] \rangle + \mathsf{ConstantToVOLE}(\boldsymbol{c})$, with $\boldsymbol{A}$ and $\boldsymbol{c}$ as defined in Equation (8), and $\cdot$ means multiplications in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ after elements in $\boldsymbol{A}$ are embedded into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
6: **end for**
7: **return** $\langle \boldsymbol{x} \rangle$

The T.InvAffine algorithm takes $\langle \boldsymbol{z}_{\mathsf{quin}} \rangle$ as inputs. It outputs the commitments to the bits $\{y_{j,0}, \ldots, y_{j,7}\}_{j\in[0,4)}$. For each j, $(y_{j,0}, \ldots, y_{j,7})$ works as the outputs of the patched inversion function in S-boxes during the encryption routine.

6.1.1 Building Blocks for Non-EM Variants

This section gives more building blocks specified for the non-EM variant of SM4th.

T.InvAffine($\langle \boldsymbol{z}_{\mathsf{quin}} \rangle$)

INPUT: For some $i \in [0, 32)$, a commitment vector to bits $(z_{i,j,k}, z_{i+1,j,k}, z_{i+2,j,k}, z_{i+3,j,k}, z_{i+4,j,k})$
OUTPUT: $\langle \boldsymbol{y} \rangle$, commitments to $y_{j,k} \in \mathbb{F}_2$ for $j \in [0, 4)$ and $k \in [0, 8)$, working as outputs of $I(\cdot)$ during the encryption routine

1: **new** $\langle \boldsymbol{y} \rangle$, commitments to $\boldsymbol{y} \in \mathbb{F}_2^{32}$
2: **new** $\langle \boldsymbol{t} \rangle$, commitments to $\boldsymbol{t} \in \mathbb{F}_2^{32}$ // temporary values
3: $\langle \boldsymbol{t} \rangle \leftarrow \langle \boldsymbol{z}_{\mathsf{quin}}[0 \,..\, N_{\mathsf{word}} - 1] \rangle + \langle \boldsymbol{z}_{\mathsf{quin}}[4N_{\mathsf{word}} \,..\, 5N_{\mathsf{word}} - 1] \rangle$
4: $\langle \boldsymbol{t} \rangle \leftarrow L^{-1}(\langle \boldsymbol{t} \rangle)$, which has the same operations over commitments as how L^{-1} operates over vectors in $\mathbb{F}_2$ defined in Equation (13), *i.e.*, $L^{-1}(\langle \boldsymbol{t} \rangle) = \langle \boldsymbol{t} \rangle \oplus (\langle \boldsymbol{t} \rangle \lll 2) \oplus (\langle \boldsymbol{t} \rangle \lll 4) \oplus (\langle \boldsymbol{t} \rangle \lll 8) \oplus (\langle \boldsymbol{t} \rangle \lll 12) \oplus (\langle \boldsymbol{t} \rangle \lll 14) \oplus (\langle \boldsymbol{t} \rangle \lll 16) \oplus (\langle \boldsymbol{t} \rangle \lll 18) \oplus (\langle \boldsymbol{t} \rangle \lll 22) \oplus (\langle \boldsymbol{t} \rangle \lll 24) \oplus (\langle \boldsymbol{t} \rangle \lll 30)$
5: **for** $j \in [0 \,..\, 4)$ **do**
6: $\quad \langle \boldsymbol{t}[8j \,..\, 8j + 7] \rangle \leftarrow \langle \boldsymbol{t}[8j \,..\, 8j + 7] \rangle + \mathsf{ConstantToVOLE}(\boldsymbol{c})$
7: $\quad \langle \boldsymbol{y}[8j \,..\, 8j + 7] \rangle \leftarrow \boldsymbol{A}^{-1} \cdot \langle \boldsymbol{t}[8j \,..\, 8j + 7] \rangle$, where $\boldsymbol{A}^{-1}$ is defined in Equation (10).
8: **end for**
9: **return** $\langle \boldsymbol{y} \rangle$

InvNorm. The function SM4th.InvNorm extracts a 4-bit representation of the norm of the patched inverse of an 8-bit SM4 field element. This operation is performed entirely inside the SM4 byte field and is independent of the proof field size $\lambda_{\mathbb{F}}$. Let $\alpha_{\mathsf{SM4}} = t \bmod m(t)$ in $\mathbb{F}_{2^8} = \mathbb{F}_2[t]/(m(t))$, where $m(t)$ is given in Equation (9). The input $\boldsymbol{x} \in \{0,1\}^8$ is interpreted as $x = \mathsf{ToSM4Field}(\boldsymbol{x}) = \sum_{j=0}^{7} \boldsymbol{x}[j]\alpha_{\mathsf{SM4}}^j$. The inverse conversion is $\mathsf{ToSM4Bits}(\sum_{j=0}^{7} b_j \alpha_{\mathsf{SM4}}^j) = b_0 \| \cdots \| b_7$ for $b_j \in \mathbb{F}_2$. For an SM4 S-box input u, this input is $x = A_{\mathrm{SM4}}(u)$. With $I(0) = 0$ and $I(x) = x^{-1}$ for $x \neq 0$, the norm witness is

$$h = N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(I(x)) = I(x)^{17} = x^{238} \in \mathbb{F}_{2^4}.$$

For $x \neq 0$, this also equals x^{-17} because $x^{255} = 1$.

We use the basis $b = (1, \beta, \beta^2, \beta^3)$ of $\mathbb{F}_{2^4}$, where $\beta = \alpha_{\mathsf{SM4}}^3 + \alpha_{\mathsf{SM4}}^2$ and

$$(1, \beta, \beta^2, \beta^3) = (\texttt{0x01}, \texttt{0x0c}, \texttt{0x50}, \texttt{0x2a}).$$

The output $\boldsymbol{y} = (y_0, y_1, y_2, y_3)$ is the unique representation satisfying

$$h = y_0 + y_1\beta + y_2\beta^2 + y_3\beta^3, \qquad y_i \in \mathbb{F}_2.$$

Equivalently, if $\boldsymbol{y}' = \mathsf{ToSM4Bits}(h)$, then

$$\boldsymbol{y} = (y'[0], y'[2], y'[4], y'[1]).$$

For $x = 0$, the output is $(0, 0, 0, 0)$. These four bits are committed as Boolean witness values and embedded into the proof field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ through the surrounding VOLE/QuickSilver operations.

SM4th.InvNorm($\boldsymbol{x}$)

INPUT: $\boldsymbol{x} \in \{0,1\}^8$
OUTPUT: $\boldsymbol{y} \in \{0,1\}^4$

1: $x \leftarrow \mathsf{ToSM4Field}(\boldsymbol{x})$
2: **if** $x = 0$ **then**
3: $\quad$ **return** $(0, 0, 0, 0)$
4: **end if**
5: $\boldsymbol{y}' \leftarrow \mathsf{ToSM4Bits}(x^{238})$
6: $\boldsymbol{y} \leftarrow (y'[0], y'[2], y'[4], y'[1])$
7: **return** $\boldsymbol{y}$

AffineToConjugates. The SM4th.AffineToConjugates algorithm converts a committed SM4 S-box input byte vector into the conjugate representation used by the norm constraints. Given a

commitment $\langle \boldsymbol{u} \rangle$ to an 8-bit vector, it first applies the SM4 affine map and obtains the committed byte vector $\langle \boldsymbol{x}^{(0)} \rangle = \boldsymbol{A} \cdot \langle \boldsymbol{u} \rangle + \boldsymbol{c}$. It then repeatedly applies the SM4 byte-field squaring map at the bit-vector level, producing committed byte vectors $\langle \boldsymbol{x}^{(r)} \rangle$ representing $\langle (\boldsymbol{A}\boldsymbol{u} + \boldsymbol{c})^{2^r} \rangle$ for $r \in [0\,..\,8)$. For each r, the algorithm embeds $\langle \boldsymbol{x}^{(r)} \rangle$ into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ using ByteCombine and outputs $\langle \hat{x}^{2^r} \rangle$. This embedding uses the fixed SM4-compatible copy of $\mathbb{F}_{2^8}$ inside $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$, so the bit-level squaring map is still the SM4 byte-field squaring map and is independent of the value of $\lambda_\mathbb{F}$. The resulting tuple $(\langle \hat{x} \rangle, \langle \hat{x}^2 \rangle, \ldots, \langle \hat{x}^{128} \rangle)$ allows the protocol to access the Frobenius conjugates required by the norm check, in particular $\langle \hat{x} \rangle$, $\langle \hat{x}^2 \rangle$, and $\langle \hat{x}^{16} \rangle$. The full tuple is also used when reconstructing the full output bytes of S-boxes from the norm witness through subsequent affine SM4 transformations (Recall that one S-box includes 2 affine transformations and 1 inverse operation).

SM4th.AffineToConjugates($\langle \boldsymbol{u} \rangle$)

Input: Commitment to one byte $\boldsymbol{u} \in \{0,1\}^8$
Output: $(\langle x \rangle, \langle x^2 \rangle, \ldots, \langle x^{128} \rangle)$, where $x = \phi(\mathsf{ToSM4Field}(\boldsymbol{A}\boldsymbol{u} + \boldsymbol{c}))$

$\langle \boldsymbol{a}^{(0)} \rangle \leftarrow \boldsymbol{A} \cdot \langle \boldsymbol{u} \rangle + \boldsymbol{c}$ // $\boldsymbol{A}$ and $\boldsymbol{c}$ are defined as in Equation (8)
for $r \in [0\,..\,8)$ **do**
  $\langle x^{2^r} \rangle \leftarrow$ ByteCombine$(\langle a_0^{(r)} \rangle, \ldots, \langle a_7^{(r)} \rangle)$
  if $r < 7$ **then**
    // Square in the SM4 byte field
    $\langle a_0^{(r+1)} \rangle \leftarrow \langle a_0^{(r)} \rangle \oplus \langle a_4^{(r)} \rangle$
    $\langle a_1^{(r+1)} \rangle \leftarrow \langle a_5^{(r)} \rangle \oplus \langle a_7^{(r)} \rangle$
    $\langle a_2^{(r+1)} \rangle \leftarrow \langle a_1^{(r)} \rangle \oplus \langle a_4^{(r)} \rangle \oplus \langle a_5^{(r)} \rangle$
    $\langle a_3^{(r+1)} \rangle \leftarrow \langle a_5^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle \oplus \langle a_7^{(r)} \rangle$
    $\langle a_4^{(r+1)} \rangle \leftarrow \langle a_2^{(r)} \rangle \oplus \langle a_4^{(r)} \rangle \oplus \langle a_5^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle$
    $\langle a_5^{(r+1)} \rangle \leftarrow \langle a_4^{(r)} \rangle \oplus \langle a_5^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle$
    $\langle a_6^{(r+1)} \rangle \leftarrow \langle a_3^{(r)} \rangle \oplus \langle a_4^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle$
    $\langle a_7^{(r+1)} \rangle \leftarrow \langle a_4^{(r)} \rangle \oplus \langle a_6^{(r)} \rangle$
  end if
end for
return $(\langle x \rangle, \langle x^2 \rangle, \ldots, \langle x^{128} \rangle)$

InvNormToConjugates. The SM4th.InvNormToConjugates algorithm expands a committed 4-bit norm vector into the conjugates of the corresponding subfield value. Given the commitments $\langle n_0 \rangle$, $\ldots, \langle n_3 \rangle$, it interprets the 4 bits as coordinates with respect to the basis $(1, \beta, \beta^2, \beta^3)$ of the SM4 subfield $\mathbb{F}_{2^4} \subseteq \mathbb{F}_{2^8}$. Thus the represented SM4 subfield value is $h = n_0 + n_1\beta + n_2\beta^2 + n_3\beta^3$. In the proof, these basis constants are embedded into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ using the fixed SM4-compatible embedding from Definition 3. Equivalently, if $\hat{\beta} = \alpha_8^3 + \alpha_8^2 \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$, then the embedded value is $\hat{h} = n_0 + n_1\hat{\beta} + n_2\hat{\beta}^2 + n_3\hat{\beta}^3$. The algorithm then computes the Frobenius conjugates $\hat{h}^{2^r}$ for $r \in [0\,..\,4)$ by squaring the embedded basis constants and keeping the committed coordinates unchanged. The output $(\langle \hat{h} \rangle, \langle \hat{h}^2 \rangle, \langle \hat{h}^4 \rangle, \langle \hat{h}^8 \rangle)$ is used in the norm check and in the reconstruction of the inverse byte from the norm witness.

SM4th.InvNormToConjugates($\langle \boldsymbol{n} \rangle$)

Input: Commitment to norm bits $\boldsymbol{n} = (n_0, n_1, n_2, n_3) \in \{0,1\}^4$
Output: $(\langle \hat{h} \rangle, \langle \hat{h}^2 \rangle, \langle \hat{h}^4 \rangle, \langle \hat{h}^8 \rangle)$

$(\hat{b}_0, \hat{b}_1, \hat{b}_2, \hat{b}_3) \leftarrow (1, \hat{\beta}, \hat{\beta}^2, \hat{\beta}^3)$, where $\hat{\beta} = \alpha_8^3 + \alpha_8^2 \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$
for $r \in [0\,..\,4)$ **do**
  $\langle \hat{h}^{2^r} \rangle \leftarrow \sum_{t=0}^{3} \langle n_t \rangle \cdot \hat{b}_t^{2^r}$
end for
return $(\langle \hat{h} \rangle, \langle \hat{h}^2 \rangle, \langle \hat{h}^4 \rangle, \langle \hat{h}^8 \rangle)$

AffineOut. The SM4th.AffineOut algorithm applies the output affine layer of the SM4 S-box to a value already represented by its Frobenius conjugates. It receives $(\langle q\rangle, \langle q^2\rangle, \ldots, \langle q^{128}\rangle)$, where q is the patched inverse output in the SM4 byte field, embedded into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ through the fixed SM4-compatible embedding. Using the affine map $\boldsymbol{s} = \boldsymbol{A}\cdot\boldsymbol{q} + \boldsymbol{c}$ from Equation (8), the algorithm evaluates the same affine transformation in its linearized-polynomial form over the embedded copy of $\mathbb{F}_{2^8}$. This avoids converting the committed field representation back into committed bits. More precisely, let

$$\mathcal{A}_{\mathrm{out}}(X) = \hat{c} + \sum_{j=0}^{7} \hat{a}_j X^{2^j}$$

be the unique linearized-polynomial representation of the SM4 output affine map after embedding its constants into $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Its coefficients are $\hat{c} = \phi(\mathtt{D3})$ and $\hat{a}_j = \phi(a_j)$, where

$$(a_0, \ldots, a_7) = (\mathtt{18}, \mathtt{06}, \mathtt{91}, \mathtt{E9}, \mathtt{F4}, \mathtt{C8}, \mathtt{E6}, \mathtt{77}). \tag{33}$$

Here hexadecimal values denote SM4 byte-field elements with the least significant bit as the constant coefficient. Then $s = \mathcal{A}_{\mathrm{out}}(q)$, and its Frobenius conjugates are computed as

$$s^{2^r} = \hat{c}^{2^r} + \sum_{j=0}^{7} \hat{a}_j^{2^r} q^{2^{j+r}},$$

where the exponent index is reduced modulo 8 because q belongs to the embedded copy of $\mathbb{F}_{2^8}$. For each $r \in [0\,..\,8)$, the algorithm computes the conjugate $\langle s^{2^r}\rangle$ directly from the conjugates of q. The output $(\langle s\rangle, \langle s^2\rangle, \ldots, \langle s^{128}\rangle)$ gives the committed conjugate representation of the SM4 S-box output byte, which can then be used in the key expansion or encryption routines.

SM4th.AffineOut$((\langle q^{2^r}\rangle^D)_{r=0}^7)$

Input: Committed conjugates $(\langle q\rangle^D, \langle q^2\rangle^D, \ldots, \langle q^{128}\rangle^D)$ of the patched inverse output
Output: Committed conjugates $(\langle s\rangle^D, \langle s^2\rangle^D, \ldots, \langle s^{128}\rangle^D)$, where $\boldsymbol{s} = \boldsymbol{A}\cdot\boldsymbol{q} + \boldsymbol{c}$

1: Let D be the degree of the input commitments and $\mathcal{A}_{\mathrm{out}}(X) = \hat{c} + \sum_{j=0}^{7} \hat{a}_j X^{2^j}$ be the linearized-polynomial form of the SM4 output affine map over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$
2: **for** $r \in [0\,..\,8)$ **do**
3: $\quad \langle s^{2^r}\rangle^D \leftarrow \hat{c}^{2^r}\,\mathsf{Lift}_D(\mathsf{ConstantToVOLE}(1)) + \sum_{j=0}^{7} \hat{a}_j^{2^r} \cdot \langle q^{2^{(j+r) \bmod 8}}\rangle^D$
4: **end for**
5: **return** $(\langle s\rangle^D, \langle s^2\rangle^D, \ldots, \langle s^{128}\rangle^D)$

ConjToBits. For committed byte conjugates $(\langle s^{2^r}\rangle^D)_{r=0}^7$ of degree D, define

$$\langle s_b\rangle^D = \sum_{r=0}^{7} \phi(T_{b,r})\langle s^{2^r}\rangle^D, \qquad b \in [0, 8). \tag{34}$$

The hexadecimal entries of T are given in Table 13, and the output bits retain degree D. The map ConjToBits returns $(\langle s_0\rangle^D, \ldots, \langle s_7\rangle^D)$. In the anchor branches, it is applied to each of the four bytes before SM4th.L′ or SM4th.L.

Lprime. The SM4th.L′ algorithm applies the linear diffusion layer used in the SM4 key expansion. Given a committed 32-bit word $\langle \boldsymbol{s}\rangle$, it computes the committed word $\langle \boldsymbol{t}\rangle = \langle \boldsymbol{s}\rangle \oplus \mathsf{RotlWord}(\langle \boldsymbol{s}\rangle, 13) \oplus \mathsf{RotlWord}(\langle \boldsymbol{s}\rangle, 23)$. This corresponds to the SM4 key-schedule linear map $L'(\boldsymbol{s}) = \boldsymbol{s} \oplus (\boldsymbol{s} \lll 13) \oplus (\boldsymbol{s} \lll 23)$. The output is used to reconstruct a key-schedule word as $\langle \bar{\boldsymbol{k}}[i]\rangle = \langle \bar{\boldsymbol{k}}[i-4]\rangle \oplus \mathsf{SM4th.L'}(\langle \boldsymbol{s}\rangle)$ after the S-box output has been obtained.

LprimeInv. The SM4th.L′Inv algorithm applies the inverse of the SM4 key expansion linear map L'. Given a committed 32-bit word $\langle \boldsymbol{v}\rangle$, it computes $\langle L'^{-1}(\boldsymbol{v})\rangle$ using the fixed XOR-of-rotations

Table 13: SM4 byte-field coefficients for ConjToBits

$b\backslash r$	0	1	2	3	4	5	6	7
0	C7	E7	D9	46	EC	9C	AF	61
1	99	BE	95	EE	98	BF	94	EF
2	71	32	CF	A7	21	3F	9E	AB
3	C2	F6	2D	6F	93	FA	7D	62
4	A6	20	3E	9F	AA	70	33	CE
5	94	EF	99	BE	95	EE	98	BF
6	8D	5B	48	B8	81	0B	45	E9
7	7B	76	27	2B	7B	76	27	2B

SM4th.L′($\langle \boldsymbol{s} \rangle^D$)

INPUT: $D \geq 1$; Commitment $\langle \boldsymbol{s} \rangle^D$ to a 32-bit word
OUTPUT: Commitment $\langle L'(\boldsymbol{s}) \rangle^D$
1: $\langle \boldsymbol{t} \rangle^D \leftarrow \langle \boldsymbol{s} \rangle^D \oplus \mathsf{RotlWord}(\langle \boldsymbol{s} \rangle^D, 13) \oplus \mathsf{RotlWord}(\langle \boldsymbol{s} \rangle^D, 23)$
2: **return** $\langle \boldsymbol{t} \rangle^D$

expression for the inverse map. This routine is used for full non-anchor key words. In that case, the witness contains the full word $\bar{\boldsymbol{k}}[i]$, so the protocol first computes $\langle \boldsymbol{v} \rangle = \langle \bar{\boldsymbol{k}}[i-4] \rangle \oplus \langle \bar{\boldsymbol{k}}[i] \rangle$ and then applies SM4th.L′Inv to recover the committed S-box output candidate. The recovered value is then checked against the forward S-box input using the patched inversion checks.

SM4th.L′Inv($\langle \boldsymbol{v} \rangle^D$)

INPUT: $D \geq 1$; Commitment $\langle \boldsymbol{v} \rangle^D$ to a 32-bit word
OUTPUT: Commitment $\langle L'^{-1}(\boldsymbol{v}) \rangle^D$
1: $\langle \boldsymbol{t} \rangle^D \leftarrow \langle \boldsymbol{v} \rangle^D \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 2) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 4) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 8) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 11) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 12) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 14) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 17) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 22) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 23) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 24) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 30) \oplus \mathsf{RotlWord}(\langle \boldsymbol{v} \rangle^D, 31)$
2: **return** $\langle \boldsymbol{t} \rangle^D$

T'AffineByte. The SM4th.T′AffineByte algorithm computes the affine input representation used by one SM4 S-box constraint. Given a committed byte vector $\langle \boldsymbol{u} \rangle$, it first applies the SM4 affine map and obtains $\langle \boldsymbol{x} \rangle = \boldsymbol{A} \cdot \langle \boldsymbol{u} \rangle + \boldsymbol{c}$. It then embeds this committed byte vector into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ using ByteCombine to obtain $\langle x \rangle$. The algorithm also computes $\langle x^2 \rangle$ by applying SquareAndCombine to the same committed byte vector. The pair $(\langle x \rangle, \langle x^2 \rangle)$ is used in the patched inversion checks for full non-anchor key words.

SM4th.T′AffineByte($\langle \boldsymbol{u} \rangle^D$)

INPUT: $D \geq 1$; Commitment $\langle \boldsymbol{u} \rangle^D$ to one S-box input byte
OUTPUT: $(\langle x \rangle^D, \langle x^2 \rangle^D)$ where $\boldsymbol{x} = \boldsymbol{A} \cdot \boldsymbol{u} + \boldsymbol{c}$
1: $\langle \boldsymbol{x} \rangle^D \leftarrow \boldsymbol{A} \cdot \langle \boldsymbol{u} \rangle^D + \boldsymbol{c}$
2: $\langle x \rangle^D \leftarrow \mathsf{ByteCombine}(\langle x_0 \rangle^D, \ldots, \langle x_7 \rangle^D)$
3: $\langle x^2 \rangle^D \leftarrow \mathsf{SquareAndCombine}(\langle x_0 \rangle^D, \ldots, \langle x_7 \rangle^D)$
4: **return** $(\langle x \rangle^D, \langle x^2 \rangle^D)$

T'InvAffineByte. The SM4th.T′InvAffineByte algorithm computes the inverse affine representation used by the patched inversion checks. Given a committed byte vector $\langle \boldsymbol{v} \rangle$ corresponding to an

SM4 S-box output, it applies the inverse of the output affine layer and obtains $\langle \boldsymbol{y} \rangle = \boldsymbol{A}^{-1} \cdot (\langle \boldsymbol{v} \rangle + \boldsymbol{c})$. It then embeds this committed byte vector into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ using ByteCombine to obtain $\langle y \rangle$. The algorithm also computes $\langle y^2 \rangle$ by applying SquareAndCombine to the same committed byte vector. The resulting pair $(\langle y \rangle, \langle y^2 \rangle)$ is used together with $(\langle x \rangle, \langle x^2 \rangle)$ from SM4th.T′AffineByte to enforce the patched inverse relation $y = I(x)$, where $I(0) = 0$ and $I(x) = x^{-1}$ for $x \neq 0$.

SM4th.T′InvAffineByte($\langle \boldsymbol{v} \rangle^D$)

INPUT: $D \geq 1$; Commitment $\langle \boldsymbol{v} \rangle^D$ to one byte, where $\boldsymbol{v} = S(\boldsymbol{u})$
OUTPUT: $(\langle y \rangle^D, \langle y^2 \rangle^D)$ where $\boldsymbol{y} = \boldsymbol{A}^{-1} \cdot (\boldsymbol{v} + \boldsymbol{c})$

1: $\langle \boldsymbol{y} \rangle^D \leftarrow \boldsymbol{A}^{-1} \cdot (\langle \boldsymbol{v} \rangle^D + \boldsymbol{c})$
2: $\langle y \rangle^D \leftarrow$ ByteCombine$(\langle y_0 \rangle^D, \ldots, \langle y_7 \rangle^D)$
3: $\langle y^2 \rangle^D \leftarrow$ SquareAndCombine$(\langle y_0 \rangle^D, \ldots, \langle y_7 \rangle^D)$
4: **return** $(\langle y \rangle^D, \langle y^2 \rangle^D)$

L. The SM4th.L algorithm applies the SM4 encryption linear layer to a committed 32-bit word. Given $\langle \boldsymbol{s} \rangle$, it computes $\langle \boldsymbol{t} \rangle = \langle \boldsymbol{s} \rangle \oplus$ RotlWord$(\langle \boldsymbol{s} \rangle, 2) \oplus$ RotlWord$(\langle \boldsymbol{s} \rangle, 10) \oplus$ RotlWord$(\langle \boldsymbol{s} \rangle, 18) \oplus$ RotlWord$(\langle \boldsymbol{s} \rangle, 24)$. This is the diffusion layer used after the four bytewise SM4 S-box outputs in each encryption round. The output is used to reconstruct the next state word as $\langle \boldsymbol{z}[i+4] \rangle = \langle \boldsymbol{z}[i] \rangle \oplus$ SM4th.L$(\langle \boldsymbol{s} \rangle)$.

SM4th.L($\langle \boldsymbol{s} \rangle^D$)

INPUT: $D \geq 1$; Commitment $\langle \boldsymbol{s} \rangle^D$ to a 32-bit word
OUTPUT: Commitment $\langle L(\boldsymbol{s}) \rangle^D$

1: $\langle \boldsymbol{t} \rangle^D \leftarrow \langle \boldsymbol{s} \rangle^D \oplus$ RotlWord$(\langle \boldsymbol{s} \rangle^D, 2) \oplus$ RotlWord$(\langle \boldsymbol{s} \rangle^D, 10) \oplus$ RotlWord$(\langle \boldsymbol{s} \rangle^D, 18) \oplus$ RotlWord$(\langle \boldsymbol{s} \rangle^D, 24)$
2: **return** $\langle \boldsymbol{t} \rangle^D$

LInv. The SM4th.LInv algorithm applies the inverse of the SM4 encryption linear layer. Given a committed 32-bit word $\langle \boldsymbol{t} \rangle$, it computes the committed preimage $\langle \boldsymbol{s} \rangle = L^{-1}(\langle \boldsymbol{t} \rangle)$ using the fixed XOR-of-rotations expression for L^{-1}. This routine is used for full non-anchor encryption state words. In that case, the witness contains the full word $\boldsymbol{z}[i+4]$, so the protocol first computes $\langle \boldsymbol{t} \rangle = \langle \boldsymbol{z}[i] \rangle \oplus \langle \boldsymbol{z}[i+4] \rangle$ and then applies SM4th.LInv to recover the claimed S-box output vector. The recovered vector is then checked against the forward S-box input using the patched inversion checks.

SM4th.LInv($\langle \boldsymbol{t} \rangle^D$)

INPUT: $D \geq 1$; Commitment $\langle \boldsymbol{t} \rangle^D$ to a 32-bit word
OUTPUT: Commitment $\langle L^{-1}(\boldsymbol{t}) \rangle^D$

1: $\langle \boldsymbol{s} \rangle^D \leftarrow \langle \boldsymbol{t} \rangle^D \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 2) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 4) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 8) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 12) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 14) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 16) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 18) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 22) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 24) \oplus$ RotlWord$(\langle \boldsymbol{t} \rangle^D, 30)$
2: **return** $\langle \boldsymbol{s} \rangle^D$

TAffineByte. The function SM4th.TAffineByte is an alias for SM4th.T′AffineByte and uses the same input and output types. Its output pair is used in the patched inversion checks for full non-anchor encryption rounds.

TInvAffineByte. The function SM4th.TInvAffineByte is an alias for SM4th.T′InvAffineByte and uses the same input and output types. The output pairs of these aliases are used together in the patched inversion checks for full non-anchor encryption rounds.

6.2 Witness Extension

This section gives two witness extension algorithms for SM4th and SM4th-EM, which are named as SM4th.ExtendWitness and SM4th.EM.ExtendWitness, respectively. For SM4th, the algorithm takes the SM4 key $\boldsymbol{k}$, the SM4 plaintext $\boldsymbol{in}$, and the instance parameters param, returning an extended witness $\boldsymbol{w} \in \{0,1\}^{\ell}$ for later use. For SM4th-EM, whose OWF is $\mathrm{SM4}_{\boldsymbol{x}}(\boldsymbol{k}) \oplus \boldsymbol{k}$, the algorithm takes the secret input block $\boldsymbol{k}$ and the public SM4 key $\boldsymbol{x}$. The witness length ℓ is a bit length determined by the SM4 trace and is independent of the proof field size $\lambda_{\mathbb{F}}$. A word W is stored as $\mathrm{BE32}(W)$; byte $j \in [0,4)$ contributes its bits from least to most significant to witness positions $8j, \ldots, 8j+7$. Norm nibbles follow increasing anchor-word index and then increasing byte index j. Their bits are (y_0, y_1, y_2, y_3), with the even-indexed nibble in the low half of its byte. Table 14 gives the resulting bit offsets.

Table 14: SM4th witness layout, where offsets are in bits

Variant	Interval	Contents
Non-EM	$[0, 128)$	Initial whitened key words $\bar{\boldsymbol{k}}[0], \ldots, \bar{\boldsymbol{k}}[3]$
	$[128, 256)$	Key-anchor norm bits
	$[256, 1024)$	Remaining expanded key words, in increasing order
	$[1024, 1136)$	Encryption-anchor norm bits
	$[1136, 1808)$	Remaining words among $\boldsymbol{z}[4], \ldots, \boldsymbol{z}[31]$
EM	$[0, 128)$	Secret input block $\boldsymbol{k}$
	$[128, 1024)$	$\boldsymbol{z}[4], \ldots, \boldsymbol{z}[31]$

Choice of witness representation. The non-EM variant of SM4th proves knowledge of an SM4 secret key for the relation $\mathrm{SM4}_{\boldsymbol{k}}(\boldsymbol{in}) = \boldsymbol{out}$. Therefore, the zero-knowledge circuit must cover both the SM4 key expansion and the SM4 encryption routine. If all intermediate key words and encryption state words were stored as full bit witnesses, the resulting signature would be larger. For this reason, the non-EM construction replaces selected key-expansion and encryption state words by compact norm witnesses. Each anchor word stores only four 4-bit norm values, one for each S-box input byte, and the omitted 32-bit word is reconstructed virtually from the norm witness and the neighboring state words inside the constraint system. This reduces the witness length and the amount of data opened in the VOLEitH proof, at the cost of using degree-3 norm constraints and a more involved conjugate representation.

The role of the norm witness is similar in spirit to the norm-based compression used in FAEST for AES, but the construction is SM4-specific. Both the SM4 key expansion and the SM4 encryption routine update one 32-bit word at a time using a four-word feedback relation. An omitted anchor word must still be reconstructed from neighboring words, the corresponding S-box input, and the output of the SM4 linear layer. Consequently, anchor words cannot be placed too densely: enough adjacent full words must remain available to define later S-box inputs and to support the patched inversion checks on non-anchor rounds. In addition, each SM4 round contains only four byte S-boxes before the word-level linear layer L or L', so replacing one anchor word saves only one 32-bit word while still requiring four 4-bit norm witnesses and several conjugate-based reconstruction constraints.

The EM variants prove the relation $\mathrm{SM4}_{\boldsymbol{x}}(\boldsymbol{k}) \oplus \boldsymbol{k} = \boldsymbol{y}$, where the SM4 key $\boldsymbol{x}$ is public and the secret witness is the input block $\boldsymbol{k}$. The round keys are derived from public data and do not need to be stored as private witness bits. Thus, the private trace consists only of the SM4 input block and the intermediate encryption states. For this smaller witness, the ordinary patched-inversion constraints are preferable: they use direct bit commitments to the relevant intermediate states and give degree-2 checks over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. With the same seven encryption anchors $\boldsymbol{z}[5], \boldsymbol{z}[9], \ldots, \boldsymbol{z}[29]$, norm witnesses reduce ℓ from 1024 to 912 bits but increase d from 2 to 3. With all other parameters fixed, $\hat{\ell} = \ell + d\lambda_{\mathbb{F}} + B$ increases by $\lambda_{\mathbb{F}} - 112$ bits. Thus, Equation (6) gives a signature-length

increase of $\tau(\lambda_{\mathbb{F}} - 112)/8$ bytes: 22, 32, 84, and 126 for the s-lo, f-lo, s-ti, and f-ti parameter sets of SM4th-EM, respectively.

Algorithms. The extended witness of SM4th.ExtendWitness consists of three components, corresponding to the initial phase, the key expansion phase and the encryption phase. It uses norm witnesses to replace some full state words. We call such compressed words anchor words.

- **Initial key bits**: the first 4 words obtained from the SM4 key expansion, namely $\bar{\boldsymbol{k}}[0\,..\,4)$, are stored as full 32-bit words. This contributes $32 \cdot 4$ bits.
- **Norm key bits**: instead of storing all expanded key words as full 32-bit words, we replace the words $\bar{\boldsymbol{k}}[4], \bar{\boldsymbol{k}}[8], \ldots, \bar{\boldsymbol{k}}[32]$ by norm witnesses, which we call anchor words. For each such anchor word, the algorithm stores one 4-bit value for each of the four S-box input bytes. Since there are 8 anchor words, this contributes $4 \cdot 4 \cdot 8$ bits, *i.e.*, $\boldsymbol{w}_{\mathsf{ke},N}$ in SM4th.ExtendWitness.
- **Full expanded key bits**: the remaining expanded key words are stored as ordinary full 32-bit words. These are the R expanded key words excluding the 8 norm anchor words. Hence this contributes $32 \cdot (R - 8)$ bits, *i.e.*, $\boldsymbol{w}_{\mathsf{ke},F}$ in SM4th.ExtendWitness.
- **Norm encryption bits**: for the encryption trace, we similarly replace the anchor words $\boldsymbol{z}[5], \boldsymbol{z}[9], \ldots, \boldsymbol{z}[29]$ by norm witnesses. For each such anchor state, the algorithm stores one 4-bit value for each of the four S-box input bytes. Since there are 7 encryption anchor states, this contributes $4 \cdot 4 \cdot 7$ bits, *i.e.*, $\boldsymbol{w}_{\mathsf{enc},N}$ in SM4th.ExtendWitness.
- **Full encryption bits**: the remaining private intermediate encryption states are stored as full 32-bit words. In the Non-EM SM4th setting, the private encryption witness consists of $\boldsymbol{z}[4\,..\,32)$, excluding the 7 norm anchor states. Therefore, this contributes $32 \cdot ((R-4)-7)$ bits, *i.e.*, $\boldsymbol{w}_{\mathsf{enc},F}$ in SM4th.ExtendWitness.

The extended witness of SM4th.EM.ExtendWitness consists of two components, corresponding to the initial input block and the encryption phase.

- **Initial input bits**: the secret EM input block $\boldsymbol{k}$, viewed as four 32-bit words.
- **Encryption bits**: the intermediate encryption states $\boldsymbol{z}[4\,..\,32)$, each of size 32 bits, corresponding to the outputs of the round function. This contributes $32 \cdot (R - 4)$ bits.

Therefore, the total size of the extended witness for EM variants is $32 \cdot 32 = 1024$ bits.

Commitment degrees. The initial, full-witness, and public words have degree 1, while reconstructed anchor words have degree 2. Write $\langle \bar{\boldsymbol{k}}[i] \rangle^{d_i}$ and $\langle \boldsymbol{z}[a] \rangle^{e_a}$, where $d_i = 2$ for key anchors and $e_a = 2$ for encryption anchors, and 1 otherwise. Linear maps preserve the degree, and word additions lift their operands to the largest input degree using Lift from Section 5.2. In norm branches, $D_x = 1$. For every non-anchor key word, the affine input and inverse-affine output degrees are $(D_x, D_y) = (2, 1)$. For an encryption round producing word a, they are $(1, 2)$ when $a \equiv 1 \pmod 4$, $(2, 1)$ for the other words $a \leq 33$, and $(1, 1)$ for $a \in \{34, 35\}$. Every constraint is lifted to the proof degree d before it is appended to the constraint vector.

6.3 Deriving Constraints for the Key Expansion

This section gives the SM4th.KeyExpCstrnts algorithm, which derives the constraint values required for the QuickSilver proof for the SM4 circuit corresponding to the key expansion routine. Note that this only applies to the non-EM variant.

The SM4th.KeyExpCstrnts algorithm checks the key expansion routine of the norm-based SM4th witness. The key-expansion witness is parsed according to the layout $K_0, \ldots, K_3$, followed by the norm bits for the anchor words, followed by the full bits of the non-anchor words. The set $\mathcal{A}_{\mathsf{ke}} = \{4, 8, 12, 16, 20, 24, 28, 32\}$ specifies the key-expansion words that are represented by norm witnesses instead of full 32-bit words. The algorithm first loads the initial four key words from the witness, and then reconstructs the remaining key words sequentially. For each word index i, it forms the SM4 key-expansion S-box input $\boldsymbol{u} = \bar{\boldsymbol{k}}[i-3] \oplus \bar{\boldsymbol{k}}[i-2] \oplus \bar{\boldsymbol{k}}[i-1] \oplus CK_{i-4}$. If i is an anchor index, the algorithm reads only a 4-bit norm witness for each byte of $\boldsymbol{u}$. For each byte, SM4th.AffineToConjugates computes the conjugates of the embedded value x corresponding to

SM4th.ExtendWitness($\boldsymbol{k}, \boldsymbol{in}$; param)

INPUT: $\boldsymbol{k} \in \{0,1\}^{128}$, $\boldsymbol{in} = \boldsymbol{in}[0, ..3] \in (\{0,1\}^{32})^4$
OUTPUT: $\boldsymbol{w} \in \{0,1\}^{\ell}$: extended witness

// Initialize empty extended witness arrays, where $\boldsymbol{w}_N$ stores norm bits and $\boldsymbol{w}_F$ stores full bits
new $\boldsymbol{w} \in \{0,1\}^{\leq \ell}$
new $\boldsymbol{w}_{\mathsf{ke},N} \in \{0,1\}^{\leq 128}$; **new** $\boldsymbol{w}_{\mathsf{ke},F} \in \{0,1\}^{\leq 768}$
new $\boldsymbol{w}_{\mathsf{enc},N} \in \{0,1\}^{\leq 112}$; **new** $\boldsymbol{w}_{\mathsf{enc},F} \in \{0,1\}^{\leq 672}$
// Flags for using norms in the key expansion and encryption routines, respectively
$\mathcal{A}_{\mathsf{ke}} \leftarrow \{4, 8, 12, 16, 20, 24, 28, 32\}$
$\mathcal{A}_{\mathsf{enc}} \leftarrow \{5, 9, 13, 17, 21, 25, 29\}$
// Store round keys as array of 32-bit words
new $\bar{\boldsymbol{k}} \in [\{0,1\}^{32}; 4 + R]$
$\bar{\boldsymbol{k}} \leftarrow$ SM4.KeyExpansion($\boldsymbol{k}$, param)
// Key expansion routine
for $i \in [0 .. 4)$ **do**
    $\boldsymbol{w} \leftarrow \boldsymbol{w} \| \bar{\boldsymbol{k}}[i]$
end for
for $i \in [4 .. R + 4)$ **do**
    if $i \in \mathcal{A}_{\mathsf{ke}}$ **then**
        for $j \in [0 .. 4)$ **do**
            $\boldsymbol{u} \leftarrow \bar{\boldsymbol{k}}[i-3][j] \oplus \bar{\boldsymbol{k}}[i-2][j] \oplus \bar{\boldsymbol{k}}[i-1][j] \oplus CK_{i-4}[j]$
            $\boldsymbol{w}_{\mathsf{ke},N} \leftarrow \boldsymbol{w}_{\mathsf{ke},N} \| \text{SM4th.InvNorm}(\boldsymbol{A} \cdot \boldsymbol{u} + \boldsymbol{c})$
        end for
    else
        $\boldsymbol{w}_{\mathsf{ke},F} \leftarrow \boldsymbol{w}_{\mathsf{ke},F} \| \bar{\boldsymbol{k}}[i]$
    end if
end for
$\boldsymbol{w} \leftarrow \boldsymbol{w} \| \boldsymbol{w}_{\mathsf{ke},N} \| \boldsymbol{w}_{\mathsf{ke},F}$
// Encryption routine
new $\mathsf{tmp} \in \{0,1\}^{32}$
new $\boldsymbol{z} \in [\{0,1\}^{32}; 4 + R]$
for $i \in [0 .. 4)$ **do**
    $\boldsymbol{z}[i] \leftarrow \boldsymbol{in}[i]$
end for
for $i \in [0 .. R)$ **do**
    $\mathsf{tmp} \leftarrow \boldsymbol{z}[i+1] \oplus \boldsymbol{z}[i+2] \oplus \boldsymbol{z}[i+3] \oplus \bar{\boldsymbol{k}}[i+4]$
    $\mathsf{tmp} \leftarrow \text{SubWord}(\mathsf{tmp})$
    $\mathsf{tmp} \leftarrow \mathsf{tmp} \oplus \text{RotlWord}(\mathsf{tmp}, 2) \oplus \text{RotlWord}(\mathsf{tmp}, 10) \oplus \text{RotlWord}(\mathsf{tmp}, 18) \oplus \text{RotlWord}(\mathsf{tmp}, 24)$
    $\boldsymbol{z}[i+4] \leftarrow \boldsymbol{z}[i] \oplus \mathsf{tmp}$
end for
for $i \in [4 .. R)$ **do**
    if $i \in \mathcal{A}_{\mathsf{enc}}$ **then**
        for $j \in [0 .. 4)$ **do**
            $\boldsymbol{u} \leftarrow \boldsymbol{z}[i-3][j] \oplus \boldsymbol{z}[i-2][j] \oplus \boldsymbol{z}[i-1][j] \oplus \bar{\boldsymbol{k}}[i][j]$
            $\boldsymbol{w}_{\mathsf{enc},N} \leftarrow \boldsymbol{w}_{\mathsf{enc},N} \| \text{SM4th.InvNorm}(\boldsymbol{A} \cdot \boldsymbol{u} + \boldsymbol{c})$
        end for
    else
        $\boldsymbol{w}_{\mathsf{enc},F} \leftarrow \boldsymbol{w}_{\mathsf{enc},F} \| \boldsymbol{z}[i]$
    end if
end for
$\boldsymbol{w} \leftarrow \boldsymbol{w} \| \boldsymbol{w}_{\mathsf{enc},N} \| \boldsymbol{w}_{\mathsf{enc},F}$
return $\boldsymbol{w} \in \{0,1\}^{\ell}$

SM4th.EM.ExtendWitness($\boldsymbol{k}, \boldsymbol{x}$; param)

INPUT: $\boldsymbol{k} = \boldsymbol{k}[0, ..3] \in (\{0,1\}^{32})^4$, $\boldsymbol{x} \in \{0,1\}^{128}$
OUTPUT: $\boldsymbol{w} \in \{0,1\}^{\ell}$: extended witness

// Initialize empty extended witness array, eventually of length ℓ
new $\boldsymbol{w} \in \{0,1\}^{\leq \ell}$
// In EM mode, $\boldsymbol{x}$ is the public SM4 key and $\boldsymbol{k}$ is the secret input block
new $\bar{\boldsymbol{x}} \in [\{0,1\}^{32}; 4+R]$
$\bar{\boldsymbol{x}} \leftarrow$ SM4.KeyExpansion($\boldsymbol{x}$, param)
$\boldsymbol{w} \leftarrow \boldsymbol{w} \| \boldsymbol{k}$
// Encryption routine and saving witness bits
new tmp $\in \{0,1\}^{32}$
new $\boldsymbol{z} \in [\{0,1\}^{32}; 4+R]$
for $i \in [0..4)$ **do**
    $\boldsymbol{z}[i] \leftarrow \boldsymbol{k}[i]$
end for
for $i \in [0..R)$ **do**
    tmp $\leftarrow \boldsymbol{z}[i+1] \oplus \boldsymbol{z}[i+2] \oplus \boldsymbol{z}[i+3] \oplus \bar{\boldsymbol{x}}[i+4]$
    tmp $\leftarrow$ SubWord(tmp)
    tmp $\leftarrow$ tmp $\oplus$ RotlWord(tmp, 2) $\oplus$ RotlWord(tmp, 10) $\oplus$ RotlWord(tmp, 18) $\oplus$ RotlWord(tmp, 24)
    $\boldsymbol{z}[i+4] \leftarrow \boldsymbol{z}[i] \oplus$ tmp
end for
for $i \in [4..R)$ **do**
    $\boldsymbol{w} \leftarrow \boldsymbol{w} \| \boldsymbol{z}[i]$
end for
return $\boldsymbol{w} \in \{0,1\}^{\ell}$

$\boldsymbol{A}\boldsymbol{u}+\boldsymbol{c}$, and SM4th.InvNormToConjugates computes the conjugates of the committed norm value h. These embedded values live in the proof field $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$, while the byte arithmetic itself is still the SM4 byte-field arithmetic. The norm correctness constraint $\langle o_N \rangle^3_{\lambda_\mathbb{F}} = \langle h \rangle_{\lambda_\mathbb{F}} \cdot \langle x^{16} \rangle_{\lambda_\mathbb{F}} \cdot \langle x^2 \rangle_{\lambda_\mathbb{F}} + \mathsf{Lift}_3(\langle x \rangle_{\lambda_\mathbb{F}})$ is appended to the output constraint vector. For $x \neq 0$, the constraint gives $h = x^{-17}$, while for $x = 0$, the reconstructed inverse byte is zero. The algorithm then reconstructs the conjugates of the inverse value by setting $\langle q^{2^r} \rangle^2_{\lambda_\mathbb{F}} = \langle x^{2^{(r+4) \bmod 8}} \rangle_{\lambda_\mathbb{F}} \cdot \langle h^{2^{r \bmod 4}} \rangle_{\lambda_\mathbb{F}}$ for $r \in [0..8)$. Applying SM4th.AffineOut to these conjugates gives the committed conjugates of the S-box output byte. The algorithm applies ConjToBits to each output byte and applies the key-schedule linear layer L' to the resulting committed word. Thus the anchor word $\bar{\boldsymbol{k}}[i]$ is reconstructed virtually as $\bar{\boldsymbol{k}}[i-4] \oplus L'(\boldsymbol{s})$, without storing its full 32 bits in the witness. If i is not an anchor index, the algorithm reads the full word $\bar{\boldsymbol{k}}[i]$ from the witness. It then applies SM4th.L′Inv to $\bar{\boldsymbol{k}}[i-4] \oplus \bar{\boldsymbol{k}}[i]$ to recover the claimed S-box output $\boldsymbol{v}$. For each byte, SM4th.T′AffineByte computes the affine input representation $(\langle x \rangle^{D_x}_{\lambda_\mathbb{F}}, \langle x^2 \rangle^{D_x}_{\lambda_\mathbb{F}})$, while SM4th.T′InvAffineByte computes the inverse affine output representation $(\langle y \rangle^{D_y}_{\lambda_\mathbb{F}}, \langle y^2 \rangle^{D_y}_{\lambda_\mathbb{F}})$. The patched inversion checks for $x^2y + x = 0$ and $xy^2 + y = 0$ lift the linear summand to degree $D_x + D_y$, then lift the resulting constraint to degree 3 and append it to the constraint vector in that order. All constraint values committed in the returned vector $\langle \boldsymbol{o} \rangle^3_{\lambda_\mathbb{F}}$ must be zero for the committed key-expansion witness to be valid. Equivalently, the coefficient of X^3 in each constraint polynomial must vanish.

6.4 Deriving Constraints for the Encryption

This section gives two algorithms for deriving the OWF constraints in the encryption routine, *i.e.*, SM4th.EncCstrnts for SM4th and SM4th.EM.EncCstrnts for SM4th-EM.

The SM4th.EncCstrnts algorithm checks the encryption routine of the norm-based non-EM SM4th witness. The encryption witness is parsed according to the layout consisting of norm bits for the anchor state words, followed by the full bits of the non-anchor state words. The set $\mathcal{A}_{\mathsf{enc}} = \{5, 9, 13, 17, 21, 25, 29\}$ specifies the encryption state words that are represented by norm witnesses instead of full 32-bit words. The algorithm first loads the public input words as $\boldsymbol{z}[0], \ldots, \boldsymbol{z}[3]$ and the

SM4th.KeyExpCstrnts($\langle \boldsymbol{w} \rangle$; param)

INPUT: Commitment to the key-expansion part of the extended witness $\boldsymbol{w} \in \{0,1\}^{\ell_{\mathsf{ke}}}$
OUTPUT: Round-key commitments $(\langle \bar{\boldsymbol{k}}[j] \rangle^{d_j})_{j=0}^{R+3}$ and the degree-3 constraint vector $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3$

// Extended witness layout: $K_0, \ldots, K_3$ full bits, then norm bits, then full non-anchor bits
$\mathcal{A}_{\mathsf{ke}} \leftarrow \{4, 8, 12, 16, 20, 24, 28, 32\}$
$\langle \boldsymbol{w}_{\mathsf{ke},N} \rangle \leftarrow \langle \boldsymbol{w}[128 \,..\, 256) \rangle$
$\langle \boldsymbol{w}_{\mathsf{ke},F} \rangle \leftarrow \langle \boldsymbol{w}[256 \,..\, \ell_{\mathsf{ke}}) \rangle$
new $\langle \bar{\boldsymbol{k}}[0] \rangle^{d_0}, \ldots, \langle \bar{\boldsymbol{k}}[R+3] \rangle^{d_{R+3}}$
new $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3$
$p_N \leftarrow 0$
$p_F \leftarrow 0$
for $i \in [0 \,..\, 4)$ **do**
    $\langle \bar{\boldsymbol{k}}[i] \rangle^{d_i} \leftarrow \langle \boldsymbol{w}[32i \,..\, 32(i+1)) \rangle$
end for
for $i \in [4 \,..\, R+4)$ **do**
    $\langle \boldsymbol{u} \rangle^{D_x} \leftarrow \langle \bar{\boldsymbol{k}}[i-3] \rangle^{d_{i-3}} \oplus \langle \bar{\boldsymbol{k}}[i-2] \rangle^{d_{i-2}} \oplus \langle \bar{\boldsymbol{k}}[i-1] \rangle^{d_{i-1}} \oplus CK_{i-4}$
    if $i \in \mathcal{A}_{\mathsf{ke}}$ **then**
        new $(\langle s_j \rangle_{\lambda_{\mathbb{F}}}^2, \langle s_j^2 \rangle_{\lambda_{\mathbb{F}}}^2, \ldots, \langle s_j^{128} \rangle_{\lambda_{\mathbb{F}}}^2)$ for $j \in [0 \,..\, 4)$
        for $j \in [0 \,..\, 4)$ **do**
            $\langle \boldsymbol{n} \rangle \leftarrow \langle \boldsymbol{w}_{\mathsf{ke},N}[4p_N \,..\, 4p_N + 4) \rangle$
            $p_N \leftarrow p_N + 1$
            $(\langle x \rangle_{\lambda_{\mathbb{F}}}, \langle x^2 \rangle_{\lambda_{\mathbb{F}}}, \ldots, \langle x^{128} \rangle_{\lambda_{\mathbb{F}}}) \leftarrow$ SM4th.AffineToConjugates$(\langle \boldsymbol{u}[j] \rangle^{D_x})$
            $(\langle h \rangle_{\lambda_{\mathbb{F}}}, \langle h^2 \rangle_{\lambda_{\mathbb{F}}}, \langle h^4 \rangle_{\lambda_{\mathbb{F}}}, \langle h^8 \rangle_{\lambda_{\mathbb{F}}}) \leftarrow$ SM4th.InvNormToConjugates$(\langle \boldsymbol{n} \rangle)$
            $\langle o_N \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \langle h \rangle_{\lambda_{\mathbb{F}}} \cdot \langle x^{16} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle x^2 \rangle_{\lambda_{\mathbb{F}}} + \mathsf{Lift}_3(\langle x \rangle_{\lambda_{\mathbb{F}}})$
            $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \| \langle o_N \rangle_{\lambda_{\mathbb{F}}}^3$
            for $r \in [0 \,..\, 8)$ **do**
                $\langle q^{2^r} \rangle_{\lambda_{\mathbb{F}}}^2 \leftarrow \langle x^{2^{(r+4) \bmod 8}} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle h^{2^r \bmod 4} \rangle_{\lambda_{\mathbb{F}}}$
            end for
            $(\langle s_j \rangle_{\lambda_{\mathbb{F}}}^2, \langle s_j^2 \rangle_{\lambda_{\mathbb{F}}}^2, \ldots, \langle s_j^{128} \rangle_{\lambda_{\mathbb{F}}}^2) \leftarrow$ SM4th.AffineOut$(\langle q \rangle_{\lambda_{\mathbb{F}}}^2, \langle q^2 \rangle_{\lambda_{\mathbb{F}}}^2, \ldots, \langle q^{128} \rangle_{\lambda_{\mathbb{F}}}^2)$
        end for
        $\langle \boldsymbol{t} \rangle^2 \leftarrow$ SM4th.L′(ConjToBits$((\langle s_j^{2^r} \rangle_{\lambda_{\mathbb{F}}}^2)_{r=0}^7)_{j=0}^3$)
        $\langle \bar{\boldsymbol{k}}[i] \rangle^2 \leftarrow \langle \bar{\boldsymbol{k}}[i-4] \rangle^{d_{i-4}} \oplus \langle \boldsymbol{t} \rangle^2$
    else
        $\langle \bar{\boldsymbol{k}}[i] \rangle^{d_i} \leftarrow \langle \boldsymbol{w}_{\mathsf{ke},F}[32p_F \,..\, 32(p_F+1)) \rangle$
        $p_F \leftarrow p_F + 1$
        $\langle \boldsymbol{v} \rangle^{D_y} \leftarrow$ SM4th.L′Inv$(\langle \bar{\boldsymbol{k}}[i-4] \rangle^{d_{i-4}} \oplus \langle \bar{\boldsymbol{k}}[i] \rangle^{d_i})$
        for $j \in [0 \,..\, 4)$ **do**
            $(\langle x \rangle_{\lambda_{\mathbb{F}}}^{D_x}, \langle x^2 \rangle_{\lambda_{\mathbb{F}}}^{D_x}) \leftarrow$ SM4th.T′AffineByte$(\langle \boldsymbol{u}[j] \rangle^{D_x})$
            $(\langle y \rangle_{\lambda_{\mathbb{F}}}^{D_y}, \langle y^2 \rangle_{\lambda_{\mathbb{F}}}^{D_y}) \leftarrow$ SM4th.T′InvAffineByte$(\langle \boldsymbol{v}[j] \rangle^{D_y})$
            $D \leftarrow D_x + D_y$, where D_x, D_y are the degrees of $\langle x \rangle^{D_x}, \langle y \rangle^{D_y}$
            $\langle o_0 \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \mathsf{Lift}_3(\langle x^2 \rangle_{\lambda_{\mathbb{F}}}^{D_x} \cdot \langle y \rangle_{\lambda_{\mathbb{F}}}^{D_y} + \mathsf{Lift}_D(\langle x \rangle_{\lambda_{\mathbb{F}}}^{D_x}))$
            $\langle o_1 \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \mathsf{Lift}_3(\langle x \rangle_{\lambda_{\mathbb{F}}}^{D_x} \cdot \langle y^2 \rangle_{\lambda_{\mathbb{F}}}^{D_y} + \mathsf{Lift}_D(\langle y \rangle_{\lambda_{\mathbb{F}}}^{D_y}))$
            $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \| \langle o_0 \rangle_{\lambda_{\mathbb{F}}}^3$
            $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \| \langle o_1 \rangle_{\lambda_{\mathbb{F}}}^3$
        end for
    end if
end for
return $((\langle \bar{\boldsymbol{k}}[j] \rangle^{d_j})_{j=0}^{R+3}, \langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3)$

public output words as $\boldsymbol{z}[R], \ldots, \boldsymbol{z}[R+3]$ in the SM4 final-output order. For each round i, it forms the SM4 encryption S-box input $\boldsymbol{u} = \boldsymbol{z}[i+1] \oplus \boldsymbol{z}[i+2] \oplus \boldsymbol{z}[i+3] \oplus \bar{\boldsymbol{k}}[i+4]$. If the target state index $a = i+4$ is an anchor index, the algorithm reads only a 4-bit norm witness for each byte of $\boldsymbol{u}$. For each byte, SM4th.AffineToConjugates computes the conjugates of the embedded value x corresponding to $\boldsymbol{A}\boldsymbol{u}+\boldsymbol{c}$, and SM4th.InvNormToConjugates computes the conjugates of the committed norm value h. These embedded values live in the proof field $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$, while the byte arithmetic itself is still the SM4 byte-field arithmetic. The norm correctness constraint $\langle o_N \rangle^3_{\lambda_\mathbb{F}} = \langle h \rangle_{\lambda_\mathbb{F}} \cdot \langle x^{16} \rangle_{\lambda_\mathbb{F}} \cdot \langle x^2 \rangle_{\lambda_\mathbb{F}} + \mathsf{Lift}_3(\langle x \rangle_{\lambda_\mathbb{F}})$ is appended to the output constraint vector. For $x \neq 0$, the constraint gives $h = x^{-17}$, while for $x = 0$, the reconstructed inverse byte is zero. The algorithm then reconstructs the conjugates of the inverse value by setting $\langle q^{2^r} \rangle^2_{\lambda_\mathbb{F}} = \langle x^{2^{(r+4) \bmod 8}} \rangle_{\lambda_\mathbb{F}} \cdot \langle h^{2^{r \bmod 4}} \rangle_{\lambda_\mathbb{F}}$ for $r \in [0\,..\,8)$. Applying SM4th.AffineOut to these conjugates gives the committed conjugates of the S-box output byte. The algorithm applies ConjToBits to each output byte and applies the SM4 encryption linear layer L to the resulting committed word. Thus, the anchor state word $\boldsymbol{z}[a]$ is reconstructed virtually as $\boldsymbol{z}[i] \oplus L(\boldsymbol{s})$, without storing its full 32 bits in the witness. If a is not an anchor index, the algorithm uses the full state word $\boldsymbol{z}[a]$. For internal state words $a < R$, this word is read from the full encryption witness, while for the final boundary words it has already been loaded from the public output. The algorithm then applies SM4th.LInv to $\boldsymbol{z}[i] \oplus \boldsymbol{z}[a]$ to recover the claimed S-box output vector $\boldsymbol{v}$. For each byte, SM4th.TAffineByte computes the affine input representation $(\langle x \rangle^{D_x}_{\lambda_\mathbb{F}}, \langle x^2 \rangle^{D_x}_{\lambda_\mathbb{F}})$, while SM4th.TInvAffineByte computes the inverse affine output representation $(\langle y \rangle^{D_y}_{\lambda_\mathbb{F}}, \langle y^2 \rangle^{D_y}_{\lambda_\mathbb{F}})$. The patched inversion checks for $x^2y+x = 0$ and $xy^2+y = 0$ lift the linear summand to degree D_x+D_y, then lift the resulting constraint to degree 3 and append it to the constraint vector in that order. All constraint values committed in the returned vector $\langle \boldsymbol{o} \rangle^3_{\lambda_\mathbb{F}}$ must be zero for the committed encryption witness to be valid. Equivalently, the coefficient of X^3 in each constraint polynomial must vanish.

The SM4th.EM.EncCstrnts algorithm checks the encryption routine of the SM4th-EM witness. It derives the commitments to the inputs and outputs of the patched inversion function in S-boxes via the T.Affine and T.InvAffine algorithms. The witness words are first expanded by concatenating the committed SM4 input boundary words, the committed intermediate state words, and the committed SM4 output boundary words before invoking T.Affine and T.InvAffine. It then extracts the commitments to each 8 bits of $\langle \boldsymbol{x} \rangle$ and $\langle \boldsymbol{y} \rangle$ and applies ByteCombine and SquareAndCombine functions to them, obtaining the combined SM4-compatible commitment in $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ and the combined commitment to its square in $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$. The square can be obtained by the $\mathbb{F}_2$-linearity of the squaring map. For example, let the commitments to the combined input be $\langle \hat{x} \rangle \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$ and its combined square be $\langle \hat{x}_{\mathsf{sq}} \rangle \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$. Let the commitments to the output be $\langle \hat{y} \rangle \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$ and its square be $\langle \hat{y}_{\mathsf{sq}} \rangle \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$. Finally, the algorithm generates two degree-2 VOLE commitments, of the form

$$\langle \hat{o}_1 \rangle^2 = \langle \hat{x}_{\mathsf{sq}} \rangle \cdot \langle \hat{y} \rangle + \mathsf{Lift}_2(\langle \hat{x} \rangle), \quad \langle \hat{o}_2 \rangle^2 = \langle \hat{x} \rangle \cdot \langle \hat{y}_{\mathsf{sq}} \rangle + \mathsf{Lift}_2(\langle \hat{y} \rangle).$$

By the correctness, $\langle \hat{o}_1 \rangle^2$ and $\langle \hat{o}_2 \rangle^2$ should be commitments to zero. This helps to check

$$\hat{x}^2 \cdot \hat{y} + \hat{x} = 0, \quad \hat{x} \cdot \hat{y}^2 + \hat{y} = 0.$$

6.5 Constraints for the OWF

This section describes the remaining steps needed to prove with the QuickSilver zero-knowledge proof system that $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$ is consistent with the witness $\boldsymbol{w}$ under the SM4-based one-way function. There are three algorithms:

1. the SM4th.OWFConstraints algorithm that combines the secret-domain restriction, key expansion, and encryption constraints,
2. the signer-side procedure SM4th.OWFProve; and
3. the verifier-side procedure SM4th.OWFVerify.

SM4th.OWFConstraints($\langle \boldsymbol{w} \rangle, \mathsf{pk}; \mathsf{param}$). This is the main algorithm that combines the secret-domain restriction with the constraints from the key schedule and encryption routines. Concretely:

SM4th.EncCstrnts($\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, (\langle \bar{\boldsymbol{k}}[j] \rangle^{d_j})_{j=4}^{R+3}, \langle \boldsymbol{w} \rangle$; param)

INPUT: Commitment to the encryption part of the extended witness $\boldsymbol{w} \in \{0,1\}^{\ell_{\mathsf{enc}}}$, committed round-key expressions $\langle \bar{\boldsymbol{k}}[4] \rangle^{d_4}, \ldots, \langle \bar{\boldsymbol{k}}[R+3] \rangle^{d_{R+3}}$, and public SM4 input/output words $\boldsymbol{in}, \boldsymbol{out} \in (\{0,1\}^{32})^4$
OUTPUT: $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3$: constraint vector for degree-3 checks

// Encryption witness layout: norm bits first, then full non-anchor state bits
$\mathcal{A}_{\mathsf{enc}} \leftarrow \{5, 9, 13, 17, 21, 25, 29\}$
$\langle \boldsymbol{w}_{\mathsf{enc},N} \rangle \leftarrow \langle \boldsymbol{w}[0 \,..\, 112) \rangle$
$\langle \boldsymbol{w}_{\mathsf{enc},F} \rangle \leftarrow \langle \boldsymbol{w}[112 \,..\, \ell_{\mathsf{enc}}) \rangle$
new $\langle \boldsymbol{z}[0] \rangle^{e_0}, \ldots, \langle \boldsymbol{z}[R+3] \rangle^{e_{R+3}}$
new $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3$
$p_N \leftarrow 0$
$p_F \leftarrow 0$
// Load public input and output boundary words
for $i \in [0 \,..\, 4)$ **do**
  $\langle \boldsymbol{z}[i] \rangle^{e_i} \leftarrow \langle \boldsymbol{in}[i] \rangle$
  $\langle \boldsymbol{z}[R+i] \rangle^{e_{R+i}} \leftarrow \langle \boldsymbol{out}[3-i] \rangle$
end for
for $i \in [0 \,..\, R)$ **do**
  $a \leftarrow i + 4$
  $\langle \boldsymbol{u} \rangle^{D_x} \leftarrow \langle \boldsymbol{z}[i+1] \rangle^{e_{i+1}} \oplus \langle \boldsymbol{z}[i+2] \rangle^{e_{i+2}} \oplus \langle \boldsymbol{z}[i+3] \rangle^{e_{i+3}} \oplus \langle \bar{\boldsymbol{k}}[i+4] \rangle^{d_{i+4}}$
  if $a \in \mathcal{A}_{\mathsf{enc}}$ **then**
    new $(\langle s_j \rangle_{\lambda_{\mathbb{F}}}^2, \langle s_j^2 \rangle_{\lambda_{\mathbb{F}}}^2, \ldots, \langle s_j^{128} \rangle_{\lambda_{\mathbb{F}}}^2)$ for $j \in [0 \,..\, 4)$
    for $j \in [0 \,..\, 4)$ **do**
      $\langle \boldsymbol{n} \rangle \leftarrow \langle \boldsymbol{w}_{\mathsf{enc},N}[4p_N \,..\, 4p_N + 4) \rangle$
      $p_N \leftarrow p_N + 1$
      $(\langle x \rangle_{\lambda_{\mathbb{F}}}, \langle x^2 \rangle_{\lambda_{\mathbb{F}}}, \ldots, \langle x^{128} \rangle_{\lambda_{\mathbb{F}}}) \leftarrow$ SM4th.AffineToConjugates$(\langle \boldsymbol{u}[j] \rangle^{D_x})$
      $(\langle h \rangle_{\lambda_{\mathbb{F}}}, \langle h^2 \rangle_{\lambda_{\mathbb{F}}}, \langle h^4 \rangle_{\lambda_{\mathbb{F}}}, \langle h^8 \rangle_{\lambda_{\mathbb{F}}}) \leftarrow$ SM4th.InvNormToConjugates$(\langle \boldsymbol{n} \rangle)$
      $\langle o_N \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \langle h \rangle_{\lambda_{\mathbb{F}}} \cdot \langle x^{16} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle x^2 \rangle_{\lambda_{\mathbb{F}}} + \mathsf{Lift}_3(\langle x \rangle_{\lambda_{\mathbb{F}}})$
      $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \| \langle o_N \rangle_{\lambda_{\mathbb{F}}}^3$
      for $r \in [0 \,..\, 8)$ **do**
        $\langle q^{2^r} \rangle_{\lambda_{\mathbb{F}}}^2 \leftarrow \langle x^{2^{(r+4) \bmod 8}} \rangle_{\lambda_{\mathbb{F}}} \cdot \langle h^{2^r \bmod 4} \rangle_{\lambda_{\mathbb{F}}}$
      end for
      $(\langle s_j \rangle_{\lambda_{\mathbb{F}}}^2, \langle s_j^2 \rangle_{\lambda_{\mathbb{F}}}^2, \ldots, \langle s_j^{128} \rangle_{\lambda_{\mathbb{F}}}^2) \leftarrow$ SM4th.AffineOut$(\langle q \rangle_{\lambda_{\mathbb{F}}}^2, \langle q^2 \rangle_{\lambda_{\mathbb{F}}}^2, \ldots, \langle q^{128} \rangle_{\lambda_{\mathbb{F}}}^2)$
    end for
    $\langle \boldsymbol{t} \rangle^2 \leftarrow$ SM4th.L(ConjToBits$((\langle s_j^{2^r} \rangle_{\lambda_{\mathbb{F}}}^2)_{r=0}^7)_{j=0}^3)$
    $\langle \boldsymbol{z}[a] \rangle^2 \leftarrow \langle \boldsymbol{z}[i] \rangle^{e_i} \oplus \langle \boldsymbol{t} \rangle^2$
  else
    if $a < R$ **then**
      $\langle \boldsymbol{z}[a] \rangle^{e_a} \leftarrow \langle \boldsymbol{w}_{\mathsf{enc},F}[32p_F \,..\, 32(p_F+1)) \rangle$
      $p_F \leftarrow p_F + 1$
    end if
    $\langle \boldsymbol{v} \rangle^{D_y} \leftarrow$ SM4th.LInv$(\langle \boldsymbol{z}[i] \rangle^{e_i} \oplus \langle \boldsymbol{z}[a] \rangle^{e_a})$
    for $j \in [0 \,..\, 4)$ **do**
      $(\langle x \rangle_{\lambda_{\mathbb{F}}}^{D_x}, \langle x^2 \rangle_{\lambda_{\mathbb{F}}}^{D_x}) \leftarrow$ SM4th.TAffineByte$(\langle \boldsymbol{u}[j] \rangle^{D_x})$
      $(\langle y \rangle_{\lambda_{\mathbb{F}}}^{D_y}, \langle y^2 \rangle_{\lambda_{\mathbb{F}}}^{D_y}) \leftarrow$ SM4th.TInvAffineByte$(\langle \boldsymbol{v}[j] \rangle^{D_y})$
      $D \leftarrow D_x + D_y$, where D_x, D_y are the degrees of $\langle x \rangle^{D_x}, \langle y \rangle^{D_y}$
      $\langle o_0 \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \mathsf{Lift}_3(\langle x^2 \rangle_{\lambda_{\mathbb{F}}}^{D_x} \cdot \langle y \rangle_{\lambda_{\mathbb{F}}}^{D_y} + \mathsf{Lift}_D(\langle x \rangle_{\lambda_{\mathbb{F}}}^{D_x}))$
      $\langle o_1 \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \mathsf{Lift}_3(\langle x \rangle_{\lambda_{\mathbb{F}}}^{D_x} \cdot \langle y^2 \rangle_{\lambda_{\mathbb{F}}}^{D_y} + \mathsf{Lift}_D(\langle y \rangle_{\lambda_{\mathbb{F}}}^{D_y}))$
      $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \| \langle o_0 \rangle_{\lambda_{\mathbb{F}}}^3$
      $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \leftarrow \langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3 \| \langle o_1 \rangle_{\lambda_{\mathbb{F}}}^3$
    end for
  end if
end for
return $\langle \boldsymbol{o} \rangle_{\lambda_{\mathbb{F}}}^3$

SM4th-EM.EncCstrnts($\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, \langle \bar{\boldsymbol{k}} \rangle, \langle \boldsymbol{w} \rangle$; param)

Input: Commitments to SM4 input/output blocks $\boldsymbol{in}, \boldsymbol{out} \in \{0,1\}^{128}$, middle encryption witness words $\boldsymbol{w} \in \mathbb{F}_2^{32(R-4)}$, and expanded keys $\bar{\boldsymbol{k}} \in (\mathbb{F}_2^{32})^{4+R}$
Output: $\langle \boldsymbol{o} \rangle^2_{\lambda_\mathbb{F}}$: constraint vector $\boldsymbol{o} \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}^{2S_{\mathsf{enc}}}$

new $\langle \boldsymbol{x} \rangle$, commitments to $\boldsymbol{x} \in \mathbb{F}_2^{8S_{\mathsf{enc}}}$ // Patched inverse input bits
new $\langle \boldsymbol{y} \rangle$, commitments to $\boldsymbol{y} \in \mathbb{F}_2^{8S_{\mathsf{enc}}}$ // Patched inverse output bits
let $\langle \boldsymbol{w} \rangle \leftarrow \langle \boldsymbol{in} \rangle \| \langle \boldsymbol{w} \rangle \| \langle \boldsymbol{out}[96 \,..\, 127] \rangle \| \langle \boldsymbol{out}[64 \,..\, 95] \rangle \| \langle \boldsymbol{out}[32 \,..\, 63] \rangle \| \langle \boldsymbol{out}[0 \,..\, 31] \rangle$
for $i \in [0, R)$ **do**
    $\langle \boldsymbol{x}[iN_{\mathsf{word}} \,..\, (i+1)N_{\mathsf{word}} - 1] \rangle \leftarrow$ T.Affine($\langle \boldsymbol{w}[iN_{\mathsf{word}} \,..\, (i+5)N_{\mathsf{word}} - 1] \rangle, \langle \bar{\boldsymbol{k}}[iN_{\mathsf{word}} \,..\, (i+5)N_{\mathsf{word}} - 1] \rangle$)
    $\langle \boldsymbol{y}[iN_{\mathsf{word}} \,..\, (i+1)N_{\mathsf{word}} - 1] \rangle \leftarrow$ T.InvAffine($\langle \boldsymbol{w}[iN_{\mathsf{word}} \,..\, (i+5)N_{\mathsf{word}} - 1] \rangle$)
end for
for $j \in [0 \,..\, S_{\mathsf{enc}})$ **do**
    new $\langle \hat{x} \rangle_{\lambda_\mathbb{F}}, \langle \hat{x}_{\mathsf{sq}} \rangle_{\lambda_\mathbb{F}}, \langle \hat{y} \rangle_{\lambda_\mathbb{F}}, \langle \hat{y}_{\mathsf{sq}} \rangle_{\lambda_\mathbb{F}}$, each a commitment to an element in $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$
    $\langle \hat{x} \rangle_{\lambda_\mathbb{F}} \leftarrow$ ByteCombine($\langle x[8j \,..\, 8j+7] \rangle$)
    $\langle \hat{x}_{\mathsf{sq}} \rangle_{\lambda_\mathbb{F}} \leftarrow$ SquareAndCombine($\langle x[8j \,..\, 8j+7] \rangle$)
    $\langle \hat{y} \rangle_{\lambda_\mathbb{F}} \leftarrow$ ByteCombine($\langle y[8j \,..\, 8j+7] \rangle$)
    $\langle \hat{y}_{\mathsf{sq}} \rangle_{\lambda_\mathbb{F}} \leftarrow$ SquareAndCombine($\langle y[8j \,..\, 8j+7] \rangle$)
    $\langle o_{2j} \rangle^2_{\lambda_\mathbb{F}} \leftarrow \langle \hat{x}_{\mathsf{sq}} \rangle_{\lambda_\mathbb{F}} \cdot \langle \hat{y} \rangle_{\lambda_\mathbb{F}} +$ Lift$_2(\langle \hat{x} \rangle_{\lambda_\mathbb{F}})$
    $\langle o_{2j+1} \rangle^2_{\lambda_\mathbb{F}} \leftarrow \langle \hat{x} \rangle_{\lambda_\mathbb{F}} \cdot \langle \hat{y}_{\mathsf{sq}} \rangle_{\lambda_\mathbb{F}} +$ Lift$_2(\langle \hat{y} \rangle_{\lambda_\mathbb{F}})$
end for
return $(\langle o_0 \rangle^2_{\lambda_\mathbb{F}}, \ldots, \langle o_{2S_{\mathsf{enc}}-1} \rangle^2_{\lambda_\mathbb{F}})$

1. It parses $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$ and allocates a new $\langle \boldsymbol{z} \rangle^d_{\lambda_\mathbb{F}}$ for the accumulated constraints.
2. It first appends Lift$_d(\langle \kappa_0 \rangle \cdot \langle \kappa_1 \rangle)$ to enforce the secret-domain restriction $k[0]k[1] = 0$ used by SM4th.KeyGen, where κ_0, κ_1 are bits $0, 1$ of the secret block $\boldsymbol{k}$. In the EM variant, $\kappa_b = w[b]$ and in the non-EM variant, $\kappa_b = w[b] + 1$ for $b \in \{0, 1\}$, undoing the corresponding whitening bits of $FK_0 =$ `A3B1BAC6`, whose first byte has both low bits set.
3. If EM = `true`, it interprets pk_1 as the public SM4 key, runs SM4.KeyExpansion, and then sets $\langle \bar{\boldsymbol{k}} \rangle$ using ConstantToVOLE. It also sets $\langle \boldsymbol{w}[0 \,..\, 128) \rangle$ as the commitment to the secret SM4 input block, and sets $\langle \boldsymbol{w}[0 \,..\, 128) \rangle +$ ConstantToVOLE(pk_2) as the commitment to the corresponding SM4 encryption output block.
   If EM = `false`, the algorithm calls ConstantToVOLE(pk_1) to get a bitwise-committed input block $\langle \boldsymbol{in} \rangle$ and calls ConstantToVOLE(pk_2) to get the committed output block $\langle \boldsymbol{out} \rangle$ accordingly. For the key schedule, it calls SM4th.KeyExpCstrnts($\langle \boldsymbol{w} \rangle$; param), obtaining both round-key commitments $(\langle \bar{\boldsymbol{k}}[j] \rangle^{d_j})_{j=0}^{R+3}$ and a degree-d constraint vector $\langle \tilde{\boldsymbol{z}} \rangle^d_{\lambda_\mathbb{F}}$. This is later appended to the main buffer.
4. In the encryption routine, it extracts the portion of $\langle \boldsymbol{w} \rangle$ dedicated to the encryption bits, and calls SM4th.EncCstrnts($\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, \ldots$) or SM4th.EM.EncCstrnts($\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, \ldots$) to produce the encryption constraints. These are all collected and appended into the same $\langle \boldsymbol{z} \rangle^d_{\lambda_\mathbb{F}}$.
5. Finally, the algorithm returns $\langle \boldsymbol{z} \rangle^d_{\lambda_\mathbb{F}}$.

SM4th.OWFProve and SM4th.OWFVerify. The SM4th.OWFProve describes the signer's algorithm:

1. It converts each bit $\boldsymbol{w}[i]$ into a VOLE commitment, using a row in $\boldsymbol{V}$ as tags. It also parses $\boldsymbol{u}$ into $(d-1)$ proof-response mask elements and parses the last $(d-1)\lambda_\mathbb{F}$ rows of $\boldsymbol{V}$ into the ZK-mask elements $v^*_0, \ldots, v^*_{d-2}$.
2. Then, it invokes SM4th.OWFConstraints($\langle \boldsymbol{w} \rangle, \mathsf{pk}$), which returns a polynomial vector $\langle \boldsymbol{z} \rangle^d_{\lambda_\mathbb{F}}$ of degree d. This vector is then parsed as a vector $(\boldsymbol{a}_0, \boldsymbol{a}_1, \ldots, \boldsymbol{a}_{d-1}) \in (\mathbb{F}^C_{2^{\lambda_\mathbb{F}}})^d$.
3. Finally, it generates the QuickSilver response $(\tilde{a}_0, \tilde{a}_1, \ldots, \tilde{a}_{d-1})$ by hashing $\boldsymbol{a}_j \|$(masks) with chall_2 via ZKHash. All d response components enter the final Fiat–Shamir hash; the signature carries only $(\tilde{a}_1, \ldots, \tilde{a}_{d-1})$.

The SM4th.OWFVerify algorithm describes the corresponding verifier procedure. It reconstructs the commitment values $\langle \boldsymbol{w}[i] \rangle$ from $\boldsymbol{d}_{\mathsf{w}}[i]$ and $\boldsymbol{Q}$. It then, from the verifier's side, recomputes the constraints $\langle \boldsymbol{z} \rangle^d_{\lambda_\mathbb{F}}$ using SM4th.OWFConstraints($\langle \boldsymbol{w} \rangle, \mathsf{pk}$), and checks consistency of the hashes

SM4th.OWFConstraints($\langle \boldsymbol{w} \rangle$, pk; param)

INPUT: Committed witness $\boldsymbol{w} \in \mathbb{F}_2^{\ell}$
OUTPUT: Committed constraints $\langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}}$, for $\boldsymbol{z} \in \mathbb{F}^{C}_{2^{\lambda_{\mathbb{F}}}}$ (will check that $\boldsymbol{z} = \boldsymbol{0}$)

Parse $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2) \in \{0,1\}^{128} \times \{0,1\}^{128}$
new $\langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}}$, $\boldsymbol{z} \in \mathbb{F}^{\leq C}_{2^{\lambda_{\mathbb{F}}}}$
$\eta \leftarrow 0$ if EM = true, else $\eta \leftarrow 1$
$\langle \kappa_b \rangle \leftarrow \langle \boldsymbol{w}[b] \rangle + \mathsf{ConstantToVOLE}(\eta)$ for $b \in \{0,1\}$
$\langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}} \leftarrow \langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}} \parallel \mathsf{Lift}_d(\langle \kappa_0 \rangle \cdot \langle \kappa_1 \rangle)$
if EM = true **then**
    $\bar{\boldsymbol{k}} \leftarrow \mathsf{SM4.KeyExpansion}(\mathsf{pk}_1, \mathsf{param})$ // $(R+4) \cdot 32$-bit round key $\bar{\boldsymbol{k}} \in (\{0,1\}^{32})^{R+4}$
    $\langle \bar{\boldsymbol{k}} \rangle \leftarrow \mathsf{ConstantToVOLE}(\bar{\boldsymbol{k}})$
    $\langle \boldsymbol{in} \rangle \leftarrow \langle \boldsymbol{w}[0 \,..\, 128) \rangle$ // secret SM4 input block
    $\langle \boldsymbol{out} \rangle \leftarrow \langle \boldsymbol{w}[0 \,..\, 128) \rangle + \mathsf{ConstantToVOLE}(\mathsf{pk}_2)$ // SM4 output block, since $\mathsf{pk}_2 = \mathrm{SM4}_{\mathsf{pk}_1}(\boldsymbol{in}) \oplus \boldsymbol{in}$
else
    $\langle \boldsymbol{in} \rangle \leftarrow \mathsf{ConstantToVOLE}(\mathsf{pk}_1)$ // convert each bit separately
    $\langle \boldsymbol{out} \rangle \leftarrow \mathsf{ConstantToVOLE}(\mathsf{pk}_2)$
    $((\langle \bar{\boldsymbol{k}}[j] \rangle^{d_j})_{j=0}^{R+3}, \langle \tilde{\boldsymbol{z}} \rangle^d_{\lambda_{\mathbb{F}}}) \leftarrow \mathsf{SM4th.KeyExpCstrnts}(\langle \boldsymbol{w}[0 \,..\, \ell_{\mathsf{ke}}) \rangle; \mathsf{param})$
    $\langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}} \leftarrow \langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}} \parallel \langle \tilde{\boldsymbol{z}} \rangle^d_{\lambda_{\mathbb{F}}}$ // Saving constraints
end if
$p \leftarrow 128$ if EM = true, else $p \leftarrow \ell_{\mathsf{ke}}$
$\langle \tilde{\boldsymbol{w}} \rangle \leftarrow \langle \boldsymbol{w}[p \,..\, \ell) \rangle$ // Selecting middle encryption words or the non-EM encryption witness
if EM = true **then**
    $\langle \tilde{\boldsymbol{z}} \rangle^d_{\lambda_{\mathbb{F}}} \leftarrow \mathsf{SM4th.EM.EncCstrnts}(\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, \langle \bar{\boldsymbol{k}} \rangle, \langle \tilde{\boldsymbol{w}} \rangle; \mathsf{param})$ // $d = 2$
else
    $\langle \tilde{\boldsymbol{z}} \rangle^d_{\lambda_{\mathbb{F}}} \leftarrow \mathsf{SM4th.EncCstrnts}(\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, (\langle \bar{\boldsymbol{k}}[j] \rangle^{d_j})_{j=4}^{R+3}, \langle \tilde{\boldsymbol{w}} \rangle; \mathsf{param})$ // $d = 3$
end if
$\langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}} \leftarrow \langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}} \parallel \langle \tilde{\boldsymbol{z}} \rangle^d_{\lambda_{\mathbb{F}}}$ // Appending values
return $\langle \boldsymbol{z} \rangle^d_{\lambda_{\mathbb{F}}}$

with the prover's $\{\tilde{a}_i\}_{i \in [1 \,..\, d)}$. In particular, it computes $\tilde{q} = \mathsf{ZKHash}(\mathsf{chall}_2, \boldsymbol{b} \| q^*)$, then returns $\tilde{q} + \sum_{i \in [1 \,..\, d)} \tilde{a}_i \cdot \Delta^i$. If the proof is correct, this final expression should be the same as $\tilde{a}_0$.

6.6 Construction of SM4th

Based on the building blocks and functions described above, this section gives the formal construction of the SM4th and SM4th-EM signature schemes.

6.6.1 Encoding and Input Lengths

The public key is the 32-byte string $\mathrm{OS}(\boldsymbol{x}) \| \mathrm{OS}(\boldsymbol{y})$, and the secret key is $\mathrm{OS}(\boldsymbol{x}) \| \mathrm{OS}(\boldsymbol{k})$. The bits $k[0], k[1]$ are bits $0, 1$ of the first byte of $\boldsymbol{k}$, in least-significant-bit-first order. Signing randomness has length $\xi = 384$ bits, encoded as a 48-byte ρ suffix, for every SM4th parameter set, independently of the PRG width N_{block}.

The signature uses the byte layout of Table 5, of total length sigsize(param). The hash-input encodings are specified in Section 3.3.

Verification rejects if the public key is not 32 bytes, the signature is not sigsize(param) bytes, or the message exceeds 2^{60} bytes. Signing applies the same message-length bound, requires a 32-byte secret key with $k[0]k[1] = 0$, and derives the associated public key by evaluating the OWF. The public-key argument of SM4th.Sign is this derived key. A failure to obtain the required randomness or complete signing returns $\perp$.

6.6.2 Key Generation

The SM4th.KeyGen algorithm describes the key generation procedure for the SM4th schemes. All SM4 keys, inputs, and outputs in this algorithm have length 128 bits.

SM4th.OWFProve($\boldsymbol{w}, \boldsymbol{u}, \boldsymbol{V}$, pk, chall_2; param)

INPUT: $\boldsymbol{w} \in \{0,1\}^{\ell}$, $\boldsymbol{u} \in \{0,1\}^{(d-1)\lambda_{\mathbb{F}}}$, $\boldsymbol{V} \in \{0,1\}^{(\ell+(d-1)\lambda_{\mathbb{F}})\times\lambda_{\mathbb{F}}}$, public key $\mathsf{pk} \in \{0,1\}^{128} \times \{0,1\}^{128}$, $\mathsf{chall}_2 \in \{0,1\}^{3\lambda_{\mathbb{F}}+64}$, and public parameters param from Section 3
OUTPUT: $(\tilde{a}_0, \tilde{a}_1, \ldots, \tilde{a}_{d-1}) \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{d}$

for $i \in [0\,..\,\ell)$ **do**
  Let $\langle \boldsymbol{w}[i] \rangle \leftarrow (\mathsf{ToField}(\boldsymbol{V}|_i; \lambda_{\mathbb{F}}), \boldsymbol{w}[i])$
end for
// embed VOLE masks
for $i \in [0\,..\,(d-1)\lambda_{\mathbb{F}})$ **do**
  $\boldsymbol{v}[i] \leftarrow \mathsf{ToField}(\boldsymbol{V}|_{\ell+i}; \lambda_{\mathbb{F}})$
end for
for $j \in [0\,..\,d-1)$ **do**
  $u_j^* \leftarrow \mathsf{ToField}(\boldsymbol{u}[j\lambda_{\mathbb{F}}\,..\,(j+1)\lambda_{\mathbb{F}}); \lambda_{\mathbb{F}})$
end for
for $j \in [0\,..\,d-1)$ **do**
  // $\alpha_{\lambda_{\mathbb{F}}} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, basis element over $\mathbb{F}_2$
  $v_j^* \leftarrow \sum_{i \in [0\,..\,\lambda_{\mathbb{F}})} \boldsymbol{v}[i+\lambda_{\mathbb{F}} j]\alpha_{\lambda_{\mathbb{F}}}^i$
end for
// Compute the constraints
// (polynomial $\sum_{i\in[0\,..\,d-1]} a_i x^i + z x^d; z = 0$)
$\langle \boldsymbol{z} \rangle_{\lambda_{\mathbb{F}}}^{d} \leftarrow \mathsf{SM4th.OWFConstraints}(\langle \boldsymbol{w} \rangle, \mathsf{pk})$
Parse $\langle \boldsymbol{z} \rangle_{\lambda_{\mathbb{F}}}^{d}$ as $(\boldsymbol{a}_0, \boldsymbol{a}_1, \ldots, \boldsymbol{a}_{d-1}) \in (\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{C})^d$
if $d = 2$ **then**
  $\tilde{a}_0 \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{a}_0 \| v_0^*))$
  $\tilde{a}_1 \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{a}_1 \| u_0^*))$
else if $d = 3$
  $\tilde{a}_0 \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{a}_0 \| v_0^*))$
  $\tilde{a}_1 \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{a}_1 \| v_1^* + u_0^*))$
  $\tilde{a}_2 \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{a}_2 \| u_1^*))$
end if
return $(\tilde{a}_0, \tilde{a}_1, \ldots, \tilde{a}_{d-1})$

SM4th.OWFVerify($\boldsymbol{d}_{\mathsf{w}}, \boldsymbol{Q}$, pk, chall_2, chall_3, $\tilde{a}_1, \ldots, \tilde{a}_{d-1}$; param)

INPUT: $\boldsymbol{d}_{\mathsf{w}} \in \{0,1\}^{\ell}$, $\boldsymbol{Q} \in \{0,1\}^{(\ell+(d-1)\lambda_{\mathbb{F}})\times\lambda_{\mathbb{F}}}$, public key $\mathsf{pk} \in \{0,1\}^{128} \times \{0,1\}^{128}$, $\mathsf{chall}_2 \in \{0,1\}^{3\lambda_{\mathbb{F}}+64}$, $\mathsf{chall}_3 \in \{0,1\}^{\lambda_{\mathbb{F}}}$, $(\tilde{a}_1, \ldots, \tilde{a}_{d-1}) \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{d-1}$, and public parameters param from Section 3
OUTPUT: $\tilde{a}_0 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$

$\Delta \leftarrow \mathsf{ToField}(\mathsf{chall}_3; \lambda_{\mathbb{F}})$
for $i \in [0\,..\,\ell)$ **do**
  Let $\langle \boldsymbol{w}[i] \rangle \leftarrow \mathsf{ToField}(\boldsymbol{Q}|_i; \lambda_{\mathbb{F}}) + \boldsymbol{d}_{\mathsf{w}}[i] \cdot \Delta$
end for
// embed VOLE masks
for $i \in [0\,..\,(d-1)\lambda_{\mathbb{F}})$ **do**
  $\boldsymbol{q}[i] \leftarrow \mathsf{ToField}(\boldsymbol{Q}|_{\ell+i}; \lambda_{\mathbb{F}})$
end for
for $j \in [0\,..\,d-1)$ **do**
  // $\alpha_{\lambda_{\mathbb{F}}} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, basis element over $\mathbb{F}_2$
  $q_j^* \leftarrow \sum_{i \in [0\,..\,\lambda_{\mathbb{F}})} \boldsymbol{q}[i+\lambda_{\mathbb{F}} j]\alpha_{\lambda_{\mathbb{F}}}^i$
end for
$q^* \leftarrow q_0^*$ if $d = 2$, else $q^* \leftarrow q_0^* + q_1^* \Delta$
$\langle \boldsymbol{z} \rangle_{\lambda_{\mathbb{F}}}^{d} \leftarrow \mathsf{SM4th.OWFConstraints}(\langle \boldsymbol{w} \rangle, \mathsf{pk})$
Parse $\langle \boldsymbol{z} \rangle_{\lambda_{\mathbb{F}}}^{d}$ as $\boldsymbol{b} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}^{C}$
$\tilde{q} \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{b} \| q^*))$
// Reproduce prover's $\tilde{a}_0$
return $\tilde{q} + \sum_{i \in [1\,..\,d)} \tilde{a}_i \cdot \Delta^i$

The output depends on whether the encryption mode flag EM is set to true. If $\mathsf{EM} = \mathsf{true}$, the algorithm samples a public SM4 key $\boldsymbol{x} \in \{0,1\}^{128}$ uniformly at random and samples a secret input block $\boldsymbol{k} \in \{0,1\}^{128}$ subject to the condition that the first two bits do not simultaneously satisfy $k[0] \wedge k[1] = 1$. It computes $\boldsymbol{y} \leftarrow \mathsf{SM4}_{\boldsymbol{x}}(\boldsymbol{k}) \oplus \boldsymbol{k}$, sets the secret key to $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$, and defines the public key as $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$. In this case, $\boldsymbol{x}$ serves as the public SM4 key and $\boldsymbol{k}$ serves as the secret input block. Otherwise, when $\mathsf{EM} = \mathsf{false}$, the algorithm samples a secret SM4 key $\boldsymbol{k} \in \{0,1\}^{128}$ uniformly at random, subject to the condition that the first two bits do not simultaneously satisfy $k[0] \wedge k[1] = 1$. It then samples a public input block $\boldsymbol{x} \in \{0,1\}^{128}$ uniformly at random, computes $\boldsymbol{y} \leftarrow \mathsf{SM4}_{\boldsymbol{k}}(\boldsymbol{x})$, sets $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$, and outputs $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$. In this case, $\boldsymbol{k}$ is the secret SM4 key and $\boldsymbol{x}$ is the public input block. Finally, the algorithm returns the pair $(\mathsf{sk}, \mathsf{pk})$.

SM4th.KeyGen(param)

INPUT: Public parameters param
OUTPUT: Secret key $\mathsf{sk} \in (\{0,1\}^{128}, \{0,1\}^{128})$ and public key $\mathsf{pk} \in (\{0,1\}^{128}, \{0,1\}^{128})$

if $\mathsf{EM} = \mathsf{true}$ **then**
  $\boldsymbol{x} \leftarrow \{0,1\}^{128}$ // public SM4 key
  repeat
    $\boldsymbol{k} \leftarrow \{0,1\}^{128}$ // secret input block
  until $\boldsymbol{k}[0] \wedge \boldsymbol{k}[1] = 0$
  $\boldsymbol{y} \leftarrow \mathsf{SM4}_{\boldsymbol{x}}(\boldsymbol{k}) \oplus \boldsymbol{k}$
  $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$ // public SM4 key, secret input block
  $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$ // public SM4 key, public output
else
  repeat
    $\boldsymbol{k} \leftarrow \{0,1\}^{128}$ // secret SM4 key
  until $\boldsymbol{k}[0] \wedge \boldsymbol{k}[1] = 0$
  $\boldsymbol{x} \leftarrow \{0,1\}^{128}$ // public input block
  $\boldsymbol{y} \leftarrow \mathsf{SM4}_{\boldsymbol{k}}(\boldsymbol{x})$
  $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$ // public input block, secret SM4 key
  $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$ // public input, public output
end if
return $(\mathsf{sk}, \mathsf{pk})$

6.6.3 Signing

The SM4th.Sign algorithm proceeds in four conceptually separate phases.

Phase I: message binding and IV derivation. On input a message msg, a secret key $\mathsf{sk} = (\mathsf{sk}_1, \mathsf{sk}_2)$, a public key $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$, and the public parameters param, the signer first initializes a 32-bit counter $\mathsf{ctr} \leftarrow 0$ and samples fresh signing randomness $\rho \leftarrow \{0,1\}^{\xi}$ uniformly at random. It then computes $\mu \leftarrow \mathsf{H}_2^0(D_{\mathsf{p}} \,\|\, \mathsf{pk} \,\|\, \mathsf{msg}; 2\lambda)$, which binds the signing transcript to both the message and the public key. Next, it derives $(r, \mathsf{iv}^{\mathsf{pre}}) \leftarrow \mathsf{H}_3(\mathsf{sk}_2 \,\|\, \mu \,\|\, \rho)$ and $\mathsf{iv} \leftarrow \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}}; \lambda_{\mathsf{iv}})$. Here, $r \in \{0,1\}^{N_{\mathsf{block}}}$ is the root seed used to seed the subsequent BAVC/VOLE commitment generation, while $\mathsf{iv} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$ serves as the initialization value for the derived pseudorandom process.

Phase II: VOLE commitment and first challenge. Using (r, iv), the signer invokes

$$(\mathsf{com}, \mathsf{decom}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \boldsymbol{u}, \boldsymbol{V}) \leftarrow \mathsf{VOLECommit}(r, \mathsf{iv}, \hat{\ell}; \mathsf{param}),$$

where $\hat{\ell} = \ell + d\lambda_{\mathbb{F}} + B$. This gives a VOLE commitment, the corresponding decommitment information, and the correlated vectors required later in the proof. We have $d = 3$ for the non-EM variant SM4th and $d = 2$ for the EM variant SM4th-EM.

SM4th.Sign($\mathsf{msg}, \mathsf{sk}, \mathsf{pk}; \mathsf{param}$)

INPUT: Message msg, secret key $\mathsf{sk} = (\mathsf{sk}_1, \mathsf{sk}_2)$, public key $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$, public parameters param
OUTPUT: Signature σ or $\perp$

if $|\mathsf{sk}| \neq 32$ bytes or $|\mathsf{msg}| > 2^{60}$ bytes or $\mathsf{sk}_2[0]\mathsf{sk}_2[1] \neq 0$ **then**
    return $\perp$
end if
$\mathsf{ctr} \leftarrow 0$ // 32-bit int
$\rho \leftarrow \{0,1\}^{\xi}$ // Signature randomness
$\mu \leftarrow \mathsf{H}_2^0(D_{\mathsf{p}} \,\|\, \mathsf{pk} \,\|\, \mathsf{msg}; 2\lambda)$
$(r, \mathsf{iv}^{\mathsf{pre}}) \leftarrow \mathsf{H}_3(\mathsf{sk}_2 \,\|\, \mu \,\|\, \rho) \in \{0,1\}^{N_{\mathsf{block}}} \times \{0,1\}^{\lambda_{\mathsf{iv}}}$
$\mathsf{iv} \leftarrow \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}}; \lambda_{\mathsf{iv}})$
$\hat{\ell} \leftarrow \ell + d\lambda_{\mathbb{F}} + B$
$(\mathsf{com}, \mathsf{decom}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \boldsymbol{u}, \boldsymbol{V}) \leftarrow \mathsf{VOLECommit}(r, \mathsf{iv}, \hat{\ell}; \mathsf{param})$
$\mathsf{chall}_1 \leftarrow \mathsf{H}_2^1(\mu \,\|\, \mathsf{com} \,\|\, \boldsymbol{c}_1 \,\|\, \cdots \,\|\, \boldsymbol{c}_{\tau-1} \,\|\, \mathsf{iv}; 5\lambda_{\mathbb{F}} + 64)$
// VOLE consistency check
$\tilde{\boldsymbol{u}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{u}) \in \{0,1\}^{\lambda_{\mathbb{F}}+B}$
$\tilde{\boldsymbol{V}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{V}) \in \{0,1\}^{(\lambda_{\mathbb{F}}+B)\times\lambda_{\mathbb{F}}}$
if $\mathsf{EM} = \mathsf{true}$ **then**
    $\boldsymbol{w} \leftarrow \mathsf{SM4th.EM.ExtendWitness}(\mathsf{sk}_2, \mathsf{sk}_1; \mathsf{param}) \in \{0,1\}^{\ell}$
else
    $\boldsymbol{w} \leftarrow \mathsf{SM4th.ExtendWitness}(\mathsf{sk}_2, \mathsf{sk}_1; \mathsf{param}) \in \{0,1\}^{\ell}$
end if
$\boldsymbol{d}_{\mathsf{w}} \leftarrow \boldsymbol{w} \oplus \boldsymbol{u}_{[0,\ell)}$ // Mask extended witness
$\mathsf{chall}_2 \leftarrow \mathsf{H}_2^2(\mathsf{chall}_1 \,\|\, \tilde{\boldsymbol{u}} \,\|\, \tilde{\boldsymbol{V}} \,\|\, \boldsymbol{d}_{\mathsf{w}}; 3\lambda_{\mathbb{F}} + 64)$
// QuickSilver proof
$\boldsymbol{u}_{\mathsf{zk}} \leftarrow \boldsymbol{u}_{[\ell\,..\,\ell+(d-1)\lambda_{\mathbb{F}})}$
$\boldsymbol{V}_{\mathsf{zk}} \leftarrow \boldsymbol{V}_{[0\,..\,\ell+(d-1)\lambda_{\mathbb{F}})} \in \{0,1\}^{(\ell+(d-1)\lambda_{\mathbb{F}})\times\lambda_{\mathbb{F}}}$
$(\tilde{a}_0, \ldots, \tilde{a}_{d-1}) \leftarrow \mathsf{SM4th.OWFProve}(\boldsymbol{w}, \boldsymbol{u}_{\mathsf{zk}}, \boldsymbol{V}_{\mathsf{zk}}, \mathsf{pk}, \mathsf{chall}_2; \mathsf{param})$
repeat
    $\mathsf{chall}_3 \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \cdots \,\|\, \tilde{a}_{d-1} \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda_{\mathbb{F}})$
    $I \leftarrow \mathsf{DecodeChall}(\mathsf{chall}_3; \mathsf{param})$
    $\mathsf{decom}_I \leftarrow \perp$
    if $I \neq \perp$ **then**
        $\mathsf{decom}_I \leftarrow \mathsf{BAVC.Open}(\mathsf{decom}, I; \mathsf{param})$
    end if
    if $\mathsf{decom}_I = \perp$ **then**
        if $\mathsf{ctr} = 2^{32} - 1$ **then**
            return $\perp$
        end if
        $\mathsf{ctr} \leftarrow \mathsf{ctr} + 1$
    end if
until $\mathsf{decom}_I \neq \perp$
return $\sigma \leftarrow (\{\boldsymbol{c}_i\}_{i\in[1,\tau)}, \tilde{\boldsymbol{u}}, \boldsymbol{d}_{\mathsf{w}}, \tilde{a}_1, \ldots, \tilde{a}_{d-1}, \mathsf{decom}_I, \mathsf{chall}_3, \mathsf{iv}^{\mathsf{pre}}, \mathsf{ctr})$

After that, the signer derives the first Fiat–Shamir challenge

$$\mathsf{chall}_1 \leftarrow \mathsf{H}_2^1(\mu \,\|\, \mathsf{com} \,\|\, \boldsymbol{c}_1 \,\|\, \cdots \,\|\, \boldsymbol{c}_{\tau-1} \,\|\, \mathsf{iv}; 5\lambda_{\mathbb{F}} + 64).$$

This challenge binds the entire VOLE commitment transcript to the message-dependent digest μ and will be used to compress the VOLE correlations.

Phase III: VOLE consistency check and second challenge. Given chall_1, the signer performs the VOLE consistency check by hashing the correlated values:

$$\tilde{\boldsymbol{u}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{u}) \in \{0,1\}^{\lambda_{\mathbb{F}}+B}, \qquad \tilde{\boldsymbol{V}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{V}) \in \{0,1\}^{(\lambda_{\mathbb{F}}+B)\times\lambda_{\mathbb{F}}}.$$

These hashed values serve as compressed consistency certificates for VOLE correlations.

Next, the signer derives the extended witness $\boldsymbol{w} \in \{0,1\}^{\ell}$ from the secret key. If $\mathsf{EM} = \mathsf{true}$, it sets $\boldsymbol{w} \leftarrow \mathsf{SM4th.EM.ExtendWitness}(\mathsf{sk}_2, \mathsf{sk}_1; \mathsf{param})$, where sk_1 is the public SM4 key stored in the secret key and sk_2 is the secret SM4 input block. If $\mathsf{EM} = \mathsf{false}$, it sets $\boldsymbol{w} \leftarrow \mathsf{SM4th.ExtendWitness}(\mathsf{sk}_2, \mathsf{sk}_1; \mathsf{param})$, where sk_1 is the public input block and sk_2 is the secret SM4 key. It then masks the witness using the first ℓ bits of $\boldsymbol{u}$: $\boldsymbol{d}_{\mathsf{w}} \leftarrow \boldsymbol{w} \oplus \boldsymbol{u}_{[0..\ell)}$.

Finally, the signer derives the second challenge

$$\mathsf{chall}_2 \leftarrow \mathsf{H}_2^2(\mathsf{chall}_1 \,\|\, \tilde{\boldsymbol{u}} \,\|\, \tilde{\boldsymbol{V}} \,\|\, \boldsymbol{d}_{\mathsf{w}}; 3\lambda_{\mathbb{F}} + 64).$$

This challenge binds together the VOLE consistency check and the masked witness, and will be used as the challenge for the subsequent ZKP.

Phase IV: ZKP and opening challenge. To prepare the QuickSilver proof, the signer truncates the VOLE correlations to the portion actually needed by the proof system. The vectors $\boldsymbol{u}$ and $\boldsymbol{V}$ retain $(d-1)$ ZK-mask blocks: $\boldsymbol{u} \leftarrow \boldsymbol{u}_{[\ell\,..\,\ell+(d-1)\lambda_{\mathbb{F}})}$ and $\boldsymbol{V} \leftarrow \boldsymbol{V}_{[0\,..\,\ell+(d-1)\lambda_{\mathbb{F}})} \in \{0,1\}^{(\ell+(d-1)\lambda_{\mathbb{F}})\times\lambda_{\mathbb{F}}}$. It then computes the proof components

$$(\tilde{a}_0, \tilde{a}_1, \ldots, \tilde{a}_{d-1}) \leftarrow \mathsf{SM4th.OWFProve}(\boldsymbol{w}, \boldsymbol{u}, \boldsymbol{V}, \mathsf{pk}, \mathsf{chall}_2; \mathsf{param}).$$

After the proof is generated, the signer derives the third challenge

$$\mathsf{chall}_3 \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \tilde{a}_1 \,\|\, \cdots \,\|\, \tilde{a}_{d-1} \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda_{\mathbb{F}})$$

inside a grinding loop, and checks whether its last w_{grind} bits are all zero. If this condition fails, the signer increments ctr and retries, or returns $\perp$ when the counter is already $2^{32}-1$. Otherwise, it decodes $I \leftarrow \mathsf{DecodeChall}(\mathsf{chall}_3; \mathsf{param})$, and computes the corresponding BAVC opening $\mathsf{decom}_I \leftarrow \mathsf{BAVC.Open}(\mathsf{decom}, I; \mathsf{param})$. If $\mathsf{decom}_I = \perp$, the signer increments ctr and retries. After an unsuccessful trial with $\mathsf{ctr} = 2^{32}-1$, it returns $\perp$.

Once successful, the signer outputs the signature

$$\sigma \leftarrow \big(\{\boldsymbol{c}_i\}_{i\in[1,\tau)}, \tilde{\boldsymbol{u}}, \boldsymbol{d}_{\mathsf{w}}, \tilde{a}_1, \ldots, \tilde{a}_{d-1}, \mathsf{decom}_I, \mathsf{chall}_3, \mathsf{iv}^{\mathsf{pre}}, \mathsf{ctr}\big),$$

which consists of the VOLE correction vectors $\{\boldsymbol{c}_i\}_{i\in[1,\tau)}$, the hashed VOLE consistency value $\tilde{\boldsymbol{u}}$, the masked witness $\boldsymbol{d}_{\mathsf{w}}$, the ZKP components from $\tilde{a}_1$ to $\tilde{a}_{d-1}$, the BAVC opening decom_I, the opening challenge chall_3, the preimage $\mathsf{iv}^{\mathsf{pre}}$ used to derive iv, and the grinding counter ctr.

6.6.4 Verifying

The verification algorithm SM4th.Verify also proceeds in four conceptually separate phases.

Phase I: parsing and preliminary checks. On input a message msg, a public key pk, a signature

$$\sigma = \big(\{\boldsymbol{c}_i\}_{i\in[1,\tau)}, \tilde{\boldsymbol{u}}, \boldsymbol{d}_{\mathsf{w}}, \tilde{a}_1, \ldots, \tilde{a}_{d-1}, \mathsf{decom}_I, \mathsf{chall}_3, \mathsf{iv}^{\mathsf{pre}}, \mathsf{ctr}\big),$$

and the public parameters param, the verifier checks the input lengths from Section 6.6.1 and recomputes $\mu \leftarrow \mathsf{H}_2^0(D_{\mathsf{p}} \,\|\, \mathsf{pk} \,\|\, \mathsf{msg}; 2\lambda)$ and derives $\mathsf{iv} \leftarrow \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}}; \lambda_{\mathsf{iv}})$. It then checks whether the grinding condition is satisfied, namely whether the last w_{grind} bits of chall_3 are all zero. If this condition fails, the verifier immediately rejects.

Phase II: VOLE reconstruction and first challenge. Next, the verifier reconstructs the committed VOLE transcript from the transmitted partial commitment information and the selective opening. Concretely, it invokes

$$(\mathsf{com}, \boldsymbol{Q}) \leftarrow \mathsf{VOLEReconstruct}(\mathsf{chall}_3, \mathsf{decom}_I, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv}, \hat{\ell}; \mathsf{param}).$$

If this reconstruction procedure returns $\perp$, the verifier rejects. Otherwise, it recomputes the first Fiat–Shamir challenge as

$$\mathsf{chall}_1 \leftarrow \mathsf{H}_2^1(\mu \,\|\, \mathsf{com} \,\|\, \boldsymbol{c}_1 \,\|\, \cdots \,\|\, \boldsymbol{c}_{\tau-1} \,\|\, \mathsf{iv}; 5\lambda_{\mathbb{F}} + 64).$$

Thus, the verifier recovers exactly the same first challenge as in signing, provided that the reconstructed commitment transcript is consistent with the transmitted signature.

Phase III: VOLE consistency check and second challenge. Using chall_1, the verifier computes the hashed VOLE value

$$\tilde{\boldsymbol{Q}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{Q}) \in \{0,1\}^{(\lambda_{\mathbb{F}}+B)\times\lambda_{\mathbb{F}}}.$$

It then parses chall_3 as a bit string $(\delta_0, \ldots, \delta_{\lambda_{\mathbb{F}}-1}) \in \{0,1\}^{\lambda_{\mathbb{F}}}$ and reconstructs the consistency matrix

$$\tilde{\boldsymbol{V}}' \leftarrow \tilde{\boldsymbol{Q}} \oplus [\delta_0 \cdot \tilde{\boldsymbol{u}} \;\; \cdots \;\; \delta_{\lambda_{\mathbb{F}}-1} \cdot \tilde{\boldsymbol{u}}].$$

Intuitively, this step re-applies the VOLE correction values determined by chall_3 and the transmitted compressed consistency term $\tilde{\boldsymbol{u}}$. The verifier then derives the second challenge

$$\mathsf{chall}_2 \leftarrow \mathsf{H}_2^2(\mathsf{chall}_1 \,\|\, \tilde{\boldsymbol{u}} \,\|\, \tilde{\boldsymbol{V}}' \,\|\, \boldsymbol{d}_{\mathsf{w}}; 3\lambda_{\mathbb{F}} + 64).$$

If the transmitted values are valid, the verifier reconstructs the same second challenge as the signer.

Phase IV: ZKP verification and transcript consistency. In the final phase, the verifier recomputes the missing proof component by running

$$\tilde{a}_0 \leftarrow \mathsf{SM4th.OWFVerify}(\boldsymbol{d}_{\mathsf{w}}, \boldsymbol{Q}_{[0,\ell+(d-1)\lambda_{\mathbb{F}})}, \mathsf{pk}, \mathsf{chall}_2, \mathsf{chall}_3, \tilde{a}_1, \ldots, \tilde{a}_{d-1}; \mathsf{param}).$$

It then recomputes the final Fiat–Shamir challenge

$$\mathsf{chall}_3' \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \tilde{a}_1 \,\|\, \cdots \,\|\, \tilde{a}_{d-1} \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda_{\mathbb{F}}).$$

The verifier accepts if and only if $\mathsf{chall}_3' = \mathsf{chall}_3$; otherwise, it rejects.

7 Algorithm Description: the uBlockith Signature Scheme

This section presents the whole construction of uBlockith. For uBlockith, the proof field size is $\lambda_{\mathbb{F}} = 256$, which coincides with the classical security parameter λ. Nevertheless, we write the VOLE and QuickSilver operations over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$ to keep the notation consistent with the parameter separation used throughout the paper.

7.1 Zero-Knowledge Building Blocks for uBlockith

We give several zero-knowledge building blocks in this section.

VOLESubWord and VOLEInvSubWord. Given degree-1 VOLE commitments to a witness $\boldsymbol{x} \in \{0,1\}^{4n}$, the uBlockith.VOLESubWord algorithm computes degree-3 VOLE commitments to $\mathsf{SubDWord}(\boldsymbol{x})$ when $n = 16$ and to $\mathsf{SubQWord}(\boldsymbol{x})$ when $n = 32$. Given degree-1 VOLE commitments to a witness $\boldsymbol{y} \in \{0,1\}^{4n}$, the uBlockith.VOLEInvSubWord algorithm computes degree-3 VOLE commitments to $\mathsf{SubDWord}^{-1}(\boldsymbol{y})$ when $n = 16$ and to $\mathsf{SubQWord}^{-1}(\boldsymbol{y})$ when $n = 32$. Each nibble is ordered from its least significant bit to its most significant bit, and consecutive groups of four serialized bits form consecutive nibbles. For the key permutation PK, convert these serialized indices

SM4th.Verify($\mathsf{msg}, \mathsf{pk}, \sigma; \mathsf{param}$)

INPUT: Message msg, public key pk, signature σ, public parameters param
OUTPUT: true or false

if $|\mathsf{pk}| \neq 32$ bytes or $|\sigma| \neq \mathsf{sigsize}(\mathsf{param})$ bytes or $|\mathsf{msg}| > 2^{60}$ bytes **then**
  return false
end if
$\hat{\ell} \leftarrow \ell + d\lambda_{\mathbb{F}} + B$
Parse $\sigma = \left(\{\boldsymbol{c}_i\}_{i\in[1,\tau)}, \tilde{\boldsymbol{u}}, \boldsymbol{d}_{\mathsf{w}}, \tilde{a}_1, \ldots, \tilde{a}_{d-1}, \mathsf{decom}_I, \mathsf{chall}_3, \mathsf{iv}^{\mathsf{pre}}, \mathsf{ctr}\right)$
$\mu \leftarrow \mathsf{H}_2^0(D_{\mathsf{p}} \,\|\, \mathsf{pk} \,\|\, \mathsf{msg}; 2\lambda)$
$\mathsf{iv} \leftarrow \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}}; \lambda_{\mathsf{iv}})$
if $\mathsf{chall}_3[\lambda_{\mathbb{F}} - w_{\mathsf{grind}}, \lambda_{\mathbb{F}}) \neq 0^{w_{\mathsf{grind}}}$ **then**
  return false
end if
// Reconstruct VOLEs and check commitment
$(\mathsf{com}, \boldsymbol{Q}) \leftarrow$ VOLEReconstruct$(\mathsf{chall}_3, \mathsf{decom}_I, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv}, \hat{\ell}; \mathsf{param})$
if VOLEReconstruct returned $\perp$ **then**
  return false
end if
$\mathsf{chall}_1 \leftarrow \mathsf{H}_2^1(\mu \,\|\, \mathsf{com} \,\|\, \boldsymbol{c}_1 \,\|\, \cdots \,\|\, \boldsymbol{c}_{\tau-1} \,\|\, \mathsf{iv}; 5\lambda_{\mathbb{F}} + 64)$
// Apply the VOLE correction values and check consistency
$\tilde{\boldsymbol{Q}} \leftarrow$ VOLEHash$(\mathsf{chall}_1, \boldsymbol{Q}) \in \{0,1\}^{(\lambda_{\mathbb{F}}+B)\times\lambda_{\mathbb{F}}}$
Parse $\mathsf{chall}_3 = (\delta_0, \ldots, \delta_{\lambda_{\mathbb{F}}-1}) \in \{0,1\}^{\lambda_{\mathbb{F}}}$
$\tilde{\boldsymbol{V}}' \leftarrow \tilde{\boldsymbol{Q}} \oplus [\delta_0 \cdot \tilde{\boldsymbol{u}} \;\cdots\; \delta_{\lambda_{\mathbb{F}}-1} \cdot \tilde{\boldsymbol{u}}]$
$\mathsf{chall}_2 \leftarrow \mathsf{H}_2^2(\mathsf{chall}_1 \,\|\, \tilde{\boldsymbol{u}} \,\|\, \tilde{\boldsymbol{V}}' \,\|\, \boldsymbol{d}_{\mathsf{w}}; 3\lambda_{\mathbb{F}} + 64)$
// Compute $\tilde{a}_0$
$\tilde{a}_0 \leftarrow$ SM4th.OWFVerify$(\boldsymbol{d}_{\mathsf{w}}, \boldsymbol{Q}_{[0,\ell+(d-1)\lambda_{\mathbb{F}})}, \mathsf{pk}, \mathsf{chall}_2, \mathsf{chall}_3, \tilde{a}_1, \ldots, \tilde{a}_{d-1}; \mathsf{param})$
Compute $\mathsf{chall}_3' \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \tilde{a}_1 \,\|\, \cdots \,\|\, \tilde{a}_{d-1} \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda_{\mathbb{F}})$
return true if $\mathsf{chall}_3 = \mathsf{chall}_3'$ else false

uBlockith.VOLESubWord$(\langle \boldsymbol{x} \rangle, n)$

INPUT: $\langle \boldsymbol{x} \rangle$: degree-1 commitments to $\boldsymbol{x} \in \{0,1\}^{4n}$
OUTPUT: $\langle \boldsymbol{y} \rangle^3$: degree-3 commitments to $\boldsymbol{y} \in \{0,1\}^{4n}$

Parse $\langle \boldsymbol{x} \rangle = (\langle \boldsymbol{x}[0] \rangle, \ldots, \langle \boldsymbol{x}[n-1] \rangle)$
new $(\langle \boldsymbol{y}[0] \rangle^3, \ldots, \langle \boldsymbol{y}[n-1] \rangle^3)$, n tuples of four commitments to bits
for $i \in [0, n)$ **do**
  new $(\langle y_0 \rangle^3, \langle y_1 \rangle^3, \langle y_2 \rangle^3, \langle y_3 \rangle^3)$, commitments to bits
  Parse $\langle \boldsymbol{x}[i] \rangle = (\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle)$
  $\langle y_0 \rangle^3 \leftarrow \mathsf{Lift}_3\left(\text{ConstantToVOLE}(1) + \langle x_0 \rangle + \langle x_2 \rangle \cdot \langle x_3 \rangle\right)$
  $\langle y_1 \rangle^3 \leftarrow \mathsf{Lift}_3\left(\text{ConstantToVOLE}(1) + \langle x_0 \rangle + \langle x_1 \rangle + \langle x_0 \rangle \cdot \langle x_1 \rangle + \langle x_0 \rangle \cdot \langle x_2 \rangle + \langle x_0 \rangle \cdot \langle x_1 \rangle \cdot \langle x_2 \rangle + \langle x_0 \rangle \cdot \langle x_3 \rangle + \langle x_2 \rangle \cdot \langle x_3 \rangle\right)$
  $\langle y_2 \rangle^3 \leftarrow \mathsf{Lift}_3\left(\text{ConstantToVOLE}(1) + \langle x_1 \rangle + \langle x_0 \rangle \cdot \langle x_1 \rangle + \langle x_2 \rangle + \langle x_1 \rangle \cdot \langle x_2 \rangle \cdot \langle x_3 \rangle\right)$
  $\langle y_3 \rangle^3 \leftarrow \mathsf{Lift}_3\left(\langle x_1 \rangle + \langle x_2 \rangle + \langle x_3 \rangle + \langle x_1 \rangle \cdot \langle x_2 \rangle\right)$
  $\langle \boldsymbol{y}[i] \rangle^3 \leftarrow (\langle y_0 \rangle^3, \langle y_1 \rangle^3, \langle y_2 \rangle^3, \langle y_3 \rangle^3)$
end for
return $(\langle \boldsymbol{y}[0] \rangle^3, \ldots, \langle \boldsymbol{y}[n-1] \rangle^3)$

$\mathsf{uBlockith.VOLEInvSubWord}(\langle \boldsymbol{y} \rangle, n)$

INPUT: $\langle \boldsymbol{y} \rangle$: degree-1 commitments to $\boldsymbol{y} \in \{0,1\}^{4n}$
OUTPUT: $\langle \boldsymbol{z} \rangle^3$: degree-3 commitments to $\boldsymbol{z} \in \{0,1\}^{4n}$ such that $\boldsymbol{z} = \mathsf{SubDWord}^{-1}(\boldsymbol{y})$ for $n = 16$, or $\boldsymbol{z} = \mathsf{SubQWord}^{-1}(\boldsymbol{y})$ for $n = 32$

Parse $\langle \boldsymbol{y} \rangle = (\langle \boldsymbol{y}[0] \rangle, \ldots, \langle \boldsymbol{y}[n-1] \rangle)$
new $(\langle \boldsymbol{z}[0] \rangle^3, \ldots, \langle \boldsymbol{z}[n-1] \rangle^3)$, n tuples of four commitments to bits
for $i \in [0, n)$ **do**
    new $(\langle z_0 \rangle^3, \langle z_1 \rangle^3, \langle z_2 \rangle^3, \langle z_3 \rangle^3)$, commitments to bits
    Parse $\langle \boldsymbol{y}[i] \rangle = (\langle y_0 \rangle, \langle y_1 \rangle, \langle y_2 \rangle, \langle y_3 \rangle)$
    $\langle z_0 \rangle^3 \leftarrow \mathsf{Lift}_3\big(\langle y_0 \rangle \cdot \langle y_1 \rangle + \langle y_2 \rangle + \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_3 \rangle + \langle y_2 \rangle \cdot \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_1 \rangle \cdot \langle y_3 \rangle\big)$
    $\langle z_1 \rangle^3 \leftarrow \mathsf{Lift}_3\big(\langle y_0 \rangle + \langle y_1 \rangle + \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_3 \rangle\big)$
    $\langle z_2 \rangle^3 \leftarrow \mathsf{Lift}_3\big(\mathsf{ConstantToVOLE}(1) + \langle y_0 \rangle + \langle y_2 \rangle + \langle y_0 \rangle \cdot \langle y_1 \rangle\big)$
    $\langle z_3 \rangle^3 \leftarrow \mathsf{Lift}_3\big(\mathsf{ConstantToVOLE}(1) + \langle y_2 \rangle + \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_2 \rangle + \langle y_1 \rangle \cdot \langle y_2 \rangle + \langle y_2 \rangle \cdot \langle y_3 \rangle + \langle y_0 \rangle \cdot \langle y_2 \rangle \cdot \langle y_3 \rangle\big)$
    $\langle \boldsymbol{z}[i] \rangle^3 \leftarrow (\langle z_0 \rangle^3, \langle z_1 \rangle^3, \langle z_2 \rangle^3, \langle z_3 \rangle^3)$
end for
return $(\langle \boldsymbol{z}[0] \rangle^3, \ldots, \langle \boldsymbol{z}[n-1] \rangle^3)$

using Equation (19), apply Table 10, and convert back to serialized indices. The Boolean ANFs are evaluated with the degree lifting defined for VOLE commitments; all four output commitments are lifted to degree 3.

The VOLE.T function. Given degree-1 VOLE commitments to a witness $\boldsymbol{x} \in \{0,1\}^{64}$, the $\mathsf{uBlockith.VOLE.T}$ algorithm computes degree-1 VOLE commitments to $T(\boldsymbol{x})$, where $T(\cdot)$ multiplies each of the sixteen 4-bit nibbles of $\boldsymbol{x}$ by the field element represented by hexadecimal 2 in $\mathrm{GF}(2^4)$. The concrete definition of $T(\cdot)$ is described in Equation (21) and Equation (22). Since T is $\mathbb{F}_2$-linear, it does not increase the VOLE commitment degree.

$\mathsf{uBlockith.VOLE.T}(\langle \boldsymbol{x} \rangle)$

INPUT: $\langle \boldsymbol{x} \rangle$: degree-1 commitments to $\boldsymbol{x} \in \{0,1\}^{64}$
OUTPUT: $\langle \boldsymbol{y} \rangle$: degree-1 commitments to $\boldsymbol{y} \in \{0,1\}^{64}$

Parse $\langle \boldsymbol{x} \rangle = (\langle \boldsymbol{x}[0] \rangle, \ldots, \langle \boldsymbol{x}[15] \rangle)$
new $(\langle \boldsymbol{y}[0] \rangle, \ldots, \langle \boldsymbol{y}[15] \rangle)$, 16 tuples of four commitments to bits
for $i \in [0, 16)$ **do**
    new $(\langle y_0 \rangle, \langle y_1 \rangle, \langle y_2 \rangle, \langle y_3 \rangle)$, commitments to bits
    Parse $\langle \boldsymbol{x}[i] \rangle = (\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle)$
    $\langle y_0 \rangle \leftarrow \langle x_3 \rangle$
    $\langle y_1 \rangle \leftarrow \langle x_0 \rangle + \langle x_3 \rangle$
    $\langle y_2 \rangle \leftarrow \langle x_1 \rangle$
    $\langle y_3 \rangle \leftarrow \langle x_2 \rangle$
    $\langle \boldsymbol{y}[i] \rangle \leftarrow (\langle y_0 \rangle, \langle y_1 \rangle, \langle y_2 \rangle, \langle y_3 \rangle)$
end for
return $(\langle \boldsymbol{y}[0] \rangle, \ldots, \langle \boldsymbol{y}[15] \rangle)$

7.2 Witness Extension

The algorithm $\mathsf{uBlockith.ExtendWitness}$ takes as input the uBlock key $\boldsymbol{k}$, the uBlock plaintext $\boldsymbol{in} \in \{0,1\}^{N_{\mathsf{block}}}$, and the instance parameters param, and outputs an extended witness $\boldsymbol{w} \in \{0,1\}^{\ell}$ for later use in the proof. For $\mathsf{uBlockith}$, both the uBlock block length and key length are 256 bits, matching the corresponding lengths of the Ballet PRGs. We denote this common length by N_{block}.

The construction of $\boldsymbol{w}$ consists of three main parts: the initial phase, the key expansion phase, and the encryption phase.

– **Initial bits** (`ini_bits`): the first N_{block} bits obtained from the initial key for the Non-EM variant or from the plaintext for the EM variant.
– **Expanded key bits** (`exp_key_bits`): for each round of the Non-EM variant, the algorithm appends the first $N_{\mathsf{block}}/4$ bits of the expanded key to the extended witness, resulting in $N_{\mathsf{block}}/4 \cdot R$ bits. For the EM variant, this part is the empty string and contributes 0 bits, since the round keys are derived from the public key.
– **Encryption bits** (`enc_bits`): the intermediate encryption state pairs $(\boldsymbol{x}_0, \boldsymbol{x}_1)$ are stored after every two rounds, except for the final boundary state. Here $\boldsymbol{x}_0, \boldsymbol{x}_1 \in \{0,1\}^{N_{\mathsf{block}}/2}$, so each stored pair has total length N_{block} bits. This contributes $N_{\mathsf{block}} \cdot (R/2 - 1)$ bits for both the Non-EM and EM variants.

Therefore, the extended witness can be written as:

$$\boldsymbol{w} = \underbrace{\texttt{ini_bits}}_{N_{\mathsf{block}}} \parallel \underbrace{\texttt{exp_key_bits}}_{N_{\mathsf{block}}/4 \cdot R} \parallel \underbrace{\texttt{enc_bits}}_{N_{\mathsf{block}} \cdot (R/2-1)} .$$

For the EM variant, the middle component `exp_key_bits` has length 0. The key words and state words are serialized as big-endian 32-bit words; within each byte, witness bits are numbered from the least significant bit. Each state pair stores the four words of $\boldsymbol{x}_0$ followed by the four words of $\boldsymbol{x}_1$.

7.3 Deriving Constraints for the Key Expansion

The algorithm uBlockith.KeyExpCstrnts derives the constraint values required for the QuickSilver proof for the uBlock circuit corresponding to the key expansion routine. In addition, it simultaneously computes the degree-1 VOLE commitments to all round keys.

The algorithm starts by parsing the initial round key $\langle \bar{\boldsymbol{k}}[0] \rangle = (\langle \boldsymbol{k}_0^{(0)} \rangle, \langle \boldsymbol{k}_1^{(0)} \rangle, \langle \boldsymbol{k}_2^{(0)} \rangle, \langle \boldsymbol{k}_3^{(0)} \rangle)$ from the first 256 bits of the committed witness $\langle \boldsymbol{w} \rangle$. Then, for each round i, it computes two temporary committed words $(\langle \boldsymbol{t}_0 \rangle, \langle \boldsymbol{t}_1 \rangle)$ by applying the linear map $PK(\cdot)$ to $(\langle \boldsymbol{k}_0^{(i)} \rangle, \langle \boldsymbol{k}_1^{(i)} \rangle)$. Using these values, the algorithm derives the non-linear intermediate value $\langle \boldsymbol{t}_2 \rangle^3 \leftarrow \langle \boldsymbol{k}_2^{(i)} \rangle + \mathsf{uBlockith.VOLESubWord}\big(\langle \boldsymbol{t}_0 \rangle + \mathsf{ConstantToVOLE}(\widehat{RC}[i{+}1]), 16\big)$, as well as the linear update $\langle \boldsymbol{t}_3 \rangle \leftarrow \langle \boldsymbol{k}_3^{(i)} \rangle + \mathsf{uBlockith.VOLE.T}(\langle \boldsymbol{t}_1 \rangle)$. The next-round word $\langle \boldsymbol{k}_0^{(i+1)} \rangle$ is explicitly parsed from the committed witness $\langle \boldsymbol{w} \rangle$, rather than being carried forward as $\langle \boldsymbol{t}_2 \rangle^3$. This keeps the round-key commitments themselves degree-1. The consistency between the witness-provided value and the key expansion logic is enforced by the degree-3 constraint

$$\langle \boldsymbol{o}[64i \,..\, 64i + 63] \rangle^3 \leftarrow \langle \boldsymbol{k}_0^{(i+1)} \rangle + \langle \boldsymbol{t}_2 \rangle^3,$$

where the degree-1 commitment $\langle \boldsymbol{k}_0^{(i+1)} \rangle$ is naturally lifted to degree 3 before addition. Finally, the next round key commitment is assembled as

$$\langle \bar{\boldsymbol{k}}[i+1] \rangle = \big(\langle \boldsymbol{k}_0^{(i+1)} \rangle, \langle \boldsymbol{t}_3 \rangle, \langle \boldsymbol{t}_1 \rangle, \langle \boldsymbol{t}_0 \rangle\big).$$

As a result, all round-key commitments remain degree-1, while all non-linear relations are isolated into the degree-3 constraint vector $\langle \boldsymbol{o} \rangle^3$, which is later verified within the QuickSilver proof system.

7.4 Deriving Constraints for the Encryption

The algorithm uBlockith.EncCstrnts derives the degree-3 constraint values required by the QuickSilver proof for the uBlock encryption circuit. Given VOLE commitments to the plaintext, ciphertext, witness, and expanded round keys, it first augments the encryption witness with the supplied commitments to the input and output. It then processes the encryption two rounds at a time. For each even round index i, let $j = i/2$. The algorithm parses the committed state pair $(\langle \boldsymbol{x}_0 \rangle, \langle \boldsymbol{x}_1 \rangle)$ from the j-th stored state and loads the corresponding round key from $\langle \bar{\boldsymbol{k}}[i] \rangle$.

From this state, the algorithm evaluates the forward round computation on VOLE commitments: it applies uBlockith.VOLESubWord to the key-added state, followed by the prescribed linear

uBlockith.ExtendWitness($\boldsymbol{k}, \boldsymbol{in}$; param)

INPUT: $\boldsymbol{k} \in \{0,1\}^{N_{\mathsf{block}}}$, $\boldsymbol{in} \in \{0,1\}^{N_{\mathsf{block}}}$
OUTPUT: $\boldsymbol{w} \in \{0,1\}^{\ell}$: extended witness

// Initialize empty extended witness array, eventually of length ℓ
new $\boldsymbol{w} \in \{0,1\}^{\leq \ell}$
// Store round keys as array of N_{block}-bit words
new $\bar{\boldsymbol{k}} \in [\{0,1\}^{N_{\mathsf{block}}}; R+1]$
$\bar{\boldsymbol{k}} \leftarrow$ uBlock.KeyExpansion$(\boldsymbol{k}, \{RC_i\}_{i\in[R]}, PK(\cdot), \mathsf{param})$
if EM = false **then** // extract key schedule witness components
  $\boldsymbol{w} \leftarrow \boldsymbol{w} \| \bar{\boldsymbol{k}}[0]$ // Saving the initial N_{block}-bit key word
  for $i \in [1\,..\,R]$ **do**
    $\boldsymbol{w} \leftarrow \boldsymbol{w} \| \bar{\boldsymbol{k}}[i][0\,..\,N_{\mathsf{block}}/4 - 1]$ // Saving the first $N_{\mathsf{block}}/4$ bits of each expanded key word
  end for
else
  $\boldsymbol{w} \leftarrow \boldsymbol{w} \| \boldsymbol{in}$ // For EM, $\boldsymbol{in}$ is secret
end if
// Encryption routine and saving witness bits
new $\boldsymbol{x}_0 \in \{0,1\}^{N_{\mathsf{block}}/2}$, **new** $\boldsymbol{x}_1 \in \{0,1\}^{N_{\mathsf{block}}/2}$
$\boldsymbol{x}_0 \leftarrow \boldsymbol{in}[0\,..\,N_{\mathsf{block}}/2 - 1]$, $\boldsymbol{x}_1 \leftarrow \boldsymbol{in}[N_{\mathsf{block}}/2\,..\,N_{\mathsf{block}} - 1]$
for $i \in [0\,..\,R-1]$ **do**
  Parse $\bar{\boldsymbol{k}}[i] = (\boldsymbol{k}_0, \boldsymbol{k}_1)$ with $\boldsymbol{k}_0, \boldsymbol{k}_1 \in \{0,1\}^{N_{\mathsf{block}}/2}$
  $\boldsymbol{x}_0 \leftarrow$ SubQWord$(\boldsymbol{x}_0 \oplus \boldsymbol{k}_0)$ // defined in Equation (18)
  $\boldsymbol{x}_1 \leftarrow$ SubQWord$(\boldsymbol{x}_1 \oplus \boldsymbol{k}_1)$
  $\boldsymbol{x}_1 \leftarrow \boldsymbol{x}_1 \oplus \boldsymbol{x}_0$
  $\boldsymbol{x}_0 \leftarrow \boldsymbol{x}_0 \oplus (\boldsymbol{x}_1(\lll 4)_{32})$ // 32-bit word-wise cyclic left shifts
  $\boldsymbol{x}_1 \leftarrow \boldsymbol{x}_1 \oplus (\boldsymbol{x}_0(\lll 8)_{32})$
  $\boldsymbol{x}_0 \leftarrow \boldsymbol{x}_0 \oplus (\boldsymbol{x}_1(\lll 8)_{32})$
  $\boldsymbol{x}_1 \leftarrow \boldsymbol{x}_1 \oplus (\boldsymbol{x}_0(\lll 20)_{32})$
  $\boldsymbol{x}_0 \leftarrow \boldsymbol{x}_0 \oplus \boldsymbol{x}_1$
  $\boldsymbol{x}_0 \leftarrow PL(\boldsymbol{x}_0)$ // defined in Table 9
  $\boldsymbol{x}_1 \leftarrow PR(\boldsymbol{x}_1)$ // defined in Table 9
  if $i \mod 2 = 1 \wedge i < R-1$ **then**
    $\boldsymbol{w} \leftarrow \boldsymbol{w} \| \boldsymbol{x}_0 \| \boldsymbol{x}_1$ // Saving one N_{block}-bit state pair every two rounds
  end if
end for
return $\boldsymbol{w} \in \{0,1\}^{\ell}$

uBlockith.KeyExpCstrnts($\langle \boldsymbol{w} \rangle$; param)

INPUT: Commitments to the witness $\langle \boldsymbol{w} \rangle$ where $\boldsymbol{w} \in \{0,1\}^{\ell_{\mathsf{ke}}}$
OUTPUT: $\langle \bar{\boldsymbol{k}} \rangle$, where $\bar{\boldsymbol{k}} \in (\{0,1\}^{64}, \{0,1\}^{64}, \{0,1\}^{64}, \{0,1\}^{64})^{R+1}$; $\langle \boldsymbol{o} \rangle^3_{\lambda_{\mathbb{F}}}$ for the constraint vector $\boldsymbol{o} \in \mathbb{F}_2^{4S_{\mathsf{ke}}}$, with VOLE polynomial coefficients in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$

new $\langle \boldsymbol{t}_0 \rangle, \langle \boldsymbol{t}_1 \rangle, \langle \boldsymbol{t}_2 \rangle^3, \langle \boldsymbol{t}_3 \rangle$ // temporary commitments to keys
parse $\langle \bar{\boldsymbol{k}}[0] \rangle = (\langle \boldsymbol{k}_0^{(0)} \rangle, \langle \boldsymbol{k}_1^{(0)} \rangle, \langle \boldsymbol{k}_2^{(0)} \rangle, \langle \boldsymbol{k}_3^{(0)} \rangle)$ from $\langle \boldsymbol{w}[0\,..\,255] \rangle$
for $i \in [0\,..\,R-1]$ **do**
  $(\langle \boldsymbol{t}_0 \rangle, \langle \boldsymbol{t}_1 \rangle) \leftarrow PK(\langle \boldsymbol{k}_0^{(i)} \rangle, \langle \boldsymbol{k}_1^{(i)} \rangle)$ // apply Equation (19) to the commitment indices
  $\langle \boldsymbol{t}_2 \rangle^3 \leftarrow \langle \boldsymbol{k}_2^{(i)} \rangle +$ uBlockith.VOLESubWord$\big(\langle \boldsymbol{t}_0 \rangle +$ ConstantToVOLE$(\widehat{RC}[i+1]), 16\big)$
  $\langle \boldsymbol{t}_3 \rangle \leftarrow \langle \boldsymbol{k}_3^{(i)} \rangle +$ uBlockith.VOLE.T$(\langle \boldsymbol{t}_1 \rangle)$
  parse $\langle \boldsymbol{k}_0^{(i+1)} \rangle \leftarrow \langle \boldsymbol{w}[64 \cdot i + 256\,..\,64 \cdot i + 319] \rangle$
  $\langle \boldsymbol{o}[64 \cdot i\,..\,64 \cdot i + 63] \rangle^3 \leftarrow \langle \boldsymbol{k}_0^{(i+1)} \rangle + \langle \boldsymbol{t}_2 \rangle^3$ // lift $\langle \boldsymbol{k}_0^{(i+1)} \rangle$ to degree 3
  $\langle \bar{\boldsymbol{k}}[i+1] \rangle \leftarrow (\langle \boldsymbol{k}_0^{(i+1)} \rangle, \langle \boldsymbol{t}_3 \rangle, \langle \boldsymbol{t}_1 \rangle, \langle \boldsymbol{t}_0 \rangle)$
  $(\langle \boldsymbol{k}_1^{(i+1)} \rangle, \langle \boldsymbol{k}_2^{(i+1)} \rangle, \langle \boldsymbol{k}_3^{(i+1)} \rangle) \leftarrow (\langle \boldsymbol{t}_3 \rangle, \langle \boldsymbol{t}_1 \rangle, \langle \boldsymbol{t}_0 \rangle)$
end for
return $(\langle \bar{\boldsymbol{k}} \rangle, \langle \boldsymbol{o} \rangle^3)$

operations, rotations, and permutations, and then adds $\langle \bar{\boldsymbol{k}}[i+1] \rangle$ to obtain $\langle \boldsymbol{y}_0 \rangle^3, \langle \boldsymbol{y}_1 \rangle^3$, which represent the forward-derived inputs to the S-boxes of round $i+1$. To enforce consistency, the algorithm also derives a backward representation $\langle \boldsymbol{z}_0 \rangle^3, \langle \boldsymbol{z}_1 \rangle^3$ from the next committed state by inverting the linear layer and applying uBlockith.VOLEInvSubWord. In the final iteration, this next state is reconstructed from the ciphertext after removing the last round key.

Finally, the algorithm appends $\langle \boldsymbol{y}_0 \rangle^3 + \langle \boldsymbol{z}_0 \rangle^3$ and $\langle \boldsymbol{y}_1 \rangle^3 + \langle \boldsymbol{z}_1 \rangle^3$ to the constraint vector $\langle \boldsymbol{o} \rangle^3$. Hence, $\langle \boldsymbol{o} \rangle^3$ collects all bit-level constraints ensuring that the committed intermediate states, expanded keys, and input/output together describe a valid execution of the uBlock encryption routine. The VOLE tags associated with these commitments are elements of the proof field $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$.

7.5 Constraints for the OWF

This section describes the remaining steps needed to prove that $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$ is consistent with the witness $\boldsymbol{w}$ under the uBlock-based one-way function. We have three algorithms:

1. uBlockith.OWFConstraints that combines both the key expansion and encryption constraints;
2. the signer-side procedure uBlockith.OWFProve; and
3. the verifier-side procedure uBlockith.OWFVerify.

uBlockith.OWFConstraints. This is the main algorithm that combines all sub-constraints from the key-schedule and encryption routines. Concretely:

1. It parses the public key pk as $(\mathsf{pk}_1, \mathsf{pk}_2)$, where both components are N_{block}-bit strings, and allocates a new $\langle \boldsymbol{z} \rangle^3$ for the accumulated VOLE commitments of bit-level constraints.
2. It appends the secret-witness keyspace-reduction constraint $\mathsf{Lift}_3(\langle w[0] \rangle \cdot \langle w[1] \rangle) = 0$. In the EM variant, these are the first two bits of the secret input block. In the non-EM variant, they are the first two bits of the secret uBlock key.
3. If $\mathsf{EM} = \texttt{true}$, then pk_1 is interpreted as the public uBlock key. It runs the key expansion algorithm uBlock.KeyExpansion on pk_1 and obtains public round keys, which are converted to VOLE commitments by ConstantToVOLE. In this case, the first N_{block} bits of the witness are the secret uBlock input block, and the OWF relation is $\mathsf{pk}_2 = E_{\mathsf{pk}_1}(\boldsymbol{in}) \oplus \boldsymbol{in}$. Therefore, the committed encryption input is $\langle \boldsymbol{in} \rangle = \langle \boldsymbol{w}[0 \,..\, N_{\mathsf{block}} - 1] \rangle$, while the committed block-cipher output is $\langle \boldsymbol{out} \rangle = \langle \boldsymbol{w}[0 \,..\, N_{\mathsf{block}} - 1] \rangle + \mathsf{ConstantToVOLE}(\mathsf{pk}_2)$.
4. If $\mathsf{EM} = \texttt{false}$, then pk_1 and pk_2 are respectively the public uBlock input and output blocks. The algorithm obtains $\langle \boldsymbol{in} \rangle$ and $\langle \boldsymbol{out} \rangle$ by applying ConstantToVOLE to pk_1 and pk_2. It also calls uBlockith.KeyExpCstrnts$(\langle \boldsymbol{w}[0 \,..\, \ell_{\mathsf{ke}} - 1] \rangle; \mathsf{param})$ to obtain the round-key commitments $\langle \bar{\boldsymbol{k}} \rangle$ and the key-schedule constraint vector $\langle \tilde{\boldsymbol{z}} \rangle^3$. These constraints are appended to the main constraint vector $\langle \boldsymbol{z} \rangle^3$.
5. Next, it extracts the encryption part of the witness and calls uBlockith.EncCstrnts to produce the encryption constraints. In the non-EM variant, this slice starts after the key-expansion witness. In the EM variant, the slice is $\boldsymbol{w}[N_{\mathsf{block}}, \ell)$, after the secret initial block supplied separately as $\boldsymbol{in}$. The resulting constraints are appended to $\langle \boldsymbol{z} \rangle^3$.
6. Finally, the algorithm returns $\langle \boldsymbol{z} \rangle^3$ in the order keyspace, key expansion, encryption. The EM variant has no key-expansion constraints.

uBlockith.OWFProve. The uBlockith.OWFProve algorithm describes the signer's procedure. It first converts each witness bit w_i into a VOLE commitment using the corresponding row of $\boldsymbol{V}$ as the tag. It also parses $\boldsymbol{u}$ and the last $2\lambda_\mathbb{F}$ rows of $\boldsymbol{V}$ to obtain mask elements u_0^*, u_1^* and v_0^*, v_1^* in the proof field $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$. Then it invokes uBlockith.OWFConstraints$(\langle \boldsymbol{w} \rangle, \mathsf{pk})$, which returns a degree-3 polynomial vector $\langle \boldsymbol{z} \rangle^3$. The prover parses the coefficient vectors as $(\boldsymbol{a}_0, \boldsymbol{a}_1, \boldsymbol{a}_2) \in (\mathbb{F}_{2^{\lambda_\mathbb{F}}}^C)^3$ and computes the QuickSilver responses by hashing them with the corresponding mask terms via ZKHash.

uBlockith.OWFVerify. The uBlockith.OWFVerify algorithm describes the corresponding verifier procedure. It reconstructs the verifier-side VOLE values from $\boldsymbol{d}_{\mathsf{w}}$, $\boldsymbol{Q}$, and the challenge $\Delta \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}$. It then recomputes the constraints using uBlockith.OWFConstraints$(\langle \boldsymbol{w} \rangle, \mathsf{pk})$ and obtains the evaluated constraint vector $\boldsymbol{b} \in \mathbb{F}_{2^{\lambda_\mathbb{F}}}^C$. Finally, it computes $\tilde{q} = \mathsf{ZKHash}(\mathsf{chall}_2, \boldsymbol{b} \| q^*)$ and returns $\tilde{q} + \tilde{a}_1 \Delta + \tilde{a}_2 \Delta^2$, which should equal the prover's $\tilde{a}_0$ for a valid proof.

uBlockith.EncCstrnts($\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, \langle \bar{\boldsymbol{k}} \rangle, \langle \boldsymbol{w} \rangle$; param)

INPUT: Commitments to uBlock plaintext $\boldsymbol{in} \in \{0,1\}^{N_{\text{block}}}$, uBlock ciphertext $\boldsymbol{out} \in \{0,1\}^{N_{\text{block}}}$, witness $\boldsymbol{w} \in \{0,1\}^{N_{\text{block}}(R/2-1)}$, and expanded keys $\bar{\boldsymbol{k}} \in (\{0,1\}^{N_{\text{block}}})^{R+1}$
OUTPUT: $\langle \boldsymbol{o} \rangle^3$: constraint vector $\boldsymbol{o} \in \mathbb{F}_2^{C_{\text{enc}}}$

new $\langle \boldsymbol{o} \rangle^3$
new $\langle \boldsymbol{x}_0 \rangle, \langle \boldsymbol{x}_1 \rangle \in (\langle \mathbb{F}_2 \rangle)^{N_{\text{block}}/2}$ // State of the encryption routine
new $\langle \boldsymbol{y}_0 \rangle^3, \langle \boldsymbol{y}_1 \rangle^3 \in (\langle \mathbb{F}_2 \rangle)^{N_{\text{block}}/2}$ // Forward-derived input side of S-boxes
new $\langle \boldsymbol{z}_0 \rangle^3, \langle \boldsymbol{z}_1 \rangle^3 \in (\langle \mathbb{F}_2 \rangle)^{N_{\text{block}}/2}$ // Backward-derived input side of S-boxes
new $\langle \boldsymbol{k}_0 \rangle, \langle \boldsymbol{k}_1 \rangle \in (\langle \mathbb{F}_2 \rangle)^{N_{\text{block}}/2}$ // Expanded key halves
$\langle \boldsymbol{w} \rangle^\star \leftarrow \langle \boldsymbol{in} \rangle \| \langle \boldsymbol{w} \rangle \| \langle \boldsymbol{out} \rangle$
for $i \in [0 \,..\, R-2]$ with $i \mod 2 = 0$ **do**
    $j \leftarrow i/2$
    $(\langle \boldsymbol{x}_0 \rangle, \langle \boldsymbol{x}_1 \rangle) \leftarrow (\langle \boldsymbol{w}^\star[jN_{\text{block}} \,..\, jN_{\text{block}} + N_{\text{block}}/2 - 1] \rangle, \langle \boldsymbol{w}^\star[jN_{\text{block}} + N_{\text{block}}/2 \,..\, (j+1)N_{\text{block}} - 1] \rangle)$
    $(\langle \boldsymbol{k}_0 \rangle, \langle \boldsymbol{k}_1 \rangle) \leftarrow \langle \bar{\boldsymbol{k}}[i] \rangle$
    $\langle \boldsymbol{x}_0 \rangle^3 \leftarrow$ uBlockith.VOLESubWord$(\langle \boldsymbol{x}_0 \rangle + \langle \boldsymbol{k}_0 \rangle, 32)$
    $\langle \boldsymbol{x}_1 \rangle^3 \leftarrow$ uBlockith.VOLESubWord$(\langle \boldsymbol{x}_1 \rangle + \langle \boldsymbol{k}_1 \rangle, 32)$
    $\langle \boldsymbol{x}_1 \rangle^3 \leftarrow \langle \boldsymbol{x}_1 \rangle^3 + \langle \boldsymbol{x}_0 \rangle^3$
    $\langle \boldsymbol{x}_0 \rangle^3 \leftarrow \langle \boldsymbol{x}_0 \rangle^3 + (\langle \boldsymbol{x}_1 \rangle^3 \lll 4)_{32}$
    $\langle \boldsymbol{x}_1 \rangle^3 \leftarrow \langle \boldsymbol{x}_1 \rangle^3 + (\langle \boldsymbol{x}_0 \rangle^3 \lll 8)_{32}$
    $\langle \boldsymbol{x}_0 \rangle^3 \leftarrow \langle \boldsymbol{x}_0 \rangle^3 + (\langle \boldsymbol{x}_1 \rangle^3 \lll 8)_{32}$
    $\langle \boldsymbol{x}_1 \rangle^3 \leftarrow \langle \boldsymbol{x}_1 \rangle^3 + (\langle \boldsymbol{x}_0 \rangle^3 \lll 20)_{32}$
    $\langle \boldsymbol{x}_0 \rangle^3 \leftarrow \langle \boldsymbol{x}_0 \rangle^3 + \langle \boldsymbol{x}_1 \rangle^3$
    $\langle \boldsymbol{x}_0 \rangle^3 \leftarrow PL(\langle \boldsymbol{x}_0 \rangle^3)$ // $PL(\cdot)$ is defined in Table 9, which can be naturally extended to VOLE commitments for bits
    $\langle \boldsymbol{x}_1 \rangle^3 \leftarrow PR(\langle \boldsymbol{x}_1 \rangle^3)$ // $PR(\cdot)$ is defined in Table 9, which can be naturally extended to VOLE commitments for bits
    $(\langle \boldsymbol{k}_0 \rangle, \langle \boldsymbol{k}_1 \rangle) \leftarrow \langle \bar{\boldsymbol{k}}[i+1] \rangle$
    $\langle \boldsymbol{y}_0 \rangle^3 \leftarrow \langle \boldsymbol{x}_0 \rangle^3 + \langle \boldsymbol{k}_0 \rangle$ // compute the $i+1$-th round first inputs of S-boxes
    $\langle \boldsymbol{y}_1 \rangle^3 \leftarrow \langle \boldsymbol{x}_1 \rangle^3 + \langle \boldsymbol{k}_1 \rangle$ // compute the $i+1$-th round second inputs of S-boxes
    if $i = R-2$ **then**
        $(\langle \boldsymbol{k}_0 \rangle, \langle \boldsymbol{k}_1 \rangle) \leftarrow \langle \bar{\boldsymbol{k}}[i+2] \rangle$
        $\langle \boldsymbol{x}_0 \rangle \leftarrow \langle \boldsymbol{w}^\star[(j+1)N_{\text{block}} \,..\, (j+1)N_{\text{block}} + N_{\text{block}}/2 - 1] \rangle + \langle \boldsymbol{k}_0 \rangle$ // $\boldsymbol{out} = \bar{\boldsymbol{k}}[R] \oplus (\boldsymbol{x}_0, \boldsymbol{x}_1)$
        $\langle \boldsymbol{x}_1 \rangle \leftarrow \langle \boldsymbol{w}^\star[(j+1)N_{\text{block}} + N_{\text{block}}/2 \,..\, (j+2)N_{\text{block}} - 1] \rangle + \langle \boldsymbol{k}_1 \rangle$ // $\boldsymbol{out} = \bar{\boldsymbol{k}}[R] \oplus (\boldsymbol{x}_0, \boldsymbol{x}_1)$
    else
        $(\langle \boldsymbol{x}_0 \rangle, \langle \boldsymbol{x}_1 \rangle) \leftarrow (\langle \boldsymbol{w}^\star[(j+1)N_{\text{block}} \,..\, (j+1)N_{\text{block}} + N_{\text{block}}/2 - 1] \rangle, \langle \boldsymbol{w}^\star[(j+1)N_{\text{block}} + N_{\text{block}}/2 \,..\, (j+2)N_{\text{block}} - 1] \rangle)$
    end if
    $\langle \boldsymbol{x}_1 \rangle \leftarrow PR^{-1}(\langle \boldsymbol{x}_1 \rangle)$
    $\langle \boldsymbol{x}_0 \rangle \leftarrow PL^{-1}(\langle \boldsymbol{x}_0 \rangle)$
    $\langle \boldsymbol{x}_0 \rangle \leftarrow \langle \boldsymbol{x}_0 \rangle + \langle \boldsymbol{x}_1 \rangle$
    $\langle \boldsymbol{x}_1 \rangle \leftarrow \langle \boldsymbol{x}_1 \rangle + (\langle \boldsymbol{x}_0 \rangle \lll 20)_{32}$
    $\langle \boldsymbol{x}_0 \rangle \leftarrow \langle \boldsymbol{x}_0 \rangle + (\langle \boldsymbol{x}_1 \rangle \lll 8)_{32}$
    $\langle \boldsymbol{x}_1 \rangle \leftarrow \langle \boldsymbol{x}_1 \rangle + (\langle \boldsymbol{x}_0 \rangle \lll 8)_{32}$
    $\langle \boldsymbol{x}_0 \rangle \leftarrow \langle \boldsymbol{x}_0 \rangle + (\langle \boldsymbol{x}_1 \rangle \lll 4)_{32}$
    $\langle \boldsymbol{x}_1 \rangle \leftarrow \langle \boldsymbol{x}_1 \rangle + \langle \boldsymbol{x}_0 \rangle$
    $\langle \boldsymbol{z}_0 \rangle^3 \leftarrow$ uBlockith.VOLEInvSubWord$(\langle \boldsymbol{x}_0 \rangle, 32)$
    $\langle \boldsymbol{z}_1 \rangle^3 \leftarrow$ uBlockith.VOLEInvSubWord$(\langle \boldsymbol{x}_1 \rangle, 32)$
    $\langle \boldsymbol{o} \rangle^3 \leftarrow \langle \boldsymbol{o} \rangle^3 \| \langle \boldsymbol{y}_0 \rangle^3 + \langle \boldsymbol{z}_0 \rangle^3 \| \langle \boldsymbol{y}_1 \rangle^3 + \langle \boldsymbol{z}_1 \rangle^3$
end for
return $\langle \boldsymbol{o} \rangle^3$

uBlockith.OWFConstraints($\langle \boldsymbol{w} \rangle$, pk; param)

INPUT: Committed witness $\langle \boldsymbol{w} \rangle$ for $\boldsymbol{w} \in \mathbb{F}_2^{\ell}$
OUTPUT: Committed constraints $\langle \boldsymbol{z} \rangle^3$, for $\boldsymbol{z} \in \mathbb{F}_2^{C}$

Parse $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2) \in \{0,1\}^{N_{\mathsf{block}}} \times \{0,1\}^{N_{\mathsf{block}}}$
// Allocate space for constraint commitments
new $\langle \boldsymbol{z} \rangle^3$, $\boldsymbol{z} \in \mathbb{F}_2^{\leq C}$
$\langle \boldsymbol{z} \rangle^3 \leftarrow \langle \boldsymbol{z} \rangle^3 \parallel \mathsf{Lift}_3(\langle w[0] \rangle \cdot \langle w[1] \rangle)$ // secret-witness keyspace reduction
// Define committed input and output blocks for the OWF relation
if EM = true **then**
    $\bar{\boldsymbol{k}} \leftarrow$ uBlock.KeyExpansion$(\mathsf{pk}_1, \{RC_i\}_{i \in [R]}, PK(\cdot), \mathsf{param})$
    $\langle \bar{\boldsymbol{k}} \rangle \leftarrow$ ConstantToVOLE$(\bar{\boldsymbol{k}})$
    $\langle \boldsymbol{in} \rangle \leftarrow \langle \boldsymbol{w}[0 \,..\, N_{\mathsf{block}} - 1] \rangle$
    $\langle \boldsymbol{out} \rangle \leftarrow \langle \boldsymbol{w}[0 \,..\, N_{\mathsf{block}} - 1] \rangle +$ ConstantToVOLE(pk_2)
else
    $\langle \boldsymbol{in} \rangle \leftarrow$ ConstantToVOLE(pk_1)
    $\langle \boldsymbol{out} \rangle \leftarrow$ ConstantToVOLE(pk_2)
    $(\langle \bar{\boldsymbol{k}} \rangle, \langle \tilde{\boldsymbol{z}} \rangle^3) \leftarrow$ uBlockith.KeyExpCstrnts$(\langle \boldsymbol{w}[0 \,..\, \ell_{\mathsf{ke}} - 1] \rangle; \mathsf{param})$
    $\langle \boldsymbol{z} \rangle^3 \leftarrow \langle \boldsymbol{z} \rangle^3 \parallel \langle \tilde{\boldsymbol{z}} \rangle^3$
end if
if EM = true **then**
    $\langle \tilde{\boldsymbol{w}} \rangle \leftarrow \langle \boldsymbol{w}[N_{\mathsf{block}}, \ell) \rangle$
else
    $\langle \tilde{\boldsymbol{w}} \rangle \leftarrow \langle \boldsymbol{w}[\ell_{\mathsf{ke}}, \ell) \rangle$
end if
$\langle \tilde{\boldsymbol{z}} \rangle^3 \leftarrow$ uBlockith.EncCstrnts$(\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, \langle \bar{\boldsymbol{k}} \rangle, \langle \tilde{\boldsymbol{w}} \rangle; \mathsf{param})$
$\langle \boldsymbol{z} \rangle^3 \leftarrow \langle \boldsymbol{z} \rangle^3 \parallel \langle \tilde{\boldsymbol{z}} \rangle^3$
return $\langle \boldsymbol{z} \rangle^3$

uBlockith.OWFProve$(\boldsymbol{w}, \boldsymbol{u}, \boldsymbol{V}, \mathsf{pk}, \mathsf{chall}_2; \mathsf{param})$

INPUT: $\boldsymbol{w} \in \{0,1\}^{\ell}$, $\boldsymbol{u} \in \{0,1\}^{2\lambda_{\mathbb{F}}}$, $\boldsymbol{V} \in \{0,1\}^{(\ell + 2\lambda_{\mathbb{F}}) \times \lambda_{\mathbb{F}}}$
OUTPUT: $\tilde{a}_0, \tilde{a}_1, \tilde{a}_2 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$

for $i \in [0 \,..\, \ell - 1]$ **do**
    Let $\langle w_i \rangle \leftarrow (\mathsf{ToField}(\boldsymbol{V}|_i; \lambda_{\mathbb{F}}), w_i)$
end for
// Embed VOLE masks
for $i \in [0 \,..\, 2\lambda_{\mathbb{F}} - 1]$ **do**
    $\boldsymbol{v}[i] \leftarrow \mathsf{ToField}(\boldsymbol{V}|_{\ell + i}; \lambda_{\mathbb{F}})$
end for
for $j \in [0 \,..\, 1]$ **do**
    $u_j^* \leftarrow \mathsf{ToField}(\boldsymbol{u}[j\lambda_{\mathbb{F}} \,..\, (j+1)\lambda_{\mathbb{F}} - 1]; \lambda_{\mathbb{F}})$
    // $\alpha_{\lambda_{\mathbb{F}}} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, basis element over $\mathbb{F}_2$
    $v_j^* \leftarrow \sum_{i \in [0 \,..\, \lambda_{\mathbb{F}} - 1]} \boldsymbol{v}[i + \lambda_{\mathbb{F}} j] \cdot \alpha^i_{\lambda_{\mathbb{F}}}$
end for
// Compute the constraints
// (poly $a_0 + a_1 X + a_2 X^2 + z X^3;\ z = 0$)
$\langle \boldsymbol{z} \rangle^3 \leftarrow$ uBlockith.OWFConstraints$(\langle \boldsymbol{w} \rangle, \mathsf{pk})$
Parse $\langle \boldsymbol{z} \rangle^3$ as $(\boldsymbol{a}_0, \boldsymbol{a}_1, \boldsymbol{a}_2) \in (\mathbb{F}^C_{2^{\lambda_{\mathbb{F}}}})^3$
$\tilde{a}_0 \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{a}_0 \| v_0^*))$
$\tilde{a}_1 \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{a}_1 \| (u_0^* + v_1^*)))$
$\tilde{a}_2 \leftarrow \mathsf{ZKHash}(\mathsf{chall}_2, (\boldsymbol{a}_2 \| u_1^*))$
return $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2)$

uBlockith.OWFVerify($\boldsymbol{d}_{\mathsf{w}}, \boldsymbol{Q}, \mathsf{pk}, \mathsf{chall}_2, \mathsf{chall}_3, \tilde{a}_1, \tilde{a}_2; \mathsf{param}$)

INPUT: $\boldsymbol{d}_{\mathsf{w}} \in \{0,1\}^{\ell}$, $\boldsymbol{Q} \in \{0,1\}^{(\ell+2\lambda_{\mathbb{F}})\times\lambda_{\mathbb{F}}}$
OUTPUT: $\tilde{a}_0 \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$

$\Delta \leftarrow$ ToField($\mathsf{chall}_3; \lambda_{\mathbb{F}}$)
for $i \in [0\,..\,\ell-1]$ **do**
    Let $\langle w_i \rangle \leftarrow$ ToField($\boldsymbol{Q}|_i; \lambda_{\mathbb{F}}$) $+ \boldsymbol{d}_{\mathsf{w}}[i] \cdot \Delta$
end for
// Embed VOLE masks
for $i \in [0\,..\,2\lambda_{\mathbb{F}}-1]$ **do**
    $\boldsymbol{q}[i] \leftarrow$ ToField($\boldsymbol{Q}|_{\ell+i}; \lambda_{\mathbb{F}}$)
end for
for $j \in [0\,..\,1]$ **do**
    // $\alpha_{\lambda_{\mathbb{F}}} \in \mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$, basis element over $\mathbb{F}_2$
    $q_j^* \leftarrow \sum_{i\in[0\,..\,\lambda_{\mathbb{F}}-1]} \boldsymbol{q}[i+\lambda_{\mathbb{F}} j] \cdot \alpha^i_{\lambda_{\mathbb{F}}}$
end for
$q^* \leftarrow q_0^* + \Delta \cdot q_1^*$
// Compute the evaluated constraints
$\langle \boldsymbol{z} \rangle^3 \leftarrow$ uBlockith.OWFConstraints($\langle \boldsymbol{w} \rangle, \mathsf{pk}$)
Parse $\langle \boldsymbol{z} \rangle^3$ as $\boldsymbol{b} \in \mathbb{F}^C_{2^{\lambda_{\mathbb{F}}}}$
$\tilde{q} \leftarrow$ ZKHash($\mathsf{chall}_2, (\boldsymbol{b} \| q^*)$)
// Reproduce prover's $\tilde{a}_0$
return $\tilde{q} + \tilde{a}_1 \cdot \Delta + \tilde{a}_2 \cdot \Delta^2$

7.6 Construction of uBlockith

Based on the building blocks and functions described above, this section gives the formal construction of the uBlockith and uBlockith-EM signature schemes.

7.6.1 Key Generation

The uBlockith.KeyGen algorithm describes the key generation procedure for the uBlock-based schemes. On input the public parameters param, the algorithm first samples a secret vector $\boldsymbol{k} \in \{0,1\}^{N_{\mathsf{block}}}$ uniformly at random, subject to the condition that the first two bits do not simultaneously satisfy $k[0] \wedge k[1] = 1$. It then samples an independent vector $\boldsymbol{x} \in \{0,1\}^{N_{\mathsf{block}}}$ uniformly at random. The roles of these two strings depend on whether the encryption mode flag EM is true.

If $\mathsf{EM} = \mathsf{true}$, the algorithm computes $\boldsymbol{y} \leftarrow \mathsf{uBlock}_{\boldsymbol{x}}(\boldsymbol{k}) \oplus \boldsymbol{k}$, sets the secret key to $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$, and defines the public key as $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$. In this case, $\boldsymbol{k}$ is the secret input block, while $\boldsymbol{x}$ is the public uBlock key. Otherwise, when $\mathsf{EM} = \mathsf{false}$, the algorithm computes $\boldsymbol{y} \leftarrow \mathsf{uBlock}_{\boldsymbol{k}}(\boldsymbol{x})$, sets $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$, and outputs $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$. In this case, $\boldsymbol{k}$ is the secret uBlock key, while $\boldsymbol{x}$ is the public input block. Finally, the algorithm returns the pair $(\mathsf{sk}, \mathsf{pk})$.

7.6.2 Signing

The public-key argument is the public key derived from the secret key by the corresponding OWF.

Both uBlockith variants use degree-3 constraints, so the VOLE length is $\hat{\ell} = \ell + 3\lambda_{\mathbb{F}} + B$ and the signature contains two QuickSilver responses, $\tilde{a}_1$ and $\tilde{a}_2$. The witness and OWF-proof routines are the uBlock-specific algorithms uBlockith.ExtendWitness and uBlockith.OWFProve defined above.

7.6.3 Verifying

The verification algorithm of uBlockith differs from that of SM4th in a fully analogous way. First, the verifier reconstructs the VOLE transcript under the parameter choice $\hat{\ell} = \ell + 3\lambda_{\mathbb{F}} + B$. Second, after reconstructing $\boldsymbol{Q}$, the verifier invokes the uBlock-specific OWF verification algorithm

$$\tilde{a}_0 \leftarrow \text{uBlockith.OWFVerify}(\boldsymbol{d}_{\mathsf{w}}, \boldsymbol{Q}_{[0,\ell+2\lambda_{\mathbb{F}})}, \mathsf{pk}, \mathsf{chall}_2, \mathsf{chall}_3, \tilde{a}_1, \tilde{a}_2; \mathsf{param}),$$

uBlockith.KeyGen(param)

INPUT: Public parameters param
OUTPUT: Secret key $\mathsf{sk} \in (\{0,1\}^{N_{\mathsf{block}}}, \{0,1\}^{N_{\mathsf{block}}})$ and public key $\mathsf{pk} \in (\{0,1\}^{N_{\mathsf{block}}}, \{0,1\}^{N_{\mathsf{block}}})$, or $\perp$

repeat
  $\boldsymbol{k} \leftarrow \{0,1\}^{N_{\mathsf{block}}}$
  if sampling fails **then**
    return $\perp$
  end if
until $\boldsymbol{k}[0] \wedge \boldsymbol{k}[1] = 0$
$\boldsymbol{x} \leftarrow \{0,1\}^{N_{\mathsf{block}}}$
if sampling fails **then**
  return $\perp$
end if
if $\mathsf{EM} = \mathsf{true}$ **then**
  $\boldsymbol{y} \leftarrow \mathsf{uBlock}_{\boldsymbol{x}}(\boldsymbol{k}) \oplus \boldsymbol{k}$
  $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$ // public uBlock key, secret input block
  $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$ // public key block, OWF output
else
  $\boldsymbol{y} \leftarrow \mathsf{uBlock}_{\boldsymbol{k}}(\boldsymbol{x})$
  $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$ // public input block, secret uBlock key
  $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$ // public input block, OWF output
end if
return $(\mathsf{sk}, \mathsf{pk})$

which now takes both transmitted proof components $\tilde{a}_1$ and $\tilde{a}_2$ as input. Finally, it recomputes

$$\mathsf{chall}_3' \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \tilde{a}_1 \,\|\, \tilde{a}_2 \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda_{\mathbb{F}}),$$

and, after all preceding checks succeed, accepts if and only if $\mathsf{chall}_3' = \mathsf{chall}_3$.

Both uBlockith and uBlockith-EM use degree-3 constraints, an extended VOLE length of $\ell + 3\lambda_{\mathbb{F}} + B$, and the transmitted responses $\tilde{a}_1, \tilde{a}_2$.

8 Algorithm Description: the Vistrutith Signature Scheme

We present the construction of Vistrutith in this section.

8.1 Zero-Knowledge Building Blocks for Vistrutith

This section presents several VOLEitH-based ZKP building blocks for Vistrutah operations. For Vistrutith, we have $\lambda = \lambda_{\mathbb{F}} = N_{\mathsf{block}} = 512$, so we use λ to represent all of them for ease of notation. Byte arithmetic uses $m(t) = t^8 + t^4 + t^3 + t + 1$ from Equation (25).

ByteCombine. The Vistrutith.ByteCombine function maps eight committed bits to a commitment to the corresponding byte embedded in $\mathbb{F}_{2^\lambda}$, using a Vistrutah-compatible embedding analogous to Definition 3.

Let the map $\phi : \mathrm{GF}(2^8) \to \mathbb{F}_2[\beta_8]$ defined by $\phi\left(\sum_{i=0}^{7} b_i x^i\right) = \sum_{i=0}^{7} b_i \beta_8^i$. Similar to Lemma 5, for any $a, b, c \in \mathbb{F}_2[\beta_8]$ and $A = \phi^{-1}(a), B = \phi^{-1}(b), C = \phi^{-1}(c) \in \mathrm{GF}(2^8)$, we have

$$a \cdot_\lambda b = c \quad \Longleftrightarrow \quad A \cdot_8 B = C,$$

where $\cdot_8$ and $\cdot_\lambda$ denote multiplications in $\mathrm{GF}(2^8)$ modulo $m(t)$ as in Equation (25) and $\mathbb{F}_{2^\lambda}$. Hence, to prove $A \cdot_8 B = C$ (similar to the form of patched inversion), it suffices to prove $a \cdot_\lambda b = c$ using the QuickSilver proof given the VOLE commitments $\langle a \rangle_\lambda, \langle b \rangle_\lambda$ and $\langle c \rangle_\lambda$. Table 15 gives the Vistrutah-compatible generator β_8 in the similar form with Table 12.

uBlockith.Sign($\mathsf{msg}, \mathsf{sk}, \mathsf{pk}; \mathsf{param}$)

INPUT: Message msg, secret key $\mathsf{sk} = (\mathsf{sk}_1, \mathsf{sk}_2)$, public key $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$, public parameters param
OUTPUT: Signature σ or $\perp$

if $|\mathsf{msg}|_{\mathsf{bytes}} > 2^{60}$ or $\mathsf{sk}_2[0] \wedge \mathsf{sk}_2[1] = 1$ **then**
  return $\perp$
end if
$\rho \leftarrow \{0,1\}^{\xi}$ // Signature randomness; $\xi = 384$
if sampling fails **then**
  return $\perp$
end if
$\mu \leftarrow \mathsf{H}_2^0(D_{\mathsf{p}} \,\|\, \mathsf{pk} \,\|\, \mathsf{msg}; 2\lambda)$
$(r, \mathsf{iv}^{\mathsf{pre}}) \leftarrow \mathsf{H}_3(\mathsf{sk}_2 \,\|\, \mu \,\|\, \rho; N_{\mathsf{block}} + \lambda_{\mathsf{iv}})$
$\mathsf{iv} \leftarrow \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}}; \lambda_{\mathsf{iv}})$
$(\mathsf{com}, \mathsf{decom}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \boldsymbol{u}, \boldsymbol{V}) \leftarrow \mathsf{VOLECommit}(r, \mathsf{iv}, \ell + 3\lambda_{\mathbb{F}} + B; \mathsf{param})$ // $d = 3$ for uBlockith
$\mathsf{chall}_1 \leftarrow \mathsf{H}_2^1(\mu \,\|\, \mathsf{com} \,\|\, \boldsymbol{c}_1 \,\|\, \cdots \,\|\, \boldsymbol{c}_{\tau-1} \,\|\, \mathsf{iv}; 5\lambda_{\mathbb{F}} + 64)$
// VOLE consistency check
$\tilde{\boldsymbol{u}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{u}) \in \{0,1\}^{\lambda_{\mathbb{F}}+B}$
$\tilde{\boldsymbol{V}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{V}) \in \{0,1\}^{(\lambda_{\mathbb{F}}+B)\times\lambda_{\mathbb{F}}}$
if $\mathsf{EM} = \mathsf{true}$ **then**
  $\boldsymbol{w} \leftarrow \mathsf{uBlockith.ExtendWitness}(\mathsf{sk}_1, \mathsf{sk}_2; \mathsf{param}) \in \{0,1\}^{\ell}$
else
  $\boldsymbol{w} \leftarrow \mathsf{uBlockith.ExtendWitness}(\mathsf{sk}_2, \mathsf{sk}_1; \mathsf{param}) \in \{0,1\}^{\ell}$
end if
$\boldsymbol{d}_{\mathsf{w}} \leftarrow \boldsymbol{w} \oplus \boldsymbol{u}_{[0,\ell)}$ // Mask extended witness
$\mathsf{chall}_2 \leftarrow \mathsf{H}_2^2(\mathsf{chall}_1 \,\|\, \tilde{\boldsymbol{u}} \,\|\, \tilde{\boldsymbol{V}} \,\|\, \boldsymbol{d}_{\mathsf{w}}; 3\lambda_{\mathbb{F}} + 64)$
// QuickSilver proof
$\boldsymbol{u} \leftarrow \boldsymbol{u}_{[\ell,\ell+2\lambda_{\mathbb{F}})}$ // keep the two mask blocks
$\boldsymbol{V} \leftarrow \boldsymbol{V}_{[0,\ell+2\lambda_{\mathbb{F}})} \in \{0,1\}^{(\ell+2\lambda_{\mathbb{F}})\times\lambda_{\mathbb{F}}}$ // drop the last $\lambda_{\mathbb{F}} + B$ rows
$(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2) \leftarrow \mathsf{uBlockith.OWFProve}(\boldsymbol{w}, \boldsymbol{u}, \boldsymbol{V}, \mathsf{pk}, \mathsf{chall}_2; \mathsf{param})$
for $\mathsf{ctr} \in [0, 2^{32})$ in increasing order **do**
  $\mathsf{chall}_3 \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \tilde{a}_1 \,\|\, \tilde{a}_2 \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda_{\mathbb{F}})$
  if $\mathsf{chall}_3[\lambda_{\mathbb{F}} - w_{\mathsf{grind}}, \lambda_{\mathbb{F}}) = 0^{w_{\mathsf{grind}}}$ **then**
    $I \leftarrow \mathsf{DecodeChall}(\mathsf{chall}_3; \mathsf{param})$
    $\mathsf{decom}_I \leftarrow \mathsf{BAVC.Open}(\mathsf{decom}, I; \mathsf{param})$
    if $\mathsf{decom}_I \neq \perp$ **then**
      return $\sigma \leftarrow (\{\boldsymbol{c}_i\}_{i\in[1,\tau)}, \tilde{\boldsymbol{u}}, \boldsymbol{d}_{\mathsf{w}}, \tilde{a}_1, \tilde{a}_2, \mathsf{decom}_I, \mathsf{chall}_3, \mathsf{iv}^{\mathsf{pre}}, \mathsf{ctr})$
    end if
  end if
end for
return $\perp$

uBlockith.Verify($\mathsf{msg}, \mathsf{pk}, \sigma; \mathsf{param}$)

INPUT: Message msg, public key pk, signature σ, public parameters param
OUTPUT: true or false

if $|\mathsf{msg}|_{\mathsf{bytes}} > 2^{60}$ or $|\mathsf{pk}|_{\mathsf{bytes}} \neq 64$ or $|\sigma|_{\mathsf{bytes}} \neq \mathsf{sigsize}(\mathsf{param})$ **then**
  return false
end if
$\hat{\ell} \leftarrow \ell + 3\lambda_{\mathbb{F}} + B$
Parse $\sigma = \left(\{\boldsymbol{c}_i\}_{i\in[1,\tau)}, \tilde{\boldsymbol{u}}, \boldsymbol{d}_{\mathsf{w}}, \tilde{a}_1, \tilde{a}_2, \mathsf{decom}_I, \mathsf{chall}_3, \mathsf{iv}^{\mathsf{pre}}, \mathsf{ctr}\right)$
$\mu \leftarrow \mathsf{H}_2^0(D_{\mathsf{p}} \,\|\, \mathsf{pk} \,\|\, \mathsf{msg}; 2\lambda)$
$\mathsf{iv} \leftarrow \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}}; \lambda_{\mathsf{iv}})$
if $\mathsf{chall}_3[\lambda_{\mathbb{F}} - w_{\mathsf{grind}}, \lambda_{\mathbb{F}}) \neq 0^{w_{\mathsf{grind}}}$ **then**
  return false
end if
// Reconstruct VOLEs and check commitment
$(\mathsf{com}, \boldsymbol{Q}) \leftarrow$ VOLEReconstruct$(\mathsf{chall}_3, \mathsf{decom}_I, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv}, \hat{\ell}; \mathsf{param})$
if VOLEReconstruct returned $\perp$ **then**
  return false
end if
$\mathsf{chall}_1 \leftarrow \mathsf{H}_2^1(\mu \,\|\, \mathsf{com} \,\|\, \boldsymbol{c}_1 \,\|\, \cdots \,\|\, \boldsymbol{c}_{\tau-1} \,\|\, \mathsf{iv}; 5\lambda_{\mathbb{F}} + 64)$
// Apply the VOLE correction values and check consistency
$\tilde{\boldsymbol{Q}} \leftarrow$ VOLEHash$(\mathsf{chall}_1, \boldsymbol{Q}) \in \{0,1\}^{(\lambda_{\mathbb{F}}+B)\times\lambda_{\mathbb{F}}}$
Parse $\mathsf{chall}_3 = (\delta_0, \ldots, \delta_{\lambda_{\mathbb{F}}-1}) \in \{0,1\}^{\lambda_{\mathbb{F}}}$
$\tilde{\boldsymbol{V}}' \leftarrow \tilde{\boldsymbol{Q}} \oplus [\delta_0 \cdot \tilde{\boldsymbol{u}} \;\cdots\; \delta_{\lambda_{\mathbb{F}}-1} \cdot \tilde{\boldsymbol{u}}]$
$\mathsf{chall}_2 \leftarrow \mathsf{H}_2^2(\mathsf{chall}_1 \,\|\, \tilde{\boldsymbol{u}} \,\|\, \tilde{\boldsymbol{V}}' \,\|\, \boldsymbol{d}_{\mathsf{w}}; 3\lambda_{\mathbb{F}} + 64)$
// Compute $\tilde{a}_0$
$\tilde{a}_0 \leftarrow$ uBlockith.OWFVerify$(\boldsymbol{d}_{\mathsf{w}}, \boldsymbol{Q}_{[0,\ell+2\lambda_{\mathbb{F}})}, \mathsf{pk}, \mathsf{chall}_2, \mathsf{chall}_3, \tilde{a}_1, \tilde{a}_2; \mathsf{param})$
Compute $\mathsf{chall}_3' \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \tilde{a}_1 \,\|\, \tilde{a}_2 \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda_{\mathbb{F}})$
return true if $\mathsf{chall}_3 = \mathsf{chall}_3'$ else false

Vistrutith.ByteCombine$(\langle x_0\rangle, \ldots, \langle x_7\rangle; \lambda)$

INPUT: Commitment $\langle x_i\rangle$ where $x_i \in \mathbb{F}_2$, for $i \in [0\,..\,7]$; the security parameter λ
OUTPUT: Commitment $\langle x\rangle_\lambda$ to $x \in \mathbb{F}_{2^\lambda}$, containing an embedded $\mathbb{F}_{2^8}$ element

let $\beta_8 \in \mathbb{F}_{2^\lambda}$ // Vistrutah-compatible basis element of $\mathbb{F}_{2^8}$ within $\mathbb{F}_{2^\lambda}$
return $\langle x\rangle_\lambda = \sum_{i=0}^{7} \langle x_i\rangle \cdot \beta_8^i$

Vistrutith.SquareBits$(\langle x_0\rangle, \ldots, \langle x_7\rangle)$

INPUT: Committed bits $\langle x_i\rangle$ for $i \in [0\,..\,7]$
OUTPUT: Commitments $\langle y_i\rangle$ to the bits of the squared byte, for $i \in [0\,..\,7]$

$\langle y_0\rangle \leftarrow \langle x_0\rangle + \langle x_4\rangle + \langle x_6\rangle$
$\langle y_1\rangle \leftarrow \langle x_4\rangle + \langle x_6\rangle + \langle x_7\rangle$
$\langle y_2\rangle \leftarrow \langle x_1\rangle + \langle x_5\rangle$
$\langle y_3\rangle \leftarrow \langle x_4\rangle + \langle x_5\rangle + \langle x_6\rangle + \langle x_7\rangle$
$\langle y_4\rangle \leftarrow \langle x_2\rangle + \langle x_4\rangle + \langle x_7\rangle$
$\langle y_5\rangle \leftarrow \langle x_5\rangle + \langle x_6\rangle$
$\langle y_6\rangle \leftarrow \langle x_3\rangle + \langle x_5\rangle$
$\langle y_7\rangle \leftarrow \langle x_6\rangle + \langle x_7\rangle$
return $\langle y_0\rangle, \ldots, \langle y_7\rangle$

Table 15: Vistrutah-Compatible Generator β_8 for Vistrutith.ByteCombine when $\lambda = 512$

Hexadecimal (LSB)	Polynomial Form in $\mathbb{F}_{2^\lambda}$
`48 ab f0 fb db 90 0a d1 86 e2 84 b7 31`	$z^{511} + z^{510} + z^{509} + z^{505} + z^{504} + z^{501} + z^{500} + z^{498}$
`8a 91 c6 02 e6 ad 2a 69 70 be fe 70 d1`	$z^{497} + z^{496} + z^{495} + z^{492} + z^{491} + z^{487} + z^{486} + z^{485}$
`92 ee 0a d8 66 50 6f a1 f0 f5 80 10 f5`	$z^{484} + z^{482} + z^{479} + z^{478} + z^{477} + z^{475} + z^{474} + z^{468}$
`98 1f d7 06 e4 0a 54 b1 40 cb 76 c7 0f`	$z^{467} + z^{466} + z^{463} + z^{461} + z^{460} + z^{459} + z^{457} + z^{456}$
`21 42 e0 3b d9 bb 1c ec f4 98 37 e3`	$z^{455} + z^{454} + z^{452} + z^{451} + z^{448} + z^{445} + z^{444} + z^{443}$
	$z^{441} + z^{440} + z^{439} + z^{438} + z^{437} + z^{430} + z^{425} + z^{421}$
	$z^{416} + z^{411} + z^{410} + z^{409} + z^{408} + z^{407} + z^{406} + z^{402}$
	$z^{401} + z^{400} + z^{398} + z^{397} + z^{396} + z^{394} + z^{393} + z^{391}$
	$z^{390} + z^{387} + z^{385} + z^{384} + z^{382} + z^{375} + z^{373} + z^{372}$
	$z^{368} + z^{366} + z^{364} + z^{362} + z^{355} + z^{353} + z^{351} + z^{350}$
	$z^{349} + z^{346} + z^{338} + z^{337} + z^{335} + z^{334} + z^{332} + z^{330}$
	$z^{329} + z^{328} + z^{324} + z^{323} + z^{322} + z^{321} + z^{320} + z^{319}$
	$z^{316} + z^{315} + z^{311} + z^{310} + z^{309} + z^{308} + z^{306} + z^{304}$
	$z^{300} + z^{295} + z^{287} + z^{286} + z^{285} + z^{284} + z^{282} + z^{280}$
	$z^{279} + z^{278} + z^{277} + z^{276} + z^{271} + z^{269} + z^{264} + z^{262}$
	$z^{261} + z^{259} + z^{258} + z^{257} + z^{256} + z^{254} + z^{252} + z^{246}$
	$z^{245} + z^{242} + z^{241} + z^{239} + z^{238} + z^{236} + z^{235} + z^{227}$
	$z^{225} + z^{223} + z^{222} + z^{221} + z^{219} + z^{218} + z^{217} + z^{215}$
	$z^{212} + z^{209} + z^{207} + z^{206} + z^{204} + z^{200} + z^{198} + z^{197}$
	$z^{196} + z^{191} + z^{190} + z^{189} + z^{188} + z^{187} + z^{186} + z^{185}$
	$z^{183} + z^{181} + z^{180} + z^{179} + z^{178} + z^{177} + z^{174} + z^{173}$
	$z^{172} + z^{166} + z^{165} + z^{163} + z^{160} + z^{157} + z^{155} + z^{153}$
	$z^{151} + z^{149} + z^{147} + z^{146} + z^{144} + z^{143} + z^{142} + z^{141}$
	$z^{138} + z^{137} + z^{129} + z^{127} + z^{126} + z^{122} + z^{121} + z^{119}$
	$z^{116} + z^{112} + z^{111} + z^{107} + z^{105} + z^{101} + z^{100} + z^{96}$
	$z^{95} + z^{93} + z^{92} + z^{90} + z^{89} + z^{88} + z^{87} + z^{82}$
	$z^{79} + z^{78} + z^{77} + z^{73} + z^{71} + z^{66} + z^{65} + z^{63}$
	$z^{62} + z^{60} + z^{56} + z^{51} + z^{49} + z^{47} + z^{44} + z^{39}$
	$z^{38} + z^{36} + z^{35} + z^{33} + z^{32} + z^{31} + z^{30} + z^{29}$
	$z^{28} + z^{27} + z^{25} + z^{24} + z^{23} + z^{22} + z^{21} + z^{20}$
	$z^{15} + z^{13} + z^{11} + z^9 + z^8 + z^6 + z^3$

SquareBits. The function Vistrutith.SquareBits takes commitments to the eight bits of a byte x and returns commitments to the eight bits representing x^2 in the AES byte field. Byte-field squaring is $\mathbb{F}_2$-linear and preserves the commitment degree.

SquareAndCombine. The Vistrutith.SquareAndCombine function first calls Vistrutith.SquareBits to square the committed byte. It then embeds the result in $\mathbb{F}_{2^\lambda}$ using Vistrutith.ByteCombine, returning a commitment to the squared byte. It differs from SquareAndCombine in the byte-field polynomial given in Equation (25) and the embedding into the proof field.

Vistrutith.SquareAndCombine($\langle x_0\rangle, \ldots, \langle x_7\rangle$)

INPUT: Committed bits $\langle x_i\rangle$ for $i \in [0\,..\,7]$
OUTPUT: Commitment $\langle y\rangle_\lambda$ to the squared field element $y \in \mathbb{F}_{2^\lambda}$

1: $\langle y_0\rangle, \ldots, \langle y_7\rangle \leftarrow$ Vistrutith.SquareBits($\langle x_0\rangle, \ldots, \langle x_7\rangle$)
2: **return** Vistrutith.ByteCombine($\langle y_0\rangle, \ldots, \langle y_7\rangle$)

StateToBytes. The function Vistrutith.StateToBytes groups the 128 committed state bits into sixteen consecutive bytes and applies Vistrutith.ByteCombine to each group, preserving their order.

Vistrutith.StateToBytes($\langle \boldsymbol{s}\rangle$)

INPUT: $\langle \boldsymbol{s}\rangle$, a state array of 128 committed bits
OUTPUT: $\langle \boldsymbol{out}\rangle_\lambda$, an array of 16 committed bytes

1: **new** $\langle \boldsymbol{out}\rangle_\lambda$, an array of 16 committed bytes
2: **for** $i \in [0\,..\,16)$ **do**
3: $\quad\langle \boldsymbol{out}[i]\rangle_\lambda \leftarrow$ Vistrutith.ByteCombine($\langle \boldsymbol{s}[8i]\rangle, \ldots, \langle \boldsymbol{s}[8i+7]\rangle$)
4: **end for**
5: **return** $\langle \boldsymbol{out}\rangle$

InvNorm. For a finite field extension $\mathbb{F}_{q^m}/\mathbb{F}_q$, the field norm is the map

$$N_{\mathbb{F}_{q^m}/\mathbb{F}_q}(u) = u^{1+q+q^2+\cdots+q^{m-1}}.$$

Its output always lies in the base field $\mathbb{F}_q$.

The function Vistrutith.InvNorm extracts a 4-bit representation of the norm of the inverse of an 8-bit field element. More precisely, the input $\boldsymbol{x} \in \{0,1\}^8$ is first interpreted as an element x of $\mathbb{F}_{2^8}$ via ToField($\boldsymbol{x}, 8$), where $\mathbb{F}_{2^8}$ is viewed as a degree-2 extension of the subfield $\mathbb{F}_{2^4}$. If $x = 0$, then Vistrutith.InvNorm returns $(0,0,0,0)$. Otherwise, it computes the norm of x^{-1}, *i.e.*, x^{-17}, from $\mathbb{F}_{2^8}$ down to $\mathbb{F}_{2^4}$. Recall that for an element $u \in \mathbb{F}_{2^8}$, the field norm from $\mathbb{F}_{2^8}$ to $\mathbb{F}_{2^4}$ is defined by

$$N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(u) = u^{1+2^4} = u^{17}.$$

Hence, for any non-zero $x \in \mathbb{F}_{2^8}$,

$$N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(x^{-1}) = (x^{-1})^{17} = x^{-17}.$$

Since the norm always lies in the subfield $\mathbb{F}_{2^4}$, x^{-17} can be represented using only four bits.

To obtain these four bits, we fix a basis $b = (1, \beta, \beta^2, \beta^3)$ of $\mathbb{F}_{2^4}$, where $\beta = \alpha^6 + \alpha^4$ and α is the basis element of $\mathbb{F}_{2^8}$. The role of this basis is to provide a coordinate system for elements of the subfield $\mathbb{F}_{2^4}$. That is, every element of $\mathbb{F}_{2^4}$ can be written uniquely as

$$y_0 \cdot 1 + y_1 \cdot \beta + y_2 \cdot \beta^2 + y_3 \cdot \beta^3, \qquad y_i \in \mathbb{F}_2.$$

Thus, if $\boldsymbol{y} = (y_0, y_1, y_2, y_3) =$ Vistrutith.InvNorm($\boldsymbol{x}$), then $\boldsymbol{y}$ should be understood as the coordinate vector of $N_{\mathbb{F}_{2^8}/\mathbb{F}_{2^4}}(x^{-1})$ with respect to the basis b.

Concretely, after computing the bit decomposition $\boldsymbol{y}' = \mathsf{ToBits}(1/x^{17})$, the algorithm outputs

$$\boldsymbol{y} = (y'[0], y'[6], y'[7], y'[2]).$$

These four selected bits are exactly the coordinates of the norm value with respect to the chosen basis b. In this way, one can compute $x^{-17} \in \mathbb{F}_{2^4} \subseteq \mathbb{F}_{2^8}$.

$\mathsf{Vistrutith.InvNorm}(\boldsymbol{x})$

Input: $\boldsymbol{x} \in \{0,1\}^8$
Output: $\boldsymbol{y} \in \{0,1\}^4$

1: $x \leftarrow \mathsf{ToField}(\boldsymbol{x}, 8)$
2: **if** $x = 0$ **then**
3: **return** $(0,0,0,0)$
4: **end if**
5: $\boldsymbol{y}' \leftarrow \mathsf{ToBits}(1/x^{17})$
6: $\boldsymbol{y} \leftarrow (\boldsymbol{y}'[0], \boldsymbol{y}'[6], \boldsymbol{y}'[7], \boldsymbol{y}'[2])$
7: **return** $\boldsymbol{y}$

InvNormCombine. The function Vistrutith.InvNormCombine lifts a 4-bit VOLE commitment representing an element of the subfield $\mathbb{F}_{2^4}$ to a commitment in $\mathbb{F}_{2^\lambda}$. Let $\beta_8 \in \mathbb{F}_{2^\lambda}$ be the generator used in Vistrutith.ByteCombine, and define $\beta_4 = \beta_8^6 + \beta_8^4$. Then $(1, \beta_4, \beta_4^2, \beta_4^3)$ forms a basis of the embedded copy of $\mathbb{F}_{2^4}$ inside $\mathbb{F}_{2^\lambda}$. Recall that this commitment is Vistrutah-compatible. When the input is the output of Vistrutith.InvNorm, the resulting commitment corresponds to the element x^{-17} embedded in $\mathbb{F}_{2^\lambda}$.

$\mathsf{Vistrutith.InvNormCombine}(\langle x_0\rangle, \langle x_1\rangle, \langle x_2\rangle, \langle x_3\rangle)$

Input: $\langle x_0\rangle, \langle x_1\rangle, \langle x_2\rangle, \langle x_3\rangle$, where $x_i \in \mathbb{F}_2$
Output: $\langle y\rangle_\lambda$, where $y \in \mathbb{F}_{2^\lambda}$

1: Let $\beta_8 \in \mathbb{F}_{2^\lambda}$ be the generator used in Vistrutith.ByteCombine
2: Let $\beta_4 \leftarrow \beta_8^6 + \beta_8^4$
3: **return** $\langle y\rangle_\lambda \leftarrow \sum_{i=0}^{3} \langle x_i\rangle \cdot \beta_4^i$

InvNormToConjugates. Let $x \in \mathbb{F}_{2^m}$. The Frobenius map over $\mathbb{F}_2$ is the field automorphism $x \mapsto x^2$. The *Frobenius conjugates* of x are the elements obtained by repeated applications of this map, namely $x,\ x^2,\ x^{2^2},\ \ldots,\ x^{2^{m-1}}$. In particular, for $x \in \mathbb{F}_{2^4}$, its conjugates are $x,\ x^2,\ x^4,\ x^8$, and for $x \in \mathbb{F}_{2^8}$, its conjugates are $x,\ x^2,\ x^4,\ x^8,\ x^{16},\ x^{32},\ x^{64},\ x^{128}$.

The function Vistrutith.InvNormToConjugates lifts a 4-bit VOLE commitment, representing an element of the subfield $\mathbb{F}_{2^4}$, to $\mathbb{F}_{2^\lambda}$ and computes all of its Frobenius conjugates. The input $\langle \boldsymbol{x}\rangle = (\langle x_0\rangle, \langle x_1\rangle, \langle x_2\rangle, \langle x_3\rangle)$ is interpreted as the coordinate vector of an element $y = x_0 + x_1\beta_4 + x_2\beta_4^2 + x_3\beta_4^3 \in \mathbb{F}_{2^4}$, where $\beta_4 = \beta_8^6 + \beta_8^4$ and $\beta_8 \in \mathbb{F}_{2^\lambda}$ is the generator used in Vistrutith.ByteCombine. The function Vistrutith.InvNormCombine returns a commitment $\langle y\rangle_\lambda$. The function Vistrutith.InvNormToConjugates returns the four commitments $(\langle y_0\rangle_\lambda, \langle y_1\rangle_\lambda, \langle y_2\rangle_\lambda, \langle y_3\rangle_\lambda)$, where $y_0 = y$, $y_1 = y^2$, $y_2 = y^4$, and $y_3 = y^8$. This follows from the fact that in characteristic 2, the Frobenius map $u \mapsto u^2$ acts coefficient-wise on the basis expansion, so repeated squaring transforms the basis elements as

$$1,\ \beta_4,\ \beta_4^2,\ \beta_4^3 \quad \mapsto \quad 1,\ \beta_4^{2^j},\ \beta_4^{2^{j+1}},\ \beta_4^{3\cdot 2^j}.$$

StateToConjugates. The function Vistrutith.StateToConjugates computes the Frobenius conjugates of every byte in a committed 128-bit state. The input $\langle \boldsymbol{s}\rangle$ is viewed as 16 consecutive committed bytes, each represented by 8 committed bits. For the i-th byte, the algorithm first extracts $\langle \boldsymbol{x}_0\rangle = (\langle \boldsymbol{s}[8i]\rangle, \ldots, \langle \boldsymbol{s}[8i+7]\rangle)$, which is interpreted as an element $x_i \in \mathbb{F}_{2^8}$.

Vistrutith.InvNormToConjugates($\langle \boldsymbol{x} \rangle$)

INPUT: $\langle \boldsymbol{x} \rangle = (\langle x_0 \rangle, \langle x_1 \rangle, \langle x_2 \rangle, \langle x_3 \rangle)$, where $x_i \in \mathbb{F}_2$
OUTPUT: $(\langle y_0 \rangle_\lambda, \langle y_1 \rangle_\lambda, \langle y_2 \rangle_\lambda, \langle y_3 \rangle_\lambda)$, where $y_j \in \mathbb{F}_{2^\lambda}$

Let $\beta_8 \in \mathbb{F}_{2^\lambda}$ be the generator used in Vistrutith.ByteCombine
Let $\beta_4 \leftarrow \beta_8^6 + \beta_8^4$
for $j \in [0\,..\,4)$ **do**
    $\langle y_j \rangle_\lambda \leftarrow \sum_{i=0}^{3} \langle x_i \rangle \cdot \beta_4^{i \cdot 2^j}$
end for
return $(\langle y_0 \rangle_\lambda, \langle y_1 \rangle_\lambda, \langle y_2 \rangle_\lambda, \langle y_3 \rangle_\lambda)$

It then computes the sequence

$$x_i,\ x_i^2,\ x_i^4,\ x_i^8,\ x_i^{16},\ x_i^{32},\ x_i^{64},\ x_i^{128}$$

by repeated squaring in $\mathbb{F}_{2^8}$. At each stage, the current 8-bit representation is lifted to a VOLE commitment in $\mathbb{F}_{2^\lambda}$ using Vistrutith.ByteCombine. Thus, for each byte index i, the output consists of the 8 commitments $\langle y_{i,0} \rangle_\lambda, \langle y_{i,1} \rangle_\lambda, \ldots, \langle y_{i,7} \rangle_\lambda$, where $y_{i,j} = x_i^{2^j}$.

Vistrutith.StateToConjugates($\langle \boldsymbol{s} \rangle$)

INPUT: $\langle \boldsymbol{s} \rangle$, an array of 128 VOLE commitments to the bits of a state
OUTPUT: $\langle y_{i,j} \rangle_\lambda$ for $i \in [0\,..\,16)$ and $j \in [0\,..\,8)$

for $i \in [0\,..\,16)$ **do**
    $\langle \boldsymbol{x}_0 \rangle \leftarrow (\langle \boldsymbol{s}[8i] \rangle, \ldots, \langle \boldsymbol{s}[8i+7] \rangle)$
    for $j \in [0\,..\,8)$ **do**
        $\langle y_{i,j} \rangle_\lambda \leftarrow$ Vistrutith.ByteCombine($\langle \boldsymbol{x}_j \rangle$)
        if $j < 7$ **then**
            $\langle \boldsymbol{x}_{j+1} \rangle \leftarrow$ Vistrutith.SquareBits($\langle \boldsymbol{x}_j \rangle$)
        end if
    end for
end for
return $(\langle y_{0,0} \rangle_\lambda, \ldots, \langle y_{15,7} \rangle_\lambda)$

SBoxAffine. The function Vistrutith.SBoxAffine applies the affine component of the AES S-box to committed outputs of the patched field inversion. Its input is not given as raw bytes, but as a collection of Frobenius conjugates. More precisely, for each position $i \in [0..n)$, the input contains a tuple

$$\langle \boldsymbol{x}_i \rangle_\lambda^2 = (\langle x_{i,0} \rangle_\lambda^2, \ldots, \langle x_{i,7} \rangle_\lambda^2),$$

where $x_{i,j} = x_{i,0}^{2^j}$. Thus, each input byte is represented by the full conjugate family

$$x_{i,0},\ x_{i,0}^2,\ x_{i,0}^{2^2},\ \ldots,\ x_{i,0}^{2^7}.$$

Each $x_{i,j}$ lies in the embedded copy of $\mathbb{F}_{2^8}$ inside $\mathbb{F}_{2^\lambda}$.

The reason for using such a conjugate representation is that the affine part of the AES S-box is a byte-wise affine transformation over $\mathbb{F}_2$. Equivalently, when the byte is viewed as an element z of $\mathbb{F}_{2^8}$, every $\mathbb{F}_2$-linear map on $\mathbb{F}_{2^8}$ can be written as a *linearized polynomial* of the form

$$L(z) = \sum_{k=0}^{7} \chi_k \cdot z^{2^k},$$

where $\chi_k \in \mathbb{F}_{2^8}$. Hence, the S-box affine map can be written as

$$A_{\mathsf{aff}}(z) = \chi_8 + \sum_{k=0}^{7} \chi_k \cdot z^{2^k}. \tag{35}$$

Solving the Moore-matrix linear system corresponding to Equation (24) yields

$$(\chi_0, \ldots, \chi_7) = (\texttt{05}, \texttt{09}, \texttt{f9}, \texttt{25}, \texttt{f4}, \texttt{01}, \texttt{b5}, \texttt{8f}), \qquad \chi_8 = \texttt{63}.$$

The Frobenius square of the affine output is

$$\big(A_{\mathsf{aff}}(z)\big)^2 = \chi_8^2 + \sum_{k=0}^{7} \chi_k^2 \cdot z^{2^{k+1}}.$$

Here, $(\chi_0^2, \ldots, \chi_8^2)$ can be computed as

$$(\chi_0^2, \ldots, \chi_7^2) = (\texttt{11}, \texttt{41}, \texttt{07}, \texttt{7d}, \texttt{56}, \texttt{01}, \texttt{fc}, \texttt{cf}), \qquad \chi_8^2 = \texttt{c2}.$$

When z is embedded into an element of $\mathbb{F}_{2^\lambda}$ via the Vistrutah-compatible transformation, Equation (35) holds with the embedded coefficients

$$\zeta_k = \phi(\chi_k), \qquad k \in [0, 9). \tag{36}$$

Vistrutith.SBoxAffine computes the affine layer by taking a linear combination of the committed conjugates with fixed coefficients $\zeta_0, \ldots, \zeta_7$, together with a constant term ζ_8. If the flag sq is set to false, the ordinary affine map is evaluated. If sq is set to true, the algorithm instead evaluates the Frobenius square of the affine map. This is achieved by squaring the coefficients and shifting the conjugate index by one.

Vistrutith.SBoxAffine($\langle \boldsymbol{x} \rangle_\lambda^2$; n, sq)

INPUT: $\langle \boldsymbol{x} \rangle_\lambda^2 = (\langle \boldsymbol{x}_0 \rangle_\lambda^2, \ldots, \langle \boldsymbol{x}_{n-1} \rangle_\lambda^2)$, where each $\langle \boldsymbol{x}_i \rangle_\lambda^2 = (\langle x_{i,0} \rangle_\lambda^2, \ldots, \langle x_{i,7} \rangle_\lambda^2)$ is a tuple of degree-2 commitments to the conjugates of some $x_{i,0} \in \mathbb{F}_{2^8} \subset \mathbb{F}_{2^\lambda}$, namely $x_{i,j} = x_{i,0}^{2^j}$ for $j \in [0..8)$; a flag $sq \in \{\mathsf{false}, \mathsf{true}\}$
OUTPUT: $\langle \boldsymbol{y} \rangle_\lambda^2 = (\langle y_0 \rangle_\lambda^2, \ldots, \langle y_{n-1} \rangle_\lambda^2)$, where each $y_i \in \mathbb{F}_{2^\lambda}$

1: **if** $sq = \mathsf{true}$ **then**
2: $\quad s \leftarrow 2, \quad t \leftarrow 1$
3: **else**
4: $\quad s \leftarrow 1, \quad t \leftarrow 0$
5: **end if**
6: Let $\zeta_0, \ldots, \zeta_8 \in \mathbb{F}_{2^\lambda}$ be the constants defining the affine map in Equation (36)
7: **for** $i \in [0..n)$ **do**
8: $\quad \langle y_i \rangle_\lambda^2 \leftarrow \zeta_8^s \cdot \mathsf{Lift}_2(\text{ConstantToVOLE}(1))$
9: $\quad$ **for** $k \in [0..8)$ **do**
10: $\quad\quad \langle y_i \rangle_\lambda^2 \leftarrow \langle y_i \rangle_\lambda^2 + \zeta_k^s \cdot \langle x_{i,(k+t) \bmod 8} \rangle_\lambda^2$ // shifting 1 means the square by definition
11: $\quad$ **end for**
12: **end for**
13: **return** $\langle \boldsymbol{y} \rangle_\lambda^2$

SBoxInvAffine. The function Vistrutith.SBoxInvAffine applies the inverse affine component of the AES S-box to each committed byte of the state. For each byte index i, the input consists of eight degree-1 commitments $\langle x_{i,0} \rangle, \ldots, \langle x_{i,7} \rangle$ to the bits of that byte. The output is another tuple of eight committed bits $\langle y_{i,0} \rangle, \ldots, \langle y_{i,7} \rangle$, corresponding to the inverse affine transformation.

By Equation (26), the inverse affine map is given bitwise by

$$y_{i,j} = x_{i,(j-1) \bmod 8} \oplus x_{i,(j-3) \bmod 8} \oplus x_{i,(j-6) \bmod 8} \oplus c'_j, \qquad 0 \le j < 8,$$

where $c'_j = 1$ for $j \in \{0, 2\}$ and $c'_j = 0$ otherwise. In the committed setting, the constant bit c'_j is converted into a VOLE commitment using ConstantToVOLE, and the whole transformation is evaluated bytewise over the state.

MixColumns. The function Vistrutith.MixColumns applies the MixColumns transformation to a sequence of committed state bytes. Its input is a vector $\langle \boldsymbol{x} \rangle_\lambda^2 = (\langle x_0 \rangle_\lambda^2, \ldots, \langle x_{15} \rangle_\lambda^2)$, where each

Vistrutith.SBoxInvAffine($\langle \boldsymbol{s} \rangle$)

INPUT: For each byte $i \in [0..16)$, commitments $\langle x_{i,0} \rangle, \ldots, \langle x_{i,7} \rangle$ to bits in $\mathbb{F}_2$
OUTPUT: For each byte i, commitments $\langle y_{i,0} \rangle, \ldots, \langle y_{i,7} \rangle$ to bits in $\mathbb{F}_2$

Parse $\langle \boldsymbol{s} \rangle$ as $\langle x_{i,j} \rangle = \langle \boldsymbol{s}[8i+j] \rangle$ for $i \in [0..16)$ and $j \in [0..8)$
for $i \in [0..16)$ **do**
    for $j \in [0..8)$ **do**
        $c_j \leftarrow 1$ if $j \in \{0, 2\}$, else 0
        $\langle y_{i,j} \rangle \leftarrow \langle x_{i,(j-1) \bmod 8} \rangle + \langle x_{i,(j-3) \bmod 8} \rangle + \langle x_{i,(j-6) \bmod 8} \rangle +$ ConstantToVOLE(c_j) // defined in Equation (26)
    end for
end for
return $(\langle y_{0,0} \rangle, \ldots, \langle y_{15,7} \rangle)$

entry is a degree-2 commitment to an element of $\mathbb{F}_{2^8}$, embedded in $\mathbb{F}_{2^\lambda}$. The output is another vector of degree-2 commitments $\langle \boldsymbol{y} \rangle^2_\lambda$, obtained by applying the column-wise linear transformation. The public coefficients are $\nu_1 = \phi(\mathsf{01})$, $\nu_2 = \phi(\mathsf{02})$, and $\nu_3 = \phi(\mathsf{03})$.

For $sq = \mathsf{false}$, the routine applies the ordinary MixColumns matrix. For $sq = \mathsf{true}$, it applies the matrix with each coefficient squared. On commitments to the bytewise squares $\boldsymbol{x}^{[2]}$, this branch returns commitments to $(\mathsf{MixColumns}(\boldsymbol{x}))^{[2]}$.

Vistrutith.MixColumns($\langle \boldsymbol{x} \rangle^2_\lambda$, sq)

INPUT: $\langle \boldsymbol{x} \rangle^2_\lambda = (\langle x_0 \rangle^2_\lambda, \ldots, \langle x_{15} \rangle^2_\lambda)$, where each $\langle x_i \rangle^2_\lambda$ is a degree-2 commitment to $x_i \in \mathbb{F}_{2^8} \subseteq \mathbb{F}_{2^\lambda}$; a flag $sq \in \{\mathsf{true}, \mathsf{false}\}$
OUTPUT: $\langle \boldsymbol{y} \rangle^2_\lambda = (\langle y_0 \rangle^2_\lambda, \ldots, \langle y_{15} \rangle^2_\lambda)$, where each $\langle y_i \rangle^2_\lambda$ is a degree-2 commitment to $y_i \in \mathbb{F}_{2^8} \subseteq \mathbb{F}_{2^\lambda}$

Let $\nu_1 \leftarrow \phi(\mathsf{01})$
Let $\nu_2 \leftarrow \phi(\mathsf{02})$
Let $\nu_3 \leftarrow \phi(\mathsf{03})$
if $sq = 1$ **then**
    $\nu_1 \leftarrow \nu_1^2, \quad \nu_2 \leftarrow \nu_2^2, \quad \nu_3 \leftarrow \nu_3^2$
end if
for $c \in [0..4)$ **do**
    $i_0 \leftarrow 4c, \quad i_1 \leftarrow 4c+1, \quad i_2 \leftarrow 4c+2, \quad i_3 \leftarrow 4c+3$
    $\langle y_{i_0} \rangle^2_\lambda \leftarrow \nu_2 \cdot \langle x_{i_0} \rangle^2_\lambda + \nu_3 \cdot \langle x_{i_1} \rangle^2_\lambda + \nu_1 \cdot \langle x_{i_2} \rangle^2_\lambda + \nu_1 \cdot \langle x_{i_3} \rangle^2_\lambda$
    $\langle y_{i_1} \rangle^2_\lambda \leftarrow \nu_1 \cdot \langle x_{i_0} \rangle^2_\lambda + \nu_2 \cdot \langle x_{i_1} \rangle^2_\lambda + \nu_3 \cdot \langle x_{i_2} \rangle^2_\lambda + \nu_1 \cdot \langle x_{i_3} \rangle^2_\lambda$
    $\langle y_{i_2} \rangle^2_\lambda \leftarrow \nu_1 \cdot \langle x_{i_0} \rangle^2_\lambda + \nu_1 \cdot \langle x_{i_1} \rangle^2_\lambda + \nu_2 \cdot \langle x_{i_2} \rangle^2_\lambda + \nu_3 \cdot \langle x_{i_3} \rangle^2_\lambda$
    $\langle y_{i_3} \rangle^2_\lambda \leftarrow \nu_3 \cdot \langle x_{i_0} \rangle^2_\lambda + \nu_1 \cdot \langle x_{i_1} \rangle^2_\lambda + \nu_1 \cdot \langle x_{i_2} \rangle^2_\lambda + \nu_2 \cdot \langle x_{i_3} \rangle^2_\lambda$
end for
return $\langle \boldsymbol{y} \rangle^2_\lambda$

BitInvMixColumns. The function Vistrutith.BitInvMixColumns implements the InvMixColumns transformation directly on committed state bits. The input is a committed 128-bit state $\langle \boldsymbol{s} \rangle$, viewed as 16 committed bytes arranged into 4 columns. For each column $c \in [0..4)$, the algorithm first extracts the four byte vectors $\langle \boldsymbol{a}^{(0)} \rangle, \ldots, \langle \boldsymbol{a}^{(3)} \rangle$, each consisting of 8 committed bits.

For each such byte $\langle \boldsymbol{a}^{(r)} \rangle$, the InvMixColumns step requires multiplication by the AES-field constants 09, 0b, 0d, and 0e. These multiplications are represented by fixed $\mathbb{F}_2$-linear maps $\Gamma_{\mathsf{09}}, \Gamma_{\mathsf{0b}}, \Gamma_{\mathsf{0d}}, \Gamma_{\mathsf{0e}} : \mathbb{F}_2^8 \to \mathbb{F}_2^8$. Concretely, for each column, the output bytes $\langle \boldsymbol{o}_{4c} \rangle, \ldots, \langle \boldsymbol{o}_{4c+3} \rangle$ are computed as linear combinations of the four input bytes after applying the corresponding maps Γ_η, according to the InvMixColumns matrix.

The concrete constructions of $\Gamma_{\mathsf{09}}, \Gamma_{\mathsf{0b}}, \Gamma_{\mathsf{0d}}, \Gamma_{\mathsf{0e}}$ are given below.

ShiftRows. The function Vistrutith.ShiftRows applies the AES ShiftRows transformation to a committed state represented as an ordered list of 16 committed bytes. The input is a tuple $\langle \boldsymbol{s} \rangle^2_\lambda$ of degree-2 VOLE commitments, where each committed value lies in $\mathbb{F}_{2^8} \subseteq \mathbb{F}_{2^\lambda}$. Internally, the state

Vistrutith.BitInvMixColumns($\langle \boldsymbol{s} \rangle$)

INPUT: $\langle \boldsymbol{s} \rangle = (\langle s_0 \rangle, \ldots, \langle s_{127} \rangle)$, a tuple of committed state bits
OUTPUT: $\langle \boldsymbol{o} \rangle = (\langle o_0 \rangle, \ldots, \langle o_{127} \rangle)$, the committed bits after inverse MixColumns

1: **for** $c \in [0\,..\,4)$ **do**
2: **for** $r \in [0\,..\,4)$ **do**
3: $\langle \boldsymbol{a}^{(r)} \rangle \leftarrow (\langle s_{32c+8r} \rangle, \ldots, \langle s_{32c+8r+7} \rangle)$
4: **end for**
5: $\langle \boldsymbol{o}_{4c+0} \rangle \leftarrow \Gamma_{\mathtt{0e}}(\langle \boldsymbol{a}^{(0)} \rangle) + \Gamma_{\mathtt{0b}}(\langle \boldsymbol{a}^{(1)} \rangle) + \Gamma_{\mathtt{0d}}(\langle \boldsymbol{a}^{(2)} \rangle) + \Gamma_{\mathtt{09}}(\langle \boldsymbol{a}^{(3)} \rangle)$
6: $\langle \boldsymbol{o}_{4c+1} \rangle \leftarrow \Gamma_{\mathtt{09}}(\langle \boldsymbol{a}^{(0)} \rangle) + \Gamma_{\mathtt{0e}}(\langle \boldsymbol{a}^{(1)} \rangle) + \Gamma_{\mathtt{0b}}(\langle \boldsymbol{a}^{(2)} \rangle) + \Gamma_{\mathtt{0d}}(\langle \boldsymbol{a}^{(3)} \rangle)$
7: $\langle \boldsymbol{o}_{4c+2} \rangle \leftarrow \Gamma_{\mathtt{0d}}(\langle \boldsymbol{a}^{(0)} \rangle) + \Gamma_{\mathtt{09}}(\langle \boldsymbol{a}^{(1)} \rangle) + \Gamma_{\mathtt{0e}}(\langle \boldsymbol{a}^{(2)} \rangle) + \Gamma_{\mathtt{0b}}(\langle \boldsymbol{a}^{(3)} \rangle)$
8: $\langle \boldsymbol{o}_{4c+3} \rangle \leftarrow \Gamma_{\mathtt{0b}}(\langle \boldsymbol{a}^{(0)} \rangle) + \Gamma_{\mathtt{0d}}(\langle \boldsymbol{a}^{(1)} \rangle) + \Gamma_{\mathtt{09}}(\langle \boldsymbol{a}^{(2)} \rangle) + \Gamma_{\mathtt{0e}}(\langle \boldsymbol{a}^{(3)} \rangle)$
9: **end for**
10: **return** $\langle \boldsymbol{o}_0 \rangle \parallel \langle \boldsymbol{o}_1 \rangle \parallel \cdots \parallel \langle \boldsymbol{o}_{15} \rangle$

$\Gamma_{\mathtt{09}}(\langle x_0 \rangle, \ldots, \langle x_7 \rangle)$

INPUT: Committed bits $\langle x_i \rangle$ for $i \in [0\,..\,7]$
OUTPUT: Committed bits $\langle y_i \rangle$ for $i \in [0\,..\,7]$

1: $\langle y_0 \rangle \leftarrow \langle x_0 \rangle + \langle x_5 \rangle$
2: $\langle y_1 \rangle \leftarrow \langle x_1 \rangle + \langle x_5 \rangle + \langle x_6 \rangle$
3: $\langle y_2 \rangle \leftarrow \langle x_2 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
4: $\langle y_3 \rangle \leftarrow \langle x_0 \rangle + \langle x_3 \rangle + \langle x_5 \rangle + \langle x_7 \rangle$
5: $\langle y_4 \rangle \leftarrow \langle x_1 \rangle + \langle x_4 \rangle + \langle x_5 \rangle + \langle x_6 \rangle$
6: $\langle y_5 \rangle \leftarrow \langle x_2 \rangle + \langle x_5 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
7: $\langle y_6 \rangle \leftarrow \langle x_3 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
8: $\langle y_7 \rangle \leftarrow \langle x_4 \rangle + \langle x_7 \rangle$
9: **return** $(\langle y_0 \rangle, \ldots, \langle y_7 \rangle)$

$\Gamma_{\mathtt{0b}}(\langle x_0 \rangle, \ldots, \langle x_7 \rangle)$

INPUT: Committed bits $\langle x_i \rangle$ for $i \in [0\,..\,7]$ OUTPUT: Committed bits $\langle y_i \rangle$ for $i \in [0\,..\,7]$

1: $\langle y_0 \rangle \leftarrow \langle x_0 \rangle + \langle x_5 \rangle + \langle x_7 \rangle$
2: $\langle y_1 \rangle \leftarrow \langle x_0 \rangle + \langle x_1 \rangle + \langle x_5 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
3: $\langle y_2 \rangle \leftarrow \langle x_1 \rangle + \langle x_2 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
4: $\langle y_3 \rangle \leftarrow \langle x_0 \rangle + \langle x_2 \rangle + \langle x_3 \rangle + \langle x_5 \rangle$
5: $\langle y_4 \rangle \leftarrow \langle x_1 \rangle + \langle x_3 \rangle + \langle x_4 \rangle + \langle x_5 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
6: $\langle y_5 \rangle \leftarrow \langle x_2 \rangle + \langle x_4 \rangle + \langle x_5 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
7: $\langle y_6 \rangle \leftarrow \langle x_3 \rangle + \langle x_5 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
8: $\langle y_7 \rangle \leftarrow \langle x_4 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
9: **return** $(\langle y_0 \rangle, \ldots, \langle y_7 \rangle)$

$\Gamma_{\mathtt{0d}}(\langle x_0 \rangle, \ldots, \langle x_7 \rangle)$

INPUT: Committed bits $\langle x_i \rangle$ for $i \in [0\,..\,7]$ OUTPUT: Committed bits $\langle y_i \rangle$ for $i \in [0\,..\,7]$

1: $\langle y_0 \rangle \leftarrow \langle x_0 \rangle + \langle x_5 \rangle + \langle x_6 \rangle$
2: $\langle y_1 \rangle \leftarrow \langle x_1 \rangle + \langle x_5 \rangle + \langle x_7 \rangle$
3: $\langle y_2 \rangle \leftarrow \langle x_0 \rangle + \langle x_2 \rangle + \langle x_6 \rangle$
4: $\langle y_3 \rangle \leftarrow \langle x_0 \rangle + \langle x_1 \rangle + \langle x_3 \rangle + \langle x_5 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
5: $\langle y_4 \rangle \leftarrow \langle x_1 \rangle + \langle x_2 \rangle + \langle x_4 \rangle + \langle x_5 \rangle + \langle x_7 \rangle$
6: $\langle y_5 \rangle \leftarrow \langle x_2 \rangle + \langle x_3 \rangle + \langle x_5 \rangle + \langle x_6 \rangle$
7: $\langle y_6 \rangle \leftarrow \langle x_3 \rangle + \langle x_4 \rangle + \langle x_6 \rangle + \langle x_7 \rangle$
8: $\langle y_7 \rangle \leftarrow \langle x_4 \rangle + \langle x_5 \rangle + \langle x_7 \rangle$
9: **return** $(\langle y_0 \rangle, \ldots, \langle y_7 \rangle)$

$\Gamma_{\mathsf{0e}}(\langle x_0\rangle, \ldots, \langle x_7\rangle)$

INPUT: Committed bits $\langle x_i\rangle$ for $i \in [0\,..\,7]$ OUTPUT: Committed bits $\langle y_i\rangle$ for $i \in [0\,..\,7]$

$\langle y_0\rangle \leftarrow \langle x_5\rangle + \langle x_6\rangle + \langle x_7\rangle$
$\langle y_1\rangle \leftarrow \langle x_0\rangle + \langle x_5\rangle$
$\langle y_2\rangle \leftarrow \langle x_0\rangle + \langle x_1\rangle + \langle x_6\rangle$
$\langle y_3\rangle \leftarrow \langle x_0\rangle + \langle x_1\rangle + \langle x_2\rangle + \langle x_5\rangle + \langle x_6\rangle$
$\langle y_4\rangle \leftarrow \langle x_1\rangle + \langle x_2\rangle + \langle x_3\rangle + \langle x_5\rangle$
$\langle y_5\rangle \leftarrow \langle x_2\rangle + \langle x_3\rangle + \langle x_4\rangle + \langle x_6\rangle$
$\langle y_6\rangle \leftarrow \langle x_3\rangle + \langle x_4\rangle + \langle x_5\rangle + \langle x_7\rangle$
$\langle y_7\rangle \leftarrow \langle x_4\rangle + \langle x_5\rangle + \langle x_6\rangle$
return $(\langle y_0\rangle, \ldots, \langle y_7\rangle)$

is viewed as a 4×4 byte matrix in column-major order, so that the byte $\langle s_{4c+r}\rangle^2_\lambda$ corresponds to the entry in row r and column c.

The transformation ShiftRows rotates each row r to the left by exactly r positions. In terms of the byte ordering, this means that for each row index $r \in [0\,..\,4)$ and column index $c \in [0\,..\,4)$, the output byte is given by

$$\langle s'_{4c+r}\rangle^2_\lambda = \langle s_{4((c+r) \bmod 4)+r}\rangle^2_\lambda.$$

Vistrutith.ShiftRows$(\langle \boldsymbol{s}\rangle^2_\lambda)$

INPUT: $\langle \boldsymbol{s}\rangle^2_\lambda = (\langle s_0\rangle^2_\lambda, \ldots, \langle s_{15}\rangle^2_\lambda)$ in byte order, where each $\langle s_i\rangle^2_\lambda$ is a degree-2 commitment to $s_i \in \mathbb{F}_{2^8}$
OUTPUT: $\langle \boldsymbol{s}'\rangle^2_\lambda = (\langle s'_0\rangle^2_\lambda, \ldots, \langle s'_{15}\rangle^2_\lambda)$

for $r \in [0\,..\,4)$ **do**
  for $c \in [0\,..\,4)$ **do**
    $\langle s'_{4c+r}\rangle^2_\lambda \leftarrow \langle s_{4((c+r) \bmod 4)+r}\rangle^2_\lambda$
  end for
end for
return $(\langle s'_0\rangle^2_\lambda, \ldots, \langle s'_{15}\rangle^2_\lambda)$

BitInvShiftRows. The function Vistrutith.BitInvShiftRows reverses the AES ShiftRows transformation on a committed bit-level state. Its input is a committed 128-bit state $\langle \boldsymbol{s}\rangle$. The output is another 128-bit committed state $\langle \boldsymbol{s}'\rangle$, obtained by undoing the row shifts in ShiftRows.

The forward ShiftRows transformation rotates row r to the left by r positions. Accordingly, the inverse transformation rotates row r to the right by r positions, which is equivalently expressed as a left rotation by $-r \bmod 4$. Thus, for each row index $r \in [0\,..\,4)$ and column index $c \in [0\,..\,4)$, the byte occupying position (r, c) in the inverse-shifted state is copied from position $i = 4((c-r) \bmod 4)+r$ in the input state.

Vistrutith.BitInvShiftRows$(\langle \boldsymbol{s}\rangle)$

INPUT: $\langle \boldsymbol{s}\rangle = (\langle s_0\rangle, \ldots, \langle s_{127}\rangle)$, a tuple of VOLE commitments to state bits
OUTPUT: $\langle \boldsymbol{s}'\rangle = (\langle s'_0\rangle, \ldots, \langle s'_{127}\rangle)$

for $r \in [0\,..\,4)$ **do**
  for $c \in [0\,..\,4)$ **do**
    $i \leftarrow 4((c-r) \bmod 4) + r$
    $(\langle s'_{8(4c+r)}\rangle, \ldots, \langle s'_{8(4c+r)+7}\rangle) \leftarrow (\langle s_{8i}\rangle, \ldots, \langle s_{8i+7}\rangle)$
  end for
end for
return $(\langle s'_0\rangle, \ldots, \langle s'_{127}\rangle)$

BitAddRoundKey. The function Vistrutith.BitAddRoundKey implements the AES AddRoundKey step directly on committed state bits. Its input consists of two 128-bit tuples of VOLE commitments, namely the committed state $\langle \boldsymbol{s} \rangle$ and the committed round key $\langle \boldsymbol{k} \rangle$. The output is a new tuple $\langle \boldsymbol{o} \rangle$ of committed bits representing the updated state.

For each bit position $i \in [0\,..\,128)$, the output bit is computed as

$$\langle o_i \rangle = \langle s_i \rangle + \langle k_i \rangle,$$

where the addition is over $\mathbb{F}_2$, i.e., it corresponds to bitwise XOR. Thus, Vistrutith.BitAddRoundKey performs the key addition independently on every committed bit of the state.

Vistrutith.BitAddRoundKey($\langle \boldsymbol{s} \rangle, \langle \boldsymbol{k} \rangle$)

INPUT: $\langle \boldsymbol{s} \rangle = (\langle s_0 \rangle, \ldots, \langle s_{127} \rangle)$ and $\langle \boldsymbol{k} \rangle = (\langle k_0 \rangle, \ldots, \langle k_{127} \rangle)$, tuples of VOLE commitments to state bits and round-key bits

OUTPUT: $\langle \boldsymbol{o} \rangle = (\langle o_0 \rangle, \ldots, \langle o_{127} \rangle)$, a tuple of VOLE commitments to state bits after the AddRoundKey step

1: **for** $i \in [0\,..\,128)$ **do**
2: $\quad \langle o_i \rangle \leftarrow \langle s_i \rangle + \langle k_i \rangle$
3: **end for**
4: **return** $(\langle o_0 \rangle, \ldots, \langle o_{127} \rangle)$

AddRoundKey. The function Vistrutith.AddRoundKey implements the AES AddRoundKey step at the byte level. Its input consists of a tuple $\langle \boldsymbol{s} \rangle^2_\lambda$ of degree-2 commitments to the 16 state bytes, together with a tuple $\langle \boldsymbol{k} \rangle^d_\lambda$ of commitments to the corresponding round-key bytes, where the key commitments have degree $d \in \{1, 2\}$. The output is a tuple $\langle \boldsymbol{o} \rangle^2_\lambda$ of committed bytes representing the updated state after key addition.

For each byte position $i \in [0\,..\,16)$, the output byte is computed as $\langle o_i \rangle^2_\lambda = \langle s_i \rangle^2_\lambda + \mathsf{Lift}_2(\langle k_i \rangle^d_\lambda)$, where the addition is the field addition in $\mathbb{F}_{2^\lambda}$, which corresponds to bytewise XOR on the embedded $\mathbb{F}_{2^8}$ values.

Vistrutith.AddRoundKey($\langle \boldsymbol{s} \rangle^2_\lambda, \langle \boldsymbol{k} \rangle^d_\lambda$)

INPUT: $\langle \boldsymbol{s} \rangle^2_\lambda = (\langle s_0 \rangle^2_\lambda, \ldots, \langle s_{15} \rangle^2_\lambda)$, a tuple of degree-2 commitments to state bytes; $\langle \boldsymbol{k} \rangle^d_\lambda = (\langle k_0 \rangle^d_\lambda, \ldots, \langle k_{15} \rangle^d_\lambda)$, a tuple of degree-$d$ commitments to round-key bytes, where $d \in \{1, 2\}$

OUTPUT: $\langle \boldsymbol{o} \rangle^2_\lambda = (\langle o_0 \rangle^2_\lambda, \ldots, \langle o_{15} \rangle^2_\lambda)$, a tuple of degree-2 commitments to state bytes after AddRoundKey

1: **for** $i \in [0\,..\,16)$ **do**
2: $\quad \langle o_i \rangle^2_\lambda \leftarrow \langle s_i \rangle^2_\lambda + \mathsf{Lift}_2(\langle k_i \rangle^d_\lambda)$
3: **end for**
4: **return** $(\langle o_0 \rangle^2_\lambda, \ldots, \langle o_{15} \rangle^2_\lambda)$

BitInvMix. The function Vistrutith.BitInvMix implements the bit-level inverse of the public bytewise inter-branch permutation Vistrutah.Mix on the 512-bit state. Its input is a tuple $\langle \boldsymbol{s} \rangle$ of committed state bits, which is viewed as 64 consecutive committed bytes $\langle \boldsymbol{a}_0 \rangle, \ldots, \langle \boldsymbol{a}_{63} \rangle$, each consisting of 8 committed bits.

Recall that Vistrutah.Mix takes four 128-bit branches, each containing 16 bytes, and rearranges them according to the fixed permutation $\pi_{\mathsf{mix}} = (0, 1, 2, 3, 8, 9, 10, 11, 4, 5, 6, 7, 12, 13, 14, 15)$, together with the interleaving of the four branches. Accordingly, Vistrutith.BitInvMix undoes this rearrangement by moving the byte $\langle \boldsymbol{a}_{4j+b} \rangle$ back to position $16b + \pi_{\mathsf{mix}}[j]$, for each $j \in [0\,..\,16)$ and branch index $b \in \{0, 1, 2, 3\}$.

8.2 Witness Extension

The Vistrutith.ExtendWitness algorithm takes the Vistrutah master key $\boldsymbol{k}$, the plaintext $\boldsymbol{in}$, and the instance parameters param, and returns an extended witness $\boldsymbol{w} \in \{0, 1\}^\ell$ for later use. The extended

Vistrutith.BitInvMix($\langle \boldsymbol{s} \rangle$)

INPUT: $\langle \boldsymbol{s} \rangle = (\langle s_0 \rangle, \ldots, \langle s_{511} \rangle)$, a tuple of VOLE commitments to state bits
OUTPUT: $\langle \boldsymbol{o} \rangle = (\langle o_0 \rangle, \ldots, \langle o_{511} \rangle)$, a tuple of VOLE commitments to state bits after Vistrutah.Mix^{-1}

Let $\pi_{\mathsf{mix}} = (0, 1, 2, 3, 8, 9, 10, 11, 4, 5, 6, 7, 12, 13, 14, 15)$.
Parse $\langle \boldsymbol{s} \rangle$ as 64 committed bytes $\langle \boldsymbol{a}_0 \rangle, \langle \boldsymbol{a}_1 \rangle, \ldots, \langle \boldsymbol{a}_{63} \rangle$, $\quad \boldsymbol{a}_i \in \{0,1\}^8$.
for $j \in [0 \,..\, 16)$ **do**
    $u \leftarrow \pi_{\mathsf{mix}}[j]$
    $\langle \boldsymbol{o}_{16 \cdot 0 + u} \rangle \leftarrow \langle \boldsymbol{a}_{4j+0} \rangle$
    $\langle \boldsymbol{o}_{16 \cdot 1 + u} \rangle \leftarrow \langle \boldsymbol{a}_{4j+1} \rangle$
    $\langle \boldsymbol{o}_{16 \cdot 2 + u} \rangle \leftarrow \langle \boldsymbol{a}_{4j+2} \rangle$
    $\langle \boldsymbol{o}_{16 \cdot 3 + u} \rangle \leftarrow \langle \boldsymbol{a}_{4j+3} \rangle$
end for
return $\langle \boldsymbol{o}_0 \rangle \,\|\, \langle \boldsymbol{o}_1 \rangle \,\|\, \cdots \,\|\, \langle \boldsymbol{o}_{63} \rangle$

witness consists of three components, corresponding to the key part, the initial intermediate phase, and the intermediate encryption phases.

- **Key bits** (`key_bits`): the full 512-bit master key $\boldsymbol{k}$. This contributes 512 bits.
- **Initial norm bits** (`init_norm_bits`): after the initial whitening step, the algorithm obtains the 512-bit state $P = P_0 \,\|\, P_1 \,\|\, P_2 \,\|\, P_3$, and computes the inverse norm of each of its 64 bytes by applying Vistrutith.InvNorm. Since each inverse norm contributes 4 bits, this part contributes $64 \cdot 4 = 256$ bits.
- **Intermediate encryption bits** (`enc_bits`): for each intermediate step $i \in [1 \,..\, R/2)$, the algorithm first computes the 512-bit state

$$\boldsymbol{M}_i = M_{i,0} \,\|\, M_{i,1} \,\|\, M_{i,2} \,\|\, M_{i,3}$$

  obtained after the zero-key AESRound layer, the mixing layer, the round-key update, and the round-key addition. It then appends both the full state $\boldsymbol{M}_i$ and the packed inverse norms $\boldsymbol{N}_i$, where $\boldsymbol{N}_i$ contains the 4-bit output of Vistrutith.InvNorm for each of the 64 bytes of $\boldsymbol{M}_i$. Thus, each intermediate step contributes $512 + 64 \cdot 4 = 768$ bits, and over all intermediate steps this contributes $768 \cdot (R/2 - 1)$ bits.

Therefore, the extended witness can be written as

$$\boldsymbol{w} = \underbrace{\texttt{key_bits}}_{512} \,\|\, \underbrace{\texttt{init_norm_bits}}_{256} \,\|\, \underbrace{\texttt{enc_bits}}_{768 \cdot (R/2-1)}.$$

For Vistrutah where $R = 18$, we have $\ell = 512 + 256 + 768 \cdot 8 = 6912$. Bytes are stored in branch order and bits within each byte are numbered from the least significant bit. The inverse-norm hint for byte $2j$ occupies the low nibble of hint byte j, and the hint for byte $2j + 1$ occupies its high nibble.

8.3 Deriving Constraints for the Encryption

This section describes how to derive constraints for the encryption routine in Vistrutith.EncCstrnts. Below, we first introduce several sub-functions.

ForwardByNorm. The function Vistrutith.ForwardByNorm reconstructs one fixed-key AES-like round from a committed 512-bit state together with the corresponding inverse-norm witness bits. Its input consists of a committed state $\langle \boldsymbol{S} \rangle$, the committed inverse-norm bits $\langle \boldsymbol{N} \rangle$, and the committed fixed-key branches $\langle FK_0 \rangle, \ldots, \langle FK_3 \rangle$. Its output is the committed byte state $\langle \boldsymbol{A} \rangle_\lambda^2$ after the fixed-key AESRound, its bytewise Frobenius square $\langle \boldsymbol{A}^{[2]} \rangle_\lambda^2$, and an ordered list $\mathcal{Z}_{\mathsf{norm}}$ of degree-3 constraint commitments, each of which must open to zero.

The algorithm proceeds branch by branch. For each 128-bit branch $\langle \boldsymbol{S}_b \rangle$, it first computes, for every byte, all Frobenius conjugates of the input byte by calling Vistrutith.StateToConjugates.

Vistrutith.ExtendWitness($\boldsymbol{in}, \boldsymbol{k}$; param)

INPUT: $\boldsymbol{in} \in \{0,1\}^{512}$, $\boldsymbol{k} \in \{0,1\}^{512}$
OUTPUT: $\boldsymbol{w} \in \{0,1\}^{\ell}$

Parse $\boldsymbol{in} = \boldsymbol{in}_0 \| \boldsymbol{in}_1 \| \boldsymbol{in}_2 \| \boldsymbol{in}_3$, $\boldsymbol{k} = \boldsymbol{k}_0 \| \boldsymbol{k}_1 \| \boldsymbol{k}_2 \| \boldsymbol{k}_3$ // all branches are 128 bits
$FK_0 \leftarrow \boldsymbol{k}_0$, $FK_1 \leftarrow \boldsymbol{k}_1$
$(FK_2 \| FK_3) \leftarrow$ Vistrutah.Shuffle$(\boldsymbol{k}_2 \| \boldsymbol{k}_3)$
$RK_0 \leftarrow FK_1$, $RK_1 \leftarrow FK_0$, $RK_2 \leftarrow FK_3$, $RK_3 \leftarrow FK_2$
$\boldsymbol{w} \leftarrow \boldsymbol{k}$
// Initial whitening state
$P_0 \leftarrow \boldsymbol{in}_0 \oplus RK_0$
$P_1 \leftarrow \boldsymbol{in}_1 \oplus RK_1$
$P_2 \leftarrow \boldsymbol{in}_2 \oplus RK_2$
$P_3 \leftarrow \boldsymbol{in}_3 \oplus RK_3$
$P \leftarrow P_0 \| P_1 \| P_2 \| P_3$
Parse P as 64 bytes $(P_0^{\mathsf{byte}}, \ldots, P_{63}^{\mathsf{byte}})$.
new $\boldsymbol{Q}_0 \in \{0,1\}^{256}$
for $t \in [0\,..\,64)$ **do**
    $\boldsymbol{Q}_0[4t\,..\,4t+3] \leftarrow$ Vistrutith.InvNorm(P_t^{byte}) // Store the inverse norm of P
end for
$\boldsymbol{w} \leftarrow \boldsymbol{w} \| \boldsymbol{Q}_0$
// Compute the first fixed-key AESRound, but do not store its output
$A_0 \leftarrow$ AESRound(P_0, FK_0)
$A_1 \leftarrow$ AESRound(P_1, FK_1)
$A_2 \leftarrow$ AESRound(P_2, FK_2)
$A_3 \leftarrow$ AESRound(P_3, FK_3)
// Intermediate Vistrutah steps
for $i \in [1\,..\,R/2)$ **do**
    $S_0 \leftarrow$ AESRound$(A_0, \mathbf{0}^{128})$
    $S_1 \leftarrow$ AESRound$(A_1, \mathbf{0}^{128})$
    $S_2 \leftarrow$ AESRound$(A_2, \mathbf{0}^{128})$
    $S_3 \leftarrow$ AESRound$(A_3, \mathbf{0}^{128})$
    $(S_0, S_1, S_2, S_3) \leftarrow$ Vistrutah.Mix(S_0, S_1, S_2, S_3)
    $RK_0 \leftarrow$ Vistrutah.RotByte$_5(RK_0)$
    $RK_1 \leftarrow$ Vistrutah.RotByte$_{10}(RK_1)$
    $RK_2 \leftarrow$ Vistrutah.RotByte$_5(RK_2)$
    $RK_3 \leftarrow$ Vistrutah.RotByte$_{10}(RK_3)$
    $M_{i,0} \leftarrow S_0 \oplus RK_0 \oplus RC_{i-1}$
    $M_{i,1} \leftarrow S_1 \oplus RK_1$
    $M_{i,2} \leftarrow S_2 \oplus RK_2$
    $M_{i,3} \leftarrow S_3 \oplus RK_3$
    $\boldsymbol{M}_i \leftarrow M_{i,0} \| M_{i,1} \| M_{i,2} \| M_{i,3}$
    Parse $\boldsymbol{M}_i$ as 64 bytes $(M_{i,0}^{\mathsf{byte}}, \ldots, M_{i,63}^{\mathsf{byte}})$.
    new $\boldsymbol{N}_i \in \{0,1\}^{256}$
    for $t \in [0\,..\,64)$ **do**
        $\boldsymbol{N}_i[4t\,..\,4t+3] \leftarrow$ Vistrutith.InvNorm$(M_{i,t}^{\mathsf{byte}})$
    end for
    $\boldsymbol{w} \leftarrow \boldsymbol{w} \| \boldsymbol{M}_i \| \boldsymbol{N}_i$
    $A_0 \leftarrow$ AESRound$(M_{i,0}, FK_0)$
    $A_1 \leftarrow$ AESRound$(M_{i,1}, FK_1)$
    $A_2 \leftarrow$ AESRound$(M_{i,2}, FK_2)$
    $A_3 \leftarrow$ AESRound$(M_{i,3}, FK_3)$
end for
//The final output is not appended
return $\boldsymbol{w}$

Thus, for each byte position u, it obtains the committed values $S_{b,u}$, $S_{b,u}^2$, $S_{b,u}^4$, $\ldots$, $S_{b,u}^{128}$. Let $h_{b,u}$ denote the subfield element encoded by the corresponding 4-bit inverse-norm hint. In parallel, Vistrutith.InvNormToConjugates yields the conjugates of $h_{b,u}$.

Using these two families of conjugates, the algorithm reconstructs the conjugates of $S_{b,u}^{16} h_{b,u}$. It appends to $\mathcal{Z}_{\mathsf{norm}}$ the constraint $h_{b,u} \cdot S_{b,u}^{16} \cdot S_{b,u}^2 + S_{b,u}$. For a non-zero input byte, requiring this constraint to be zero forces $h_{b,u} = S_{b,u}^{-17}$, so the reconstructed byte is $S_{b,u}^{-1}$. When the input is zero, every hint satisfies the constraint and all reconstructed inverse conjugates are zero.

After reconstructing the inverse-byte conjugates, the algorithm applies Vistrutith.SBoxAffine twice: once in the ordinary mode and once in the squared mode. These two calls produce the affine S-box output and its bytewise Frobenius square, respectively. Each of these two byte states is then processed through the committed versions of ShiftRows and MixColumns. The appropriate branch round key is then added: the ordinary branch key for the unsquared state, and the squared branch key for the squared state.

At the end, the four branch outputs are concatenated to form the committed 512-bit state $\langle \boldsymbol{A} \rangle_\lambda^2$ after the fixed-key AESRound, together with its bytewise square $\langle \boldsymbol{A}^{[2]} \rangle_\lambda^2$.

Vistrutith.ForwardByNorm serves as the main routine that turns the compressed inverse-norm witness into the full committed forward computation of one AES-like round, while simultaneously producing the algebraic checks needed to validate the reconstruction.

Vistrutith.ForwardByNorm$(\langle \boldsymbol{S} \rangle, \langle \boldsymbol{N} \rangle, \langle FK_0 \rangle, \ldots, \langle FK_3 \rangle)$

INPUT: $\langle \boldsymbol{S} \rangle$, a committed 512-bit state; $\langle \boldsymbol{N} \rangle$, commitments to $\boldsymbol{N} \in \mathbb{F}_2^{256}$, four InvNorm bits per byte; committed fixed-key branches $\langle FK_0 \rangle, \ldots, \langle FK_3 \rangle$
OUTPUT: $(\langle \boldsymbol{A} \rangle_\lambda^2, \langle \boldsymbol{A}^{[2]} \rangle_\lambda^2, \mathcal{Z}_{\mathsf{norm}})$, where $\boldsymbol{A}$ is the byte state after the fixed-key AESRound, and $\boldsymbol{A}^{[2]}$ is its bytewise Frobenius square

new $\mathcal{Z}_{\mathsf{norm}}$
for $b \in [0\,..\,4)$ **do**
  Let $\langle \boldsymbol{S}_b \rangle$ be the b-th 128-bit branch of $\langle \boldsymbol{S} \rangle$
  $\langle C_{b,u,0} \rangle_\lambda, \ldots, \langle C_{b,u,7} \rangle_\lambda \leftarrow$ Vistrutith.StateToConjugates$(\langle \boldsymbol{S}_b \rangle)$ // $C_{b,u,j} = S_{b,u}^{2^j}$ for byte index $u \in [0, 16)$, *i.e.*, $S_{b,u}$ is the u-th byte in $\boldsymbol{S}_b$
  for $u \in [0\,..\,16)$ **do**
    $t \leftarrow 16b + u$
    $(\langle H_{b,u,0} \rangle_\lambda, \ldots, \langle H_{b,u,3} \rangle_\lambda) \leftarrow$ Vistrutith.InvNormToConjugates$(\langle N[4t] \rangle, \ldots, \langle N[4t+3] \rangle)$.
    $\langle z \rangle^3 \leftarrow \langle H_{b,u,0} \rangle_\lambda \cdot \langle C_{b,u,4} \rangle_\lambda \cdot \langle C_{b,u,1} \rangle_\lambda + \mathsf{Lift}_3(\langle C_{b,u,0} \rangle_\lambda)$ // $h \cdot x^{16} \cdot x^2 + x = 0$, where $h = H_{b,u,0}$ and $x = S_{b,u}$
    Append $\langle z \rangle^3$ to $\mathcal{Z}_{\mathsf{norm}}$
    for $j \in [0\,..\,8)$ **do**
      $\langle P_{b,u,j} \rangle_\lambda^2 \leftarrow \langle C_{b,u,(j+4) \bmod 8} \rangle_\lambda \cdot \langle H_{b,u,j \bmod 4} \rangle_\lambda$ // conjugates of $S_{b,u}^{16} h_{b,u}$
    end for
  end for
  for $sq \in \{\mathsf{false}, \mathsf{true}\}$ **do**
    $\langle \boldsymbol{A}_b^{sq} \rangle_\lambda^2 \leftarrow$ Vistrutith.SBoxAffine$((\langle P_{b,0,0} \rangle_\lambda^2, \ldots, \langle P_{b,0,7} \rangle_\lambda^2), \ldots, (\langle P_{b,15,0} \rangle_\lambda^2, \ldots, \langle P_{b,15,7} \rangle_\lambda^2); 16, sq)$
    $\langle \boldsymbol{A}_b^{sq} \rangle_\lambda^2 \leftarrow$ Vistrutith.ShiftRows$(\langle \boldsymbol{A}_b^{sq} \rangle_\lambda^2)$
    $\langle \boldsymbol{A}_b^{sq} \rangle_\lambda^2 \leftarrow$ Vistrutith.MixColumns$(\langle \boldsymbol{A}_b^{sq} \rangle_\lambda^2, sq)$
  end for
  $\langle FK_b^{(0)} \rangle_\lambda \leftarrow$ Vistrutith.StateToBytes$(\langle FK_b \rangle)$
  for $u \in [0, 16)$ **do**
    $\langle FK_b^{(1)}[u] \rangle_\lambda \leftarrow$ Vistrutith.SquareAndCombine$(\langle FK_b[8u] \rangle, \ldots, \langle FK_b[8u+7] \rangle)$
  end for
  $\langle \boldsymbol{A}_b^0 \rangle_\lambda^2 \leftarrow$ Vistrutith.AddRoundKey$(\langle \boldsymbol{A}_b^{\mathsf{false}} \rangle_\lambda^2, \langle FK_b^{(0)} \rangle_\lambda)$
  $\langle \boldsymbol{A}_b^1 \rangle_\lambda^2 \leftarrow$ Vistrutith.AddRoundKey$(\langle \boldsymbol{A}_b^{\mathsf{true}} \rangle_\lambda^2, \langle FK_b^{(1)} \rangle_\lambda)$
end for
$\langle \boldsymbol{A} \rangle_\lambda^2 \leftarrow \langle \boldsymbol{A}_0^0 \rangle_\lambda^2 \| \langle \boldsymbol{A}_1^0 \rangle_\lambda^2 \| \langle \boldsymbol{A}_2^0 \rangle_\lambda^2 \| \langle \boldsymbol{A}_3^0 \rangle_\lambda^2$
$\langle \boldsymbol{A}^{[2]} \rangle_\lambda^2 \leftarrow \langle \boldsymbol{A}_0^1 \rangle_\lambda^2 \| \langle \boldsymbol{A}_1^1 \rangle_\lambda^2 \| \langle \boldsymbol{A}_2^1 \rangle_\lambda^2 \| \langle \boldsymbol{A}_3^1 \rangle_\lambda^2$
return $(\langle \boldsymbol{A} \rangle_\lambda^2, \langle \boldsymbol{A}^{[2]} \rangle_\lambda^2, \mathcal{Z}_{\mathsf{norm}})$

BackwardZeroRndByBits. The function Vistrutith.BackwardZeroRndByBits reconstructs, at the bit level, the outputs of the patched field inversion in the zero-key AESRound preceding the committed state $\langle \boldsymbol{M}_i \rangle$. Its input is the committed 512-bit state after the application of Vistrutah.Mix and the subsequent round-key injection, together with the committed round-key branches $\langle RK_0^{(i)} \rangle, \ldots, \langle RK_3^{(i)} \rangle$ and the public round constant RC_{i-1}. Its output is a committed 512-bit bitstring $\langle \boldsymbol{Y}_i \rangle$, representing the inputs to the S-box affine layer of that zero-key AESRound.

The algorithm first parses $\langle \boldsymbol{M}_i \rangle$ into four 128-bit branches $\langle M_{i,0} \rangle, \ldots, \langle M_{i,3} \rangle$. It then removes the round-key injection: for the first branch, this means xoring both $\langle RK_0^{(i)} \rangle$ and the round constant RC_{i-1}, while for the remaining three branches it xors only the corresponding round-key branch. This yields the intermediate branches $\langle U_0 \rangle, \ldots, \langle U_3 \rangle$.

Next, the algorithm inverts the public linear layers of the zero-key AESRound in reverse order. It first applies Vistrutith.BitInvMix to undo the inter-branch mixing step Vistrutah.Mix. Then, on each 128-bit branch separately, it applies Vistrutith.BitInvMixColumns to undo MixColumns, followed by Vistrutith.BitInvShiftRows to undo ShiftRows. At this point, each branch is in the bit representation of the affine S-box output. Finally, the algorithm applies Vistrutith.SBoxInvAffine to each branch, thereby recovering the committed bits of the patched inversions in S-boxes.

Vistrutith.BackwardZeroRndByBits($\langle \boldsymbol{M}_i \rangle, \langle RK_0^{(i)} \rangle, \ldots, \langle RK_3^{(i)} \rangle, RC_{i-1}$)

INPUT: $\langle \boldsymbol{M}_i \rangle$, the committed 512-bit state after Vistrutah.Mix and round-key injection; round-key branches $\langle RK_0^{(i)} \rangle, \ldots, \langle RK_3^{(i)} \rangle$; round constant RC_{i-1}

OUTPUT: $\langle \boldsymbol{Y}_i \rangle$, the committed bit representation of the patched field inversion outputs of the preceding zero-key AESRound

Parse $\langle \boldsymbol{M}_i \rangle = \langle M_{i,0} \rangle \| \langle M_{i,1} \rangle \| \langle M_{i,2} \rangle \| \langle M_{i,3} \rangle$.
$\langle U_0 \rangle \leftarrow \langle M_{i,0} \rangle + \langle RK_0^{(i)} \rangle + \mathsf{ConstantToVOLE}(RC_{i-1})$.
for $b \in \{1, 2, 3\}$ **do**
    $\langle U_b \rangle \leftarrow \langle M_{i,b} \rangle + \langle RK_b^{(i)} \rangle$.
end for
$(\langle U_0 \rangle, \langle U_1 \rangle, \langle U_2 \rangle, \langle U_3 \rangle) \leftarrow$ Vistrutith.BitInvMix$(\langle U_0 \rangle, \langle U_1 \rangle, \langle U_2 \rangle, \langle U_3 \rangle)$.
for $b \in [0\,..\,4)$ **do**
    $\langle U_b \rangle \leftarrow$ Vistrutith.BitInvMixColumns$(\langle U_b \rangle)$.
    $\langle U_b \rangle \leftarrow$ Vistrutith.BitInvShiftRows$(\langle U_b \rangle)$.
    $\langle Y_b \rangle \leftarrow$ Vistrutith.SBoxInvAffine$(\langle U_b \rangle)$.
end for
return $\langle \boldsymbol{Y}_i \rangle = \langle Y_0 \rangle \| \langle Y_1 \rangle \| \langle Y_2 \rangle \| \langle Y_3 \rangle$.

AppendCstrntsByBits. The function Vistrutith.AppendCstrntsByBits links a committed byte state $\langle \boldsymbol{X} \rangle_\lambda^2$ and its bytewise square $\langle \boldsymbol{X}^{[2]} \rangle_\lambda^2$ to a committed bit state $\langle \boldsymbol{Y} \rangle$. It parses $\langle \boldsymbol{Y} \rangle$ into 64 bytes, and for each byte position t, reconstructs both the byte value $\langle Y_t \rangle_\lambda$ and its bytewise square $\langle Y_t^{[2]} \rangle_\lambda$ using Vistrutith.ByteCombine and Vistrutith.SquareAndCombine, respectively. It then appends the degree-3 patched inversion checks, $\langle X_t^{[2]} \rangle_\lambda^2 \cdot \langle Y_t \rangle_\lambda + \mathsf{Lift}_3(\langle X_t \rangle_\lambda^2)$ and $\langle X_t \rangle_\lambda^2 \cdot \langle Y_t^{[2]} \rangle_\lambda + \mathsf{Lift}_3(\langle Y_t \rangle_\lambda)$, which enforce $Y_t = \mathrm{Inv}(X_t)$, where $\mathrm{Inv}(x) = x^{-1}$ for $x \neq 0$ and $\mathrm{Inv}(0) = 0$. For $X_t \neq 0$, the first check forces $Y_t = X_t^{-1}$, while for $X_t = 0$, the second forces $Y_t = 0$.

BackwardFinalRndByBits. The function Vistrutith.BackwardFinalRndByBits reconstructs, at the bit level, the outputs of the patched field inversion in the final AES round from the committed ciphertext and the committed final round keys. Its input consists of the committed 512-bit ciphertext $\langle \boldsymbol{out} \rangle$, together with the final round-key branches $\langle RK_0^{(T)} \rangle, \ldots, \langle RK_3^{(T)} \rangle$. Its output is a committed 512-bit bitstring $\langle \boldsymbol{Y}_T \rangle$, representing the inputs to the S-box affine layer of the final AES round.

The algorithm first parses the ciphertext into four 128-bit branches $\langle out_0 \rangle, \ldots, \langle out_3 \rangle$. For each branch b, it removes the final AddRoundKey operation by xoring the corresponding committed round key $\langle RK_b^{(T)} \rangle$. Since the final AES round does not contain a MixColumns layer, the algorithm

Vistrutith.AppendCstrntsByBits($\langle \boldsymbol{X} \rangle^2_\lambda, \langle \boldsymbol{X}^{[2]} \rangle^2_\lambda, \langle \boldsymbol{Y} \rangle$)

INPUT: $\langle \boldsymbol{X} \rangle^2_\lambda$, degree-2 commitments to 64 state bytes; $\langle \boldsymbol{X}^{[2]} \rangle^2_\lambda$, degree-2 commitments to their Frobenius squares; $\langle \boldsymbol{Y} \rangle$, degree-1 commitments to a 512-bit state
OUTPUT: $(\mathcal{Z}_0, \mathcal{Z}_1)$, two lists of 64 degree-3 constraints

new $\mathcal{Z}_0, \mathcal{Z}_1$
for $t \in [0\,..\,64)$ **do**
  $\langle Y_t \rangle_\lambda \leftarrow$ Vistrutith.ByteCombine$(\langle Y[8t] \rangle, \ldots, \langle Y[8t+7] \rangle)$
  $\langle Y_t^{[2]} \rangle_\lambda \leftarrow$ Vistrutith.SquareAndCombine$(\langle Y[8t] \rangle, \ldots, \langle Y[8t+7] \rangle)$
  $\langle z_t^{(0)} \rangle^3 \leftarrow \langle X_t^{[2]} \rangle^2_\lambda \cdot \langle Y_t \rangle_\lambda + \mathsf{Lift}_3(\langle X_t \rangle^2_\lambda)$
  $\langle z_t^{(1)} \rangle^3 \leftarrow \langle X_t \rangle^2_\lambda \cdot \langle Y_t^{[2]} \rangle_\lambda + \mathsf{Lift}_3(\langle Y_t \rangle_\lambda)$
  Append $\langle z_t^{(0)} \rangle^3$ to $\mathcal{Z}_0$, and $\langle z_t^{(1)} \rangle^3$ to $\mathcal{Z}_1$
end for
return $(\mathcal{Z}_0, \mathcal{Z}_1)$

then only applies the inverse row permutation Vistrutith.BitInvShiftRows, followed by the inverse affine map Vistrutith.SBoxInvAffine. This yields the committed bit representation of the patched field inversion outputs for each branch. Finally, the four branches are concatenated to obtain $\langle \boldsymbol{Y}_T \rangle$.

Vistrutith.BackwardFinalRndByBits($\langle \boldsymbol{out} \rangle, \langle RK_0^{(T)} \rangle, \ldots, \langle RK_3^{(T)} \rangle$)

INPUT: $\langle \boldsymbol{out} \rangle$, the committed 512-bit ciphertext; final round-key branches $\langle RK_0^{(T)} \rangle, \ldots, \langle RK_3^{(T)} \rangle$
OUTPUT: $\langle \boldsymbol{Y}_T \rangle$, the committed bit representation of the patched field inversion outputs of the final AES round

Parse $\langle \boldsymbol{out} \rangle = \langle out_0 \rangle \| \langle out_1 \rangle \| \langle out_2 \rangle \| \langle out_3 \rangle$.
for $b \in [0\,..\,4)$ **do**
  $\langle U_b \rangle \leftarrow \langle out_b \rangle + \langle RK_b^{(T)} \rangle$.
  $\langle U_b \rangle \leftarrow$ Vistrutith.BitInvShiftRows$(\langle U_b \rangle)$.
  $\langle Y_b \rangle \leftarrow$ Vistrutith.SBoxInvAffine$(\langle U_b \rangle)$.
end for
return $\langle \boldsymbol{Y}_T \rangle = \langle Y_0 \rangle \| \langle Y_1 \rangle \| \langle Y_2 \rangle \| \langle Y_3 \rangle$.

EncCstrnts. Based on the building blocks above, the function Vistrutith.EncCstrnts constructs the full ordered list of algebraic constraints certifying the correctness of the Vistrutah encryption relation. Its input consists of commitments to the plaintext $\boldsymbol{in}$, the ciphertext $\boldsymbol{out}$, and the extended witness $\boldsymbol{w}$. Its output is the concatenation of three ordered lists of degree-3 constraint commitments $\mathcal{Z}_{\mathsf{norm}}$, $\mathcal{Z}_{\mathsf{io},0}$, and $\mathcal{Z}_{\mathsf{io},1}$, all of which are required to open to zero.

The witness $\boldsymbol{w}$ is first parsed into the committed master key $\langle \boldsymbol{K} \rangle$, the initial inverse-norm bits $\langle \boldsymbol{Q}_0 \rangle$, and the intermediate witness blocks $(\langle \boldsymbol{M}_i \rangle, \langle \boldsymbol{N}_i \rangle)$ for $i = 1, \ldots, T-1$, where $T = R/2$. From the committed key, the algorithm reconstructs the fixed-key branches $FK_0, \ldots, FK_3$ and all rotated round-key branches $RK_0^{(j)}, \ldots, RK_3^{(j)}$ for $j = 0, \ldots, T$. Thus, all round keys used by the encryption circuit are derived deterministically from the committed witness key.

The first phase checks the initial fixed-key AESRound. Starting from the committed plaintext, the algorithm first applies the committed bit-level AddRoundKey using the initial round keys $RK_b^{(0)}$, obtaining the whitened state $\langle \boldsymbol{P} \rangle$. It then invokes Vistrutith.ForwardByNorm on $\langle \boldsymbol{P} \rangle$ and the committed inverse-norm bits $\langle \boldsymbol{Q}_0 \rangle$. This produces the committed byte state after the first fixed-key AESRound, together with its bytewise Frobenius square and an ordered list of norm-consistency constraints. These constraints are appended to $\mathcal{Z}_{\mathsf{norm}}$.

The second phase iterates through the intermediate Vistrutah steps. For each $i \in [1\,..\,T)$, the algorithm first calls Vistrutith.BackwardZeroRndByBits on the committed intermediate state $\langle \boldsymbol{M}_i \rangle$, the corresponding rotated round keys, and the public round constant RC_{i-1}. This reconstructs the committed bit-level outputs of the patched field inversion in the preceding zero-key AESRound.

The procedure Vistrutith.AppendCstrntsByBits then uses the patched inversion checks to link these outputs to their forward inputs in $\langle \boldsymbol{A} \rangle_\lambda^2$, together with its square $\langle \boldsymbol{A}^{[2]} \rangle_\lambda^2$. The resulting constraints are appended to $\mathcal{Z}_{\mathsf{io},0}$ and $\mathcal{Z}_{\mathsf{io},1}$.

After this backward check, the algorithm performs the next forward fixed-key AESRound by calling Vistrutith.ForwardByNorm on $\langle \boldsymbol{M}_i \rangle$ and the committed inverse-norm witness $\langle \boldsymbol{N}_i \rangle$. This again yields a new byte-level forward state, its square, and additional norm-consistency constraints, which are appended to $\mathcal{Z}_{\mathsf{norm}}$. In this way, each intermediate witness block is checked both backward and forward, thereby linking consecutive rounds of the encryption computation.

The final phase checks the last AES round against the committed ciphertext. The algorithm invokes Vistrutith.BackwardFinalRndByBits on $\langle \boldsymbol{out} \rangle$ and the final rotated round keys, obtaining the committed bit-level outputs of the patched field inversion in the last AES round. It then calls Vistrutith.AppendCstrntsByBits once more to link these outputs to their forward inputs in $\langle \boldsymbol{A} \rangle_\lambda^2$, together with its square $\langle \boldsymbol{A}^{[2]} \rangle_\lambda^2$, using the patched inversion checks. The constraints are appended to $\mathcal{Z}_{\mathsf{io},0}$ and $\mathcal{Z}_{\mathsf{io},1}$.

The routine Vistrutith.EncCstrnts forms three ordered lists: one for the inverse-norm checks and two for the patched inversion checks. Each list contains $64T$ constraints, and the final-round checks form the last 64 entries of each patched inversion list. Hence, the concatenation $\mathcal{Z}_{\mathsf{norm}} \| \mathcal{Z}_{\mathsf{io},0} \| \mathcal{Z}_{\mathsf{io},1}$ enforces the Vistrutah encryption relation from $\boldsymbol{in}$ to $\boldsymbol{out}$.

8.4 Complete OWF Constraints

This section describes the final QuickSilver relation for the Vistrutah one-way function. We write the public key as $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2) \in \{0,1\}^{512} \times \{0,1\}^{512}$, where pk_1 is the public 512-bit input block and pk_2 is the public 512-bit output block. The extended witness is obtained from Vistrutith.ExtendWitness, namely $\boldsymbol{w} = \boldsymbol{K} \,\|\, \boldsymbol{Q}_0 \,\|\, (\boldsymbol{M}_1 \| \boldsymbol{N}_1) \,\|\, \cdots \,\|\, (\boldsymbol{M}_{T-1} \| \boldsymbol{N}_{T-1})$ with $T = R/2$.

The key schedule of the underlying Vistrutah cipher is $\mathbb{F}_2$-linear. The first 512 witness bits already contain the master key, and all branch keys and rotated round-key branches used by the cipher are public $\mathbb{F}_2$-linear functions of it. The complete OWF relation combines the keyspace-reduction term with the encryption constraints from Vistrutith.EncCstrnts. We use Deg2To3 to lift the reduction term from degree 2 to degree 3. For this, Deg2To3 is the restriction of Lift_3 in Equation (32) to degree-2 inputs.

Deg2To3. The function Deg2To3 takes a scalar or vector of degree-2 commitments and lifts it to degree 3. It does so by multiplying each entry with a degree-1 commitment to the constant 1, obtained through ConstantToVOLE. Since 1 is the multiplicative identity in the embedded copy of $\mathbb{F}_2 \subseteq \mathbb{F}_{2^\lambda}$, the committed value is unchanged.

Vistrutith.OWFConstraints. This is the main routine that builds the complete Vistrutith OWF relation. It first parses the public key and allocates a fresh buffer $\langle \boldsymbol{z} \rangle_\lambda^3$ for the accumulated constraints. It then appends the standard keyspace-reduction term $\langle w[0] \rangle \cdot \langle w[1] \rangle$, lifted to degree 3 using Deg2To3. After that, it turns the public input and output blocks into bitwise VOLE commitments by calling ConstantToVOLE. Finally, it passes the full committed witness to Vistrutith.EncCstrnts. The output polynomial vector is ordered as

$$Z(X) = K(X) \,\|\, \mathcal{Z}_{\mathsf{norm}}(X) \,\|\, \mathcal{Z}_{\mathsf{io},0}(X) \,\|\, \mathcal{Z}_{\mathsf{io},1}(X), \qquad C = 1 + 3 \cdot 576 = 1729.$$

Within each family, entries are ordered by increasing step, branch, and byte index. For $j \in [0, C)$, write $Z_j(X) = a_{0,j} + a_{1,j} X + a_{2,j} X^2 + z_j X^3$. The prover forms $\boldsymbol{a}_h = (a_{h,0}, \ldots, a_{h,C-1})$ for each $h \in [0,3)$, and the verifier evaluates the same C polynomials at Δ.

Vistrutith.OWFProve and Vistrutith.OWFVerify. The prover and verifier follow the standard degree-3 QuickSilver interface. The prover first turns every witness bit into a VOLE commitment using one row of $\boldsymbol{V}$ as the tag. It then embeds the additional 2λ mask rows into field elements, computes Vistrutith.OWFConstraints, and parses the resulting degree-3 vector as coefficient vectors $(\boldsymbol{a}_0, \boldsymbol{a}_1, \boldsymbol{a}_2)$. These are hashed together with the corresponding masks to produce $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2)$.

The verifier reconstructs the committed witness values from $\boldsymbol{d}_{\mathsf{w}}$, $\boldsymbol{Q}$, and the challenge $\Delta = \mathsf{ToField}(\mathsf{chall}_3; \lambda)$. It also reconstructs the mask contribution $q^* = q_0^* + \Delta \cdot q_1^*$, recomputes

Vistrutith.EncCstrnts($\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, \langle \boldsymbol{w} \rangle$; param)

INPUT: Commitments to $\boldsymbol{in} \in \{0,1\}^{512}$, $\boldsymbol{out} \in \{0,1\}^{512}$, and extended witness $\boldsymbol{w}$
OUTPUT: The concatenation of three lists of $64T = 576$ degree-3 commitments $\mathcal{Z}_{\mathsf{norm}}, \mathcal{Z}_{\mathsf{io},0}, \mathcal{Z}_{\mathsf{io},1}$, all of which must open to zero

$T \leftarrow R/2$
new $\mathcal{Z}_{\mathsf{norm}}, \mathcal{Z}_{\mathsf{io},0}, \mathcal{Z}_{\mathsf{io},1}$
Parse $\langle \boldsymbol{w} \rangle = \langle \boldsymbol{K} \rangle \| \langle \boldsymbol{Q}_0 \rangle \| (\langle \boldsymbol{M}_1 \rangle \| \langle \boldsymbol{N}_1 \rangle) \| \cdots \| (\langle \boldsymbol{M}_{T-1} \rangle \| \langle \boldsymbol{N}_{T-1} \rangle)$
//$|\boldsymbol{K}| = 512, |\boldsymbol{Q}_0| = 256, |\boldsymbol{M}_i| = 512, |\boldsymbol{N}_i| = 256$
Parse $\langle \boldsymbol{K} \rangle = \langle \boldsymbol{k}_0 \rangle \| \langle \boldsymbol{k}_1 \rangle \| \langle \boldsymbol{k}_2 \rangle \| \langle \boldsymbol{k}_3 \rangle$
//Compute the rotated round-key branches
$\langle FK_0 \rangle \leftarrow \langle \boldsymbol{k}_0 \rangle, \quad \langle FK_1 \rangle \leftarrow \langle \boldsymbol{k}_1 \rangle$
$(\langle FK_2 \rangle \| \langle FK_3 \rangle) \leftarrow$ Vistrutah.Shuffle$(\langle \boldsymbol{k}_2 \rangle \| \langle \boldsymbol{k}_3 \rangle)$
for $j \in [0, T]$ **do**
    $\langle RK_0^{(j)} \rangle \leftarrow$ Vistrutah.RotByte$_5^j(\langle FK_1 \rangle)$
    $\langle RK_1^{(j)} \rangle \leftarrow$ Vistrutah.RotByte$_{10}^j(\langle FK_0 \rangle)$
    $\langle RK_2^{(j)} \rangle \leftarrow$ Vistrutah.RotByte$_5^j(\langle FK_3 \rangle)$
    $\langle RK_3^{(j)} \rangle \leftarrow$ Vistrutah.RotByte$_{10}^j(\langle FK_2 \rangle)$
end for
//Forward-check the initial fixed-key AESRound using $\boldsymbol{Q}_0$
Parse $\langle \boldsymbol{in} \rangle = \langle \boldsymbol{in}_0 \rangle \| \langle \boldsymbol{in}_1 \rangle \| \langle \boldsymbol{in}_2 \rangle \| \langle \boldsymbol{in}_3 \rangle$.
for $b \in [0\,..\,4)$ **do**
    $\langle P_b \rangle \leftarrow$ Vistrutith.BitAddRoundKey$(\langle \boldsymbol{in}_b \rangle, \langle RK_b^{(0)} \rangle)$
end for
$\langle \boldsymbol{P} \rangle \leftarrow \langle P_0 \rangle \| \langle P_1 \rangle \| \langle P_2 \rangle \| \langle P_3 \rangle$.
$(\langle \boldsymbol{A} \rangle_\lambda^2, \langle \boldsymbol{A}^{[2]} \rangle_\lambda^2, \mathcal{Z}) \leftarrow$ Vistrutith.ForwardByNorm$(\langle \boldsymbol{P} \rangle, \langle \boldsymbol{Q}_0 \rangle, \langle FK_0 \rangle, \ldots, \langle FK_3 \rangle)$
$\mathcal{Z}_{\mathsf{norm}} \leftarrow \mathcal{Z}_{\mathsf{norm}} \| \mathcal{Z}$
// For each intermediate step, check the zero-key AESRound backwards and the following fixed-key AESRound forwards
for $i \in [1\,..\,T)$ **do**
    $\langle \boldsymbol{Y}_i \rangle \leftarrow$ Vistrutith.BackwardZeroRndByBits$(\langle \boldsymbol{M}_i \rangle, \langle RK_0^{(i)} \rangle, \ldots, \langle RK_3^{(i)} \rangle, RC_{i-1})$
    $(\mathcal{Z}_0, \mathcal{Z}_1) \leftarrow$ Vistrutith.AppendCstrntsByBits$(\langle \boldsymbol{A} \rangle_\lambda^2, \langle \boldsymbol{A}^{[2]} \rangle_\lambda^2, \langle \boldsymbol{Y}_i \rangle)$
    $\mathcal{Z}_{\mathsf{io},0} \leftarrow \mathcal{Z}_{\mathsf{io},0} \| \mathcal{Z}_0, \ \mathcal{Z}_{\mathsf{io},1} \leftarrow \mathcal{Z}_{\mathsf{io},1} \| \mathcal{Z}_1$
    $(\langle \boldsymbol{A} \rangle_\lambda^2, \langle \boldsymbol{A}^{[2]} \rangle_\lambda^2, \mathcal{Z}) \leftarrow$ Vistrutith.ForwardByNorm$(\langle \boldsymbol{M}_i \rangle, \langle \boldsymbol{N}_i \rangle, \langle FK_0 \rangle, \ldots, \langle FK_3 \rangle)$
    $\mathcal{Z}_{\mathsf{norm}} \leftarrow \mathcal{Z}_{\mathsf{norm}} \| \mathcal{Z}$
end for
// Check the final AES round backwards from the ciphertext
$\langle \boldsymbol{Y}_T \rangle \leftarrow$ Vistrutith.BackwardFinalRndByBits$(\langle \boldsymbol{out} \rangle, \langle RK_0^{(T)} \rangle, \ldots, \langle RK_3^{(T)} \rangle)$
$(\mathcal{Z}_0, \mathcal{Z}_1) \leftarrow$ Vistrutith.AppendCstrntsByBits$(\langle \boldsymbol{A} \rangle_\lambda^2, \langle \boldsymbol{A}^{[2]} \rangle_\lambda^2, \langle \boldsymbol{Y}_T \rangle)$
$\mathcal{Z}_{\mathsf{io},0} \leftarrow \mathcal{Z}_{\mathsf{io},0} \| \mathcal{Z}_0, \ \mathcal{Z}_{\mathsf{io},1} \leftarrow \mathcal{Z}_{\mathsf{io},1} \| \mathcal{Z}_1$
return $\mathcal{Z}_{\mathsf{norm}} \| \mathcal{Z}_{\mathsf{io},0} \| \mathcal{Z}_{\mathsf{io},1}$

Deg2To3$(\langle \boldsymbol{x} \rangle_\lambda^2)$

INPUT: $\langle \boldsymbol{x} \rangle_\lambda^2 = (\langle x_0 \rangle_\lambda^2, \ldots, \langle x_{m-1} \rangle_\lambda^2)$, where $m \geq 1$; a scalar input is treated as the case $m = 1$
OUTPUT: $\langle \boldsymbol{y} \rangle_\lambda^3 = (\langle y_0 \rangle_\lambda^3, \ldots, \langle y_{m-1} \rangle_\lambda^3)$, where $y_i = x_i$ for all i

$\langle 1 \rangle \leftarrow$ ConstantToVOLE(1)
for $i \in [0..m)$ **do**
    $\langle y_i \rangle_\lambda^3 \leftarrow \langle x_i \rangle_\lambda^2 \cdot \langle 1 \rangle$
end for
return $\langle \boldsymbol{y} \rangle_\lambda^3$

Vistrutith.OWFConstraints($\langle \boldsymbol{w} \rangle, \mathsf{pk}; \mathsf{param}$)

INPUT: Committed witness $\langle \boldsymbol{w} \rangle$, where $\boldsymbol{w} \in \mathbb{F}_2^{\ell}$
OUTPUT: Committed constraints $\langle \boldsymbol{z} \rangle_\lambda^3$, where $\boldsymbol{z} \in \mathbb{F}_{2^\lambda}^{C}$ and the verifier checks that $\boldsymbol{z} = \boldsymbol{0}$

Parse $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2) \in \{0,1\}^{512} \times \{0,1\}^{512}$
new $\langle \boldsymbol{z} \rangle_\lambda^3$, where $\boldsymbol{z} \in \mathbb{F}_{2^\lambda}^{\leq C}$
$\langle \tilde{z} \rangle_\lambda^3 \leftarrow$ Deg2To3($\langle w[0] \rangle \cdot \langle w[1] \rangle$) // enforce $w[0]w[1] = 0$
$\langle \boldsymbol{z} \rangle_\lambda^3 \leftarrow \langle \boldsymbol{z} \rangle_\lambda^3 \parallel \langle \tilde{z} \rangle_\lambda^3$
$\langle \boldsymbol{in} \rangle \leftarrow$ ConstantToVOLE(pk_1)
$\langle \boldsymbol{out} \rangle \leftarrow$ ConstantToVOLE(pk_2)
$\langle \tilde{\boldsymbol{w}} \rangle \leftarrow \langle \boldsymbol{w}[0..\ell) \rangle$ // Vistrutith uses no separate key-expansion witness
$\langle \tilde{\boldsymbol{z}} \rangle_\lambda^3 \leftarrow$ Vistrutith.EncCstrnts($\langle \boldsymbol{in} \rangle, \langle \boldsymbol{out} \rangle, \langle \tilde{\boldsymbol{w}} \rangle; \mathsf{param}$)
$\langle \boldsymbol{z} \rangle_\lambda^3 \leftarrow \langle \boldsymbol{z} \rangle_\lambda^3 \parallel \langle \tilde{\boldsymbol{z}} \rangle_\lambda^3$
return $\langle \boldsymbol{z} \rangle_\lambda^3$

Vistrutith.OWFConstraints($\langle \boldsymbol{w} \rangle, \mathsf{pk}$), and hashes the evaluated constraint vector. Finally, it subtracts the terms involving $\tilde{a}_1$ and $\tilde{a}_2$ to recover $\tilde{a}_0$. If the proof is valid, the returned value must match the prover's $\tilde{a}_0$.

Vistrutith.OWFProve($\boldsymbol{w}, \boldsymbol{u}, \boldsymbol{V}, \mathsf{pk}, \mathsf{chall}_2; \mathsf{param}$)

INPUT: $\boldsymbol{w} \in \{0,1\}^{\ell}$, $\boldsymbol{u} \in \{0,1\}^{2\lambda}$, $\boldsymbol{V} \in \{0,1\}^{(\ell+2\lambda)\times\lambda}$, pk, and chall_2
OUTPUT: $\tilde{a}_0, \tilde{a}_1, \tilde{a}_2 \in \mathbb{F}_{2^\lambda}$

for $i \in [0, \ell)$ **do**
    Let $\langle w[i] \rangle \leftarrow$ (ToField($\boldsymbol{V}|_i; \lambda$), $\boldsymbol{w}[i]$)
end for
for $i \in [0, 2\lambda)$ **do**
    $v[i] \leftarrow$ ToField($\boldsymbol{V}|_{\ell+i}; \lambda$)
end for
for $j \in [0, 2)$ **do**
    $u_j^* \leftarrow$ ToField($\boldsymbol{u}[j\lambda..(j+1)\lambda); \lambda$)
    $v_j^* \leftarrow \sum_{i \in [0..\lambda)} v[i + \lambda j] \alpha_\lambda^i$
end for
// compute a degree-3 constraint vector
$\langle \boldsymbol{z} \rangle_\lambda^3 \leftarrow$ Vistrutith.OWFConstraints($\langle \boldsymbol{w} \rangle, \mathsf{pk}; \mathsf{param}$)
Parse $\langle \boldsymbol{z} \rangle_\lambda^3$ as $(\boldsymbol{a}_0, \boldsymbol{a}_1, \boldsymbol{a}_2) \in (\mathbb{F}_{2^\lambda}^{C})^3$
$\tilde{a}_0 \leftarrow$ ZKHash($\mathsf{chall}_2, (\boldsymbol{a}_0 \| v_0^*)$)
$\tilde{a}_1 \leftarrow$ ZKHash($\mathsf{chall}_2, (\boldsymbol{a}_1 \| (u_0^* + v_1^*))$)
$\tilde{a}_2 \leftarrow$ ZKHash($\mathsf{chall}_2, (\boldsymbol{a}_2 \| u_1^*)$)
return $(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2)$

8.5 Construction of Vistrutith

Based on the building blocks and functions described above, this section gives the formal construction of the Vistrutith signature scheme. Keys, signatures, and hash inputs use the encoding in Section 3.3 and Table 5.

8.5.1 Key Generation

The Vistrutith.KeyGen algorithm describes the key generation procedure for Vistrutith based on Vistrutah. On input the public parameters param, the algorithm first samples a key vector $\boldsymbol{k} \in \{0,1\}^\lambda$ uniformly at random, subject to the condition that the first two bits do not simultaneously satisfy $k[0] \wedge k[1] = 1$. It then samples an input vector $\boldsymbol{x} \in \{0,1\}^\lambda$ uniformly at random.

Vistrutith.OWFVerify($\boldsymbol{d}_{\mathsf{w}}, \boldsymbol{Q}, \mathsf{pk}, \mathsf{chall}_2, \mathsf{chall}_3, \tilde{a}_1, \tilde{a}_2; \mathsf{param}$)

INPUT: $\boldsymbol{d}_{\mathsf{w}} \in \{0,1\}^{\ell}$, $\boldsymbol{Q} \in \{0,1\}^{(\ell+2\lambda)\times\lambda}$, pk, chall_2, chall_3, $\tilde{a}_1$, and $\tilde{a}_2$
OUTPUT: $\tilde{a}_0 \in \mathbb{F}_{2^\lambda}$

$\Delta \leftarrow$ ToField($\mathsf{chall}_3; \lambda$)
for $i \in [0..\ell)$ **do**
    Let $\langle w[i] \rangle \leftarrow$ ToField($\boldsymbol{Q}|_i; \lambda$) $+ \boldsymbol{d}_{\mathsf{w}}[i] \cdot \Delta$
end for
for $i \in [0..2\lambda)$ **do**
    $q[i] \leftarrow$ ToField($\boldsymbol{Q}|_{\ell+i}; \lambda$)
end for
for $j \in [0..2)$ **do**
    $q_j^* \leftarrow \sum_{i\in[0..\lambda)} q[i+\lambda j]\alpha_\lambda^i$
end for
$q^* \leftarrow q_0^* + \Delta \cdot q_1^*$
// compute the evaluated constraint vector
$\langle \boldsymbol{z} \rangle_\lambda^3 \leftarrow$ Vistrutith.OWFConstraints($\langle \boldsymbol{w} \rangle, \mathsf{pk}; \mathsf{param}$)
Parse $\langle \boldsymbol{z} \rangle_\lambda^3$ as $\boldsymbol{b} \in \mathbb{F}_{2^\lambda}^C$ // $\boldsymbol{b} = Z(\Delta)$
$\tilde{q} \leftarrow$ ZKHash($\mathsf{chall}_2, (\boldsymbol{b} \| q^*)$)
// recover the prover's $\tilde{a}_0$
return $\tilde{q} - \tilde{a}_1 \cdot \Delta - \tilde{a}_2 \cdot \Delta^2$

The algorithm computes $\boldsymbol{y} \leftarrow \mathsf{Vistrutah}_{\boldsymbol{k}}(\boldsymbol{x})$, sets $\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$, and outputs $\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$, where $\boldsymbol{k}$ is treated as the secret Vistrutah key and $\boldsymbol{x}$ as the public input. The algorithm then returns $(\mathsf{sk}, \mathsf{pk})$.

Vistrutith.KeyGen(param)

INPUT: Public parameters param
OUTPUT: Secret key $\mathsf{sk} \in (\{0,1\}^\lambda, \{0,1\}^\lambda)$ and public key $\mathsf{pk} \in (\{0,1\}^\lambda, \{0,1\}^\lambda)$, or $\perp$

repeat
    $\boldsymbol{k} \leftarrow \{0,1\}^\lambda$
    if sampling fails **then**
        return $\perp$
    end if
until $\boldsymbol{k}[0] \wedge \boldsymbol{k}[1] = 0$
$\boldsymbol{x} \leftarrow \{0,1\}^\lambda$
if sampling fails **then**
    return $\perp$
end if
$\boldsymbol{y} \leftarrow \mathsf{Vistrutah}_{\boldsymbol{k}}(\boldsymbol{x})$
$\mathsf{sk} \leftarrow (\boldsymbol{x}, \boldsymbol{k})$ // public input, secret Vistrutah key
$\mathsf{pk} \leftarrow (\boldsymbol{x}, \boldsymbol{y})$ // public input, public output
return $(\mathsf{sk}, \mathsf{pk})$

8.5.2 Signing

The public-key argument is the public key derived from the secret key by the corresponding OWF.

Vistrutith.Sign($\mathsf{msg}, \mathsf{sk}, \mathsf{pk}; \mathsf{param}$)

INPUT: Message msg, secret key $\mathsf{sk} = (\mathsf{sk}_1, \mathsf{sk}_2)$, public key $\mathsf{pk} = (\mathsf{pk}_1, \mathsf{pk}_2)$, public parameters param
OUTPUT: Signature σ or $\perp$

if $|\mathsf{msg}|_{\mathsf{bytes}} > 2^{60}$ or $\mathsf{sk}_2[0] \wedge \mathsf{sk}_2[1] = 1$ **then**
  return $\perp$
end if
$\rho \leftarrow \{0,1\}^{\xi}$ // Signature randomness; $\xi = 640$
if sampling fails **then**
  return $\perp$
end if
$\mu \leftarrow \mathsf{H}_2^0(D_{\mathsf{p}} \,\|\, \mathsf{pk} \,\|\, \mathsf{msg}; 2\lambda)$
$(r, \mathsf{iv}^{\mathsf{pre}}) \leftarrow \mathsf{H}_3(\mathsf{sk}_2 \,\|\, \mu \,\|\, \rho; \lambda + \lambda_{\mathsf{iv}})$
$\mathsf{iv} \leftarrow \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}}; \lambda_{\mathsf{iv}})$
$(\mathsf{com}, \mathsf{decom}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \boldsymbol{u}, \boldsymbol{V}) \leftarrow \mathsf{VOLECommit}(r, \mathsf{iv}, \ell + 3\lambda + B; \mathsf{param})$ // $d = 3$ for Vistrutith
$\mathsf{chall}_1 \leftarrow \mathsf{H}_2^1(\mu \,\|\, \mathsf{com} \,\|\, \boldsymbol{c}_1 \,\|\, \cdots \,\|\, \boldsymbol{c}_{\tau-1} \,\|\, \mathsf{iv}; 5\lambda + 64)$
// VOLE consistency check
$\tilde{\boldsymbol{u}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{u}) \in \{0,1\}^{\lambda+B}$
$\tilde{\boldsymbol{V}} \leftarrow \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{V}) \in \{0,1\}^{(\lambda+B)\times\lambda}$
$\boldsymbol{w} \leftarrow \mathsf{Vistrutith.ExtendWitness}(\mathsf{sk}_1, \mathsf{sk}_2; \mathsf{param}) \in \{0,1\}^{\ell}$
$\boldsymbol{d}_{\mathsf{w}} \leftarrow \boldsymbol{w} \oplus \boldsymbol{u}_{[0,\ell)}$ // Mask extended witness
$\mathsf{chall}_2 \leftarrow \mathsf{H}_2^2(\mathsf{chall}_1 \,\|\, \tilde{\boldsymbol{u}} \,\|\, \tilde{\boldsymbol{V}} \,\|\, \boldsymbol{d}_{\mathsf{w}}; 3\lambda + 64)$
// QuickSilver proof
$\boldsymbol{u} \leftarrow \boldsymbol{u}_{[\ell,\ell+2\lambda)}$ // keep the two mask blocks following the witness bits
$\boldsymbol{V} \leftarrow \boldsymbol{V}_{[0,\ell+2\lambda)} \in \{0,1\}^{(\ell+2\lambda)\times\lambda}$ // drop last $\lambda + B$ rows
$(\tilde{a}_0, \tilde{a}_1, \tilde{a}_2) \leftarrow \mathsf{Vistrutith.OWFProve}(\boldsymbol{w}, \boldsymbol{u}, \boldsymbol{V}, \mathsf{pk}, \mathsf{chall}_2; \mathsf{param})$
for $\mathsf{ctr} \in [0, 2^{32})$ in increasing order **do**
  $\mathsf{chall}_3 \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \tilde{a}_1 \,\|\, \tilde{a}_2 \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda)$
  if $\mathsf{chall}_3[\lambda - w_{\mathsf{grind}}, \lambda) = 0^{w_{\mathsf{grind}}}$ **then**
    $I \leftarrow \mathsf{DecodeChall}(\mathsf{chall}_3; \mathsf{param})$
    $\mathsf{decom}_I \leftarrow \mathsf{BAVC.Open}(\mathsf{decom}, I; \mathsf{param})$
    if $\mathsf{decom}_I \neq \perp$ **then**
      return $\sigma \leftarrow (\{\boldsymbol{c}_i\}_{i\in[1,\tau)}, \tilde{\boldsymbol{u}}, \boldsymbol{d}_{\mathsf{w}}, \tilde{a}_1, \tilde{a}_2, \mathsf{decom}_I, \mathsf{chall}_3, \mathsf{iv}^{\mathsf{pre}}, \mathsf{ctr})$
    end if
  end if
end for
return $\perp$

Apart from the parameter values and the Vistrutah-specific witness and OWF routines, the signing algorithm of Vistrutith follows the same structure as that of uBlockith. Both use degree-3 constraints. Because Vistrutah has a linear key schedule, an EM variant would not remove non-linear key-expansion constraints, so we do not include one.

8.5.3 Verifying

Vistrutith.Verify($\mathsf{msg}, \mathsf{pk}, \sigma; \mathsf{param}$)

INPUT: Message msg, public key pk, signature σ, public parameters param
OUTPUT: true or false

if $|\mathsf{msg}|_{\mathsf{bytes}} > 2^{60}$ or $|\mathsf{pk}|_{\mathsf{bytes}} \neq 128$ or $|\sigma|_{\mathsf{bytes}} \neq \mathsf{sigsize}(\mathsf{param})$ **then**
  return false
end if
$\hat{\ell} \leftarrow \ell + 3\lambda + B$
Parse $\sigma = \left(\{\boldsymbol{c}_i\}_{i\in[1,\tau)}, \tilde{\boldsymbol{u}}, \boldsymbol{d}_{\mathsf{w}}, \tilde{a}_1, \tilde{a}_2, \mathsf{decom}_I, \mathsf{chall}_3, \mathsf{iv}^{\mathsf{pre}}, \mathsf{ctr}\right)$
$\mu \leftarrow \mathsf{H}_2^0(D_{\mathsf{p}} \,\|\, \mathsf{pk} \,\|\, \mathsf{msg}; 2\lambda)$
$\mathsf{iv} \leftarrow \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}}; \lambda_{\mathsf{iv}})$
if $\mathsf{chall}_3[\lambda - w_{\mathsf{grind}}, \lambda) \neq 0^{w_{\mathsf{grind}}}$ **then**
  return false
end if
// Reconstruct VOLEs and check commitment
$(\mathsf{com}, \boldsymbol{Q}) \leftarrow$ VOLEReconstruct$(\mathsf{chall}_3, \mathsf{decom}_I, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv}, \hat{\ell}; \mathsf{param})$
if VOLEReconstruct returned $\perp$ **then**
  return false
end if
$\mathsf{chall}_1 \leftarrow \mathsf{H}_2^1(\mu \,\|\, \mathsf{com} \,\|\, \boldsymbol{c}_1 \,\|\, \cdots \,\|\, \boldsymbol{c}_{\tau-1} \,\|\, \mathsf{iv}; 5\lambda + 64)$
// Apply the VOLE correction values and check consistency
$\tilde{\boldsymbol{Q}} \leftarrow$ VOLEHash$(\mathsf{chall}_1, \boldsymbol{Q}) \in \{0,1\}^{(\lambda+B)\times\lambda}$
Parse $\mathsf{chall}_3 = (\delta_0, \ldots, \delta_{\lambda-1}) \in \{0,1\}^{\lambda}$
$\tilde{\boldsymbol{V}}' \leftarrow \tilde{\boldsymbol{Q}} \oplus [\delta_0 \cdot \tilde{\boldsymbol{u}} \;\cdots\; \delta_{\lambda-1} \cdot \tilde{\boldsymbol{u}}]$
$\mathsf{chall}_2 \leftarrow \mathsf{H}_2^2(\mathsf{chall}_1 \,\|\, \tilde{\boldsymbol{u}} \,\|\, \tilde{\boldsymbol{V}}' \,\|\, \boldsymbol{d}_{\mathsf{w}}; 3\lambda + 64)$
// Compute $\tilde{a}_0$
$\tilde{a}_0 \leftarrow$ Vistrutith.OWFVerify$(\boldsymbol{d}_{\mathsf{w}}, \boldsymbol{Q}_{[0,\ell+2\lambda)}, \mathsf{pk}, \mathsf{chall}_2, \mathsf{chall}_3, \tilde{a}_1, \tilde{a}_2; \mathsf{param})$
Compute $\mathsf{chall}_3' \leftarrow \mathsf{H}_2^3(\mathsf{chall}_2 \,\|\, \tilde{a}_0 \,\|\, \tilde{a}_1 \,\|\, \tilde{a}_2 \,\|\, \mathrm{LE32}(\mathsf{ctr}); \lambda)$
return true if $\mathsf{chall}_3 = \mathsf{chall}_3'$ else false

Apart from the parameter values, the verification algorithm of Vistrutith uses Vistrutith.OWFVerify in place of uBlockith.OWFVerify and follows the same structure as that of uBlockith.

9 Security Statements and Analysis

This section presents the security statements and analyses of our signature schemes.

9.1 Theoretical Security

9.1.1 Security Targets and Model

We analyze *existential unforgeability against chosen-message attacks* (EUF-CMA) in the ROM and the QROM under the primitive and multi-user conditions stated below. We first establish *existential unforgeability against key-only attacks* (EUF-KO), where the adversary is given only the public key and must output a valid forgery. We then extend this notion to EUF-CMA, where the adversary is additionally given access to a signing oracle, which returns a valid signature given a message.

Definition 4 (EUF-CMA and EUF-KO Security in the ROM or QROM). *Let* $\mathsf{M} \in \{\mathrm{ROM},\mathrm{QROM}\}$. *A signature scheme*

$$\mathsf{SIG} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$$

is EUF-CMA *(resp.,* EUF-KO*) secure in model* M *if every efficient adversary has at most negligible advantage of winning the* EUF-CMA *(resp.,* EUF-KO*) security experiment in Figure 3.*

$\boxed{\mathsf{ExpEUFCMA}}$ $\mathsf{ExpEUFKO}$

$1:\ \mathcal{M} := \emptyset$

$2:\ (\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(\mathsf{param})$

$3:\ (\mathsf{msg}, \sigma) \leftarrow \mathcal{A}^{\mathsf{H_M}, \boxed{\mathsf{OSign}}}(\mathsf{param}, \mathsf{pk})$

$4:\ \textbf{return}\ \mathsf{Verify}^{\mathsf{H_M}}(\mathsf{msg}, \mathsf{pk}, \sigma; \mathsf{param}) \wedge \mathsf{msg} \notin \mathcal{M}$

$\mathsf{OSign}(\mathsf{msg})$

$1:\ \mathcal{M} := \mathcal{M} \cup \{\mathsf{msg}\}$

$2:\ \textbf{return}\ \mathsf{Sign}^{\mathsf{H_M}}(\mathsf{sk}, \mathsf{msg})$

Fig. 3: The $\boxed{\mathsf{EUF\text{-}CMA}}$ and $\mathsf{EUF\text{-}KO}$ security experiments in the ROM/QROM. Here $\mathsf{M} \in \{\mathrm{ROM},\mathrm{QROM}\}$, and $\mathsf{H_M}$ denotes classical access to the random oracle(s) in the ROM and quantum access in the QROM. Only in $\mathsf{EUF\text{-}CMA}$ does $\mathcal{A}$ get access to OSign.

In the ROM, the adversary is a PPT algorithm with classical oracle access to the random oracle(s). In the QROM, the adversary is a QPT algorithm with quantum access to the random oracle(s). In both models, the signing oracle OSign *is classical. We write*

$$\mathsf{AdvEUFCMA}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}} = \Pr[\mathsf{ExpEUFCMA}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}} = 1],$$

and analogously

$$\mathsf{AdvEUFKO}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}} = \Pr[\mathsf{ExpEUFKO}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}} = 1].$$

The security proofs follow the same high-level strategy as FAEST. In the ROM, we first prove $\mathsf{EUF\text{-}KO}$ security by viewing a valid signature forgery as an accepting non-interactive ZKP. The proof first replaces the honestly generated public key by an independent uniform instance using pseudorandomness of the actual public-key distribution, as defined in Section 9.1.3. The restricted secret-key domain gives a constant fraction of uniform instances with no valid witness; the lossy-key argument then relates forgery to soundness of the proof system, with the additional losses from random-oracle events and the Fiat–Shamir transformation stated below. The $\mathsf{EUF\text{-}KO}$ result is then lifted to $\mathsf{EUF\text{-}CMA}$ security by simulating the signing oracle using the honest-verifier zero-knowledge property of the proof system and programming the Fiat–Shamir random oracles. Intuitively, the honest-verifier zero-knowledge property guarantees that such transcripts can be generated without knowing the witness, namely the secret key, while remaining indistinguishable from honestly generated transcripts. Therefore, in the security reduction, the simulator can answer signing queries even though it only receives the public key and does not know the corresponding secret key. To make the simulated transcript pass verification after applying the Fiat–Shamir transform, the simulator programs the random oracles so that the hash outputs coincide with the simulated challenges. The unpredictability of the commitments ensures that the adversary has not queried these random-oracle inputs in advance, except with negligible probability, and hence the programming is not detected.

In the QROM, the same conceptual reduction from $\mathsf{EUF\text{-}CMA}$ to $\mathsf{EUF\text{-}KO}$ is used, but the proof must account for quantum superposition queries to the random oracles. Hence, ordinary random-oracle programming is replaced by standard QROM reprogramming tools, such as adaptive reprogramming and compressed-oracle arguments. Following the FAEST proof, the non-interactive proof is interpreted as a Fiat–Shamir transform of a suitable multi-round interactive proof, so that round-by-round (RBR) soundness can be applied in the presence of quantum-accessible random oracles. The resulting bounds differ from the ROM bounds mainly by the usual quantum query losses and by replacing classical unpredictability arguments with their quantum analogues.

Multi-key $\mathsf{EUF\text{-}CMA}$**.** We additionally define the multi-key $\mathsf{EUF\text{-}CMA}$ security, which captures the security of a signature scheme when an adversary is given several independently generated public keys and may adaptively request signatures under any of them. The adversary wins if it outputs a valid signature on a fresh message under one of these public keys.

Following FAEST-v2 [22, Sections 9.1.2 and 9.3.4], we use 2^{64} signing queries per key as a conservative reference budget for the lo variants of $\mathsf{SM4th}$ and $\mathsf{SM4th\text{-}EM}$, retaining the 128-bit IV and the existing SM4-based PRG. FAEST-v2 observes that independently sampled 128-bit IVs

$\mathsf{ExpMK\text{-}EUFCMA}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}}(U)$

1 : $\mathcal{M} := \emptyset$
2 : **for** $i \in [U]$ **do**
$(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KeyGen}(\mathsf{param})$
3 : $(i^\star, \mathsf{msg}^\star, \sigma^\star) \leftarrow \mathcal{A}^{\mathsf{H_M}, \mathsf{OSign}}(\mathsf{param}, \mathsf{pk}_1, \ldots, \mathsf{pk}_U)$
4 : **return** $i^\star \in [U] \wedge \mathsf{Verify}^{\mathsf{H_M}}(\mathsf{msg}^\star, \mathsf{pk}_{i^\star}, \sigma^\star; \mathsf{param})$
$\wedge (i^\star, \mathsf{msg}^\star) \notin \mathcal{M}$

$\mathsf{OSign}(i, \mathsf{msg})$

1 : **if** $i \notin [U]$ **then return** $\perp$
2 : $\mathcal{M} := \mathcal{M} \cup \{(i, \mathsf{msg})\}$
3 : **return** $\mathsf{Sign}^{\mathsf{H_M}}(\mathsf{sk}_i, \mathsf{msg})$

Fig. 4: The multi-key EUF-CMA security experiment in the ROM/QROM for U keys, where $[U] = \{1, \ldots, U\}$. Here $\mathsf{M} \in \{\text{ROM,QROM}\}$, and $\mathsf{H_M}$ denotes classical access to the random oracle(s) in the ROM and quantum access in the QROM. The signing oracle is classical and records queried pairs (i, msg).

are all distinct with good probability below this budget. For a fixed tweak and output-block index, distinct IVs give distinct cipher inputs, preventing one candidate-seed evaluation from being reused across those signatures.

For the multi-key evaluation, we consider $U = 2^{16}$ independently generated key pairs, and each key is used for at most $q = 2^{64}$ signing queries. The total number of signing queries is then $Q_{\mathsf{sig}} = U \cdot q = 2^{16} \cdot 2^{64} = 2^{80}$.

Repeated IVs can create several PRG search targets with the same IV and tweak. A refined analysis for larger single-key signing budgets can account for the multiplicities of targets sharing the same effective cipher input, together with the corresponding CMA losses in Equation (65). The multi-hiding experiment of FAEST-v2 [22, Lemma 9.20] samples IVs independently and permits repetitions, while the signing-simulation losses are accounted for in that equation.

Definition 5 (Multi-key EUF-CMA Security in the ROM or QROM). *Let $U = U(\lambda)$ be the number of keys and $\mathsf{M} \in \{\text{ROM,QROM}\}$. A signature scheme*

$$\mathsf{SIG} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$$

is U-key EUF-CMA *secure in model* M *if every efficient adversary has at most negligible advantage of winning* $\mathsf{ExpMK\text{-}EUFCMA}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}}(U)$.

In the ROM, the adversary is a PPT algorithm with classical oracle access to the random oracle(s). In the QROM, the adversary is a QPT algorithm with quantum access to the random oracle(s). In both models, the signing oracle is classical.

We write

$$\mathsf{AdvMK\text{-}EUFCMA}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}}(U) = \Pr[\mathsf{ExpMK\text{-}EUFCMA}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}}(U) = 1].$$

According to the union bound, the advantage of any multi-key adversary $\mathcal{A}$ can be bounded by the sum of its success probabilities for forging under each individual public key. Therefore, for suitable single-key adversaries $\mathcal{B}_i$ derived from $\mathcal{A}$, we have

$$\mathsf{AdvMK\text{-}EUFCMA}^{\mathsf{SIG}}_{\mathcal{A},\mathsf{M}}(U) \leq \sum_{i \in [U]} \mathsf{AdvEUFCMA}^{\mathsf{SIG}}_{\mathcal{B}_i,\mathsf{M}} \leq U \cdot \max_{i \in [U]} \mathsf{AdvEUFCMA}^{\mathsf{SIG}}_{\mathcal{B}_i,\mathsf{M}}. \tag{37}$$

9.1.2 Building Blocks

This subsection introduces the notation and primitive security notions used in the subsequent analysis.

PRP advantage. We use the standard PRP advantage notation. Let $P : \mathcal{K} \times \mathcal{M} \to \mathcal{M}$ be a block cipher, where for every key $k \in \mathcal{K}$, the map $P(k, \cdot)$ is a permutation over $\mathcal{M}$. A PRP distinguisher $\mathcal{D}$ has oracle access to O and outputs a bit. Its goal is to distinguish whether $O =$

$P(k,\cdot)$ for a uniformly random key $k \leftarrow \mathcal{K}$, or whether $O = \pi$ for a uniformly random permutation $\pi \leftarrow \mathrm{Perm}(\mathcal{M})$, where $\mathrm{Perm}(\mathcal{M})$ denotes the set of all permutations over $\mathcal{M}$. We define the PRP advantage of $\mathcal{D}$ against P as

$$\mathsf{AdvPRP}^{P}_{\mathcal{D}} = \left|\Pr\left[\mathcal{D}^{P(k,\cdot)} = 1 : k \leftarrow \mathcal{K}\right] - \Pr[\mathcal{D}^{\pi} = 1 : \pi \leftarrow \mathrm{Perm}(\mathcal{M})]\right|. \tag{38}$$

Write $\mathsf{AdvPRP}^{P}_{\mathcal{D}}[q]$ when $\mathcal{D}$ makes at most q queries to O. Let $\mathcal{R} = (T, \boldsymbol{Q}_{\mathsf{aux}})$ be a resource budget, where T bounds the running time or circuit size in a fixed classical or quantum model, and $\boldsymbol{Q}_{\mathsf{aux}}$ bounds all auxiliary-oracle queries. We write

$$\mathsf{AdvPRP}^{P}[\mathcal{R}; q] = \sup_{\mathcal{D}:\, \mathsf{res}(\mathcal{D}) \le \mathcal{R},\ Q_{\mathcal{D}} \le q} \mathsf{AdvPRP}^{P}_{\mathcal{D}}. \tag{39}$$

The time budget includes local cipher evaluations and all reduction simulations, and in the ideal-cipher model, $\boldsymbol{Q}_{\mathsf{aux}}$ also counts forward and inverse queries under all keys.

OWF advantage. We also use the standard OWF advantage notation. Let $F : \mathcal{X} \to \mathcal{Y}$ be a deterministic function. An OWF adversary $\mathcal{A}$ is given $y = F(x)$ for a uniformly random $x \leftarrow \mathcal{X}$, and outputs a candidate preimage $x' \in \mathcal{X}$. We define the OWF advantage of $\mathcal{A}$ against F as

$$\mathsf{AdvOWF}^{F}_{\mathcal{A}} = \Pr[F(x') = y : x \leftarrow \mathcal{X},\ y := F(x),\ x' \leftarrow \mathcal{A}(y)]\,. \tag{40}$$

Almost-injective PRG. We use the almost-injective property of PRGs [22]. For a fixed IV and tweak, consider the function that maps a PRG key k to the output $\mathsf{PRG}(k, \mathsf{iv}, \mathsf{twk})$. In the almost-injectivity game, the adversary outputs an image y and succeeds if y has at least two distinct seed preimages. In our security proof, this property is used to rule out extraction failures in PRG-based leaf commitments. Throughout the almost-injectivity games, PRG denotes the two-block function $\mathsf{PRG}_{2N_{\mathsf{block}}}$. We next recall the definitions.

Definition 6 (Almost-injective PRG [22]). *Let $\mathsf{PRG}(k, \mathsf{iv}, \mathsf{twk})$ be an IV-based tweakable PRG as defined in the PRG algorithm, where $k \in \mathcal{K}_\lambda = \{0,1\}^{N_{\mathsf{block}}}$, $\mathsf{iv} \in \mathcal{I}_\lambda = \{0,1\}^{\lambda_{\mathsf{iv}}}$, and $\mathsf{twk} \in \mathcal{T}_\lambda = \{0,1\}^{32}$. Consider the following game, parameterized by Q_{out}.*

1. *The adversary $\mathcal{A}$ outputs an IV $\mathsf{iv} \in \mathcal{I}_\lambda$ together with Q_{out} tuples $\left(\mathsf{twk}_j, y_j\right)_{j \in [1..Q_{\mathsf{out}}]}$*
2. *If, for some $j \in [1..Q_{\mathsf{out}}]$, there exist two distinct keys $k \neq k'$ in $\mathcal{K}_\lambda$ such that $\mathsf{PRG}(k, \mathsf{iv}, \mathsf{twk}_j) = y_j = \mathsf{PRG}(k', \mathsf{iv}, \mathsf{twk}_j)$, then the game outputs 1; otherwise, it outputs 0.*

We define the advantage of $\mathcal{A}$ in this game as

$$\mathsf{AdvInj}^{\mathsf{PRG}}_{\mathcal{A}}[Q_{\mathsf{out}}] = \Pr[\textit{the game outputs } 1].$$

Write $\mathsf{AdvInj}^{\mathsf{PRG}}_{\mathsf{M}}[\mathcal{R}; Q_{\mathsf{out}}]$ for the supremum over adversaries in model $\mathsf{M} \in \{\mathsf{R}, \mathsf{Q}\}$ with resource budget $\mathcal{R}$. The budget records running time, memory or qubits, every public-oracle query, and classical and quantum primitive queries separately, including simulation costs.

Remark 1 (Classical heuristic for a fixed IV). For a classical adversary restricted to an IV fixed independently of the ideal-oracle randomness, the random-function heuristic of FAEST [22], Remark 9.14, with seed-domain size $m = 2^{N_{\mathsf{block}}}$ and output-space size $n = 2^{2N_{\mathsf{block}}}$ gives the coarse estimate

$$\mathsf{AdvInj}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}_{\mathcal{A}}[Q_{\mathsf{out}}] \lesssim \frac{Q_{\mathsf{C}}^2}{2^{2N_{\mathsf{block}}}} + \frac{Q_{\mathsf{out}}}{2^{N_{\mathsf{block}}}}, \tag{41}$$

where Q_{C} counts ideal-cipher queries, heuristically identified with PRG queries.

Definition 6 describes the case of a single IV, which can be naturally extended to the case of multiple IVs as described in Definition 7.

Definition 7 (Multi-IV almost-injective PRG). *Let $\mathsf{PRG}(k, \mathsf{iv}, \mathsf{twk})$ be an IV-based tweakable PRG as in Definition 6. Consider the following game, parameterized by Q_{iv} and Q_{out}.*

1. *The adversary $\mathcal{A}$ outputs Q_{iv} IVs*

$$\mathsf{iv}_i \in \mathcal{I}_\lambda \quad \textit{for } i \in [1..Q_{\mathsf{iv}}],$$

together with Q_{out} tuples for each IV,

$$\big(\mathsf{twk}_{i,j}, y_{i,j}\big)_{j\in[1..Q_{\mathsf{out}}]}.$$

2. *If there exist $i \in [1..Q_{\mathsf{iv}}]$, $j \in [1..Q_{\mathsf{out}}]$, and two distinct keys $k \neq k'$ in $\mathcal{K}_\lambda$ such that*

$$\mathsf{PRG}(k, \mathsf{iv}_i, \mathsf{twk}_{i,j}) = y_{i,j} = \mathsf{PRG}(k', \mathsf{iv}_i, \mathsf{twk}_{i,j}),$$

then the game outputs 1; otherwise, it outputs 0.

We define the advantage of $\mathcal{A}$ in this game as

$$\mathsf{AdvInj}^{\mathsf{PRG}}_{\mathcal{A}}[Q_{\mathsf{iv}}, Q_{\mathsf{out}}] = \Pr[\textit{the game outputs } 1].$$

The classical and quantum multi-IV notions use the adversary types and resource convention of Definition 6, with classical output IVs and lists. For each model, multi-IV almost injectivity requires negligible advantage for every adversary of that type and polynomial $Q_{\mathsf{iv}}, Q_{\mathsf{out}}$.

The single-IV notion of Definition 6 corresponds to the special case that $Q_{\mathsf{iv}} = 1$. By a union-bound argument, a uniform single-IV bound $\epsilon(\mathcal{R}; Q_{\mathsf{out}})$ gives a multi-IV bound $Q_{\mathsf{iv}}\epsilon(\mathcal{R}; Q_{\mathsf{out}})$.

Keyed-hash PRF advantage. Let $\mathcal{K} = \{0,1\}^\lambda$, let $\mathcal{X}$ be an input domain, and let $\mathcal{Y}$ be an output range. For a random oracle $H : \{0,1\}^* \to \mathcal{Y}$, define the keyed-hash function

$$\mathsf{enc}(x, k) = x \| k, \qquad F^H_k(x) := H(\mathsf{enc}(x, k)).$$

A PRF distinguisher $\mathcal{A}$ is given oracle access to the public random oracle H and to a challenge oracle $O : \mathcal{X} \to \mathcal{Y}$, and outputs a bit. Its goal is to distinguish whether $O = F^H_k$ for a uniformly random key $k \leftarrow \mathcal{K}$, or whether $O = R$ for a uniformly random function $R \leftarrow \mathrm{Func}(\mathcal{X}, \mathcal{Y})$ independent of H. We define

$$\begin{aligned}\mathsf{AdvPRF}^{\mathsf{KH}}_{\mathcal{A}} = \big|\, &\Pr\big[\mathcal{A}^{H, F^H_k} = 1 : H \leftarrow \mathrm{Func}(\{0,1\}^*, \mathcal{Y}),\ k \leftarrow \mathcal{K}\big] \\ &- \Pr\big[\mathcal{A}^{H,R} = 1 : H \leftarrow \mathrm{Func}(\{0,1\}^*, \mathcal{Y}),\ R \leftarrow \mathrm{Func}(\mathcal{X}, \mathcal{Y})\big]\,\big|.\end{aligned} \tag{42}$$

When the query budget is explicit, we write $\mathsf{AdvPRF}^{\mathsf{KH}}_{\mathcal{A}}[q_H, q_F]$ for adversaries making at most q_H queries to H and at most q_F queries to the challenge oracle O.

Let $\epsilon := \max_{x\in\mathcal{X},\, u\in\{0,1\}^*} \Pr_{k\leftarrow\mathcal{K}}[\mathsf{enc}(x, k) = u]$. By the standard split-key PRF bound for hash-based constructions in the ROM, we have $\mathsf{AdvPRF}^{\mathsf{KH}}_{\mathcal{A}}[q_H, q_F] \le q_H\, 2^{-\lambda}$ [46]. A looser per-query hybrid bound, useful when each of the q_F invocations uses an independent hidden seed, is

$$\mathsf{AdvPRF}^{\mathsf{KH}}_{\mathcal{A}}[q_H, q_F] \le q_H q_F\, 2^{-\lambda}.$$

In the QROM, the split-key PRF bound of Don, Fehr, and Huang [47] gives

$$\mathsf{AdvQPRF}^{\mathsf{KH}}_{\mathcal{A}}[q_H, q_F] \le 4\sqrt{2q_F^2 q_H \epsilon} + 4\sqrt{2q_H^2 q_F \epsilon}.$$

Thus, for a uniform λ-bit hidden key,

$$\mathsf{AdvQPRF}^{\mathsf{KH}}_{\mathcal{A}}[q_H, q_F] \le 4\sqrt{\frac{2q_F^2 q_H}{2^\lambda}} + 4\sqrt{\frac{2q_H^2 q_F}{2^\lambda}}. \tag{43}$$

Each signing hybrid replaces one H_3 evaluation with an independently uniform ξ-bit hidden suffix ρ, so we take $q_F = 1$ and $\epsilon = 2^{-\xi}$. Consequently, the per-signature bounds used below are

$$\begin{aligned} h_{\mathsf{R}}(q_H) &= \min\{1, q_H 2^{-\xi}\}, \\ h_{\mathsf{Q}}(q_H) &= \min\{1, 4\sqrt{2}(\sqrt{q_H} + q_H) 2^{-\xi/2}\}. \end{aligned} \tag{44}$$

9.1.3 One-Wayness and Public-Key Pseudorandomness of Block Ciphers

We analyze two security properties of the functions used to generate public keys. One-wayness concerns the difficulty of finding a valid secret-key preimage of a given public key, and hence addresses direct key-recovery attacks. Public-key pseudorandomness concerns the difficulty of distinguishing an honestly generated public key from an independent uniform instance, which justifies the public-key replacement in the lossy-key signature proof. We first recall the inversion bounds for the standard and EM constructions over the full key space, then define the public-key distribution induced by the actual restricted key generation and explain its role in the signature reduction.

Standard block ciphers. As in Lemma 6, Chiesa *et al.* [36] show that it is feasible to use a block cipher, with key size equal to the block size and viewed as a PRP, to instantiate an OWF.

Lemma 6 (Security of block-cipher-based OWFs [36]). *Let $P : \mathcal{K} \times \mathcal{M} \to \mathcal{M}$ be a PRP. For any fixed $x \in \mathcal{M}$, define*

$$F_x(k) = P(k, x).$$

Then F_x is an OWF. Concretely, any adversary $\mathcal{A}$ against the OWF F_x can be turned into an adversary $\mathcal{A}'$ against the PRP P, such that

$$\mathsf{AdvOWF}^{F_x}_{\mathcal{A}} \le \left(1 + \frac{|\mathcal{K}|}{|\mathcal{M}|}\right) \mathsf{AdvPRP}^{P}_{\mathcal{A}'} + \frac{|\mathcal{K}|}{|\mathcal{M}|(|\mathcal{M}| - 1)}.$$

The reduction is tight such that the adversary $\mathcal{A}'$ makes at most 2 queries to the PRP oracle and takes computation similar to $\mathcal{A}$ plus one extra PRP evaluation.

In Chinith, we use block ciphers with $|\mathcal{K}| = |\mathcal{M}| = 2^\lambda$ as OWFs and assume that they are PRP-secure. By Lemma 6, for every OWF adversary $\mathcal{A}$, there exists a PRP adversary $\mathcal{A}'$ such that

$$\mathsf{AdvOWF}^{F_x}_{\mathcal{A}} \le 2 \cdot \mathsf{AdvPRP}^{P}_{\mathcal{A}'} + \frac{1}{2^\lambda - 1} \le 2 \cdot \mathsf{AdvPRP}^{P}_{\mathcal{A}'} + 2^{-\lambda+1}. \tag{45}$$

Even-Mansour Constructions. The Even-Mansour construction [35] is a way to construct a block cipher F from a random permutation π. We follow the proof of [37] and give a lemma below.

Lemma 7 (Security of Even–Mansour-based OWFs [37]). *Let $\mathcal{M} = \{0,1\}^\lambda$ and let $P : \mathcal{X} \times \mathcal{M} \to \mathcal{M}$ be a block cipher. For a fixed public value $x \in \mathcal{X}$, write $\pi_x(\cdot) = P(x, \cdot)$. In the ideal-permutation model, π_x is modeled as a uniformly random permutation over $\mathcal{M}$. Define the Even–Mansour OWF*

$$F_{\pi_x}(k) = \pi_x(k) \oplus k = P(x, k) \oplus k.$$

Then F_{π_x} is an OWF in the ideal-permutation model. Concretely, for any adversary $\mathcal{A}$ making at most q adaptive queries to π_x and π_x^{-1}, we have

$$\mathsf{AdvOWF}^{F_{\pi_x}}_{\mathcal{A}} \le \frac{2(q+1)}{2^\lambda - 1} \le 4(q+1) \cdot 2^{-\lambda}.$$

The running time is that of $\mathcal{A}$ plus one extra ideal-permutation query.

Restricted keys and public-key pseudorandomness. The SM4th.KeyGen, uBlockith.KeyGen, and Vistrutith.KeyGen algorithms sample the secret key K uniformly from $S = \{k \in \{0,1\}^\lambda : k[0]k[1] = 0\}$, of density $p = 3/4$, and sample the public block X independently and uniformly. The resulting public key is $(X, F_X(K))$, where

$$F_X(K) = \begin{cases} P(K, X), & \text{for a standard variant,} \\ P(X, K) \oplus K, & \text{for an EM variant.} \end{cases}$$

Define the public-key distinguishing advantage by

$$\mathsf{AdvPRG}^{\mathsf{OWF}}_{\mathcal{D}} = \left| \Pr[\mathcal{D}(X, F_X(K)) = 1 : K \leftarrow S] - \Pr[\mathcal{D}(X, Y) = 1 : Y \leftarrow \{0,1\}^\lambda] \right|, \tag{46}$$

where X is uniform and independent in both probabilities.

For each fixed X, $|F_X(S)| \le |S| = p2^\lambda$, so a uniform Y has no preimage in S with probability at least $1-p = 1/4$. An accepting signature for such an instance violates soundness. The lossy-key argument of FAEST [22] combines the public-key replacement with this image-size bound to relate forgery to soundness, with a factor $1/(1-p) = 4$, as used in Theorems 1 and 3.

For the standard construction, the following lemma bounds Equation (46) in terms of PRP security.

Lemma 8 (Restricted-key pseudorandomness for the standard construction). *Let the key and block spaces of P both have size $M = 2^\lambda$ and let S have density p. For every distinguisher $\mathcal{D}$ in Equation (46) for $F_X(K) = P(K, X)$, there is a distinguisher $\mathcal{B}$ making two classical challenge-PRP queries such that*

$$\mathsf{AdvPRG}^{\mathsf{OWF}}_{\mathcal{D}} \le \sqrt{\frac{2\mathsf{AdvPRP}^{P}_{\mathcal{B}}}{p} + \frac{1}{4p(M-1)}}. \tag{47}$$

The reduction makes two classical challenge queries and runs $\mathcal{D}$ four times, with the corresponding running time and auxiliary-oracle queries. The statement also holds for quantum $\mathcal{D}$ receiving a classical public key and using independent public quantum random oracles, provided its initial state is independent of the challenge and can be independently prepared for each run. For $p = 3/4$, the right-hand side is $\sqrt{(8/3)\mathsf{AdvPRP}^{P}_{\mathcal{B}} + 1/(3(2^\lambda - 1))}$.

Proof. First, fix the realizations of any independent public random oracles, and the following expectations are conditional on them. Set $f(x, y) = \Pr[\mathcal{D}(x, y) = 1]$, $\mu = \mathbb{E}_{X,Y} f(X, Y)$, and $a_k = \mathbb{E}_X f(X, P(k, X)) - \mu$. Cauchy–Schwarz gives $\mathbb{E}_K a_K^2 \ge p\big(\mathsf{AdvPRG}^{\mathsf{OWF}}_{\mathcal{D}}\big)^2$ for a full-space uniform K. Given a challenge permutation O, independently sample X_1, X_2, Y_1, Y_2 and run $A_i = \mathcal{D}(X_i, O(X_i))$, $B_i = \mathcal{D}(X_i, Y_i)$. Output one with probability $(1+(A_1-B_1)(A_2-B_2))/2$. For fixed O, its expectation is $1/2 + a_O^2/2$. Hence,

$$\mathbb{E}_K a_K^2 - \mathbb{E}_\pi a_\pi^2 \le \left|\mathbb{E}_K a_K^2 - \mathbb{E}_\pi a_\pi^2\right| = 2\mathsf{AdvPRP}^{P}_{\mathcal{B}}.$$

For a uniform permutation π, double-center the matrix f as $h_{xy} = f(x, y) - \mathbb{E}_Y f(x, Y) - \mathbb{E}_X f(X, y) + \mu$. Each assignment $\pi(x) = y$ has probability $1/M$, and two assignments with distinct inputs and outputs have probability $1/(M(M-1))$. Since every row sum and column sum of h is zero, expanding the square gives the identity below:

$$\mathbb{E}_\pi a_\pi^2 = \frac{\sum_{x,y} h_{xy}^2}{M^2(M-1)} \le \frac{1}{4(M-1)},$$

where the inequality follows from $\sum_{x,y} h_{xy}^2 \le \sum_{x,y} (f(x, y) - \mu)^2 \le M^2/4$. Finally, average the two second moments over the public random oracles. Their signed difference is bounded by twice the unconditional PRP advantage of $\mathcal{B}$. Together with the preceding Cauchy–Schwarz inequality, Jensen's inequality lower-bounds the averaged $\mathbb{E}_K a_K^2$ by $p\big(\mathsf{AdvPRG}^{\mathsf{OWF}}_{\mathcal{D}}\big)^2$, while the pointwise variance bound upper-bounds the averaged $\mathbb{E}_\pi a_\pi^2$ by $1/(4(M-1))$. Thus,

$$p\big(\mathsf{AdvPRG}^{\mathsf{OWF}}_{\mathcal{D}}\big)^2 \le 2\mathsf{AdvPRP}^{P}_{\mathcal{B}} + \frac{1}{4(M-1)},$$

which gives the claimed bound. □

The public value X is the cipher key in the EM construction, so Lemma 8 does not apply to it. The EM public-key pseudorandomness in Equation (46) is an additional assumption.

9.1.4 Security Analysis of VOLE Commitments

This section claims the security properties of VOLECommit as proved in FAEST [22].

Binding. We argue that the VOLE commitments as in VOLECommit are binding, by showing that there is an extractor algorithm that is allowed to observe random oracle queries, and will successfully extract the adversary's committed message with high probability.

Lemma 9 (VOLE commitment is extractable binding [22]). *Consider the extractable binding game below for* (VOLECommit, VOLEReconstruct)*, a stateful PPT adversary* $\mathcal{A}$ *and a straight-line extractor* $\mathcal{E}$*. The record supplies the actual IV directly; the protocol's additional check* $\mathsf{iv} = \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}})$ *can only reject. The extractor uses the supplied classical query logs:*

1. *Commit-Phase: Let* $\mathcal{T} = \{\}$ *be an empty associative array.*
   (a) $C = (\mathsf{com}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv}) \leftarrow \mathcal{A}^{\mathsf{H}_1,\mathsf{H}_2,\mathsf{H}_4}(\mathsf{commit})$.
   (b) $\mathcal{D} = (\boldsymbol{u}_i^*, \boldsymbol{V}_i^*)_{i\in[0,\tau)} \leftarrow \mathcal{E}(\mathcal{L}_1, \mathcal{L}_2, \mathsf{com}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv})$*, where* $\mathcal{L}_b$ *is a set* $\{(x_i, \mathsf{H}_b(x_i))\}$ *of query–response pairs from queries* $\mathcal{A}$ *made to the random oracle* H_b*.*
   (c) *Set* $\mathcal{T}[C] = \mathcal{D}$*. Note that* $\mathcal{D} = \bot$ *is allowed.*
   (d) $\mathcal{A}$ *can choose to go to Step 1a or it outputs* $C^\star$*. If* $C^\star \notin \mathrm{dom}(\mathcal{T})$*, output* 0*; otherwise continue below.*
2. *Decommitment-Phase: Set* $\mathcal{D} = \mathcal{T}[C^\star]$ *and parse* $C^\star = (\mathsf{com}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv})$*. If* $\mathcal{D} \neq \bot$*, parse* $\mathcal{D} = (\boldsymbol{u}_i^*, \boldsymbol{V}_i^*)_{i\in[0,\tau)}$*.*
3. $[\mathsf{chall}_3, \mathsf{decom}_I] \leftarrow \mathcal{A}^{\mathsf{H}_1,\mathsf{H}_2,\mathsf{H}_4}(\mathsf{open})$*, where* $I = (\Delta_0, \ldots, \Delta_{\tau-1}) \leftarrow \mathsf{DecodeChall}(\mathsf{chall}_3)$*.*
4. *If* $I = \bot$ *then output* 0 *(failure).*
5. *For* $i \in [0\,..\,\tau)$*:* $(\delta_{i,0}, \ldots, \delta_{i,k_i-1}) \leftarrow \mathsf{BitDec}(\Delta_i, k_i)$*.*
6. $(\overline{\mathsf{com}}, Q) \leftarrow \mathsf{VOLEReconstruct}^{\mathsf{H}_1,\mathsf{H}_2}(\mathsf{chall}_3, \mathsf{decom}_I, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \mathsf{iv}, \hat{\ell}, \mathsf{param})$*.*
7. *Parse* $[Q_0 \cdots Q_{\tau-1} 0_{\hat{\ell},w}] = Q$*, where* $Q_i \in \mathbb{F}_2^{\hat{\ell}\times k_i}$*.*
8. *Set* $\boldsymbol{c}_0 = \boldsymbol{0}$*. Output* 0 *(failure) if*
   (a) $\overline{\mathsf{com}} = \bot$ *or* $\mathsf{com} \neq \overline{\mathsf{com}}$ *(i.e., verifier rejects), or*
   (b) $\mathcal{D} \neq \bot$ *and* $Q_i = \boldsymbol{V}_i^* \oplus [\delta_{i,0} \cdot (\boldsymbol{u}_i^* \oplus \boldsymbol{c}_i), \ldots, \delta_{i,k_i-1} \cdot (\boldsymbol{u}_i^* \oplus \boldsymbol{c}_i)]$ *for* $i \in [0, \tau)$ *(i.e., extraction correct).*
9. *Output* 1 *(success) otherwise.*

Suppose $\mathcal{A}$ *makes at most* M *commitment-phase calls and* Q_1, Q_2, Q_4 *queries to* $\mathsf{H}_1, \mathsf{H}_2, \mathsf{H}_4$*, respectively, including its final verification, with* $Q_4 \geq 1$*. Moreover, suppose that* $\mathcal{A}$ *checks whether* VOLEReconstruct *returns* $\bot$ *for the purported opening* decom_I*, and outputs* $\bot$ *if that happens. There is a deterministic (unbounded) straight-line extractor* $\mathcal{E}$ *which does not program any oracle, such that the advantage of* $\mathcal{A}$ *in the above game is at most*

$$\frac{(Q_1 + M + 1)^2}{2^{2N_{\mathsf{block}}}} + \mathsf{AdvInj}_{\mathcal{B}}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}[Q_4, L],$$

which is no more than

$$\frac{(Q_1 + M + 1)^2}{2^{2N_{\mathsf{block}}}} + Q_4 \cdot \mathsf{AdvInj}_{\mathcal{B}}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}[L] \tag{48}$$

according to Definition 7. The resources of $\mathcal{B}$ *include the complete simulation and its query log.*

For each record, fix the first correctly sized queried preimages of com and its group summaries; store $\bot$ and register the first missing target, if any. An H_1 collision or a later answer hitting one of at most M registered targets costs at most $\binom{Q_1}{2} + MQ_1)2^{-2N_{\mathsf{block}}}$. Otherwise, an accepted record has its structural leaf list fixed. The extractor chooses the first seed preimage of each leaf, while a missing leaf uses a zero expansion, which cancels in the affine relation if that leaf is hidden. Two missing leaves in one group force rejection. Extraction is therefore correct whenever the final selected record has no ambiguous leaf. An efficient $\mathcal{B}$ records the public structural preimages and outputs only the selected record's actual IV and at most L leaf-commitment pairs. It neither enumerates seeds nor receives the extractor's private data. Ambiguity in this record wins the adaptive-IV game, covered by $[Q_4, L]$ for $Q_4 \geq 1$; no union over the other records is needed.

Hiding. The hiding property of VOLECommit bounds a PPT adversary's advantage in distinguishing the selective-opening view in Lemma 10 from the corresponding view in which $\boldsymbol{u}$ and $(\boldsymbol{c}_i)_{i\in[1,\tau)}$ are replaced by independent uniform bit vectors.

Lemma 10 (VOLECommit is multi-hiding [22]). *Let* H_1 *be a random oracle and fix admissible* $\hat{\ell}$ *and* param*. For* $b^* \in \{0, 1\}$*, define* $G_{b^*}^*$ *as follows.*

1. *For each $j \in [1, Q]$, independently:*
   - *Sample $r^j \leftarrow \{0,1\}^{N_{\mathsf{block}}}$ and $\mathsf{iv}^j \leftarrow \{0,1\}^{\lambda_{\mathsf{iv}}}$, and compute*
   $$(\mathsf{com}^j, \mathsf{decom}^j, (\boldsymbol{c}_i^j)_{i\in[1,\tau)}, \boldsymbol{u}^j, \boldsymbol{V}^j) \leftarrow \mathsf{VOLECommit}(r^j, \mathsf{iv}^j, \hat{\ell}, \mathsf{param}).$$
   - *Sample $\mathsf{chall}_3^j \leftarrow \{0,1\}^{\lambda_{\mathbb{F}} - w_{\mathsf{grind}}} \times 0^{w_{\mathsf{grind}}}$ uniformly conditioned on opening acceptance for $I^j \leftarrow \mathsf{DecodeChall}(\mathsf{chall}_3^j; \mathsf{param})$, and compute $\mathsf{decom}_{I^j} \leftarrow \mathsf{BAVC.Open}(\mathsf{decom}^j, I^j)$.*
   - *If $b^* = 0$, set $(\overline{\boldsymbol{u}}^j, (\overline{\boldsymbol{c}}_i^j)_i) = (\boldsymbol{u}^j, (\boldsymbol{c}_i^j)_i)$, retaining the values returned by $\mathsf{VOLECommit}$. If $b^* = 1$, sample this entire tuple independently and uniformly from $(\mathbb{F}_2^{\hat{\ell}})^\tau$.*
2. *Give*
   $$\left(\mathsf{iv}^j, \mathsf{com}^j, \mathsf{decom}_{I^j}, \mathsf{chall}_3^j, \overline{\boldsymbol{u}}^j, (\overline{\boldsymbol{c}}_i^j)_{i\in[1,\tau)}\right)_{j\in[1,Q]}$$
   to $\mathcal{A}^{\mathsf{H}_1}$ and return its output bit b.

Opening acceptance depends only on I^j, so the accepted challenges can be sampled before the commitments. The same bound holds for sequentially requested records with retained public-oracle history, fresh independent uniform roots, and the actual $\mathsf{iv}^j = \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre},j})$. IVs may repeat. Each standard-cipher reduction retains this history and uses the prescribed cipher inputs, and the selected opening indices are fixed before their commitment is generated.

Write PRG_m for a call with an N_{block}-bit seed and m output bits, and put

$$\beta = 1, \quad D = \lceil \log L \rceil, \quad h = \lceil \hat{\ell}/N_{\mathsf{block}} \rceil, \qquad A_H = 4D\beta^2 + 4\beta^2 N_0^2 + h^2.$$

Let ϵ_T, $\epsilon_L(q)$ and ϵ_E denote the PRP advantage terms for the tree, leaf and expansion hybrids, respectively:

$$\epsilon_T = \mathsf{AdvPRP}_{\mathcal{B}_T}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}[1], \quad \epsilon_L(q) = \mathsf{AdvPRP}_{\mathcal{B}_L}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}[q], \quad \epsilon_E = \mathsf{AdvPRP}_{\mathcal{B}_E}^{\mathsf{PRG}_{\hat{\ell}}}[1]. \tag{49}$$

Each ϵ bounds the largest advantage among the reductions at the corresponding hybrid steps, with the resource restriction of Equation (39). The PRP-type game for PRG_m follows FAEST [22] with an N_{block}-bit seed and block size. It samples one uniform hidden seed $k \in \{0,1\}^{N_{\mathsf{block}}}$ and one uniform IV, gives the IV to the distinguisher, and permits at most q queries indexed by a tweak. The real oracle returns $\mathsf{PRG}(k, \mathsf{iv}, \mathsf{twk}; m)$. For every new tweak the ideal oracle samples $\lceil m/N_{\mathsf{block}} \rceil$ fresh N_{block}-bit blocks without replacement across all answers, concatenates them and truncates to m bits; repeated tweaks repeat the same answer. The advantage is the absolute difference in the distinguisher's output-one probabilities. Thus, q counts PRG queries under that seed and IV, and their simulation uses at most $q\lceil m/N_{\mathsf{block}} \rceil$ underlying cipher queries. For the underlying block cipher E, put

$$\gamma_m = \mathsf{AdvPRP}^E[\mathcal{R}; m] + \binom{m}{2} 2^{-N_{\mathsf{block}}},$$

where $\mathcal{R}$ includes the full simulation resources and the m challenge queries are classical. Then

$$|\Pr[G_0^* = 1] - \Pr[G_1^* = 1]| \le Q\tau\big((2D+1)\gamma_2 + \gamma_{h+2}\big). \tag{50}$$

The same statement applies with quantum PRP adversaries and quantum oracle access.

Proof. Fix the accepted opening indices before generating each commitment. Starting from G_0^*, randomize the hidden tree paths in at most $Q\tau D$ steps. For each unopened leaf, jointly randomize its two commitment blocks and h expansion blocks under the same seed; their cipher inputs are distinct under the prescribed tweak encoding ($L - 1 + \tau < 2^{31}$ and $h \le 2^{32}$ for the present PRG). These steps cost at most $Q\tau D\gamma_2 + Q\tau\gamma_{h+2}$. The unexposed VOLE sums are then uniform, and their linear map to $(\boldsymbol{u}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1})$ is invertible. Starting from G_1^*, randomize the same hidden tree paths and leaf commitments, at cost at most $Q\tau(D+1)\gamma_2$. The resulting distributions coincide. The permutation-to-random-function comparison uses classical challenge queries, also for quantum distinguishers. □

Unpredictability. We recall that VOLECommit has the unpredictability property, *i.e.*, any PPT adversary cannot predict, before the commitment is generated, the commitment string together with its associated correction vectors.

Lemma 11 (VOLECommit is unpredictable [22]). *Fix some $\hat{\ell} \in \mathbb{N}$ and admissible* param*. Let $\mathcal{A}$ be a PPT adversary in the following game* $\mathsf{ExpUnpred}_{b,\mathcal{A}}^{\mathsf{VOLECommit},\hat{\ell},\mathsf{param}}(\lambda)$*:*

1. $(\mathcal{L}_{\mathsf{com}}, \mathsf{state}) \leftarrow \mathcal{A}^{\mathsf{H}_1}(1^\lambda)$*. Here $\mathcal{L}_{\mathsf{com}}$ is a list of commitment guesses.*
2. *Retain the public-oracle history and actual* $\mathsf{iv} = \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}})$*, with* $\mathsf{iv}^{\mathsf{pre}}$ *drawn from that history or freshly sampled. After the guess list is fixed, sample $r \leftarrow \{0,1\}^{N_{\mathsf{block}}}$ independently of this history and IV.*
3. $C = (\mathsf{com}, \mathsf{decom}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}, \boldsymbol{u}, \boldsymbol{V}) \leftarrow \mathsf{VOLECommit}(r, \mathsf{iv}, \hat{\ell}; \mathsf{param})$*.*
4. *Let $b' \leftarrow \mathcal{A}^{\mathsf{H}_1}(\mathsf{state}, C)$.*
5. *If $(\mathsf{com}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1}) \in \mathcal{L}_{\mathsf{com}}$ and $b = 1$, output 0. Else output b'.*

Let G_b denote this experiment with flag b. With D and γ_m as in Lemma 10, retain the numerical constant

$$A_U = 4D + 16.$$

For at most Q_1 classical oracle queries and Q_{com} guesses, the event-only hybrid gives

$$\Pr[G_0 = 1] \le \Pr[G_1 = 1] + (D+1)\gamma_2 + (2Q_1 + Q_{\mathsf{com}})2^{-2N_{\mathsf{block}}}. \tag{51}$$

Proof. Bound $\Pr[G_0 = 1] - \Pr[G_1 = 1]$ by the probability that com hits the commitment-coordinate projection of $\mathcal{L}_{\mathsf{com}}$. Randomizing one path and its leaf costs $(D+1)\gamma_2$. The reductions test the hit event before the continuation receives C; they need not reconstruct an unknown challenge seed. The two premature-query events and the list hit cost at most $(2Q_1 + Q_{\mathsf{com}})2^{-2N_{\mathsf{block}}}$. □

9.1.5 EUF-CMA Security in the ROM

This section presents the EUF-CMA security of our signature schemes. We first claim the EUF-KO property in Theorem 1, which is later extended into EUF-CMA in Theorem 2.

Theorem 1 (EUF-KO in the ROM [22]). *Let $\mathsf{H}_1, \{\mathsf{H}_2^i\}_{i\in[0,3]}$ and H_4 be modeled as random oracles and* param *be the parameters of our signature schemes. Let $\mathcal{A}$ be an adversary which, for simplicity, runs* Verify *on its purported forgery and outputs $\perp$ if the verification fails. Let $Q_1, Q_{2,i}, Q_3$ and Q_4 denote the upper bounds on the number of queries of $\mathcal{A}$ to $\mathsf{H}_1, \mathsf{H}_2^i$, H_3, and H_4, respectively, and let $Q_2 = Q_{2,1} + Q_{2,2} + Q_{2,3}$. Let n_V, n_{64} be as in Lemma 2, and let C be the number of constraint entries in Table 2. Suppose that $(n_V - 1)(n_{64} - 1) \le 2^{14}$, $C + 1 \le 2^{13}$, and $2\log_2 \tau \le \min\{B, 50\}$. Put $\epsilon_{\mathsf{inj}} = \mathsf{AdvInj}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}[Q_4, L]$, with the budgets ordered as in Definition 7. Then, we have*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{EUF\text{-}KO}} \le \frac{1}{1 - \frac{\mathsf{dom(OWF)}}{\mathsf{cod(OWF)}}} \cdot \left(\mathsf{AdvSnd}_{\mathcal{B}_2}^{\mathsf{Sig}} + \mathsf{AdvPRG}_{\mathcal{B}_1}^{\mathsf{OWF}}\right).$$

where OWF *denotes the public-key function specified by* param*, and*

$$\begin{aligned} \mathsf{AdvSnd}_{\mathcal{B}_2}^{\mathsf{Sig}} \le{}& \frac{(Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + 4)^2}{2^{2\lambda}} \\ &+ \frac{(Q_1 + Q_2 + 5)^2}{2^{2N_{\mathsf{block}}}} + \epsilon_{\mathsf{inj}} \\ &+ \frac{(Q_{2,1} + Q_{2,2} + Q_{2,3} + 9)\cdot d}{2^{\lambda_{\mathbb{F}}}}. \end{aligned} \tag{52}$$

The term $\mathsf{AdvPRG}_{\mathcal{B}_1}^{\mathsf{OWF}}$ *is defined by Equation (46), at the resource budget of $\mathcal{B}_1$. Definition 7 gives*

$$\epsilon_{\mathsf{inj}} \le Q_4\, \mathsf{AdvInj}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}[\mathcal{R}; L], \tag{53}$$

where $\mathcal{R}$ includes the total cipher-query budget across IVs. If the classical heuristic of Equation (41) is used as a separate estimate, the corresponding coarse expression is

$$Q_4\left(\frac{Q_{\mathsf{C}}^2}{2^{2N_{\mathsf{block}}}}+\frac{L}{2^{N_{\mathsf{block}}}}\right). \tag{54}$$

In our signature schemes, the key generation algorithm KeyGen *excludes keys with bits $k[0]=k[1]=1$. Thus, we have*

$$\frac{\mathsf{dom}(\mathsf{OWF})}{\mathsf{cod}(\mathsf{OWF})}=\frac{3}{4} \quad \textit{and} \quad \frac{1}{1-\frac{\mathsf{dom}(\mathsf{OWF})}{\mathsf{cod}(\mathsf{OWF})}}=4.$$

Proof. Lemmas 2 and 3 give $\epsilon_V \le 2^{-\lambda_{\mathbb{F}}-B}(1+2^{B-50})$ and $\epsilon_{\mathsf{zk}} \le 2^{-\lambda_{\mathbb{F}}}(1+2^{-38})$. Following the round-by-round argument of FAEST [22], the three transition errors are bounded by $\binom{\tau}{2}\epsilon_V$, ϵ_{zk}, and $d/2^{\lambda_{\mathbb{F}}}$. The first two are smaller than $2/2^{\lambda_{\mathbb{F}}}$ under the stated conditions. Since $d \ge 2$, the round-by-round soundness error is bounded by $d/2^{\lambda_{\mathbb{F}}}$. The direct classical query accounting below gives Equation (52). □

All parameter sets satisfy these conditions: $(n_V-1)(n_{64}-1) \le 1905$, $C+1 \le 4610$, and $2\log_2\tau \le 12 < B = 16$. For Vistrutith, $\ell = 6912$ and $\hat{\ell} = 8464$ give a hashed prefix of 7936 bits, padded to $\ell' = \ell_{64} = 8192$ bits. Thus $n_V = 16$, $n_{64} = 128$, and the degree product is $15\cdot 127 = 1905$.

Process the classical queries, including final verification, chronologically. Create one binding record at each fresh well-formed H_2^1 input, using its actual IV; thus $M \le Q_{2,1} \le Q_2$. With $Q_{\mathrm{FS}} = \sum_{j=0}^{3} Q_{2,j}$, an output collision at H_2^j or a fresh output hitting a previously queried H_2^{j+1} prefix, for $j = 0,1,2$, costs at most $Q_{\mathrm{FS}}^2 2^{-2\lambda}$ in total. Outside these events the final transcript has a chronological parent chain. Apply binding only to its selected record, and charge each algebraic failure at its fresh challenge query, with the preceding coefficients fixed. These tests are defined before each fresh answer, without conditioning on future bad events; the H_2^3 input fixes every claimed coefficient, including the reconstructed $\tilde{a}_0$. The KOS equations partition the groups into fixed and free challenges; outside a VOLEHash collision the free groups have a common u_{eff}, fixing the effective witness before the second challenge. Degree-two and degree-three masks leave the highest coefficient unchanged. Hence each conditional algebraic error is at most $d2^{-\lambda_{\mathbb{F}}}$, giving $Q_2 d 2^{-\lambda_{\mathbb{F}}}$ in total. Together with $(Q_1+Q_{2,1}+1)^2 2^{-2N_{\mathsf{block}}}+\epsilon_{\mathsf{inj}}$, these losses are bounded by Equation (52).

We next state the EUF-CMA property as in Theorem 2.

Theorem 2 (EUF-CMA in the ROM [22]). *Let $\mathsf{H}_2^0, \mathsf{H}_1, \mathsf{H}_2^1, \mathsf{H}_2^2, \mathsf{H}_2^3$ and H_4 be modeled as random oracles. Let H_3 be a PRF. Let $\mathcal{A}$ make $s = Q_{\mathsf{sig}}$ signing queries. Let $Q_{2,0}, Q_1, Q_{2,1}, Q_{2,2}, Q_{2,3}, Q_4$ bound the corresponding queries and privately sampled or programmed entries in each reduction, including signing, reconstruction and final verification, and let Q bound their total. Count each entry once when created; later queries still count as queries. If q_{ext} bounds all external RO queries, the conservative choice*

$$Q = q_{\mathrm{ext}} + s(2\tau+T+10)+\tau+6, \qquad T \le 2^{b_{\mathrm{ctr}}},$$

suffices; the one-step H_3 budget is at most $q_{\mathrm{ext}}+s$. Let ℓ_1, ℓ_2 be the first two challenge lengths. In the present instantiation they are $5\lambda_{\mathbb{F}}+64$ and $3\lambda_{\mathbb{F}}+64$, where 64 is the universal-hash tail length, and $b_{\mathrm{ctr}} = 32$, $T = 2^{32}$. Define

$$C_{\mathsf{R}}(Q) = \binom{Q}{2}(2^{-2\lambda}+2^{-\ell_1}+2^{-\ell_2}).$$

Put

$$\eta_{\mathsf{prg}} = (D+1)\gamma_2 + \tau\big((2D+1)\gamma_2+\gamma_{h+2}\big). \tag{55}$$

There is a key-only adversary $\mathcal{B}$, within the above complete resources, such that

$$\begin{aligned}\mathsf{AdvEUFCMA}_{\mathcal{A}}^{\mathsf{Sig}}[Q_{\mathsf{sig}}] \le{}& \rho\,\mathsf{AdvEUFKO}_{\mathcal{B}}^{\mathsf{Sig}} + \rho C_{\mathsf{R}}(Q) + \rho Q_{\mathsf{sig}}\left(\mathsf{AdvPRF}^{\mathsf{H}_3}+\eta_{\mathsf{prg}}\right)\\ &+\rho Q_{\mathsf{sig}}\left(\frac{2Q_1+Q_{2,1}}{2^{2N_{\mathsf{block}}}}+\frac{Q_{\mathsf{sig}}+Q_{2,2}}{2^{\ell_1}}+\frac{Q_{\mathsf{sig}}+Q_{2,3}}{2^{\ell_2}}\right),\end{aligned} \tag{56}$$

where

$$\rho = \exp\left(\frac{Q_{\mathsf{sig}}(A_U + \tau A_H)}{2^{N_{\mathsf{block}}}}\right). \tag{57}$$

Here $\mathsf{AdvPRF}^{\mathsf{H}_3}$ *is the maximum one-step advantage from Equation (44), including all public-oracle calls of that hybrid distinguisher. The symbols* D, A_H, A_U *and all PRP terms are defined in Lemmas 10 and 11; the hiding experiment uses* $Q = Q_{\mathsf{sig}}$ *records. The additive comparisons already hold with* $\rho = 1$*; the displayed* $\rho \geq 1$ *is retained as a conservative relaxation and is at most* 2 *at the assigned signature limits. The reduction and bound apply to both non-EM and EM variants under the stated primitive assumptions.*

Proof. Replace the signing seeds by independent uniform values and apply the classical unpredictability and joint hiding bounds with the actual IV history. Fresh-prefix aborts cost at most $(s+Q_{2,2})2^{-\ell_1} + (s+Q_{2,3})2^{-\ell_2}$ per signature. Pre-sample the finite counter stream, retaining every rejected attempt and programmed entry. On exhaustion, use a fixed admissible opening index only for the hiding comparison and return $\perp$; if no index is admissible, signing always returns $\perp$. The hidden expansions make the VOLE sums uniform. The degree-two response mask is uniform; for degree three, $(u_0^*, u_1^*) \mapsto (u_0^* + \Delta u_1^*, u_1^*)$ is invertible. Computing the constant response by the verifier identity therefore gives a witness-independent simulated view, including exhaustion. A final new-message forgery first hitting a signing patch entails a message-digest collision or a hit on a sampled first- or second-challenge target. Counting all sampled values gives $C_{\mathsf{R}}(Q)$. Otherwise the simulated and original-RO verifiers agree. The executable $\mathcal{B}$ outputs the candidate for original-RO verification; simulated verification is only part of the comparison experiment. It uses a private patch table and the prescribed oracle interfaces. Summing these costs proves the bound, with the KO budget charged for the complete simulation. □

According to Theorem 2, the EUF-CMA security of our signature schemes follows from the EUF-KO security of the underlying construction, the PRF security of H_3, the pseudorandomness of the PRG calls, and the unpredictability and multi-hiding properties of VOLECommit. The concrete losses include the number of signing queries and the resource-dependent primitive advantages.

9.1.6 EUF-CMA Security in the QROM

We follow the FAEST-EM QROM proof [22].

Explicit-leaf transcript. Fix a parameter set with domain identifier D_{p}, and denote its signature scheme by Sig. Let Sig_r use independent uniform $r \in \{0,1\}^{N_{\mathsf{block}}}$ and $\mathsf{iv}^{\mathsf{pre}} \in \{0,1\}^{\lambda_{\mathsf{iv}}}$ and one counter attempt, with the same verification algorithm as Sig. The expanded verifier IP^+ uses the following transcript.

j	p_j	γ_j and its length in bits
-3	$\mathsf{iv}^{\mathsf{pre}}$	$\mathsf{iv} : \lambda_{\mathsf{iv}}$
-1	$(c_{i,j}^{\mathsf{leaf}})_{i<\tau,\, j<N_i}$	$(h_i)_{i<\tau} : 2N_{\mathsf{block}}\tau$
0	$\emptyset$	$\mathsf{com} : 2N_{\mathsf{block}}$
1	$(\boldsymbol{c}_1, \ldots, \boldsymbol{c}_{\tau-1})$	$\mathsf{chall}_1 : 5\lambda_{\mathbb{F}} + 64$
2	$(\tilde{\boldsymbol{u}}, \tilde{\boldsymbol{V}}, \boldsymbol{d}_{\mathsf{w}})$	$\mathsf{chall}_2 : 3\lambda_{\mathbb{F}} + 64$
3	$(\tilde{a}_0, \ldots, \tilde{a}_{d-1}, \mathrm{LE32}(\mathsf{ctr}))$	$\mathsf{chall}_3 : \lambda_{\mathbb{F}}$
4	decom_I	no challenge

(58)

Here, $c_{i,j}^{\mathsf{leaf}} \in \{0,1\}^{2N_{\mathsf{block}}}$ is the commitment to an N_{block}-bit seed with tweak $i + L - 1$, $\hat{\ell} = \ell + d\lambda_{\mathbb{F}} + B$, $\boldsymbol{c}_i \in \{0,1\}^{\hat{\ell}}$, $\tilde{\boldsymbol{u}} \in \{0,1\}^{\lambda_{\mathbb{F}}+B}$, $\tilde{\boldsymbol{V}} \in \{0,1\}^{(\lambda_{\mathbb{F}}+B)\times\lambda_{\mathbb{F}}}$, and $\boldsymbol{d}_{\mathsf{w}} \in \{0,1\}^{\ell}$. Each $\tilde{a}_i$ is a $\lambda_{\mathbb{F}}$-bit field encoding, and decom_I has $N_{\mathsf{block}}(2\tau + T_{\mathsf{open}})$ bits. The verifier runs BAVC.Reconstruct and VOLEReconstruct, replacing their H_1 outputs by the supplied h_i and com, and requires every reconstructed leaf commitment to equal its entry in p_{-1}. It retains the length, grinding, opening, padding, and challenge-decoding checks, verifies $\tilde{\boldsymbol{V}} = \mathsf{VOLEHash}(\mathsf{chall}_1, \boldsymbol{Q}) + \tilde{\boldsymbol{u}}\,\mathsf{chall}_3^{\mathsf{T}}$ in the specified bit representation, and compares the selected $\mathsf{OWFVerify}$ output with the $\tilde{a}_0$ in p_3. It

additionally rejects if any $c_{i,j}^{\mathsf{leaf}}$ has more than one seed preimage under $\mathsf{PRG}_{2N_{\mathsf{block}}}(\cdot, \mathsf{iv}, i+L-1)$. Define $\mathsf{Sig}_r^{\mathrm{elc}}$ to output all seven messages and to derive the challenges by

$$\begin{aligned} \mu &= \mathsf{H}_2^0(D_{\mathsf{p}} \| \mathsf{pk} \| \mathsf{msg}; 2\lambda), & \mathsf{iv} &= \mathsf{H}_4(p_{-3}; \lambda_{\mathsf{iv}}), \\ h_i &= \mathsf{H}_1(c_{i,0}^{\mathsf{leaf}} \| \cdots \| c_{i,N_i-1}^{\mathsf{leaf}}; 2N_{\mathsf{block}}), & \mathsf{com} &= \mathsf{H}_1(h_0 \| \cdots \| h_{\tau-1}; 2N_{\mathsf{block}}), \\ \mathsf{chall}_1 &= \mathsf{H}_2^1(\mu \| \mathsf{com} \| p_1 \| \mathsf{iv}; 5\lambda_{\mathbb{F}} + 64), \\ \mathsf{chall}_2 &= \mathsf{H}_2^2(\mathsf{chall}_1 \| p_2; 3\lambda_{\mathbb{F}} + 64), \\ \mathsf{chall}_3 &= \mathsf{H}_2^3(\mathsf{chall}_2 \| p_3; \lambda_{\mathbb{F}}). \end{aligned} \tag{59}$$

Acceptance-preserving conversion and resources. Given an accepted (msg, σ), run its verification once and record the leaf values hashed by BAVC.Reconstruct, the reconstructed $\tilde{\boldsymbol{V}}'$, and the returned $\tilde{a}_0$. Use these values for p_{-1}, the matrix in p_2, and the first field element in p_3, and copy the remaining messages from σ. All checks of IP^+ then hold except possibly its additional multiple-preimage test. A quantum almost-injectivity adversary outputs the derived iv and the L pairs $(i+L-1, c_{i,j}^{\mathsf{leaf}})$, and every failure of that additional test is a win in Definition 6. Its additional public-oracle queries, in the order $(\mathsf{H}_1, \mathsf{H}_2^0, \mathsf{H}_2^1, \mathsf{H}_2^2, \mathsf{H}_2^3, \mathsf{H}_4)$, are bounded by $(\tau+1, 1, 1, 1, 1, 1)$. The injectivity reduction has one output IV and L output pairs, where its resources follow Definition 6, including the verification above and all IV searches.

Algebraic transition bounds. For a fixed IV and a leaf list with no multiple preimages, complete each leaf to its unique seed when one exists and to $0^{N_{\mathsf{block}}}$ otherwise, and compute the corresponding $(\boldsymbol{u}_i, \boldsymbol{V}_i)$ by ConvertToVOLE. A leaf with no preimage must be at an unopened position in any accepted transcript, and its assigned expansion cancels from the reconstructed VOLE value by Lemma 4.

Lemma 12 (Round-by-round soundness of the expanded verifier). *Under the hash-size hypotheses of Theorem 1, IP^+ has round-by-round soundness on invalid instances of the specified OWF constraint relation, with a passive prefix through iv and transition errors*

$$\begin{aligned} \epsilon_{-3} = \epsilon_{-1} = \epsilon_0 = 0, \\ \epsilon_1 \le \binom{\tau}{2} \epsilon_V, \quad \epsilon_2 \le \epsilon_{\mathsf{zk}}, \quad \epsilon_3 \le d\, 2^{-\lambda_{\mathbb{F}}}. \end{aligned} \tag{60}$$

In particular, $\max_i \epsilon_i \le d\, 2^{-\lambda_{\mathbb{F}}}$.

Proof. Instantiate the state function of FAEST-v2 [22], Lemmas 9.34 and 9.37, on the completed VOLE pairs above and the constraint vector from SM4th.OWFConstraints, uBlockith.OWFConstraints, or Vistrutith.OWFConstraints, according to the parameter set. Accepted openings agree with the completed VOLE pairs by the argument above. For the first two transitions, apply Lemma 2 to each pair of distinct corrected secrets and take a union bound, and apply Lemma 3 to the non-zero constraint vector. For the last transition, degree lifting makes the verification polynomial have degree at most d, whereas the response $\sum_{j=0}^{d-1} \tilde{a}_j X^j$ has degree at most $d-1$; a non-zero difference has at most d roots in $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. The first two bounds are below $2 \cdot 2^{-\lambda_{\mathbb{F}}}$ under the stated hash-size hypotheses, and $d \ge 2$. The three prefix challenges perform no algebraic test, so their errors are zero and the passive-prefix argument of FAEST's Lemma 9.37 applies unchanged. The final challenge is uniform on the full field in this argument. □

Theorem 3 (EUF-KO in the QROM). *Assume the hash-size hypotheses of Theorem 1 and the public-key experiment of Equation (46). Let $\mathcal{A}^{\mathsf{H}_1, \mathsf{H}_2^0, \mathsf{H}_2^1, \mathsf{H}_2^2, \mathsf{H}_2^3, \mathsf{H}_3, \mathsf{H}_4}$ be a QROM EUF-KO adversary that makes a total of $Q_1, Q_{2,0}, Q_{2,1}, Q_{2,2}, Q_{2,3}, Q_3, Q_4$ queries to its oracles. Then, there exist a quantum public-key distinguisher $\mathcal{B}_2$ in the experiment of Equation (46), a QROM adversary $\mathcal{B}_1^{\mathsf{H}_1, \mathsf{H}_2^0, \mathsf{H}_2^1, \mathsf{H}_2^2, \mathsf{H}_2^3, \mathsf{H}_4}$ against the $\mathsf{Sig}_r^{\mathrm{elc}}$ that makes at most a total of $Q_1 + \tau + 1$, $Q_{2,0} + 1$, $Q_{2,1} + 1$,*

$Q_{2,2}+1$, $Q_{2,3}+1$, *and* Q_4+1 *queries to its oracles, and a quantum adversary* $\mathcal{B}_3$ *in the almost-injectivity game of Definition 6, such that*

$$\mathsf{AdvEUFKO}^{\mathsf{Sig}}_{\mathcal{A}} \le \frac{1}{1-\frac{\mathsf{dom(OWF)}}{\mathsf{cod(OWF)}}} \cdot \left(\mathsf{AdvSnd}^{\mathsf{Sig}^{\mathrm{elc}}_r}_{\mathcal{B}_1} + \mathsf{AdvPRG}^{\mathsf{OWF}}_{\mathcal{B}_2} + \mathsf{AdvInj}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}_{\mathcal{B}_3}[L]\right),$$

Here, $\mathsf{Sig}^{\mathrm{elc}}_r$ *is defined by Equations (58) and (59), and the injectivity advantage is quantum with the conversion resources specified above. The soundness bound is*

$$\mathsf{AdvSnd}^{\mathsf{Sig}^{\mathrm{elc}}_r}_{\mathcal{B}} \le 10(\tau+1)\cdot Q^3\cdot 2^{-2\lambda} + 10\cdot Q^2\cdot d\cdot 2^{-\lambda_{\mathbb{F}}}.$$

Here, we have

$$Q = Q_1 + Q_{2,0} + Q_{2,1} + Q_{2,2} + Q_{2,3} + Q_4 + 3\tau + 19.$$

Proof. The conversion above bounds the loss from the additional leaf test by the single-IV quantum injectivity advantage, and the public-key replacement and invalid-instance probability give the displayed lossy-key factor. For soundness, use the first-bad-transition and collision predicate of FAEST-v2 [22], Lemma 9.39, on Equation (59), with the state from Lemma 12. The collision part covers $\mathsf{H}_1, \mathsf{H}^0_2, \mathsf{H}^1_2, \mathsf{H}^2_2$; the final challenge is handled by the transition predicate. Its oracle cases give the following local bounds for a database of size at most k:

oracle domain	probability of creating a bad witness
H_4	0
H_1	$(\tau+1)k\,2^{-2N_{\mathsf{block}}}$
H^0_2	$k\,2^{-2\lambda}$
H^1_2	$k\,2^{-(5\lambda_{\mathbb{F}}+64)} + d\,2^{-\lambda_{\mathbb{F}}}$
H^2_2	$k\,2^{-(3\lambda_{\mathbb{F}}+64)} + d\,2^{-\lambda_{\mathbb{F}}}$
H^3_2	$d\,2^{-\lambda_{\mathbb{F}}}$

(61)

The H_1 case includes both tree-summary levels and the commitment field in H^1_2, and the IV is fixed in that input, so the passive-prefix argument handles H_4. Since $N_{\mathsf{block}} \ge \lambda$ and $\lambda_{\mathbb{F}} \ge \lambda$, the maximum is at most $(\tau+1)k2^{-2\lambda} + d2^{-\lambda_{\mathbb{F}}}$. Conversion, transcript evaluation, and the local witness respectively require at most $\tau+6$, $\tau+6$, and $\tau+6$ additional queries, covered by $3\tau+19$. FAEST-v2 [22], Lemma 9.38, and Cauchy–Schwarz now give $10Q\sum_{k=1}^{Q}((\tau+1)k2^{-2\lambda} + d2^{-\lambda_{\mathbb{F}}})$, which is at most the claimed bound. □

For the QROM bounds below, write $e_m = \mathsf{AdvPRP}^{E}_{\mathsf{Q}}[\mathcal{R}; m]$ for the standard cipher PRP advantage of Equation (39), with m classical chosen-block queries under one hidden uniform N_{block}-bit key and quantum distinguishing computation. All primitive advantages use the full reduction resources $\mathcal{R}$, including public cipher evaluations, oracle simulations and the actual $\mathsf{iv} = \mathsf{H}_4(\mathsf{iv}^{\mathsf{pre}})$ history. Put $D = \lceil \log_2 L \rceil$, $h = \lceil \hat{\ell}/N_{\mathsf{block}} \rceil$ and

$$a = e_2 + 2^{-N_{\mathsf{block}}}, \qquad b = e_{h+2} + \binom{h+2}{2} 2^{-N_{\mathsf{block}}}, \qquad H = \tau\big((2D+1)a + b\big).$$

Assume admissible parameters with $L - 1 + \tau < 2^{31}$ and $h \le 2^{32}$.

Lemma 13 (VOLECommit is quantum-guess unpredictable). *After an arbitrary quantum history and the actual* H_4 *evaluation, generate a commitment from a fresh uniform root. Let* L_{g} *be a quantum register of lists of at most* R *commitment guesses prepared before the root is sampled, with all public-oracle queries bounded by* R*. Let* G_0 *continue with the full commitment and auxiliary information, and* G_1 *first measure* $\mathsf{com} \in L_{\mathsf{g}}$*, abort on a hit, and otherwise use the same continuation on the post-measurement state. Then, using the quantum-list experiment of FAEST v3 [48, Lemma 10.52],*

$$\begin{aligned} |\Pr[G_0 = 1] - \Pr[G_1 = 1]| &\le 2\sqrt{p} + p \le 3\sqrt{p}, \\ p &= \min\{1, (D+1)a + 3\sqrt{R}\,2^{-N_{\mathsf{block}}} + R2^{-2N_{\mathsf{block}}}\}. \end{aligned}$$

(62)

Proof. Bound only the membership event before the continuation receives the full commitment state. A hidden path and its two-block leaf cost $(D+1)a$; resampling the two containing H_1 outputs and consistently programming their inputs costs at most $3\sqrt{R}\,2^{-N_{\mathsf{block}}}$, and a uniform commitment hits a list of size R with probability at most $R2^{-2N_{\mathsf{block}}}$. These event-only reductions retain the actual IV and never need to disclose an unknown challenge key. The binary membership measurement and abort then cost $2\sqrt{p}+p$, by the gentle-measurement bound, including the entire continuation. □

Signing budgets. Let $s = q_{\mathsf{sig}}$ count target-user signing queries and q all external RO queries. Write ℓ_1, ℓ_2 for the first two challenge lengths and $1 \le T \le 2^{b_{\mathsf{ctr}}}$ for the maximum number of attempts with a b_{ctr}-bit counter. The algorithms use $\ell_1 = 5\lambda_{\mathbb{F}} + 64$, $\ell_2 = 3\lambda_{\mathbb{F}} + 64$, $b_{\mathsf{ctr}} = 32$, $T = 2^{32}$; the 64 bits belong to the universal-hash challenge encoding. Define

$$\begin{aligned} r_3 &= q + s, & R &= 2q + s(2\tau + T + 10) + \tau + 6, \\ K &= 2q + s(2\tau + 6) + \tau + 6, & Z &= K + 3\tau + 19, \\ h_3(r) &= 4\sqrt{2}(\sqrt{r} + r)2^{-\xi/2}, & C(Z) &= 10Z^3(2^{-2\lambda} + 2^{-\ell_1} + 2^{-\ell_2}). \end{aligned} \tag{63}$$

Here R bounds intermediate-game queries, K bounds the final simulator's queries, and Z includes Theorem 3's conversion allowance. The fixed constants count reduction operations; $\mathcal{R}$ includes all T attempts and table operations.

For independent fresh challenges, let A be the acceptance set determined by the decoded indices and parameters, $\alpha = |A|/2^{\lambda_{\mathbb{F}}}$, and J the first accepted counter, or $\perp$ on exhaustion. Then

$$\Pr[J = j] = (1-\alpha)^j\alpha \quad (0 \le j < T), \qquad \Pr[J = \perp] = (1-\alpha)^T. \tag{64}$$

Theorem 4 (Single-user EUF-CMA in the QROM). *Assume the hypotheses of Theorem 3 and use the cipher PRP games defined above. Model $\mathsf{H}_1, \mathsf{H}_2^0, \mathsf{H}_2^1, \mathsf{H}_2^2, \mathsf{H}_2^3, \mathsf{H}_3, \mathsf{H}_4$ as domain-separated QROs, using Equation (44) for H_3. Let $\mathcal{A}$ be a quantum polynomial-time adversary making at most s classical signing queries and at most q external RO queries, with independent uniform randomness used for each signature. Let $P = \mathsf{AdvPRG}_{\mathsf{Q}}^{\mathsf{OWF}}$ and $I = \mathsf{AdvInj}_{\mathsf{Q}}^{\mathsf{PRG}_{2N_{\mathsf{block}}}}[L]$ denote the corresponding primitive advantages at the full reduction resources, including final verification. Then*

$$\begin{aligned} \mathsf{AdvEUFCMA}^{\mathsf{Sig}}_{\mathcal{A},\mathsf{Q}}[s] &\le F(q,s) := \min\{1, F_0(q,s)\}, \\ F_0(q,s) &= 4\left(10(\tau+1)Z^3 2^{-2\lambda} + 10dZ^2 2^{-\lambda_{\mathbb{F}}} + P + I\right) + C(Z) \\ &\quad + s\left[h_3(r_3) + H + 6\sqrt{p} + 6\sqrt{R2^{-\ell_1}} + 6\sqrt{R2^{-\ell_2}}\right]. \end{aligned} \tag{65}$$

Proof. Replace the s signing-seed derivations at cost $sh_3(r_3)$, retaining the actual H_4 history. In analysis-only compressed-oracle games, the first programming step uses the membership bound $3\sqrt{p}$; the first- and second-challenge prefixes use $3\sqrt{R2^{-\ell_1}}$ and $3\sqrt{R2^{-\ell_2}}$. Sample each prefix before programming the preceding oracle; its good projector commutes with that programming. Presample the finite counter stream with the displayed stopping law and retain every rejected attempt. On exhaustion, use a fixed admissible opening index only for the hiding comparison, reveal no opening, and return $\perp$; an empty acceptance set always returns $\perp$. The joint hidden-leaf comparison costs H and the degree-two/three affine mask transformations used in the ROM proof also simulate these classical responses, including the retained patches. After hiding, remove the analysis-only tests in reverse signature order, and within each signature in the order third prefix, second prefix, first input. At each premeasurement point the root is fresh or the prefix is uniform; all later auxiliary output is part of the continuation channel. The same three gentle bounds apply again, giving the factors 6. The executable simulator keeps its own override table; a coherent lookup uses at most two calls to the prescribed original oracle and never reads its private database. Local counter sampling costs time and storage in $\mathcal{R}$, covered by R, but requires no original-oracle calls in this final simulator. Its query budget is K. A final new-message forgery first hitting a signing patch creates a message-digest collision or a hit on a privately sampled first- or second-challenge target. Adding these samples on private auxiliary domains, the same quantum collision bound used in

Theorem 3 charges their union by $C(Z)$; the intended copying into a patch is excluded. Outside this event original-RO and simulated verification agree. The virtual verifier belongs only to this comparison experiment; the KO experiment separately uses one original-RO verifier, within K. Apply Theorem 3 to the executable adversary with $Z = K + 3\tau + 19$ and sum the losses. The compressed databases are used only for analysis. □

Additional, unproved multi-user assumption. For U independently generated keys sharing the QROs, each with at most s classical signing queries, assume that every fixed target's unconditional probability of forgery on a message not submitted to its signing oracle is at most $F(q,s)+\varepsilon_{\mathsf{cross}}$, with $\varepsilon_{\mathsf{cross}} \geq 0$, uniformly over adaptive cross-user queries and interleavings within the stated resource range. The budget q counts all external adversarial RO queries, including superpositions across users. The assumption absorbs only the additional explicit contribution of other users' internal queries; all primitive advantages use uniform bounds at the full reduction resources including the entire multi-user environment. Summing over forgery targets gives

$$\mathsf{AdvMK\text{-}EUFCMA}_{\mathsf{Q}}(U) \leq \min\{1, U[F(q,s)+\varepsilon_{\mathsf{cross}}]\}.$$

For $U = 1$, take $\varepsilon_{\mathsf{cross}} = 0$ without this assumption. The instance $U = 2^{16}$, $s = 2^{64}$ has $Us = 2^{80}$; its loose-parameter bound equals one.

9.1.7 Conditional Query Bounds

We separate random-oracle losses from primitive advantages using Theorems 2 and 4.

Query limits and assumptions. Set $U = 2^{\mu_{\mathsf{mk}}}$, let $s = q_{\mathsf{sig}}$ be the per-key signing limit, and set $S = Us$. We use

$$(\mu_{\mathsf{mk}}, \log_2 s) = \begin{cases} (16, 64), & \text{for the lo variants of SM4th and SM4th-EM,} \\ (0, 80), & \text{for the other variants} \end{cases} \tag{66}$$

such that $S = 2^{80}$ for the budgets in Equation (66).

For the ROM calculation, q bounds the total oracle queries of each non-H_3 reduction, including all users' simulations and privately sampled or programmed challenge values, as well as every external H_3 query. Simulating the other signatures adds at most $S-1$ classical H_3 calls, giving a per-hybrid query bound of $q+S$. For the QROM calculation, q is the global external query budget of Theorem 4; its internal budgets are r_3, R, K, Z.

For $\mathsf{M} \in \{\mathsf{R}, \mathsf{Q}\}$, let $\mathsf{AdvPRG}^{\mathsf{OWF}}_{\mathsf{M}}$ denote the supremum of the advantage in Equation (46) over public-key distinguishers within the maximum reduction resource bounds. Write I_{R} and I_{Q} for the almost-injectivity terms in Theorems 1 and 3, respectively. Let $\eta_{\mathsf{prg},\mathsf{R}}$ denote the classical PRG loss in Equation (55). For the ROM only, define

$$\Delta_{\mathsf{R}} = 8U\left(\mathsf{AdvPRG}^{\mathsf{OWF}}_{\mathsf{R}} + I_{\mathsf{R}}\right) + 2S\eta_{\mathsf{prg},\mathsf{R}}. \tag{67}$$

In this ROM calculation, combining the lossy-key factor 4, $\rho \leq 2$, and the multi-key union bound gives $8U$, and the signing-hybrid coefficient is bounded by $2Us = 2S$.

ROM residual bound. In the ROM, define

$$\begin{aligned} B_{\mathsf{R}}(q) = 8U &\left(\frac{(q+4)^2}{2^{2\lambda}} + \frac{(q+5)^2}{2^{2N_{\mathsf{block}}}} + \frac{d(q+9)}{2^{\lambda_{\mathbb{F}}}}\right) \\ &+ 2S\left(\frac{q+S}{2^{\xi}} + \frac{3q}{2^{2N_{\mathsf{block}}}} + \frac{s+q}{2^{\ell_1}} + \frac{s+q}{2^{\ell_2}}\right) + 2UC_{\mathsf{R}}(q). \end{aligned} \tag{68}$$

The terms account for KO security, signing simulation and the final original-RO transfer, respectively. For $U > 1$, each target reduction simulates all other users with their keys, charging their complete resources to q and the primitive advantages; each target transfer is included in $2UC_{\mathsf{R}}(q)$. Equations (37) and (56) give

$$\mathsf{AdvMK\text{-}EUFCMA}_{\mathsf{R}} \leq \min\{1, B_{\mathsf{R}}(q) + \Delta_{\mathsf{R}}\}. \tag{69}$$

Table 16: Explicit ROM query budgets with $B_{\mathsf{R}}(q) \le 1/4$. The lo rows use $U = 1$, $s = S = 2^{64}$ and have a trivial complete bound. The other rows use Equation (66) and require $\Delta_{\mathsf{R}} \le 1/4$

Scheme	Explicit $\log_2 q$	Dominant explicit term
SM4th-d3-128s-lo	121.4	Field term
SM4th-d3-128f-lo	121.4	Field term
SM4th-EM-d2-128s-lo	121.9	Field term
SM4th-EM-d2-128f-lo	121.9	Field term
SM4th-d3-128s-ti	125.4	RO chain
SM4th-d3-128f-ti	125.4	RO chain
SM4th-EM-d2-128s-ti	125.4	RO chain
SM4th-EM-d2-128f-ti	125.4	RO chain
uBlockith-d3-256s	249.4	Field term
uBlockith-d3-256f	249.4	Field term
uBlockith-EM-d3-256s	249.4	Field term
uBlockith-EM-d3-256f	249.4	Field term
Vistrutith-d3-512s	505.4	Field term
Vistrutith-d3-512f	505.4	Field term

QROM residual bound. Use r_3, R, Z from Theorem 4, and split

$$u_0 = (D+1)2^{-N_{\mathsf{block}}} + 3\sqrt{R}\,2^{-N_{\mathsf{block}}} + R2^{-2N_{\mathsf{block}}}, \quad u_P = (D+1)e_2,$$
$$H_0 = \tau\left(2D+1+\binom{h+2}{2}\right)2^{-N_{\mathsf{block}}}, \qquad H_P = \tau\big((2D+1)e_2 + e_{h+2}\big).$$

Since $\sqrt{p} \le \sqrt{u_0} + \sqrt{u_P}$, define the explicit loss and the primitive residual by

$$\begin{aligned} B_{\mathsf{Q}}(q) &= U\Big[40(\tau+1)Z^3 2^{-2\lambda} + 40dZ^2 2^{-\lambda_{\mathbb{F}}} + C(Z) \\ &\qquad + s\big(h_3(q+s) + H_0 + 6\sqrt{u_0} + 6\sqrt{R2^{-\ell_1}} + 6\sqrt{R2^{-\ell_2}}\big)\Big], \\ \Delta_{\mathsf{Q}} &= U\left[4(P+I) + s\left(H_P + 6\sqrt{u_P}\right)\right]. \end{aligned} \tag{70}$$

All primitive advantages, including those under the square root, use the full reduction resources. Under the additional multi-user assumption,

$$\mathsf{AdvMK\text{-}EUFCMA}_{\mathsf{Q}} \le \min\{1, B_{\mathsf{Q}}(q) + \Delta_{\mathsf{Q}} + U\varepsilon_{\mathsf{cross}}\}. \tag{71}$$

Setting $\varepsilon_{\mathsf{cross}} = 0$ for $U > 1$ is a stronger, unproved assumption.

A concrete conditional target. Table 16 reports $\log_2 q$ rounded down to one decimal place with $B_{\mathsf{R}}(q) \le 1/4$. A ROM success bound of $1/2$ additionally requires $\Delta_{\mathsf{R}} \le 1/4$ at the complete resources. For the lo rows, this condition fails: with $e_2 = \mathsf{AdvPRP}^E[\mathcal{R}; 2]$, testing $s-1$ candidate keys fits within the complete signing-simulation resources and gives $e_2 \ge (s-1)(2^{-N_{\mathsf{block}}} - [2^{N_{\mathsf{block}}}(2^{N_{\mathsf{block}}} - 1)]^{-1})$, and already $\Delta_{\mathsf{R}} \ge 4s\tau(D+1)e_2 > 1$. Their complete bound is therefore 1; their table entries record only the explicit-loss arithmetic. The other rows remain conditional on the stated residual budget.

9.1.8 Reference Attack-Cost Models

We estimate the circuit costs of exhaustive key search and depth-limited Grover search.

Classical reference attack costs. The NIST post-quantum standardization call evaluates concrete security using the gate complexity of classical circuits. In the classical model, NIST's reference estimates assign exhaustive key recovery against λ-bit-key AES a cost of $2^{\lambda} \cdot 2^{C_{\mathsf{AES}}}$, where C_{AES} is the base-2 logarithm of the classical gate cost of one AES evaluation in NIST's metric. From NIST's estimates, $C_{\mathsf{AES}} \approx 15$ for AES-128 and AES-192, and $C_{\mathsf{AES}} \approx 16$ for AES-256.

For these search estimates, we use the AES-128 reference normalization $C_{\mathrm{SM4}} = C_{\mathrm{Ballet}} = C_{\mathrm{uBlock}} = C_{\mathrm{Vistrutah}} = 15$.

Quantum reference attack costs. NIST [39] estimates the depth-limited AES-128 key-recovery cost as $2^{170}/\mathsf{MAXDEPTH}$ quantum gates, based on the circuit estimates of Grassl *et al.* [40]. Here, $\mathsf{MAXDEPTH}$ bounds the overall logical circuit depth such that NIST considers 2^{40}, 2^{64}, and 2^{96}. Sarah and Peter [49] additionally suggest considering maximum depths of 2^{48} and 2^{56} under a 200 ns cycle-time assumption.

Let G_{ser} and D_{ser} denote the gate count and overall logical depth of a serial Grover key search. For $D_{\mathrm{ser}} > \mathsf{MAXDEPTH}$, reducing the depth by $x = D_{\mathrm{ser}}/\mathsf{MAXDEPTH}$ uses approximately x^2 parallel instances, each with G_{ser}/x gates. The total gate count scales as

$$W^{\mathsf{Q}}_{\text{AES-128}}(\mathsf{MAXDEPTH}) \approx \frac{G_{\mathrm{ser}} D_{\mathrm{ser}}}{\mathsf{MAXDEPTH}}.$$

For SM4, we use the Toffoli-depth and width estimates of Lin *et al.* [50] to estimate the relative quantum gate count and normalize it to the NIST AES-128 reference.

Lin *et al.* report that their lowest-depth SM4 implementation, corresponding to 8 parallel S-boxes, has Toffoli depth $\Delta_{\mathrm{SM4}} = 2244$ and width $Q_{\mathrm{SM4}} = 368$. For the AES-128 implementation of Grassl *et al.* used in the same table, they report Toffoli depth $\Delta_{\mathrm{AES}} = 12672$ and width $Q_{\mathrm{AES}} = 984$. In Figure 7 of Lin *et al.* [50], two encryption circuits run in parallel to check two plaintext–ciphertext pairs and are then uncomputed in parallel. Following their depth approximation in Equation (10) and qubit count in Equation (11), we use $D_{\mathrm{iter}} \approx 2\Delta$ and $W_{\mathrm{iter}} = 2Q + 1$, where Δ and Q denote the Toffoli depth and width of one encryption circuit. Hence, we have $D_{\mathrm{SM4}} = 4488$, $W_{\mathrm{SM4}} = 737$, $D_{\mathrm{AES}} = 25344$, and $W_{\mathrm{AES}} = 1969$.

At a common Toffoli-depth limit, the logical qubit–Toffoli-layer volume of parallel Grover search is proportional to $R^2 D_{\mathrm{iter}}^2 W_{\mathrm{iter}}$, where $R = (\pi/4)2^{64}$ for 128-bit key search. The common key size and depth limit cancel in the relative comparison, giving the logarithmic ratio

$$\Delta_{\mathrm{SM4/AES}} = \log_2\left(\frac{D_{\mathrm{SM4}}^2 W_{\mathrm{SM4}}}{D_{\mathrm{AES}}^2 W_{\mathrm{AES}}}\right) = \log_2\left(\frac{4488^2 \cdot 737}{25344^2 \cdot 1969}\right) \approx -6.41.$$

Normalizing this ratio to NIST's $2^{170}/\mathsf{MAXDEPTH}$ reference gives

$$W^{\mathsf{Q}}_{\mathrm{SM4}} \approx \frac{2^{170-6.41}}{\mathsf{MAXDEPTH}} = \frac{2^{163.59}}{\mathsf{MAXDEPTH}}.$$

For $\mathsf{MAXDEPTH} = 2^{56}$, which corresponds to the number of 200ns cycles that can be completed in roughly 500 years [49], this gives an estimated quantum gate count of approximately $2^{107.59}$, or roughly 2^{108}. Under this normalization, the estimated quantum gate count of this reference attack exceeds 2^{80}.

The quantum search costs for uBlock and Vistrutah are expressed symbolically in Table 17.

Reference cost of quantum bad-challenge search. Consider a search for an input transcript or randomness value whose Fiat–Shamir challenge, generated by SM3, satisfies a target event. In this search model, set $p = d \cdot 2^{-128}$ for $d = 2$ or 3, and use $p_0 = 2^{-126} \geq p$ as a conservative reference probability. We are not aware of any better attacks than Grover's search in the quantum model. For $p = d2^{-128}$, the Grover iteration estimate is $R(p) = (\pi/4)2^{64}/\sqrt{d}$. Using the reference probability p_0 gives

$$R_0 = \frac{\pi}{4}\frac{1}{\sqrt{p_0}} = \frac{\pi}{4} \cdot 2^{63} \approx 2^{62.651}$$

iterations. For the SM3 circuit, we use the low-T-depth implementation of [38], which reports

$$D_{T,\mathrm{SM3}} = 4528, \qquad W_{\mathrm{SM3}} = 1369, \qquad T_{\mathrm{SM3}} = 977920.$$

Here $D_{T,\mathrm{SM3}}$ is the T-depth, and W_{SM3} is the number of logical qubits. It gives a total T-count $T_{\mathrm{SM3}} = 977920$, obtained by adding the 965632 T-gates from the adders and the 12288 T-gates from the Boolean components.

Using a very coarse estimate, we charge one SM3 evaluation per Grover iteration and set $D_{T,\mathrm{iter}} = D_{T,\mathrm{SM3}}$, $W_{\mathrm{iter}} = W_{\mathrm{SM3}}$, and $T_{\mathrm{iter}} = T_{\mathrm{SM3}}$. This ignores the extra cost of checking the challenge predicate, diffusion, and possible uncomputation, and therefore gives a synchronized baseline estimate in terms of the reported SM3 circuit itself. Thus $D_{T,\mathrm{iter}} = 4528 \approx 2^{12.145}$, $W_{\mathrm{iter}} = 1369 \approx 2^{10.419}$, and $T_{\mathrm{iter}} = 977920 \approx 2^{19.899}$.

Under a maximum T-depth bound $D_{\max}$, the logical qubit–T-layer volume for this reference search is

$$C_{\mathrm{FS}}(D_{\max}) = \frac{R_0^2 D_{T,\mathrm{iter}}^2 W_{\mathrm{iter}}}{D_{\max}} \approx \frac{2^{160.011}}{D_{\max}},$$

as long as $D_{\max} < R_0 \cdot D_{T,\mathrm{iter}} \approx 2^{74.796}$. For $D_{\max} = 2^{56}$, this gives

$$C_{\mathrm{FS}}(2^{56}) \approx 2^{104.011}.$$

The exponent 104.011 uses p_0; the bad-challenge branch in Table 18 uses $p = d2^{-\lambda_{\mathbb{F}}}$, giving 104.4 and 105.0 for $\lambda_{\mathbb{F}} = 128$ with $d = 3$ and 2, respectively.

9.2 Practical Security against Known Attacks

Sections 9.1.7 and 9.1.7 give conditional ROM and QROM bounds under their respective resource and primitive assumptions. This section describes selected known attack strategies under the stated cost assumptions. Their estimated costs are compared with the ICCS targets.

9.2.1 Exhaustive and quantum search for an OWF preimage

The simplest attack strategy for VOLEitH-based signatures is to invert the OWF value contained in the public key and thereby recover a valid secret key. For our schemes, this corresponds to a key-recovery attack on the underlying block-cipher-based OWF. In the standard variants, the public key contains values of $E_{\boldsymbol{k}}(\boldsymbol{x})$ for fixed public inputs $\boldsymbol{x}$, so the attack is essentially exhaustive search over the block-cipher key $\boldsymbol{k}$. In the EM variants, the public key contains values of $E_{\boldsymbol{x}}(\boldsymbol{k}) \oplus \boldsymbol{k}$, and the attack similarly amounts to finding a preimage key $\boldsymbol{k}$ for this public OWF output.

There are two minor differences from ordinary block-cipher key recovery. First, our key-generation algorithm rejects keys whose first two bits are both 1, so the effective secret-key space has size $3 \cdot 2^{\lambda-2}$, which causes a loss of only $\log_2(4/3) \approx 0.42$ bits. Second, because the public key contains only a small number of block-cipher output blocks, there may exist collisions, so an attacker may find a different key k' that is still a valid preimage of the OWF. For the reference search scenarios below, we use illustrative cost deductions of 2 bits for the non-EM variants and 3 bits for the EM variants.

Based on the above two differences, for the non-EM variants, we estimate the costs of exhaustive OWF-preimage search to be about $2^{\lambda-1-2-0.42}$ block-cipher evaluations, while for the EM variants we estimate it as about $2^{\lambda-1-3-0.42}$ evaluations. Using these heuristic allowances and the reference cost models in Section 9.1.8, we obtain Table 17. For the quantum estimate, we use the corresponding search and circuit costs. For SM4th variants, Section 9.1.8 gives a dedicated depth-limited Grover estimate for full 128-bit SM4 key recovery. Under the bound $\mathsf{MAXDEPTH} = 2^{56}$, the estimated quantum gate count is approximately $2^{107.59}$. For the SM4th public-key search estimates, we apply the same illustrative deductions and the $\log_2(4/3)$-bit keyspace adjustment to this baseline. This gives $2^{107.59-2-\log_2(4/3)}$ for the non-EM SM4th variants and $2^{107.59-3-\log_2(4/3)}$ for the EM variants.

For uBlockith and Vistrutith, we are not aware of comparable depth-limited quantum circuit estimates for uBlock or Vistrutah. We therefore do not assign a precise gate-level constant to one coherent OWF evaluation. Let C_E^{Q} denote the base-2 gate-cost exponent of one reversible OWF evaluation in the round-by-round NCT reference implementation using the S-box circuits cited below, including coherent checking and uncomputation. If the effective classical search space has exponent s, then generic Grover search requires about $2^{s/2}$ OWF evaluations, and hence has gate cost $2^{s/2+C_E^{\mathsf{Q}}}$. Here $s = \lambda - 3 - \log_2(4/3)$ for the non-EM variants and $s = \lambda - 4 - \log_2(4/3)$ for the EM variants. The table therefore keeps the uBlockith and Vistrutith quantum gate counts symbolic.

Table 17: Reference costs of classical and quantum OWF-preimage search, where the SM4th and SM4th-EM quantum entries use NIST-normalized quantum gate count estimates, and the remaining quantum entries are reversible-circuit gate counts

Scheme	Classical gate count	Quantum gate count
SM4th-d3-128s/f-lo/ti	$2^{139.6}$	$2^{105.2}$
SM4th-EM-d2-128s/f-lo/ti	$2^{138.6}$	$2^{104.2}$
uBlockith-d3-256s/f	$2^{267.6}$	$2^{126.3+C^{\mathsf{Q}}_{\mathrm{uBlock}}} > 2^{128}$
uBlockith-EM-d3-256s/f	$2^{266.6}$	$2^{125.8+C^{\mathsf{Q}}_{\mathrm{uBlock}}} > 2^{128}$
Vistrutith-d3-512s/f	$2^{523.6}$	$2^{254.3+C^{\mathsf{Q}}_{\mathrm{Vistrutah}}} > 2^{256}$

For uBlock, the SAT-based synthesis of Chen et al. [51] gives an optimal in-place quantum circuit for its 4-bit S-box using eight gates from the NCT gate set. Since $2^{2.3} < 8$, even one such S-box evaluation already has a gate-cost exponent greater than 2.3, while a complete uBlock OWF evaluation contains many S-box evaluations and additionally requires the linear layers, coherent output checking, and uncomputation. It therefore follows under this standard reversible-circuit accounting that $C^{\mathsf{Q}}_{\mathrm{uBlock}} > 2.3$. For Vistrutah, its 512-bit long variant consists of 18 AES-like rounds over four 128-bit branches and hence invokes $18 \cdot 64 = 1152$ AES S-boxes [6]. The in-place AES S-box circuit reported by Chen et al. [52] uses 92 Toffoli gates, 268 CNOT gates, and 4 NOT gates. Thus, under the corresponding NCT reference-circuit accounting, even one AES S-box evaluation costs far more than $2^{1.8}$ gates, and a complete Vistrutah OWF evaluation, including coherent checking and uncomputation, satisfies $C^{\mathsf{Q}}_{\mathrm{Vistrutah}} > 1.8$. With these reference implementations, the estimated Grover-search costs for uBlockith-EM and Vistrutith exceed 2^{128} and 2^{256}, respectively.

Under the stated search and circuit assumptions, the attacks in Table 17 have estimated costs above the corresponding ICCS targets.

9.2.2 Brute Force the PRG and Multi-Key Attacks

Another possible attack strategy is to recover the signing key by attacking the PRG calls used inside BAVC.Commit, VOLECommit, and the conversion procedures that derive VOLE-related randomness. These PRGs use SM4, Ballet-256/256, or Vistrutah-512, with a per-signature IV and domain-separated tweaks. Given one signature, an attacker may try to guess an unrevealed PRG seed and test whether the expanded output is consistent with the public transcript and the revealed openings. If the same $(\mathsf{iv}, \mathsf{twk})$ pair were reused across many PRG calls, this could lead to a multi-target attack, where one guessed seed is tested against many possible targets. Such attacks can sometimes reduce the cost compared with attacking each PRG output independently, and are a form of time-space tradeoff attack.

First, consider the length-doubling tweakable PRGs used in the tree-based vector commitment BAVC.Commit. A signature hides the τ selected leaf seeds and reveals their commitments together with the root seeds of the maximal subtrees containing no selected leaf. Given a guess g for an unrevealed node seed, the attacker can expand it using the corresponding tweaks and compare the derived subtree-root seeds or leaf commitments with the corresponding opening values. A correct guess recovers the hidden leaf seeds in its subtree. Second, PRG calls are also used when deriving the VOLE-related masks. There, all but one of the relevant seeds may be exposed, so an attacker could again guess the missing seed, expand it, and test whether the resulting mask is consistent with the transcript or with a candidate signing key. However, these two kinds of PRG calls use different tweaks, and different locations in the commitment tree or VOLE generation also use domain-separated tweaks. For a fixed IV, these tweaks separate the cipher-input ranges of the different roles. Equation (50) bounds the joint hiding advantage using resource-bounded PRP terms.

9.2.3 Attack Soundness of the ZK Protocol

Instead of recovering the secret key from the public key or from PRG seeds, an attacker may try to forge a signature by violating the soundness of the underlying ZKP. In this attack, the adversary uses an invalid witness for the public statement and attempts to produce an accepting Fiat–Shamir transcript. This type of attack is captured by the EUF-KO bound in Theorem 1.

We compare reference search subtasks associated with the explicit random-oracle losses in Equation (52):

$$\frac{(Q_{2,0}+Q_{2,1}+Q_{2,2}+Q_{2,3}+4)^2}{2^{2\lambda}} \quad \text{and} \quad \frac{(Q_1+Q_2+5)^2}{2^{2N_{\mathsf{block}}}}.$$

Their output widths are 2λ and $2N_{\mathsf{block}}$, respectively. Classical generic collision searches at those widths take order 2^{λ} and 2^{N}_{block} queries. The third term in Equation (52),

$$\frac{(Q_{2,1}+Q_{2,2}+Q_{2,3}+9)d}{2^{\lambda_{\mathbb{F}}}},$$

comes from the round-by-round soundness of the underlying proof system over the field $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. Intuitively, after the adversary fixes a false transcript, it can only pass the verifier if a Fiat–Shamir challenge falls into a small bad set. For a degree-d consistency check, this bad set has size at most a $d/2^{\lambda_{\mathbb{F}}}$ fraction of all possible challenges. A classical exhaustive search over challenges for this specified bad event requires about $2^{\lambda_{\mathbb{F}}}/d$ trials. All variants use PRG-based leaf commitments and retain the almost-injectivity advantage as a separate term.

The following reference comparison considers two generic search subtasks. The first branch searches for collisions or bad preimages in random-oracle outputs of length at least 2λ. The second branch searches for a bad Fiat–Shamir challenge for the degree-d consistency check over $\mathbb{F}_{2^{\lambda_{\mathbb{F}}}}$. In the ROM, the first branch requires about 2^{λ} classical random-oracle evaluations, while the second branch requires about $2^{\lambda_{\mathbb{F}}}/d$ evaluations. Let $C^{\mathsf{cl}}_{\mathsf{H}}$ denote the base-2 logarithm of the classical gate cost of one SM3-based random-oracle evaluation. Following the NIST normalization for SHA3-256 collision search [39], we use $C^{\mathsf{cl}}_{\mathsf{H}} = 18$. Thus, the ROM cost estimate is $G_{\mathsf{ROM}}(\lambda, \lambda_{\mathbb{F}}, d) = \min\{2^{\lambda+C^{\mathsf{cl}}_{\mathsf{H}}}, 2^{\lambda_{\mathbb{F}}-\log_2 d+C^{\mathsf{cl}}_{\mathsf{H}}}\}$. In the QROM, according to the Brassard–Hoyer–Tapp quantum collision-finding algorithm [53], the random-oracle collision branch is estimated by the generic quantum collision-search complexity $2^{2\lambda/3}$ for a 2λ-bit output. Let $C^{\mathsf{q}}_{\mathsf{H}}$ denote the base-2 logarithm of the cost of one quantum-accessible SM3 oracle evaluation. Using the SM3 quantum circuit of [38], which has $T_{\mathrm{SM3}} = 977920$, we set $C^{\mathsf{q}}_{\mathsf{H}} = \log_2(977920) \approx 19.90$. For the bad-challenge branch, we use the depth-limited parallel Grover model in Section 9.1.8. With $D_{T,\mathrm{SM3}} = 4528$, $W_{\mathrm{SM3}} = 1369$, and $\mathsf{MAXDEPTH} = 2^{56}$, the base-2 cost exponent is $\lambda_{\mathbb{F}} - \log_2 d + 2\log_2(\pi/4) + 2\log_2(4528) + \log_2(1369) - 56 = \lambda_{\mathbb{F}} - \log_2 d + 34.012 - 56$. Therefore, the QROM cost estimate is $G_{\mathsf{QROM}}(\lambda, \lambda_{\mathbb{F}}, d) = \min\{2^{2\lambda/3+C^{\mathsf{q}}_{\mathsf{H}}}, 2^{\lambda_{\mathbb{F}}-\log_2 d+34.012-56}\}$[6].

Table 18 summarizes the resulting estimates. For the SM4th-ti variants, the larger field size increases the bad-challenge branch, but the collision branch still depends on the 2λ-bit random-oracle output with $\lambda = 128$, and therefore dominates the table entry.

Under the stated search and circuit assumptions, the attacks in Table 18 have estimated costs above the corresponding ICCS targets.

10 Performance Evaluation

10.1 Performance Analysis

This section gives a performance analysis of communication costs for our signature schemes. Recall that in Section 3 we describe the main parameter param of our signature schemes, which include

$$\mathsf{param} = (\lambda, \lambda_{\mathsf{Q}}, \lambda_{\mathbb{F}}, \tau, w_{\mathsf{grind}}, T_{\mathsf{open}}, d, B, \mathrm{OWF}, \ell, \lambda_{\mathsf{iv}}, N_{\mathsf{block}}).$$

[6] The two branches use different resource metrics. Their minimum is reported as a heuristic reference.

Table 18: Estimated ROM and QROM gate-level costs of attacking ZK soundness

Scheme	ROM gate-level cost	QROM gate-level cost
SM4th-d3-128s/f-lo	$G_{\mathsf{ROM}}(128,128,3)=2^{144.4}$	$G_{\mathsf{QROM}}(128,128,3)=2^{104.4}$
SM4th-EM-d2-128s/f-lo	$G_{\mathsf{ROM}}(128,128,2)=2^{145.0}$	$G_{\mathsf{QROM}}(128,128,2)=2^{105.0}$
SM4th-d3-128s/f-ti	$G_{\mathsf{ROM}}(128,160,3)=2^{146.0}$	$G_{\mathsf{QROM}}(128,160,3)=2^{105.2}$
SM4th-EM-d2-128s/f-ti	$G_{\mathsf{ROM}}(128,160,2)=2^{146.0}$	$G_{\mathsf{QROM}}(128,160,2)=2^{105.2}$
uBlockith-d3-256s/f	$G_{\mathsf{ROM}}(256,256,3)=2^{272.4}$	$G_{\mathsf{QROM}}(256,256,3)=2^{190.6}$
uBlockith-EM-d3-256s/f	$G_{\mathsf{ROM}}(256,256,3)=2^{272.4}$	$G_{\mathsf{QROM}}(256,256,3)=2^{190.6}$
Vistrutith-d3-512s/f	$G_{\mathsf{ROM}}(512,512,3)=2^{528.4}$	$G_{\mathsf{QROM}}(512,512,3)=2^{361.2}$

Table 19: Transmission sizes for all parameter sets

Scheme	Sizes in Bytes		
	sk	pk	signature
SM4th-d3-128s-lo	32	32	5 056
SM4th-d3-128f-lo	32	32	6 724
SM4th-EM-d2-128s-lo	32	32	3 818
SM4th-EM-d2-128f-lo	32	32	4 932
SM4th-d3-128s-ti	32	32	9 176
SM4th-d3-128f-ti	32	32	11 864
SM4th-EM-d2-128s-ti	32	32	7 556
SM4th-EM-d2-128f-ti	32	32	9 450
uBlockith-d3-256s	64	64	24 144
uBlockith-d3-256f	64	64	31 556
uBlockith-EM-d3-256s	64	64	19 056
uBlockith-EM-d3-256f	64	64	25 028
Vistrutith-d3-512s	128	128	83 004
Vistrutith-d3-512f	128	128	106 788

The main parameter can determine the public key size, the secret key size, and the signature size (in bits) as below. The size of the public key is

$$\underbrace{\lambda}_{\text{public input size of OWFs}} + \underbrace{\lambda}_{\text{public output size of OWFs}}.$$

The size of the secret key is

$$\underbrace{\lambda}_{\text{public input size of OWFs}} + \underbrace{\lambda}_{\text{secret key size of OWFs}}.$$

As for the signature size, it is

$$\underbrace{\tau\cdot(\ell+(d-1)\lambda_{\mathbb{F}}+\lambda_{\mathbb{F}}+B)}_{\substack{\text{witness/mask commitment}\\ +\text{ responses to VOLE/ZK checks}}} + \underbrace{T_{\mathsf{open}}\cdot N_{\mathsf{block}}+2N_{\mathsf{block}}\tau}_{\text{GGM tree opening}} + \underbrace{\lambda_{\mathbb{F}}}_{\Delta\text{ challenge}} + \underbrace{\lambda_{\mathsf{iv}}}_{\mathsf{iv}^{\mathsf{pre}}} + \underbrace{32}_{\text{counter}}.$$

Table 19 reports the communication sizes for all signature schemes.

10.2 Detailed Configuration of Implementation Platforms

We provide two implementations, as required by NGCC of ICCS. The first is a reference implementation in standard C, designed for portability and clarity. The optimized implementation uses

Table 20: Main acceleration instructions used by the optimized implementations

Component	Intel without native SM4	Intel with native SM4
SM4 OWF	AES-NI, SSSE3	AVX2, VSM4KEY4, VSM4RNDS4
SM4 PRG (lo)	AES-NI, SSSE3, AVX2, AVX-512F/VL	AVX2, VSM4KEY4, VSM4RNDS4
Ballet PRG (ti)	AVX2, AVX-512F/VL	AVX2
Ballet PRG (uBlockith, uBlockith-EM)	AVX2	—
Vistrutah PRG (Vistrutith)	AES-NI	—
Field arithmetic	PCLMULQDQ	PCLMULQDQ

Table 21: Implementation components

Protocol	Components
SM4th-lo, SM4th-EM-lo	SM4; SM4-NI; native Intel SM4 backend; shared FAEST-style protocol components
SM4th-ti, SM4th-EM-ti	SM4; Ballet; shared FAEST-style protocol components
uBlockith, uBlockith-EM	uBlock; Ballet; shared FAEST-style protocol components
Vistrutith	Vistrutah; shared FAEST-style protocol components

architecture-specific instructions on x86-64 processors. These implementation choices preserve the protocol and signature encoding.

We use two Intel platforms: a Xeon 6982P-C system (up to 3.9 GHz) under KVM without native SM4 and a Xeon 6992E+C system (up to 3.2 GHz) with native SM4. We refer to them as Intel without native SM4 and Intel with native SM4, respectively. The former runs Linux 6.12.57 with GCC 14.2.0, while the latter runs Linux 6.12.0 with GCC 14.4.1. Each benchmark process is pinned to one logical CPU, with the operating system's frequency scaling retained.

Instruction sets. Table 20 reports the ISA used in the optimized implementations of our signature schemes. The optimized implementations select SIMD kernels according to the available instruction sets. For SM4th and SM4th-EM, SM4 evaluation uses either AES-NI-based kernels or native Intel SM4 instructions. Vistrutith additionally leverages AES-NI to accelerate AES-based Vistrutah. PCLMULQDQ is employed whenever carry-less finite-field arithmetic is required.

On Intel without native SM4, shared key setup for the loose variants also selects an AVX-512F/VL/BW specialization; single-key PRG expansion uses the baseline AES-NI/SSSE3 kernel.

Crypto libraries. Table 21 summarizes the open-source libraries and reference code sources used in our implementations. For the OWF in SM4th, the implementation uses SM4-related reference code by OpenSSL[7]. For the PRGs in the lo variants of SM4th and SM4th-EM, we use SM4-NI-based kernels[8] or a native Intel SM4 backend. The ti variants of SM4th and SM4th-EM, together with uBlockith and uBlockith-EM, use Ballet-based PRGs.

For OWFs in uBlockith, we use components from uBlock[9]. For Vistrutith, the implementation is based on the Vistrutah reference code in C[10]. All implementations also share FAEST-style protocol components for the common MPC-in-the-head and VOLE-based building blocks[11].

[7] https://github.com/openssl/openssl/blob/master/crypto/sm4/sm4.c
[8] https://github.com/mjosaarinen/sm4ni
[9] Ballet and uBlock both at https://sfjs.cacrnet.org.cn/site/term/list_76_1.html
[10] https://github.com/jedisct1/vistrutah
[11] https://github.com/faest-sign/faest-ref

10.3 Acceleration Techniques

10.3.1 Acceleration Techniques of PRGs

We accelerate PRGs using ISA-specific kernels and batching tailored to their output lengths.

General techniques for specific PRG modes. We make the following two observations: (1) PRGs in BAVC.LeafCommit and ConvertToVOLE share the same keys, so their key schedule routines only need to run once. (2) In ConvertToVOLE, there are τ sub-GGM trees. For a fixed sub-tree with index i, PRGs are evaluated in parallel for different keys but the tweak/counter base derived from i is shared. This creates a fixed-tweak, multiple-keys workload, *e.g.*., $\mathrm{PRG}(k_0, \mathsf{iv}, \mathsf{twk}_i), \ldots, \mathrm{PRG}(k_{m-1}, \mathsf{iv}, \mathsf{twk}_i)$. To achieve acceleration in the second case, we implement a fixed-tweak batched PRG core with variable output length. The key idea is to separate key-dependent work from tweak-dependent work: for a fixed index i, the counter base derived from $\mathsf{iv} + \mathsf{twk}_i$ is prepared once and reused across all lanes, while each lane derives its own round-key or prepared-key state only once and then reuses it across all counter blocks required by that PRG instance.

PRG design in SM4th and SM4th-EM. The lo variants use SM4 in counter mode, with either AES-NI-based kernels or native Intel SM4 instructions. We specialize the kernels for 32-byte seed expansions and VOLE streams of 276 bytes for SM4th and 162 bytes for SM4th-EM.

AES-NI backend. We adapt the four-block SM4-NI kernel to expand four independent keys and encrypt one block under each key in parallel. For seed expansion, two calls to the encryption kernel produce the two counter blocks required by each PRG instance. The four expanded key schedules are reused for both calls. Single-key expansion and incomplete batches use the AES-NI key-schedule kernel as well, with the portable key schedule retained when AES-NI or SSSE3 is unavailable.

For a 276-byte stream, we use a 16-block same-key kernel[12] followed by a four-block tail kernel and retain the first 20 bytes of the tail. For a 162-byte stream, we reuse four packed key schedules for both the two-block seed output and the long stream, processing each counter under four keys and truncating the final block.

Native Intel SM4 backend. We use `VSM4KEY4` for SM4 key expansion and `VSM4RNDS4` for encryption. Their 256-bit forms operate on two independent 128-bit lanes in parallel. We expand four independent keys in two packed pairs and retain the round keys in this layout. For seed expansion, each expanded key encrypts two counter blocks. For fixed-tweak leaf and VOLE generation, four-key kernels process two or six counter blocks per key, reusing the expanded keys across the short and long outputs. The final counter output is truncated to the required stream length.

Ballet-based PRGs. The ti variants of SM4th and SM4th-EM, together with uBlockith and uBlockith-EM, use a Ballet-256/256 [5] PRG. We pack four independent key schedules into AVX2 lanes and use the same layout for batched counter-mode encryption. The expanded keys are reused across all blocks of each output stream.

PRG design in Vistrutith. Vistrutith uses Vistrutah-CTR-style PRG with security level $\lambda = 512$ and 64-byte counter blocks. Vistrutah uses AES-based rounds as building blocks, so we use AES-NI for acceleration. For the 2λ-bit PRG, we prepare the Vistrutah key once and implement a dedicated 2-block encryption helper for counter and counter+1. For $\hat{\ell}$-bit outputs, we use the same fixed-tweak batched PRG core with variable output length.

10.3.2 Acceleration Techniques of Field Operations

Basic field operations. All optimized implementations use the x86 `PCLMULQDQ` instruction for carry-less multiplication over $\mathbb{F}_{2^n}$, with field elements stored in 64-bit limbs. For $\mathbb{F}_{2^{160}}$ in the ti variants of SM4th and SM4th-EM, field elements occupy three 64-bit limbs, with the upper 32 bits of the most significant limb set to zero. The unreduced 320-bit product is reduced modulo the pentanomial $f_{160}(z) = z^{160} + z^5 + z^3 + z^2 + 1$. For $\mathbb{F}_{2^{256}}$ in uBlockith, the optimized code uses a 2-way Karatsuba decomposition [54] to lower the number of 128-bit carry-less subproducts versus

[12] https://git.fh-aachen.de/tb3838s/linux/-/blob/c291c9cfd76a8fb92ef3d66567e507009236ce90/arch/x86/crypto/sm4-aesni-avx2-asm_64.S

the naive split-product formulation. For $\mathbb{F}_{2^{512}}$ in Vistrutith, we represent elements as eight 64-bit limbs and use `PCLMULQDQ` with recursive 2-way Karatsuba, followed by tailored reduction modulo $m(t) = t^{512} + t^8 + t^5 + t^2 + 1$.

Constraints in SM4th. We describe several acceleration techniques in SM4th.

Precomputing the embedding from $\mathbb{F}_{2^8}$ to $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$. In the SM4th variants, the native SM4 byte operations are defined over $\mathbb{F}_{2^8}$, whereas the ZKP constraints are evaluated over the larger field $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$. Our implementation embeds an SM4 byte $b = \sum_{i=0}^{7} b_i \alpha^i$ into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ using precomputed powers of α. We further precompute the embeddings of the fixed public byte-field constants and their Frobenius conjugates. As a result, byte reconstruction, byte squaring, and norm-basis expansion become fixed linear combinations over $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$, instead of repeatedly recomputing byte-field constants and their embeddings. In the optimized implementation, these linear combinations and the resulting $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ multiplications are accelerated with `PCLMULQDQ` and fused carry-less product accumulation.

Reducing repeated embeddings and field reductions. For SM4th-d3, we specialize the SM4 norm path for the linearized maps used by the byte-level affine layers. For such an affine map M, write $M(z) = c + \sum_{i=0}^{7} a_i z^{2^i}$ over the SM4 byte field, where $z \in \mathbb{F}_{2^8}$ is the input byte, $c \in \mathbb{F}_{2^8}$ is the affine constant, and the coefficients $a_i \in \mathbb{F}_{2^8}$ are fixed by M. For the r-th Frobenius conjugate, the embedded value is $\langle M(z)^{2^r} \rangle_{\lambda_\mathbb{F}} = \langle c^{2^r} \rangle_{\lambda_\mathbb{F}} + \sum_{i=0}^{7} \langle a_i^{2^r} \rangle_{\lambda_\mathbb{F}} \cdot \langle z^{2^{i+r}} \rangle_{\lambda_\mathbb{F}}$. After embedding into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$, each term $\langle a_i^{2^r} \rangle_{\lambda_\mathbb{F}} \cdot \langle z^{2^{i+r}} \rangle_{\lambda_\mathbb{F}}$ is evaluated as a multiplication in $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$. Concretely, the processor first computes an unreduced $2\lambda_\mathbb{F}$-bit carry-less product and then reduces it modulo the defining polynomial to obtain a $\lambda_\mathbb{F}$-bit field element. Thus, a direct evaluation of one byte-level conjugate performs one $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ multiplication and one field reduction for each nonzero coefficient a_i.

We denote the modular reduction from an unreduced $2\lambda_\mathbb{F}$-bit carry-less product to the $\lambda_\mathbb{F}$-bit field representation by $\mathrm{Red}_{\lambda_\mathbb{F}}$. A direct implementation reduces each product separately, computing $\langle c^{2^r} \rangle_{\lambda_\mathbb{F}} + \bigoplus_i \mathrm{Red}_{\lambda_\mathbb{F}}(\langle a_i^{2^r} \rangle_{\lambda_\mathbb{F}} \cdot \langle z^{2^{i+r}} \rangle_{\lambda_\mathbb{F}})$. Our implementation instead precomputes the $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ embeddings of the fixed SM4 byte-field constants and accumulates the unreduced carry-less products before applying one reduction, computing $\langle c^{2^r} \rangle_{\lambda_\mathbb{F}} + \mathrm{Red}_{\lambda_\mathbb{F}}(\bigoplus_i \langle a_i^{2^r} \rangle_{\lambda_\mathbb{F}} \cdot \langle z^{2^{i+r}} \rangle_{\lambda_\mathbb{F}})$. Since $\mathrm{Red}_{\lambda_\mathbb{F}}$ is $\mathbb{F}_2$-linear, the two expressions are equivalent, while the latter avoids repeated reductions inside each fixed linear combination. For the word-level maps L, L', L^{-1}, and $(L')^{-1}$, we apply the same idea across all products contributing to one output-byte conjugate. The norm-anchor check $h \cdot x^{16} \cdot x^2 + x = 0$ also uses the optimized $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$ multiplication path for its two field multiplications, namely first $\langle h \rangle_{\lambda_\mathbb{F}} \cdot \langle x^{16} \rangle_{\lambda_\mathbb{F}}$ and then the product with $\langle x^2 \rangle_{\lambda_\mathbb{F}}$.

Only computing and storing necessary conjugate tuples. In SM4th-d3, there are two kinds of constraint checks. The non-anchor checks verify the patched inversion relations $x^2 y + x = 0$ and $xy^2 + y = 0$ in S-boxes, where x and y are the input and output of the patched inversion function. The norm-anchor checks verify $h \cdot x^{16} \cdot x^2 + x = 0$, where x is the affine S-box input and h is the claimed norm witness corresponding to x^{-17}.

For a byte value $z \in \mathbb{F}_{2^8}$, we call $(\langle z \rangle_{\lambda_\mathbb{F}}, \langle z^2 \rangle_{\lambda_\mathbb{F}}, \ldots, \langle z^{128} \rangle_{\lambda_\mathbb{F}})$ its full conjugate tuple after embedding into $\mathbb{F}_{2^{\lambda_\mathbb{F}}}$. This tuple is only a local representation used by the constraint computation; it is not an additional component of the extended witness.

At a norm-anchor position, the full inverse byte is not stored as committed witness bits. The witness contains only the norm coordinate for $h = x^{-17}$. After checking $h \cdot x^{16} \cdot x^2 + x = 0$, the implementation reconstructs the inverse byte as $x^{-1} = x^{16} \cdot h$ and then applies the SM4 output affine map to obtain the S-box output byte. This reconstructed byte becomes a virtual key or state byte used by later SM4 rounds. Since such a virtual byte is not backed by committed output bits, its later propagation through byte-level affine maps and the word-level linear layers L and L' cannot recover the needed Frobenius conjugates by reloading witness bits. Therefore, the constraint computation materializes and carries the full conjugate tuple for these reconstructed virtual bytes.

For non-anchor positions, the corresponding byte values are already present in the extended witness. The patched-inversion check itself only needs $\langle x \rangle_{\lambda_\mathbb{F}}$, $\langle x^2 \rangle_{\lambda_\mathbb{F}}$, $\langle y \rangle_{\lambda_\mathbb{F}}$, and $\langle y^2 \rangle_{\lambda_\mathbb{F}}$. Thus, the implementation does not keep a full conjugate tuple solely for this check. If such a byte is later consumed by a linearized affine map or by the SM4 word-level linear layers, its tuple can be constructed on demand from the committed bits. This avoids carrying unnecessary conjugates for

ordinary S-box positions, while still retaining full tuples for norm-anchor bytes whose reconstructed virtual values must continue through the SM4 computation.

10.4 Performance Test Results

Table 22 reports the reference implementation on Intel without native SM4. We also report the table-based reference implementation in Table 23, including table-based SM4 adapted from OpenSSL, uBlock lookup tables, and table-based AES rounds in Vistrutah. Table 24 reports all 14 optimized parameter sets on this platform, and Table 25 reports the eight optimized SM4th and SM4th-EM parameter sets on Intel with native SM4. For the 48 configurations remeasured in the current batch, each timing result averages 600 retained sequences for optimized implementations or 60 for each reference mode. Milliseconds measure process CPU time, Mcycles count user-mode hardware CPU cycles, and the resident set size (RSS) is measured separately at system-call stops.

The term dn means the constraint systems in ZKPs are degree-n. The `s` variants have smaller signature sizes but slower running times, while the `f` variants have faster running times but larger signature sizes, achieving different trade-offs by using different VOLE parameters. The EM variants use the Even-Mansour constructions as OWFs with shorter witnesses and fewer constraints, leading to smaller signatures and lower running times in the reported measurements.

For SM4th and SM4th-EM, the sizes of the public key and the secret key are both 32 bytes. On Intel without native SM4, optimized signing takes 3.787–5.398 ms for the `f` variants and 27.564–37.495 ms for the `s` variants. Verification takes 3.530–4.637 ms and 24.586–31.524 ms, respectively. On Intel with native SM4, the lo variants sign in 1.231–1.687 ms for `f` and 9.373–11.961 ms for `s`, and verify in 0.963–1.269 ms and 6.575–7.332 ms, respectively. The ti variants on this platform sign in 5.263–6.789 ms for `f` and 41.346–49.467 ms for `s`, and verify in 5.174–6.350 ms and 38.814–44.105 ms, respectively. The smallest loose signature is 3 818 bytes for SM4th-EM-d2-128s-lo, and the largest is 6 724 bytes for SM4th-d3-128f-lo. Tight signatures are approximately 1.76–1.98 times as large as their corresponding loose signatures.

For uBlockith and uBlockith-EM, the sizes of the public key and the secret key are both 64 bytes. On Intel without native SM4, optimized signing takes 11.156–12.398 ms for the `f` variants and 87.006–89.008 ms for the `s` variants. Verification takes 10.624–11.982 ms and 81.197–89.653 ms, respectively. The smallest signature is 19 056 bytes for uBlockith-EM-d3-256s, and the largest is 31 556 bytes for uBlockith-d3-256f.

For Vistrutith, the sizes of the public key and the secret key are both 128 bytes. On Intel without native SM4, the optimized Vistrutith-d3-512f implementation signs in 52.110 ms and verifies in 47.220 ms, while Vistrutith-d3-512s takes 324.979 ms and 325.239 ms, respectively. The `s` variant has a signature size of 83 004 bytes, compared with 106 788 bytes for the `f` variant.

Table 22: Reference implementation results on Intel Xeon 6982P-C without native SM4

Scheme	Operation cost						Sizes in B			Observed RSS (KiB)
	KeyGen		Sign		Verify		sk	pk	Sig	
	ms	Mcycles	ms	Mcycles	ms	Mcycles				
SM4th-d3-128s-lo	0.025667	0.091792	5959.251	21338.656	5934.565	21250.072	32	32	5 056	10 308
SM4th-EM-d2-128s-lo	0.025733	0.091631	4191.906	15009.834	4184.809	14984.325	32	32	3 818	7 444
SM4th-d3-128f-lo	0.025383	0.090575	842.347	3016.373	818.962	2932.630	32	32	6 724	2 988
SM4th-EM-d2-128f-lo	0.025117	0.089890	572.249	2049.233	566.031	2026.888	32	32	4 932	2 480
SM4th-d3-128s-ti	0.025467	0.091441	123.678	442.509	94.580	338.336	32	32	9 176	15 364
SM4th-EM-d2-128s-ti	0.026733	0.094896	62.237	221.944	58.841	209.892	32	32	7 556	11 432
SM4th-d3-128f-ti	0.025483	0.091298	62.061	222.089	37.077	132.823	32	32	11 864	3 496
SM4th-EM-d2-128f-ti	0.025933	0.089682	9.653	33.993	8.911	31.491	32	32	9 450	2 872
uBlockith-d3-256s	0.007933	0.028287	236.775	840.170	239.586	850.269	64	64	24 144	15 584
uBlockith-EM-d3-256s	0.007817	0.027508	203.242	721.446	199.326	706.711	64	64	19 056	14 260
uBlockith-d3-256f	0.008117	0.029020	44.914	160.681	47.520	169.981	64	64	31 556	8 732
uBlockith-EM-d3-256f	0.007717	0.027188	35.830	127.312	37.622	134.180	64	64	25 028	3 480
Vistrutith-d3-512s	0.020150	0.071889	44662.880	159886.974	44610.418	159692.031	128	128	83 004	51 772
Vistrutith-d3-512f	0.020467	0.072917	5125.868	18345.856	5058.322	18106.366	128	128	106 788	8 584

Table 23: Reference table-based implementation results on Intel Xeon 6982P-C without native SM4

Scheme	Operation cost						Sizes in B			Observed RSS (KiB)
	KeyGen		Sign		Verify		sk	pk	Sig	
	ms	Mcycles	ms	Mcycles	ms	Mcycles				
SM4th-d3-128s-lo	0.005100	0.017374	90.981	325.612	75.455	270.012	32	32	5 056	10 256
SM4th-EM-d2-128s-lo	0.005050	0.017145	49.720	177.941	46.814	167.485	32	32	3 818	7 484
SM4th-d3-128f-lo	0.004300	0.015065	33.598	120.251	22.196	79.449	32	32	6 724	3 052
SM4th-EM-d2-128f-lo	0.003850	0.012628	7.791	27.176	7.318	25.790	32	32	4 932	2 488
SM4th-d3-128s-ti	0.006400	0.022468	114.937	411.240	90.466	323.620	32	32	9 176	15 416
SM4th-EM-d2-128s-ti	0.006033	0.021259	63.434	226.152	60.141	214.364	32	32	7 556	11 440
SM4th-d3-128f-ti	0.005950	0.021062	52.553	188.103	32.473	116.211	32	32	11 864	3 576
SM4th-EM-d2-128f-ti	0.004083	0.013629	9.580	33.792	8.849	31.269	32	32	9 450	2 912
uBlockith-d3-256s	0.008183	0.028715	237.820	841.896	240.556	851.531	64	64	24 144	15 560
uBlockith-EM-d3-256s	0.008250	0.029111	204.238	722.513	200.408	707.913	64	64	19 056	14 268
uBlockith-d3-256f	0.008200	0.028780	45.020	161.016	47.605	170.321	64	64	31 556	8 744
uBlockith-EM-d3-256f	0.007483	0.026310	35.939	127.449	37.649	134.169	64	64	25 028	3 512
Vistrutith-d3-512s	0.007367	0.026333	1749.675	6235.359	1726.290	6145.472	128	128	83 004	51 696
Vistrutith-d3-512f	0.006917	0.024578	328.539	1169.190	293.642	1046.312	128	128	106 788	8 624

Table 24: Optimized implementation on Intel Xeon 6982P-C without native SM4

Scheme	Operation cost						Sizes in B			Observed RSS (KiB)
	KeyGen		Sign		Verify		sk	pk	Sig	
	ms	Mcycles	ms	Mcycles	ms	Mcycles				
SM4th-d3-128s-lo	0.014640	0.051631	30.483	108.997	25.383	90.766	32	32	5 056	10 388
SM4th-EM-d2-128s-lo	0.014767	0.052196	27.564	98.567	24.586	87.944	32	32	3 818	7 592
SM4th-d3-128f-lo	0.014395	0.050673	4.312	15.331	3.861	13.740	32	32	6 724	3 124
SM4th-EM-d2-128f-lo	0.014197	0.050224	3.787	13.479	3.530	12.565	32	32	4 932	2 536
SM4th-d3-128s-ti	0.015200	0.054164	37.495	133.992	31.524	112.641	32	32	9 176	15 560
SM4th-EM-d2-128s-ti	0.015005	0.053326	32.154	114.862	29.313	104.720	32	32	7 556	11 684
SM4th-d3-128f-ti	0.015160	0.053710	5.398	19.230	4.637	16.520	32	32	11 864	3 820
SM4th-EM-d2-128f-ti	0.014840	0.052256	4.178	14.869	3.857	13.737	32	32	9 450	3 076
uBlockith-d3-256s	0.008102	0.028607	89.008	312.756	89.653	314.554	64	64	24 144	15 464
uBlockith-EM-d3-256s	0.008208	0.028842	87.006	305.114	81.197	284.855	64	64	19 056	14 308
uBlockith-d3-256f	0.008057	0.028270	12.398	43.235	11.982	41.946	64	64	31 556	3 788
uBlockith-EM-d3-256f	0.007908	0.027731	11.156	38.941	10.624	37.504	64	64	25 028	3 544
Vistrutith-d3-512s	0.005335	0.017878	324.979	1143.500	325.239	1141.299	128	128	83 004	51 664
Vistrutith-d3-512f	0.004978	0.016707	52.110	181.471	47.220	165.293	128	128	106 788	8 608

Table 25: Optimized SM4 implementations on Intel Xeon 6992E+C with native SM4

Scheme	Operation cost						Sizes in B			Observed RSS (KiB)
	KeyGen		Sign		Verify		sk	pk	Sig	
	ms	Mcycles	ms	Mcycles	ms	Mcycles				
SM4th-d3-128s-lo	0.003367	0.010501	11.961	38.169	7.332	23.396	32	32	5 056	10 588
SM4th-EM-d2-128s-lo	0.003690	0.011455	9.373	29.912	6.575	20.980	32	32	3 818	7 728
SM4th-d3-128f-lo	0.003138	0.009660	1.687	5.385	1.269	4.049	32	32	6 724	3 356
SM4th-EM-d2-128f-lo	0.003142	0.009723	1.231	3.927	0.963	3.072	32	32	4 932	2 684
SM4th-d3-128s-ti	0.003677	0.011491	49.467	157.865	44.105	140.754	32	32	9 176	15 864
SM4th-EM-d2-128s-ti	0.003540	0.010981	41.346	131.904	38.814	123.825	32	32	7 556	11 884
SM4th-d3-128f-ti	0.003178	0.009868	6.789	21.665	6.350	20.266	32	32	11 864	3 980
SM4th-EM-d2-128f-ti	0.003220	0.009848	5.263	16.797	5.174	16.512	32	32	9 450	3 332

References

[1] Goldwasser S, Micali S, Rackoff C. The knowledge complexity of interactive proof-systems (extended abstract)[C]//ACM Symposium on Theory of Computing (STOC). New York: ACM, 1985: 291–304.

[2] Information security technology—SM3 cryptographic hash algorithm: GB/T 32905-2016[S]. Standardization Administration of China, 2016.

[3] Information security technology—SM4 block cipher algorithm: GB/T 32907-2016[S]. Standardization Administration of China, 2016.

[4] Wu W, Zhang L, Zheng Y, et al. The design of uBlock block cipher (in Chinese)[EB/OL]. (2019-12-10)[2026-04-01]. https://sfjs.cacrnet.org.cn/site/term/list_76_1.html.

[5] Cui T, Wang M, Fan Y, et al. The design of Ballet block cipher (in Chinese)[EB/OL]. (2019-12-10)[2026-04-01]. https://sfjs.cacrnet.org.cn/site/term/list_76_1.html.

[6] Avanzi R, Chakraborti A, Chakraborty B, et al. The large block cipher Vistrutah[J]. IACR Trans. Symmetric Cryptol., 2025, 2025(3): 1–150.

[7] Institute of Commercial Cryptography Standards. Submission Requirements for Block Cipher Algorithms (Draft)[EB/OL]. 2026[2026-10-21]. https://niccs.org.cn/symmbzyjy/%E9%80%9A%E7%9F%A5%E5%85%AC%E5%91%8A/tHGHcTNN.pdf.

[8] Fiat A, Shamir A. How to prove yourself: Practical solutions to identification and signature problems[C]//Annual International Cryptology Conference (CRYPTO). Berlin: Springer, 1986: 186–194.

[9] Baum C, Dittmer S, Scholl P, et al. Sok: vector ole-based zero-knowledge protocols[J]. Designs, Codes and Cryptography, 2023, 91(11): 3527–3561.

[10] Baum C, Braun L, de Saint Guilhem C D, et al. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head[C]//Annual Cryptology Conference (CRYPTO). Berlin: Springer, 2023: 581–615.

[11] Yang K, Sarkar P, Weng C, et al. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field[C]//ACM SIGSAC Conference on Computer and Communications Security (CCS). New York: ACM, 2021: 2986–3001.

[12] Chase M, Derler D, Goldfeder S, et al. Post-quantum zero-knowledge and signatures from symmetric-key primitives[C]//ACM SIGSAC Conference on Computer and Communications Security (CCS). New York: ACM, 2017: 1825–1842.

[13] Ishai Y, Kushilevitz E, Ostrovsky R, et al. Zero-knowledge from secure multiparty computation[C]//ACM Symposium on Theory of Computing (STOC). New York: ACM, 2007: 21–30.

[14] Albrecht M R, Rechberger C, Schneider T, et al. Ciphers for MPC and FHE[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT). Berlin: Springer, 2015: 430–454.

[15] Feneuil T, Rivain M. Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments[J]. Journal of Cryptology, 2025, 38(3): 28.

[16] Baum C, de Saint Guilhem C D, Kales D, et al. Banquet: Short and fast signatures from AES [C]//Public-Key Cryptography (PKC). Berlin: Springer, 2021: 266–297.

[17] Adj G, Aragon N, Barbero S, et al. Mirath signature scheme: Algorithm specifications and supporting documentation, version 2.0[EB/OL]. (2025-07-25)[2026-04-01]. https://pqc-mirath.org/assets/downloads/mirath_v2.0.1.pdf.

[18] Benadjila R, Feneuil T, Rivain M. MQ on my mind: Post-quantum signatures from the non-structured multivariate quadratic problem[C]//IEEE European Symposium on Security and Privacy (EuroS&P). Los Alamitos: IEEE, 2024: 468–485.

[19] Bettaieb S, Bidoux L, Dyseryn V, et al. PERK: compact signature scheme based on a new variant of the permuted kernel problem[J]. Designs, Codes and Cryptography, 2024, 92(8): 2131–2157.

[20] Bidoux L, Chi-Domínguez J, Feneuil T, et al. RYDE: A digital signature scheme based on Rank-Syndrome-Decoding problem with MPCitH paradigm[J]. CoRR, 2023, abs/2307.08726.

[21] Melchor C A, Bettaieb S, Bidoux L, et al. The syndrome decoding in the head (SD-in-the-head) signature scheme: Algorithm specifications and supporting documentation, version 2.0 [EB/OL]. (2025-02-05)[2026-04-01]. https://sdith.org/docs/sdith-v2.0.pdf.
[22] Baum C, Beullens W, Braun L, et al. FAEST v2: Algorithm specifications version 2.0[EB/OL]. (2025)[2026-04-01]. https://faest.info/faest-spec-v2.0.pdf.
[23] Cui H, Liu H, Yan D, et al. ReSolveD: Shorter signatures from regular syndrome decoding and VOLE-in-the-head[C]//Public-Key Cryptography (PKC). Berlin: Springer, 2024.
[24] Bai S, Ducas L, Kiltz E, et al. CRYSTALS-Dilithium algorithm specifications and supporting documentation (version 3.1)[EB/OL]. (2021-02-08)[2026-04-01]. https://pq-crystals.org/dilithium/data/dilithium-specification-round3-20210208.pdf.
[25] Fouque P A, Lyubashevsky V, Hoffstein J, et al. Falcon: Fast-Fourier lattice-based compact signatures over NTRU[EB/OL]. (2021-01-10)[2026-04-01]. https://falcon-sign.info/falcon.pdf.
[26] Bos J W, Bronchain O, Ducas L, et al. HAWK: Version 1.1[EB/OL]. (2025-02-05)[2026-04-01]. https://hawk-sign.info/hawk-spec.pdf.
[27] Baldi M, Barenghi A, Battagliola M, et al. CROSS: Codes and restricted objects signature scheme, algorithm specifications and supporting documentation, version 2.2[EB/OL]. (2025-07-31)[2026-04-01]. https://www.cross-crypto.com/CROSS_Specification_v2.2.pdf.
[28] Baldi M, Barenghi A, Beckwith L, et al. LESS: Linear equivalence signature scheme[EB/OL]. 2025-02-07[2026-04-25]. https://www.less-project.com/LESS-2025-02-07.pdf.
[29] Beullens W, Campos F, Celi S, et al. MAYO: Round 2 version[EB/OL]. (2025)[2026-04-25]. https://pqmayo.org/assets/specs/mayo-round2.pdf.
[30] Aardal M A, Adj G, Aranha D F, et al. SQIsign: Algorithm specifications and supporting documentation, version 2.0.1[EB/OL]. (2025-07-07)[2026-04-25]. https://sqisign.org/spec/sqisign-20250707.pdf.
[31] Aumasson J P, Bernstein D J, Beullens W, et al. SPHINCS+: Submission to the NIST post-quantum project, v.3.1[EB/OL]. (2022-06-10)[2026-04-01]. https://sphincs.org/data/sphincs+-r3.1-specification.pdf.
[32] Goldreich O, Goldwasser S, Micali S. How to construct random functions (extended abstract)[C]//Annual Symposium on Foundations of Computer Science (FOCS). Los Alamitos: IEEE, 1984: 464–479.
[33] Baum C, Beullens W, Mukherjee S, et al. One tree to rule them all: Optimizing GGM trees and owfs for post-quantum signatures[C]//International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT). Berlin: Springer, 2024: 463–493.
[34] Roy L. Softspokenot: Quieter OT extension from small-field silent VOLE in the minicrypt model[C]//Annual International Cryptology Conference (CRYPTO). Berlin: Springer, 2022: 657–687.
[35] Even S, Mansour Y. A construction of a cipher from a single pseudorandom permutation[J]. Journal of Cryptology, 1997, 10(3): 151–162.
[36] Chiesa A, Ojha D, Spooner N. Fractal: Post-quantum and transparent recursive proofs from holography[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT). Berlin: Springer, 2020: 769–793.
[37] Dobraunig C, Kales D, Rechberger C, et al. Shorter signatures based on Tailor-Made minimalist symmetric-key crypto[C]//ACM SIGSAC Conference on Computer and Communications Security (CCS). New York: ACM, 2022: 843–857.
[38] Zou J, Guo N, Li J. Efficient quantum circuit implementations of SM3 and RIPEMD-160 hash functions (in Chinese)[J]. Journal on Communications, 2025, 46(12): 222–233.
[39] National Institute of Standards and Technology. Submission requirements and evaluation criteria for the Post-Quantum Cryptography standardization process[EB/OL]. 2016. https://csrc.nist.gov/csrc/media/projects/post-quantum-cryptography/documents/call-for-proposals-final-dec-2016.pdf.
[40] Grassl M, Langenberg B, Roetteler M, et al. Applying grover's algorithm to AES: quantum resource estimates[C]//Post-Quantum Cryptography - International Workshop (PQCrypto). Berlin: Springer, 2016: 29–43.

[41] Liu M, Chen J. Improved linear attacks on the Chinese block cipher standard[J]. Journal of Computer Science and Technology, 2014, 29(6): 1123–1133.
[42] Wang C, Cui J, Li M, et al. Improved integral cryptanalysis on block cipher uBlock (in Chinese)[J]. Journal of Electronics & Information Technology, 2024, 46(5): 2149–2158.
[43] Institute of Commercial Cryptography Standards. NGCC Public-Key API Package (API_PKC.zip)[EB/OL]. 2025[2026-10-26]. https://niccs.org.cn/niccs/Download/API/API_PKC.zip.
[44] Institute of Commercial Cryptography Standards. Submission Requirements for Public-Key Cryptographic Algorithms[EB/OL]. 2025[2026-10-26]. https://www.niccs.org.cn/niccs/Notice/1975896137741635584/lDop1mav.pdf.
[45] Liu F, Ji W, Hu L, et al. Analysis of the SMS4 block cipher[C]//Information Security and Privacy - Australasian Conference (ACISP). Berlin: Springer, 2007: 472–492.
[46] Giacon F, Heuer F, Poettering B. KEM combiners[C]//Public-Key Cryptography (PKC). Berlin: Springer, 2018: 190–218.
[47] Don J, Fehr S, Huang Y. Adaptive versus static multi-oracle algorithms, and quantum security of a split-key PRF[C]//Theory of Cryptography - International Conference (TCC). Berlin: Springer, 2022: 33–51.
[48] Baum C, Beullens W, Braun L, et al. FAEST v3: Algorithm specifications, version 3.0[EB/OL]. (2026-09-01)[2026-10-29]. https://faest.info/faest-spec-v3.0.pdf.
[49] Sarah D., Peter C. On the practical cost of Grover for AES key recovery[C]//The 5th NIST PQC Standardization Conference. 2024.
[50] Lin D, Xiang Z, Zhang R, et al. Quantum implementation of SM4 (in Chinese)[J]. Journal of Cryptologic Research, 2021, 8(6): 999–1018.
[51] Chen J, Liu Q, Fan Y, et al. New SAT-based model for quantum circuit decision problem: Searching for low-cost quantum implementation[J/OL]. IACR Communications in Cryptology, 2024, 1(1). https://cic.iacr.org/p/1/1/31. https://doi.org/10.62056/anmmp-4c2h.
[52] Chen H, Cai B, Gao F, et al. Quantum circuit for implementing AES S-box with low costs [EB/OL]. 2025. https://eprint.iacr.org/2025/454.
[53] Brassard G, Høyer P, Tapp A. Quantum cryptanalysis of hash and claw-free functions[C]//LATIN 1998: Theoretical Informatics. 1998: 163–169.
[54] Karatsuba A, Ofman Y. Multiplication of many-digital numbers by automatic computers[J]. Proceedings of the USSR Academy of Sciences, 1962, 145: 293–294.